

GeneralUpdate

中文文档合集 · Documentation

来源: GeneralUpdate-Samples / website/docs

文档数量: 29 篇

生成时间: 2026-09-30

目录 Contents

快速开始 Quick Start

- GeneralUpdate 入门实战手册
- GeneralUpdate.Tools
- GeneralUpdate.Avalonia.Android 实战手册
- GeneralUpdate.Maui.Android 实战手册

组件参考 Components

- Component Introduction
- GeneralUpdate.Core
- GeneralUpdate.Bowl
- GeneralUpdate.Differential
- GeneralUpdate.DriveLution
- GeneralUpdate.Extension
- GeneralUpdate.Avalonia.Android
- GeneralUpdate.Maui.Android
- GeneralUpdate.Core — 执行流程详解
- GeneralUpdate.Bowl — 执行流程详解
- GeneralUpdate.DriveLution — 执行流程详解
- GeneralUpdate.Extension — 执行流程详解
- GeneralUpdate.Avalonia.Android — 执行流程详解
- GeneralUpdate.Maui.Android — 执行流程详解
- GeneralSpacestation
- UpgradeHub

发布日志 Release Log

-  Release log

Agent 技能 Agent Skills

-  GeneralUpdate Agent Skills 套件
-  GeneralUpdate 集成完全指南
-  GeneralUpdate 更新界面生成指南
-  选择你的更新策略
-  GeneralUpdate 高级定制参考
-  GeneralUpdate 故障排查
-  GeneralUpdate 迁移指南
-  GeneralUpdate 安全审计指南

快速开始 Quick Start

GeneralUpdate 入门实战手册

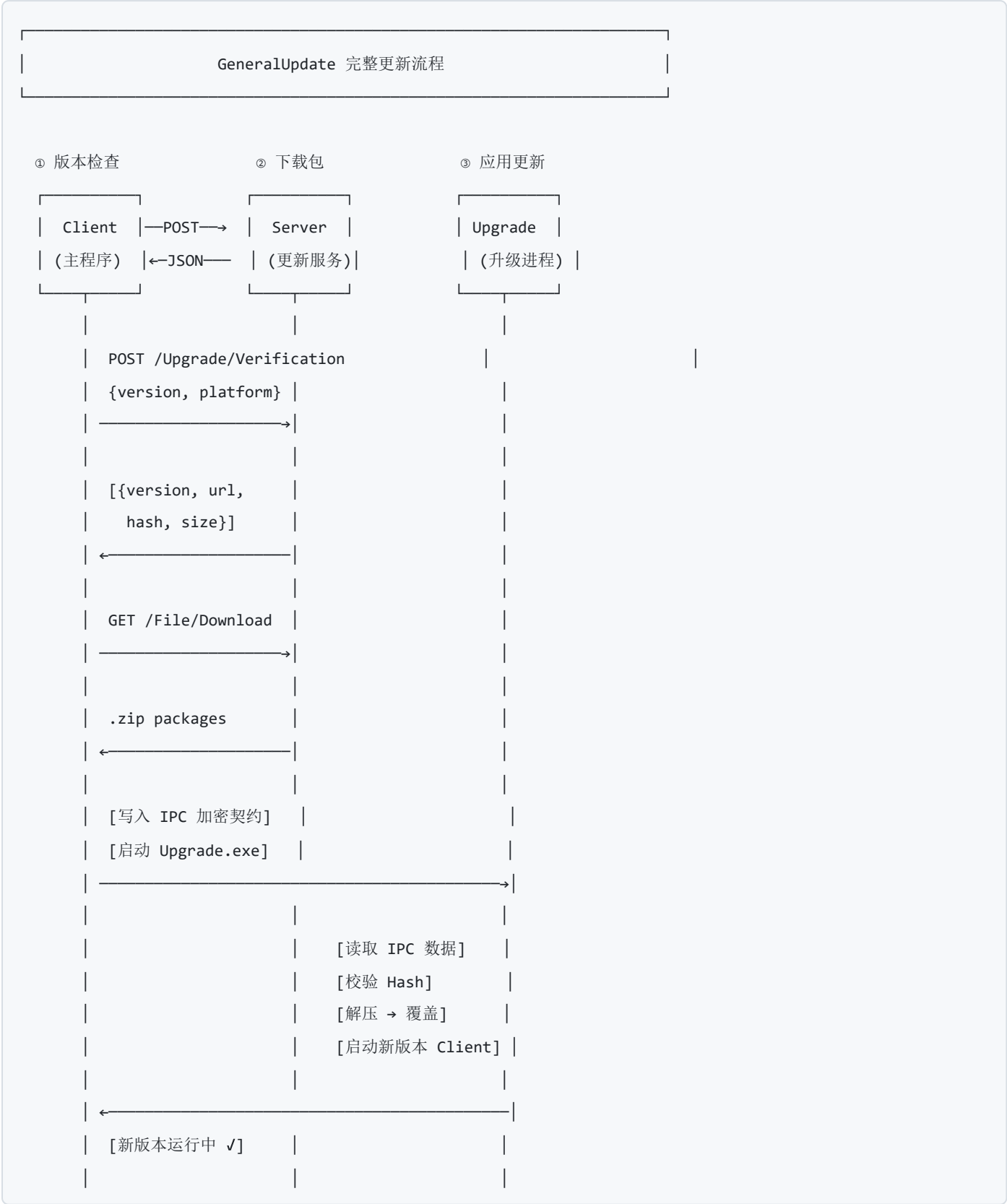
这篇手册面向第一次接触 GeneralUpdate 的开发者。目标不是一次讲完所有 API，而是让你从零开始，写完 Client → 写完 Upgrade → 用 Tools 生成配置 → 启动 Server → 一条命令跑通完整的"发现更新 → 下载 → 应用 → 回到新版本"闭环。

你将学到什么

完成这篇手册后，你会拥有一个**可运行的端到端自动更新系统**：一个能检查更新的 Client、一个能应用更新的 Upgrade、一份自动生成的 manifest、以及一个本地测试 Server。

视频演示：<https://www.bilibili.com/video/BV1EgEP6oExY>

更新流程总览



角色	在项目中的位置	负责什么
Client	MyApp.exe (你自己的主程序)	检查更新 → 下载包 → 启动 Upgrade
Upgrade	MyApp.Upgrade.exe (独立的升级进程)	校验 Hash → 解压覆盖 → 启动新版 Client
Server	GeneralUpdate-Samples/src/Server	返回版本信息、接收更新报告、提供补丁下载

角色	在项目中的位置	负责什么
Tools	GeneralUpdate.Tools	生成 <code>generalupdate.manifest.json</code> ，串联 Client 与 Upgrade

Phase 1：环境准备

开始之前

所有后续步骤都依赖这两个工具。请先用验证命令确认它们安装正确，否则 Phase 4 的 `dotnet publish` 和 Phase 5 的 Server 启动会失败。

安装清单

项	要求	验证命令
.NET SDK	8.0+（推荐 10.0）	<code>dotnet --version</code>
Git	任意版本	<code>git --version</code>

新建项目

打开终端，创建 Client 和 Upgrade 两个控制台项目：

```
dotnet new console -n MyApp
dotnet new console -n MyApp.Upgrade

cd MyApp
dotnet add package GeneralUpdate.Core

cd ../MyApp.Upgrade
dotnet add package GeneralUpdate.Core
```

预期结果：两个项目目录下都有 `GeneralUpdate.Core` 的 NuGet 引用。

Phase 2：集成 Client 代码

在 `MyApp/Program.cs` 中粘贴以下全部内容。Client 负责：检查更新 → 下载包 → 启动 Upgrade。

```

using GeneralUpdate.Core;
using GeneralUpdate.Core.Configuration;
using GeneralUpdate.Core.Event;

// =====
// Client - 主程序入口，负责检测和下载更新
// =====

// 只需设置 3 个 secret。MainAppName、ClientVersion、UpdateAppName
// 等身份字段由框架自动从 generalupdate.manifest.json 发现，无需手写。
var request = new UpdateRequest
{
    UpdateUrl    = "http://localhost:5000/Upgrade/Verification",
    ReportUrl    = "http://localhost:5000/Upgrade/Report",
    AppSecretKey = "dfeb5833-975e-4afb-88f1-6278ee9aeff6",
};

await new GeneralUpdateBootstrap()
    .SetConfig(request)
    .SetOption(Option.AppType, AppType.Client)
    .AddListenerUpdateInfo((_, e) =>
    {
        Console.WriteLine($"发现 {e.Info?.Body?.Count ?? 0} 个可用更新");
        if (e.Info?.Body != null)
        {
            foreach (var v in e.Info.Body)
                Console.WriteLine($"  {v.Version} - {v.Name} ({v.Size} bytes)");
        }
    })
    .AddListenerMultiDownloadStatistics((_, e) =>
    {
        Console.WriteLine($"\\r下载: {e.ProgressPercentage:F0}% {e.Speed}");
    })
    .AddListenerMultiDownloadCompleted((_, e) =>
    {
        Console.WriteLine();
        Console.WriteLine($"下载 {(e.IsCompleted ? "√ 完成" : "X 失败")}");
    })
    .AddListenerException((_, e) =>
    {
        Console.WriteLine($"异常: {e.Exception.Message}");
    })
    .LaunchAsync();

```

为什么只设 3 个参数？

ClientStrategy 在执行时会自动调用 AppMetadataDiscoverer.Discover()，从 generalupdate.manifest.json (Phase 4 用 Tools 生成) 中读取身份字段并补全 UpdateRequest：

字段	谁负责	说明
UpdateUrl	你写代码	Server 验证接口地址
ReportUrl	你写代码	Server 报告接口地址
AppSecretKey	你写代码	应用密钥
MainAppName	框架自动发现	从 manifest 读取
ClientVersion	框架自动发现	从 manifest 读取
UpdateAppName	框架自动发现	从 manifest 读取
ProductId	框架自动发现	从 manifest 读取
UpdatePath	框架自动发现	从 manifest 读取
InstallPath	默认值	AppDomain.CurrentDomain.BaseDirectory

如果你不想写 UpdateRequest，还可以用更简短的 SetSource()：

```
await new GeneralUpdateBootstrap()
    .SetSource("http://localhost:5000/Upgrade/Verification",
               "dfef5833-975e-4afb-88f1-6278ee9aeff6",
               "http://localhost:5000/Upgrade/Report")
    .SetOption(Option.AppType, AppType.Client)
    // ... listeners
    .LaunchAsync();
```

Client 不直接应用更新——下载完成后写 IPC 加密契约 → 启动 Upgrade → 自身退出。

关键设计

Client 只负责发现和下载，文件替换工作完全交给独立的 Upgrade 进程。这是因为运行中的进程无法覆盖自身的可执行文件——通过 IPC 加密契约把控制权交给 Upgrade，是实现"自己更新自己"的核心架构。

Phase 3：集成 Upgrade 代码

在 MyApp.Upgrade/Program.cs 中粘贴以下全部内容。Upgrade 比 Client 更简单——不需要 SetConfig()，所有参数通过加密 IPC 文件从 Client 传入。

```
using GeneralUpdate.Core;
using GeneralUpdate.Core.Configuration;
using GeneralUpdate.Core.Event;

// =====
// Upgrade - 独立的升级进程 (MyApp.Upgrade.exe)
// 无需 SetConfig(), 参数由 Client 通过加密 IPC 文件传入
// =====

await new GeneralUpdateBootstrap()
    .SetOption(Option.AppType, AppType.Upgrade)
    .AddListenerMultiDownloadStatistics((_, e) =>
    {
        Console.WriteLine($"应用补丁: {e.ProgressPercentage:F0}%");
    })
    .AddListenerMultiDownloadCompleted((_, e) =>
    {
        Console.WriteLine();
        Console.WriteLine($"补丁 {(e.IsCompleted ? "√ 完成" : "X 失败")}");
    })
    .AddListenerException((_, e) =>
    {
        Console.WriteLine($"异常: {e.Exception.Message}");
    })
    .LaunchAsync();
```

Client vs Upgrade 差异

	Client	Upgrade
AppType	AppType.Client	AppType.Upgrade
SetConfig()	✔ 必须, 设置服务器地址、版本等	✘ 不需要, 通过 IPC 从 Client 获取
职责	检查版本 → 下载包 → 启动 Upgrade	Hash 校验 → 解压覆盖 → 启动新版 Client
谁启动谁	由用户直接启动	由 Client 进程自动启动

Phase 4: 使用 Tools 生成项目结构

Client 和 Upgrade 代码写好后, 用 Tools 的配置生成器 (Config 标签页) 一键 dotnet publish 并生成 generalupdate.manifest.json 。

操作步骤

打开 GeneralUpdate.Tools → 切换到 Config 标签页 (配置生成器) :

1. **Client .csproj Path**: 点击 Browse, 选择 `MyApp.csproj`
2. **Upgrade .csproj Path**: 点击 Browse, 选择 `MyApp.Upgrade.csproj`
3. 点击 **Analyze**: Tools 解析两个 .csproj, 自动填入:
 - `MainAppName`: 来自 Client 项目的 AssemblyName (如 `MyApp.exe`)
 - `UpdateAppName`: 来自 Upgrade 项目的 AssemblyName (如 `MyApp.Upgrade.exe`)
 - `ClientVersion` / `UpgradeClientVersion`: 默认 `1.0.0`, 按需修改
4. 检查自动填入的字段, 确认 `UpdatePath` 为 `update/` (默认值)
5. 点击 **Generate Sample**: Tools 依次执行:
 - `dotnet publish` Client → 输出到根目录
 - `dotnet publish` Upgrade → 输出到 `update/` 子目录
 - 写入 `generalupdate.manifest.json` 到根目录

发布后的目录结构

Tools 可以帮助你一键 `dotnet publish` 并组装输出目录 (通过 Config 标签页的 Sample Publisher)。最终结构如下:

```
publish/
├─ MyApp.exe                # Client publish 产物 (Phase 2)
├─ MyApp.dll
├─ GeneralUpdate.Core.dll
├─ generalupdate.manifest.json # Tools 生成, 放根目录, 框架自动发现
└─ update/                  # Upgrade publish 产物 (Phase 3), 子目录名来自 manifest 的 updatePath
    ├─ MyApp.Upgrade.exe
    └─ MyApp.Upgrade.dll
```

注意: `MyApp.Upgrade.exe` 不在根目录, 而是在 `update/` 子目录下。框架根据 manifest 的 `updatePath` 字段 (默认 `"update/"`) 找到并启动它。

目录结构错误会导致升级失败

Upgrade 进程(.exe) **必须**放在 manifest 指定的 `updatePath` 子目录下。如果 Upgrade 不在该位置, Client 将无法找到并启动升级进程, 整个更新链路中断。常见错误: 把 Upgrade 放在根目录或不同名称的子目录中。

generalupdate.manifest.json 示例

```
{
  "mainAppName": "MyApp.exe",
  "clientVersion": "1.0.0.0",
  "updateAppName": "MyApp.Upgrade.exe",
  "upgradeClientVersion": "1.0.0.0",
  "appType": "Client",
  "productId": "2d974e2a-31e6-4887-9bb1-b4689e98c77a",
  "updatePath": "update/"
}
```

注意：manifest 只存身份/结构信息，不存 secret。 UpdateUrl、ReportUrl、AppSecretKey 等敏感字段始终写在代码里，由 Phase 2 的 request.XXX = "..." 提供。

Phase 5：启动 Server

Client / Upgrade / manifest 全部就绪，启动 Server 准备端到端验证。

```
git clone https://github.com/GeneralLibrary/GeneralUpdate-Samples.git
cd GeneralUpdate-Samples/src/Server
dotnet run
```

预期输出：

```
┌────────────────────────────────────────────────────────────────────────────────┐
│ GeneralUpdate Sample Upgrade Server                                           │
└────────────────────────────────────────────────────────────────────────────────┘
Verification: http://localhost:5000/Upgrade/Verification
Report:       http://localhost:5000/Upgrade/Report
Download:     http://localhost:5000/File/Download/{hash}
Packages:     N version(s) loaded
```

Server 三端点速查

端点	方法	用途
/Upgrade/Verification	POST	Client 上报当前版本 → Server 返回可用更新列表
/Upgrade/Report	POST	Client 上报更新结果（成功 / 失败）
/File/Download/{hash}	GET	下载更新包（支持 HTTP Range 断点续传）

Server 读取 wwwroot/packages/versions.json 中的包元数据并对外提供下载。

快速验证：

```
curl -X POST http://localhost:5000/Upgrade/Verification \  
-H "Content-Type: application/json" \  
-d '{"Version":"1.0.0.0"}'
```

应返回包含可用更新版本的 JSON 响应。

Phase 6: 端到端运行验证

```
# 终端 1 - 确保 Server 在运行  
cd GeneralUpdate-Samples/src/Server && dotnet run  
  
# 终端 2 - 运行 Client  
cd MyApp && dotnet run
```

完整预期输出

```
Client:  
[UpdateInfo] 发现 1 个可用更新  
v2.0.0.0 - client_1.0.0_to_2.0.0 (xxxxx bytes)  
下载: 45% 1.2MB/s  
下载: 100%  
下载 ✓ 完成  
→ 启动 MyApp.Upgrade.exe...  
  
Upgrade:  
应用补丁: 50%  
应用补丁: 100%  
补丁 ✓ 完成  
→ 启动 MyApp.exe (新版本)...  
  
Client (新版本):  
MyApp v2.0.0.0 ✓
```


排查清单

Use this table to diagnose common issues quickly. Most failures trace back to one of these five checks.

现象	检查
Client 连不上 Server	<code>curl -X POST http://localhost:5000/Upgrade/Verification</code> 确认 Server 在跑
返回"Already up to date"	<code>versions.json</code> 中是否有比 <code>ClientVersion</code> 更高的版本
下载完成但未启动 Upgrade	<code>MyApp.Upgrade.exe</code> 是否在 <code>update/</code> 子目录下, manifest 的 <code>updatePath</code> 是否正确
Upgrade 报错退出	检查目录写入权限、杀毒软件是否拦截
端口被占用	修改 Server 启动参数 <code>--UrIs http://0.0.0.0:5001</code> , 同步修改 Client 的 <code>UpdateUr1</code>

下一步

跑通这条闭环后，建议按顺序深入阅读：

- 1. [GeneralUpdate.Core](#)：更新策略、事件通知、静默更新、manifest 极简配置
- 2. [GeneralUpdate.Differential](#)：差分算法、并行处理和 Clean/Dirty
- 3. [GeneralUpdate.Bowl](#)：崩溃监控、备份和失败恢复
- 4. [GeneralUpdate.Tools](#)：补丁包、Hash、OSS Config、Simulation 的完整操作指南

安装目录注意事项

更新过程中，应用程序需要对安装目录进行读写操作。为避免触发 Windows UAC（用户账户控制）弹窗，请将应用安装在以下非系统保护的目录：

推荐目录	路径示例
用户 AppData（推荐）	<code>C:\Users\<用户名>\AppData\Local\<你的应用名></code>
用户主目录	<code>C:\Users\<用户名>\<你的应用名></code>
非系统盘自定义目录	<code>D:\<你的应用名></code>

不推荐的目录：

- `C:\Program Files\` — 需要管理员权限才能写入
- `C:\Program Files (x86)\` — 需要管理员权限才能写入
- `C:\Windows\` — 系统目录，禁止写入
- 系统盘根目录（如 `c:\`） — 可能触发权限限制

提示：如果在更新时遇到"权限不足"或 UAC 弹窗，请检查应用是否安装在不推荐的目录中。将应用迁移到用户目录通常可以解决此问题。

关于加密文件与加壳程序

注意：使用加壳工具（如 Themida、VMProtect）、代码混淆器或加密的二进制文件**无法进行二进制差分更新**。加壳/加密会导致整个文件在新旧版本间完全不同，差分算法失效，补丁包体积等同于全量文件。如需差

分更新，请确保原始发布文件未被加壳或加密。

示例仓库

仓库	地址
Samples (Server + Hub)	GeneralUpdate-Samples
Tools (GUI 配置工具)	GeneralUpdate.Tools
Core (NuGet 包源码)	GeneralUpdate

GeneralUpdate.Tools

这是什么

GeneralUpdate.Tools 是一个基于 Avalonia 12 开发的跨平台桌面工具（Windows / Linux / macOS），用于在软件发布流程中生成和管理补丁包、扩展包、版本清单以及执行本地更新仿真。它不替代你的 CI/CD 系统，而是把“打包、校验、验证”这些重复劳动收敛到一个可视化工具中。

Tools 与 CI/CD 的关系

Tools 是**桌面 GUI 工具**，适合开发者在本地交互式地生成补丁、验证更新。如果你需要在 CI/CD 流水线中自动化补丁生成，可以直接调用 `GeneralUpdate.Core.Pipeline.DiffPipeline.CleanAsync()` —— GUI 和脚本走的是同一条代码路径。

仓库地址：<https://github.com/GeneralLibrary/GeneralUpdate.Tools>

下载与运行

方式一：从源码运行（推荐开发者）

当前工具基于 .NET 10 构建。确保本机安装了 [.NET 10 SDK](#)：

```
git clone https://github.com/GeneralLibrary/GeneralUpdate.Tools.git
cd GeneralUpdate.Tools\src
dotnet run --project GeneralUpdate.Tools.csproj
```

方式二：下载预编译版本

前往 [GeneralUpdate.Tools Releases](#) 下载对应平台的可执行文件，直接运行即可。

Simulation 模块内部会调用 `dotnet publish` 构建测试应用，因此使用仿真功能时必须安装 .NET SDK。Mobile 模块的项目模式（Build & Locate）也会调用 `dotnet publish`，同样需要 .NET SDK。仅运行 Patch / Extension / OSS / Config 模块以及 Mobile 的文件模式则不需要。

七个模块速览

模块	你提供	工具产出	下游消费者
Patch	旧版本目录 + 新版本目录	<code>{name}.zip</code> （含 <code>.patch</code> 文件、新文件、 <code>generalupdate.delete.json</code> ）	Server、OSS/CDN、Core Upgrade 进程
Extension	扩展源目录 + 元数据	<code>{name}_{version}.zip</code> （含 <code>manifest.json</code> ）	<code>GeneralUpdate.Extension</code> 组件
OSS Config	包名、版本、下载 URL、本地 ZIP（计算 Hash）	<code>oss_config.json</code> （版本数组）	OSS 客户端、对象存储发布流程
Config	Client/Upgrade 的 <code>.csproj</code>	<code>generalupdate.manifest.json</code> + <code>sample_output/</code> 发布目录	Client/Upgrade 启动引导

模块	你提供	工具产出	下游消费者
Simulation	旧版本目录 + 补丁 ZIP	本地更新服务 + <code>simulation_report.md</code>	发布前质量把关
Hash	本地文件 (ZIP)	SHA256 小写十六进制字符串	完整性校验、服务端版本记录
Mobile	APK/AAB 文件或 MAUI/Avalonia Android 的 <code>.csproj</code>	<code>mobile_version_{timestamp}.json</code> 版本记录	移动端更新服务端、GeneralUpdate.Avalonia/Maui 组件

Patch：生成补丁包

解决什么问题

当你发布了一个新版本，用户不想下载整个安装包。Patch 模块比较旧版本目录和新版本目录，只输出变更内容：修改过的文件生成二进制差分 `.patch`，新增文件直接复制，删除的文件写入清单。差分包通常远小于完整发布目录。

输入

字段	必填	说明
Old Directory	✓	用户当前安装的旧版本发布目录，例如 <code>publish\v1.0.0</code>
New Directory	✓	准备发布的新版本目录，例如 <code>publish\v2.0.0</code>
Package Name	✗	输出 ZIP 文件名（不含 <code>.zip</code> ），为空时自动生成 <code>patch_yyyyMMddHHmmss</code>
Version	✓	目标版本号，格式 <code>MAJOR.MINOR.PATCH</code> （如 <code>2.0.0</code> ）或 <code>MAJOR.MINOR.PATCH-prerelease+build</code>
Output Directory	✗	ZIP 输出目录，为空时输出到桌面

工具内部做了什么

- 加密文件检测**：扫描 old/new 目录中的二进制文件，检测是否存在加壳（如 Themida、VMProtect）或加密签名。被检测到的文件将标记警告——此类文件无法生成有效差分补丁，将以全量文件形式打包。
- 校验 old/new 目录存在、version 格式合法。
- 创建临时目录 `gupatch_yyyyMMddHHmmss`。
- 递归比较 old/new：**修改文件** → 生成 `.patch` 二进制差分；**新增文件** → 直接复制；**删除文件** → 记录 hash 到 `generalupdate.delete.json`。
- 将临时目录压缩为 `{PackageName}.zip`。
- 删除临时目录，ZIP 保留在 Output Directory。

关于加密文件：加壳工具（Themida、VMProtect 等）、代码混淆或加密的二进制文件，因文件哈希在新旧版本间完全不同，差分算法对其无效。此类文件检测到后将直接以全量形式打包，而不会生成 `.patch` 差分。建议在发布前对原始文件进行去壳/解密处理，以获得最优的补丁体积。

加密/加壳文件无法差分

如果你的应用使用了加壳工具（Themida、VMProtect）、代码混淆器或加密二进制文件，**整个文件在新旧版本间会完全不同**，差分算法完全失效，补丁包体积等同于全量文件。如需差分更新，请确保原始发布文件未被加壳或加密。

底层调用的是 `GeneralUpdate.Core.Pipeline.DiffPipeline.CleanAsync(oldDir, newDir, patchDir)`，和你的 CI 脚本走的是同一条代码路径。

输出

```
{OutputDirectory}/{PackageName}.zip
├─ changed.dll.patch      ← 二进制差分
├─ new_file.dll           ← 新增文件（保持目录结构）
└─ generalupdate.delete.json ← 删除清单
```

下游如何使用

- 把 ZIP 放到 Server 的 `wwwroot/packages/` 目录，更新 `versions.json` 中的 `Url` 和 `Hash`
- 或上传到 OSS/CDN，把下载链接写入 `oss_config.json`
- Core Upgrade 进程下载 ZIP 后，`DiffPipeline.DirtyAsync` 根据 `.patch` 和删除清单完成更新

Extension：打包扩展

解决什么问题

如果你使用 `GeneralUpdate.Extension` 组件管理插件/扩展，每次发布新扩展都需要手动构建标准格式的 ZIP 包并生成 `manifest.json`。Extension 模块把你从手工压缩和手写 JSON 中解放出来。

输入

字段	必填	说明
Name	✓	扩展唯一标识，建议小写+连字符，如 <code>my-data-exporter</code>
Version	✓	语义化版本，如 <code>1.0.0</code>
Description	✗	扩展功能描述
Publisher	✗	发布者名称
License	✗	许可证标识，如 <code>MIT</code>
Extension Directory	✓	扩展源文件目录，点击 Pick 选择文件夹
Export Directory	✗	输出目录，为空时输出到桌面
Custom Properties	✗	自定义键值对，写入 <code>manifest.json.customProperties</code>

工具内部做了什么

- 校验目录存在、version 格式合法。

2. 将 Extension Directory 中所有文件打包为 ZIP。
3. 在 ZIP 根目录写入 `manifest.json`。

输出

```
{ExportDirectory}/{Name}_{Version}.zip
├─ manifest.json          ← 扩展元数据
├─ bin/
│   └─ MyExtension.dll
├─ resources/
└─ ...
```

`manifest.json` 示例：

```
{
  "name": "my-data-exporter",
  "version": "1.0.0",
  "description": "Export data to CSV/Excel/JSON",
  "publisher": "MyCompany",
  "license": "MIT",
  "dependencies": "",
  "minHostVersion": "",
  "maxHostVersion": "",
  "isPreRelease": false,
  "platform": { "displayName": "Windows", "value": 1 },
  "format": ".zip",
  "releaseDate": "2026-06-01T00:00:00",
  "customProperties": { "maxConnections": "100" }
}
```

下游如何使用

- Extension Host 调用 `ExtensionManager.QueryRemoteExtensionsAsync(...)` 获取扩展列表
- 安装时下载 ZIP，读取 `manifest.json` 进行兼容性检查和依赖解析
- 详见 [GeneralUpdate.Extension](#)

OSS Config：生成 OSS 版本清单

解决什么问题

如果你使用 OSS 模式更新（静态文件服务器），你需要维护一个 `versions.json` 或 `oss_config.json`，让客户端知道有哪些版本可以下载。OSS Config 模块帮你整理版本信息并计算 SHA256。

输入

字段	必填	说明
PackageName	☑	更新包名称
Version	☑	语义化版本号
Url	☑	客户端可下载补丁包的完整地址
SHA256	推荐	选择本地 ZIP 文件后点击 ComputeHash 自动计算

操作流程

1. 填写 PacketName、Version、Url。
2. 点击 **ComputeHash**，选择本地补丁 ZIP 文件，自动填入 SHA256。
3. 点击 **Add To List**，加入列表。
4. 重复以上步骤添加所有版本。
5. 点击 **Export**，生成 `oss_config.json`。

输出

```
[
  {
    "PackageName": "client_1.0.0_to_2.0.0",
    "Hash": "f2c7a8b3...",
    "Version": "2.0.0",
    "Url": "https://cdn.example.com/client_1.0.0_to_2.0.0.zip",
    "ReleaseDate": ""
  }
]
```

下游如何使用

- 将 `oss_config.json` 上传到 OSS bucket 或静态文件服务器
- OSS 客户端读取此文件发现可用版本，下载后校验 Hash
- 详见 [GeneralUpdate.Core](#) OSS 更新策略

Config：生成启动清单

解决什么问题

Client 和 Upgrade 启动时需要知道主程序名、升级程序名、版本号、ProductId、UpdatePath 等信息。Config 模块从 `.csproj` 中自动提取 AssemblyName 和 TargetFramework，帮你生成 `generalupdate.manifest.json`，省去手写和手误。

输入

字段	必填	说明
Client .csproj	✓	主程序的 .csproj 文件路径
Upgrade .csproj	✗	升级程序的 .csproj 路径
ClientVersion	✓	客户端当前版本，如 1.0.0
UpgradeClientVersion	✓	升级程序版本，如 1.0.0
AppType	✓	Client / Upgrade / OssClient / OssUpgrade
ProductId	✓	产品标识 GUID
UpdatePath	✓	升级程序相对主程序的子目录，如 update/

操作流程

1. 点击 Browse 选择 Client .csproj（必填）和 Upgrade .csproj（可选）。
2. 点击 **Analyze**，工具自动读取 AssemblyName 填入 MainAppName、UpdateAppName。
3. 手动填写 Version、AppType、ProductId、UpdatePath。
4. 点击 **Generate**，在工具运行目录生成 generalupdate.manifest.json。
5. 点击 **Generate Sample**，额外执行 dotnet publish 并输出可运行的发布目录。

输出

generalupdate.manifest.json：

```
{
  "mainAppName": "ClientSample.exe",
  "clientVersion": "1.0.0",
  "appType": "Client",
  "updateAppName": "UpgradeSample.exe",
  "upgradeClientVersion": "1.0.0",
  "productId": "2d974e2a-31e6-4887-9bb1-b4689e98c77a",
  "updatePath": "update/"
}
```

Generate Sample 额外输出：

```
{工具运行目录}/sample_output/
├─ ClientSample.exe          ← Client dotnet publish 输出
├─ generalupdate.manifest.json
└─ update/
    └─ UpgradeSample.exe     ← Upgrade dotnet publish 输出
```

下游如何使用

- Client 启动时读取 generalupdate.manifest.json，配合服务端地址和 AppSecretKey 即可进入更新流程
- 不需要手写 Configinfo，只需用 GeneralUpdateBootstrap 读取 manifest

Simulation：本地更新仿真

解决什么问题

你刚用 Patch 模块生成了补丁包，但不确定这个包在实际更新流程中能不能跑通。Simulation 模块在你本机启动一个临时更新服务，构建 Client/Upgrade 样例，模拟一次完整的“发现版本 → 下载补丁 → 应用更新”闭环，最后输出 PASS/FAIL 报告。

前置条件

- 已安装 .NET 10 SDK（或更高版本）
- `dotnet --version` 可正常执行
- 本地 5000 端口未被占用（可配置）

输入

字段	必填	说明
Old App Directory	✓	模拟用户当前安装的旧版本目录
Patch File	✓	Patch 模块生成的补丁 ZIP
CurrentVersion	✓	旧版本号（Simulation 内部传给 Client，用于版本校验）
TargetVersion	✓	新版本号（Simulation 内部赋值到补丁 ZIP，写入服务端版本源）
AppType	✓	ClientApp = 1 / UpgradeApp = 2
Platform	✓	Windows = 1 / Linux = 2
AppSecretKey	✓	客户端验证密钥
ProductId	✓	产品标识 GUID
UpdatePath	✓	升级程序子目录名称，如 <code>update/</code>
ServerPort	✗	本地服务端口，默认 5000

仿真过程

- 校验**：旧目录存在、Patch ZIP 存在、version 格式合法、.NET SDK 可用。
- 发布 Client**：`dotnet publish test_app/Client/ClientSample.csproj` 到 App Directory。
- 发布 Upgrade**：`dotnet publish test_app/Upgrade/UpgradeSample.csproj` 到 UpdatePath 子目录。
- 生成 manifest**：在 App Directory 写入 `generalupdate.manifest.json`。
- 准备补丁**：把 Patch ZIP 复制到内部 `.server` 目录，计算 SHA256，写入服务端版本源。
- 启动服务**：本地服务监听 `http://localhost:{ServerPort}`，提供 `POST /Upgrade/Verification`、`POST /Upgrade/Report`、`GET /patch/{filename}`。
- 运行 Client**：启动 `ClientSample.exe`，传入 `--server-url http://localhost:5000 --app-secret ... --client-version 1.0.0`。
- 停止服务**：Client 完成更新或超时时停止服务。
- 生成报告**：检查目录文件变化，输出 `simulation_report.md`。

输出

`{AppDirectory}/simulation_report.md` ← 仿真报告 (PASS/FAIL + 时间线)

报告包含配置表、PASS/FAIL 结论、Notes 和完整时间线，可以直接贴到发布记录或 CI artifact。

Hash：SHA256 校验

解决什么问题

补丁包上传到 CDN/OSS 后，必须确保客户端下载到的文件和原始包一致。Hash 功能对本地文件计算 SHA256，输出小写十六进制字符串，你应该把这个值写入版本清单或服务端返回数据。

使用方式

在 **OSS Config** 模块中，点击 **ComputeHash**，选择本地 ZIP 文件，自动填入 SHA256 字段。你也可以单独使用这个功能来校验任何文件。

Mobile：移动端打包

解决什么问题

如果你使用 `GeneralUpdate.Avalonia.Android` 或 `GeneralUpdate.Maui.Android` 组件为 Android 应用提供自动更新能力，每次发布新版本时都需要获取 APK/AAB 文件的元数据（包名、版本号、SHA256 哈希、文件大小），并将其上传到服务端进行版本管理。Mobile 模块把"解析 → 打包 → 上传 → 生成版本记录"这四步收敛到一个界面中。

视频演示：<https://www.bilibili.com/video/BV1ctJF6XEJs>

支持两种工作模式：

- **文件模式**：直接选择 APK 或 AAB 文件，自动解析 AndroidManifest.xml 提取元数据
- **项目模式**：选择 `.csproj` 项目文件，自动执行 `dotnet publish` 构建，然后定位产物并解析

输入

文件模式

字段	必填	说明
APK/AAB 文件	✓	选择 <code>.apk</code> 或 <code>.aab</code> 文件，工具自动识别格式类型
Output Directory	✗	版本记录 JSON 输出目录，为空时输出到桌面
ProductId	✓	产品标识 GUID，用于区分服务端产品线
Platform	✓	目标平台（默认 Android = 4）
Product Name	✗	产品名称，写入版本记录
Release Notes	✗	发布说明
Is Forcibly	✗	是否强制更新

项目模式

字段	必填	说明
.csproj 文件	✔	选择 MAUI 或 Avalonia Android 项目的 .csproj，工具自动解析 ApplicationId、ApplicationDisplayVersion、ApplicationVersion
Output Directory	✘	同上

操作流程

1. **选择模式**：切换 File Mode / Project Mode 开关。
 - **文件模式**：点击 **Select File** 选择 .apk 或 .aab 文件。
 - **项目模式**：点击 **Select Project** 选择 .csproj 文件，然后点击 **Build & Locate** 自动执行 dotnet publish 并定位产物。
2. **Analyze**：点击 Analyze 按钮，工具自动：
 - 识别文件格式（APK / Android App Bundle）
 - 从 AndroidManifest.xml 中提取 PackageName、VersionName、VersionCode
 - 计算 SHA256 哈希和文件大小
3. **填写上传配置**：确认或编辑自动提取的元数据，填写 ProductId、Product Name 等发布信息。
4. **Upload**：点击 Upload 上传到服务端，并自动生成版本记录 JSON 文件。

工具内部做了什么

1. **格式检测**：通过文件扩展名（.apk / .aab）和 ZIP 内部结构（AndroidManifest.xml 位置）自动识别包格式。
 - APK 检测：打开 ZIP 文件，检查根目录是否存在 AndroidManifest.xml。
 - AAB 检测：打开 ZIP 文件，检查是否存在 base/manifest/AndroidManifest.xml。
 - 格式未知时（不是 .apk 也不是 .aab）会直接返回 Unknown。
2. **元数据解析**：使用 AxmlParser 解析二进制的 AXML（Android Binary XML）格式，从 ZIP 中读取 AndroidManifest.xml。解析流程如下：
 - **字符串池提取**：读取 AXML 文件的 Chunk 头，找到类型为 0x0001 的 StringPool Chunk，从中解析出所有 UTF-16LE 编码的字符串数组。
 - **属性值提取**：遍历 XML 的 Start Element Chunk（类型 0x0102），解析属性块（每个属性 20 字节），通过属性名在字符串池中的索引匹配 package 和 versionName，再通过 rawValueIndex 从字符串池中取出属性值。
 - **versionCode 提取**：versionCode 存储为整型属性（类型 0x10 = INT_DEC 或 0x11 = INT_HEX），直接从属性的 typedValueData 字段（偏移 +16，4 字节）读取有符号整数。
 - AXML 结构与参考：<https://justanapplication.wordpress.com/category/android/android-binary-xml/>
3. **SHA256 计算**：对完整文件计算 SHA256 哈希值，输出小写十六进制字符串。
4. **文件大小**：读取文件长度并格式化为人类可读的显示（B / KB / MB / GB）。
5. **项目构建**（仅项目模式）：选择 .csproj 后会触发：

- `MobileCsprojParser.Parse()` 解析 `.csproj` 的 XML, 提取 `TargetFramework` (支持单 TFM 和多 TFM, 多 TFM 时自动选取包含 `-android` 的目标框架)、`ApplicationId` (映射为 `PackageName`)、`ApplicationDisplayVersion` (映射为 `VersionName`)、`ApplicationVersion` (映射为 `VersionCode`)、`UseMaui` (用于识别 MAUI 项目)、`AndroidPackageFormat`、`AssemblyName`。
- 调用 `dotnet publish "{csprojPath}" -c Release -o "{publishDir}"`。
- 构建完成后在 `bin/Release/{tfm}/publish/` 目录搜索 `.apk` 或 `.aab` 文件, 自动定位构建产物。
- 项目类型显示: MAUI (`true`) 或 Avalonia (没有 `UseMaui`) 。

6. 上传: 通过 HTTP multipart/form-data 将文件上传到服务端。上传表单字段和对应的数据源如下:

字段	值来源	示例
Name	ProductName (用户填写)	MyApp
Version	VersionName (自动解析)	2.0.0
Hash	SHA256 (自动计算)	a1b2c3d4...
Format	包格式 (自动识别)	.apk / .aab
Size	文件大小 (自动计算)	50000000
AppType	固定值 "1"	1
Platform	Platform (默认 4=Android)	4
ProductId	ProductId (用户填写 GUID)	2d974e2a-...
IsForcibly	强制更新开关	true / false

上传服务支持以下配置:

- **Server URL**: 服务端地址
- **Upload Endpoint**: API 路径, 默认 `/Packet/Create`
- **Timeout**: 超时时间 (秒)
- **Retry Count**: 失败重试次数 (指数退避)
- **Auth**: 认证配置, 支持 Basic / Bearer Token / API Key 三种模式 (凭据通过 DPAPI 加密存储)

输出

```
{OutputDirectory}/mobile_version_20260614120000.json
```

```
{
  "name": "MyApp",
  "version": "2.0.0",
  "hash": "a1b2c3d4e5f6...",
  "url": "https://server.example.com/packages/app-v2.0.0.apk",
  "packageName": "com.example.myapplication",
  "fileSize": 50000000,
  "format": "apk",
  "platform": 4,
  "productId": "2d974e2a-31e6-4887-9bb1-b4689e98c77a",
  "isForcibly": false,
  "releaseDate": "2026-06-14T12:00:00.000000Z"
}
```

下游如何使用

- 将生成的版本记录 JSON 导入或上传到你的更新服务端（如 GeneralSpacestation）
- 客户端（ GeneralUpdate.Avalonia.Android 或 GeneralUpdate.Maui.Android ）从服务端查询版本信息时，返回的数据结构与版本记录的内容对应
- 客户端下载 APK 后使用 hash 字段做 SHA256 完整性校验

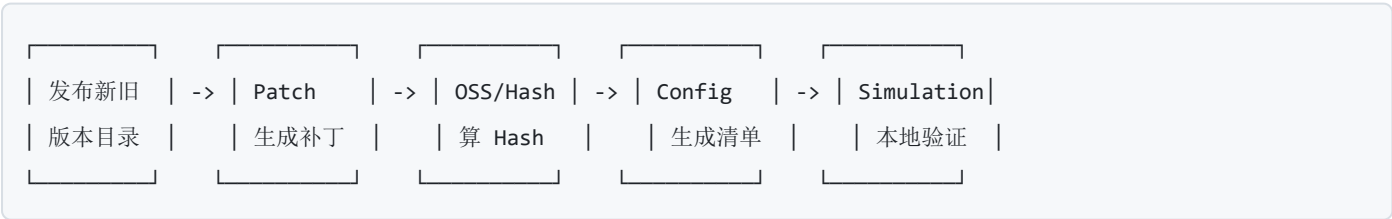
支持的 AndroidManifest 字段提取

清单属性	字段名	说明
package	PackageName	应用包名（唯一标识）
android:versionName	VersionName	展示版本号（如 2.0.0 ）
android:versionCode	VersionCode	内部版本代码（整数）

推荐发布 workflow

桌面端更新

这个顺序把 Patch、OSS、Config、Simulation 四个模块串联成一个完整的桌面应用发布流水线：



1. **构建发布目录**：在 CI 或本地 `dotnet publish` 得到 old/new 两个目录。
2. **Patch**：选择 old/new，生成补丁 ZIP。
3. **OSS Config**：对补丁 ZIP 计算 SHA256，填写下载 URL，导出 `oss_config.json`。（如果使用自建 Server，将相同信息写入 `versions.json`）。
4. **Config**：选择 Client/Upgrade 的 `.csproj`，生成 `generalupdate.manifest.json`。

5. **Simulation**：选择旧版本目录和补丁 ZIP，确认 PASS。
6. **发布上线**：上传补丁 ZIP、OSS 清单到生产环境。

移动端更新

使用 Mobile 模块串联移动端 Android 应用发布：



1. **构建产物**：在 CI 或本地构建出 `.apk` / `.aab` 文件。
2. **Mobile**：选择文件或项目，自动解析元数据、计算 SHA256、上传服务端、导出版本记录。
3. **发布上线**：确认上传成功，客户端通过 `GeneralUpdate.Avalonia.Android` 或 `GeneralUpdate.Maui.Android` 检查更新。

常见问题

现象	原因与处理
Version invalid	使用 <code>1.0.0</code> 、 <code>2.0.0-beta.1</code> 格式。不要用 <code>1.0</code> 或 <code>v1.0.0</code>
Patch 输出为空或极小	old/new 目录可能选反，或两个目录内容完全一致。确认 old 是用户已安装版本，new 是目标版本
找不到 delete_files.json	当前文件名是 <code>generalupdate.delete.json</code> ，位于 Patch ZIP 根目录
Simulation 提示找不到 SDK	安装 .NET 10 SDK 并确保命令行可执行 <code>dotnet --version</code>
Simulation 端口冲突	修改 <code>ServerPort</code> 或释放本地 5000 端口
客户端下载后 Hash 校验失败	对 最终上传到 CDN/OSS 的文件 重新计算 Hash，检查是否被中间代理重压缩
Upgrade 进程没有启动	检查 <code>generalupdate.manifest.json</code> 中 <code>updateAppName</code> 和 <code>updatePath</code> 是否与发布目录结构一致
Mobile 分析时提示 "Metadata extraction warning"	AXML 解析未找到预期属性，常见的两种原因：APK 使用资源 ID 引用而非直接文本存储属性值；或 APK 被加固/混淆后 <code>AndroidManifest.xml</code> 结构被修改。可以手动填写 <code>PackageName</code> 、 <code>VersionName</code> 、 <code>VersionCode</code>
Mobile 项目模式 Build & Locate 失败	确认已安装 .NET SDK 且版本支持目标框架（TFM），检查 <code>.csproj</code> 包含 <code>-android</code> 目标框架，输出目录 <code>bin/Release/{tfm}/publish/</code> 没有被写入保护
Mobile 项目模式 "Build output not found"	<code>dotnet publish</code> 成功执行但未在预期目录找到 <code>.apk</code> / <code>.aab</code> 。检查 <code>.csproj</code> 中 <code>AndroidPackageFormat</code> 设置是否正确（默认 <code>aab;apk</code> 优先产生 APK），以及 TFM 是否自动解析正确
Mobile 上传失败	检查服务端地址和 Endpoint 配置是否正确，确认服务端认证方式（Basic / Bearer / API Key）与工具配置一致，查看工具日志中的 HTTP 状态码和错误消息
Mobile 上传后客户端更新失败	确认服务端返回的版本记录 JSON 中 <code>Platform</code> 字段为 <code>4</code> （Android）， <code>Format</code> 字段与客户端期望的 APK/AAB 格式一致

关联文档

- [GeneralUpdate.Core](#): Client/Upgrade 更新主流程
- [GeneralUpdate.Differential](#): 差分算法 Clean/Dirty 模式
- [GeneralUpdate.Extension](#): 扩展包安装与版本管理
- [GeneralUpdate.Avalonia.Android](#): Avalonia Android APK 更新组件
- [GeneralUpdate.Maui.Android](#): MAUI Android APK 更新组件
- [入门实战手册](#): 从零跑通完整更新闭环

GeneralUpdate.Avalonia.Android 实战手册

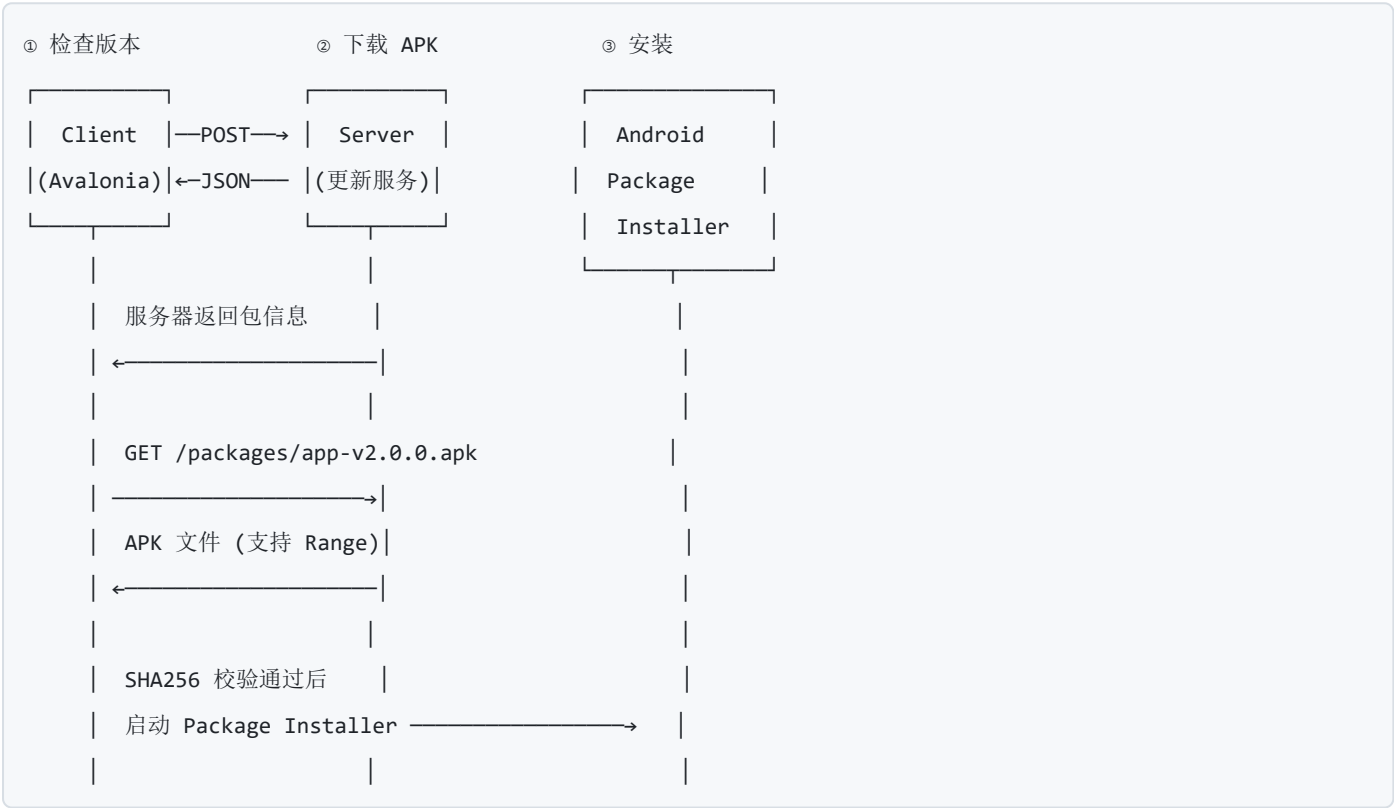
这篇手册面向需要在 Avalonia 应用中集成 Android APK 自动更新的开发者。目标是用最少的代码跑通"检查更新 → 下载 APK → 校验 SHA256 → 启动安装器"的完整流程。

视频演示: <https://www.bilibili.com/video/BV1HtJF6XET3>

前置知识

这篇手册假设你已经有一个 Avalonia 项目并配置好了 Android 目标框架。如果你还没有 Avalonia Android 项目, 请先参考 [Avalonia 官方文档](#) 创建。

更新流程



角色	定义	负责什么
Client (Avalonia)	你的 Avalonia Android 应用	检查版本 → 下载 APK → 校验 → 启动安装器
Server	更新服务 (如 Samples 中的 Server)	返回版本信息、提供 APK 下载
Android Package Installer	Android 系统组件	安装 APK (验证签名、替换应用)

Phase 1: 环境准备

安装清单

项	要求	验证命令
.NET SDK	10.0+	<code>dotnet --version</code>
Android SDK	API 26+	<code>dotnet workload list</code> (应包含 <code>android</code>)

NuGet 包

```
dotnet add package GeneralUpdate.Avalonia.Android
```

Phase 2: FileProvider 配置

APK 文件需要通过 FileProvider 传递给 Android Package Installer。在 `AndroidManifest.xml` 中添加：

```
<application>
  <provider
    android:name="androidx.core.content.FileProvider"
    android:authorities="${applicationId}.fileprovider"
    android:exported="false"
    android:grantUriPermissions="true">
    <meta-data
      android:name="android.support.FILE_PROVIDER_PATHS"
      android:resource="@xml/file_paths" />
  </provider>
</application>
```

创建 `Resources/xml/file_paths.xml`：

```
<?xml version="1.0" encoding="utf-8"?>
<paths>
  <cache-path name="cache" path="update/" ></cache-path>
</paths>
```

注意： `authorities` 中的 `${applicationId}.fileprovider` 和代码中传入的 `FileProviderAuthority` 必须一致。

Phase 3: 编写更新代码

在 Avalonia ViewModel 或 Service 中集成更新逻辑：

```
using GeneralUpdate.Avalonia.Android;
using GeneralUpdate.Avalonia.Android.Models;
using GeneralUpdate.Avalonia.Android.Abstractions;

public class UpdateViewModel : ViewModelBase
{
    private IAndroidBootstrap? _bootstrap;

    public async Task CheckAndUpdateAsync()
    {
        // 1. 创建更新引导实例
        var options = new AndroidUpdateOptions
        {
            FileProviderAuthority = "com.myapp.fileprovider"
        };

        _bootstrap = GeneralUpdateBootstrap.CreateDefault(options);

        // 2. 订阅事件
        _bootstrap.AddListenerDownloadProgressChanged += (_, e) =>
        {
            // 更新 UI 进度条
            Console.WriteLine($"Download: {e.ProgressPercentage:F1}%");
        };

        // 3. 构造更新包信息（通常从服务端获取）
        var package = new UpdatePackageInfo
        {
            Version = "2.0.0",
            DownloadUrl = "http://10.0.2.2:5000/packages/app-v2.0.0.apk",
            Sha256 = "服务端返回的SHA256哈希值",
            FileSize = 50_000_000,
            FileName = "app-v2.0.0.apk"
        };

        // 4. 检查更新
        var checkResult = await _bootstrap.ValidateAsync(package, "1.0.0");
        if (!checkResult.UpdateFound)
        {
            Console.WriteLine("已是最新版本");
            return;
        }

        // 5. 下载并校验
```

```

var downloadResult = await _bootstrap.DownloadAndVerifyAsync(package);
if (!downloadResult.Success)
{
    Console.WriteLine($"下载失败: {downloadResult.Message}");
    return;
}

// 6. 检查安装权限 (Android 8+)
if (Android.OS.Build.VERSION.SdkInt >= Android.OS.BuildVersionCodes.O)
{
    var ctx = Android.App.Application.Context;
    var pm = ctx.PackageManager;
    if (pm != null && !pm.CanRequestPackageInstalls())
    {
        // 引导用户开启"安装未知应用"权限
        var intent = new Android.Content.Intent(
            Android.Provider.Settings.ActionManageUnknownAppSources)
            .SetData(Android.Net.Uri.Parse("package:" + ctx.PackageName));
        intent.AddFlags(Android.Content.ActivityFlags.NewTask);
        ctx.StartActivity(intent);
        return;
    }
}

// 7. 启动安装器
var installResult = await _bootstrap.LaunchInstallerAsync(
    package, downloadResult.FilePath!);

if (installResult.Success)
    Console.WriteLine("安装器已启动, 请在设备上确认安装。");
}
}

```

Phase 4: 启动 Server

使用 Samples 仓库中的 Server:

```

git clone https://github.com/GeneralLibrary/GeneralUpdate-Samples.git
cd GeneralUpdate-Samples/src/Server
dotnet run

```

Server 默认在 `http://localhost:5000` 启动。Android 模拟器中请使用 `http://10.0.2.2:5000` 访问宿主机的 localhost。

Phase 5: 端到端验证

1. 将 APK 包放到 `Server/wwwroot/packages/` 目录
2. 更新 `wwwroot/packages/versions.json` 添加版本信息
3. 在模拟器或真机上运行 Avalonia Android 应用
4. 观察控制台输出: 检查更新 → 下载 → 校验 → 启动安装器

预期输出

```
Checking for updates...
Update v2.0.0 found.
Downloading... 45% 1.2MB/s
Downloading... 100%
SHA256 verified.
Installer launched.
```

下一步

- [GeneralUpdate.Avalonia.Android 组件文档](#): 完整 API 参考、配置说明和高级用法
- [Android 更新示例](#): 完整的 Avalonia Android 更新示例项目

GeneralUpdate.Maui.Android 实战手册

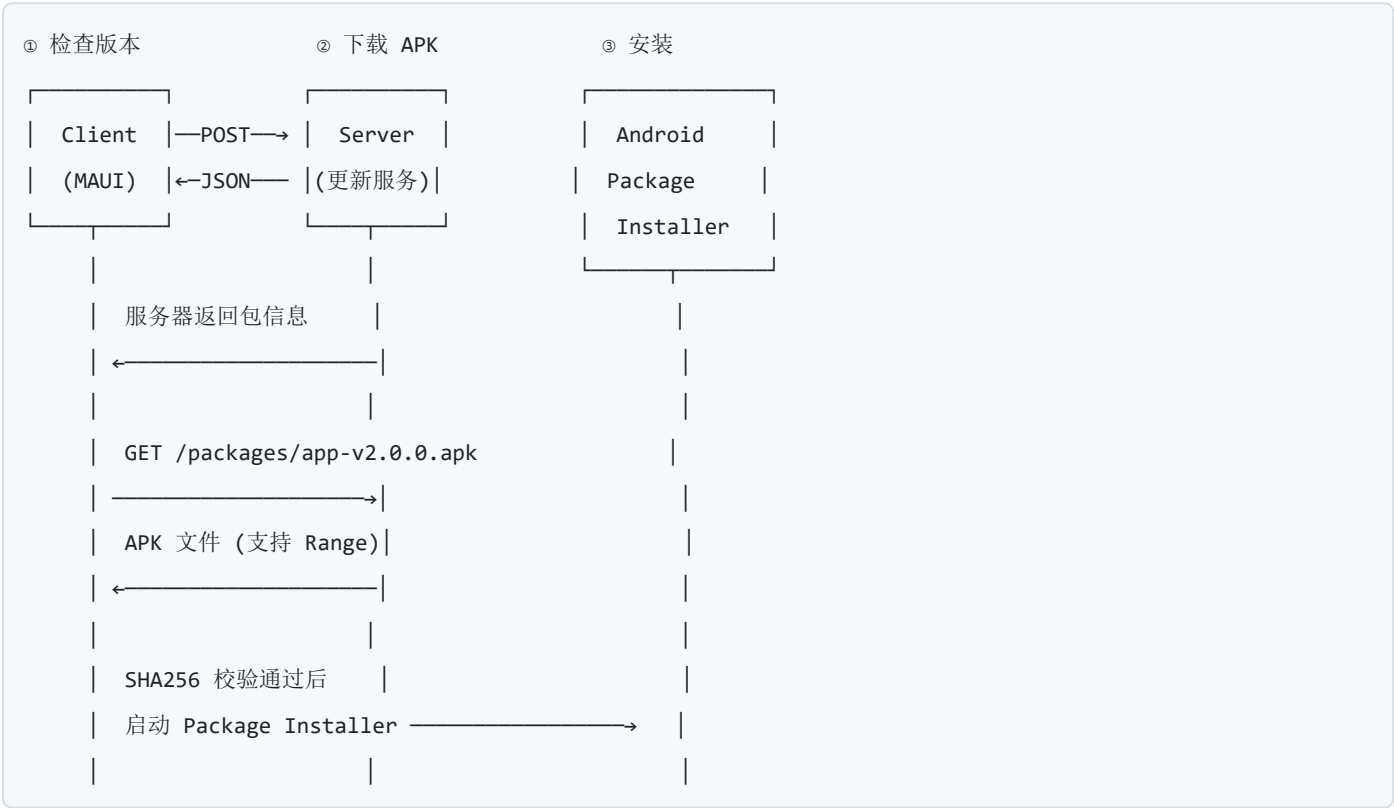
这篇手册面向需要在 MAUI 应用中集成 Android APK 自动更新的开发者。目标是用最少的代码跑通"检查更新 → 下载 APK → 校验 SHA256 → 启动安装器"的完整流程。

视频演示: <https://www.bilibili.com/video/BV1ntJF6XEvG>

前置知识

这篇手册假设你已经有一个 .NET MAUI 项目并配置好了 Android 目标框架。如果你还没有 MAUI Android 项目，请先参考 [.NET MAUI 官方文档](#) 创建。

更新流程



角色	定义	负责什么
Client (MAUI)	你的 MAUI Android 应用	检查版本 → 下载 APK → 校验 → 启动安装器
Server	更新服务 (如 Samples 中的 Server)	返回版本信息、提供 APK 下载
Android Package Installer	Android 系统组件	安装 APK (验证签名、替换应用)

Phase 1: 环境准备

安装清单

项	要求	验证命令
.NET SDK	10.0+	<code>dotnet --version</code>
MAUI 工作负载	已安装	<code>dotnet workload list</code> (应包含 <code>maui</code>)

NuGet 包

```
dotnet add package GeneralUpdate.Maui.Android
```

Phase 2: FileProvider 配置

APK 文件需要通过 FileProvider 传递给 Android Package Installer。在 `Platforms/Android/AndroidManifest.xml` 中添加：

```
<application>
  <provider
    android:name="androidx.core.content.FileProvider"
    android:authorities="${applicationId}.fileprovider"
    android:exported="false"
    android:grantUriPermissions="true">
    <meta-data
      android:name="android.support.FILE_PROVIDER_PATHS"
      android:resource="@xml/file_paths" />
  </provider>
</application>
```

创建 `Platforms/Android/Resources/xml/file_paths.xml`：

```
<?xml version="1.0" encoding="utf-8"?>
<paths>
  <cache-path name="cache" path="update/" ></cache-path>
</paths>
```

Phase 3: 编写更新代码

方式一：DI 容器集成（推荐）

在 `MauiProgram.cs` 中注册服务：

```
using GeneralUpdate.Maui.Android.Services;

public static class MauiProgram
{
    public static MauiApp CreateMauiApp()
    {
        var builder = MauiApp.CreateBuilder();
        builder
            .UseMauiApp()
            .ConfigureFonts(fonts => { });

        // 注册更新服务
        builder.Services.AddGeneralUpdateMauiAndroid();

        // 注册应用服务
        builder.Services.AddSingleton();
        builder.Services.AddSingleton();

        return builder.Build();
    }
}
```

在 ViewModel 中使用：

```

using GeneralUpdate.Maui.Android.Abstractions;
using GeneralUpdate.Maui.Android.Models;

public class MainViewModel
{
    private readonly IAndroidBootstrap _bootstrap;

    public MainViewModel(IAndroidBootstrap bootstrap)
    {
        _bootstrap = bootstrap;

        _bootstrap.AddListenerDownloadProgressChanged += (_, args) =>
        {
            MainThread.BeginInvokeOnMainThread(() =>
            {
                OnProgressChanged?.Invoke(this, args.Statistics.ProgressPercentage);
            });
        };
    }

    public async Task CheckAndUpdateAsync()
    {
        var package = new UpdatePackageInfo
        {
            Version = "2.0.0",
            DownloadUrl = "http://10.0.2.2:5000/packages/app-v2.0.0.apk",
            Sha256 = "服务端返回的SHA256哈希值",
            PackageSize = 50_000_000,
            ApkFileName = "app-v2.0.0.apk"
        };

        var options = new UpdateOptions
        {
            CurrentVersion = "1.0.0",
            InstallOptions = new AndroidInstallOptions
            {
                FileProviderAuthority = "com.mycompany.myapp.fileprovider"
            }
        };

        // 检查更新
        var checkResult = await _bootstrap.ValidateAsync(package, options, CancellationToken.None);
        if (!checkResult.IsUpdateAvailable)
        {

```



```

        await Shell.Current.DisplayAlert("提示", "已是最新版本。", "确定");
        return;
    }

    // 执行更新（下载 → 校验 → 安装）
    var result = await _bootstrap.ExecuteUpdateAsync(package, options, CancellationToken.None);

    if (result.IsSuccess)
        await Shell.Current.DisplayAlert("更新", "安装器已启动，请在设备上确认安装。", "确定");
    else
        await Shell.Current.DisplayAlert("更新失败", result.Message, "确定");
}

public event EventHandler<double>? OnProgressChanged;
}

```

方式二：直接创建（无 DI）

```

using GeneralUpdate.Maui.Android;
using GeneralUpdate.Maui.Android.Models;

var bootstrap = GeneralUpdateBootstrap.CreateDefault();

var package = new UpdatePackageInfo
{
    Version = "2.0.0",
    DownloadUrl = "http://10.0.2.2:5000/packages/app-v2.0.0.apk",
    Sha256 = "服务端返回的SHA256哈希值"
};

var options = new UpdateOptions
{
    CurrentVersion = "1.0.0",
    InstallOptions = new AndroidInstallOptions
    {
        FileProviderAuthority = "com.mycompany.myapp.fileprovider"
    }
};

// 二合一 API
var result = await bootstrap.ExecuteUpdateAsync(package, options, CancellationToken.None);
Console.WriteLine(result.IsSuccess ? "更新完成" : $"更新失败: {result.Message}");

```

Phase 4: 启动 Server

使用 Samples 仓库中的 Server:

```
git clone https://github.com/GeneralLibrary/GeneralUpdate-Samples.git
cd GeneralUpdate-Samples/src/Server
dotnet run
```

Server 默认在 `http://localhost:5000` 启动。Android 模拟器中请使用 `http://10.0.2.2:5000` 访问宿主机的 localhost。

Phase 5: 端到端验证

1. 将 APK 包放到 `Server/wwwroot/packages/` 目录
2. 更新 `wwwroot/packages/versions.json` 添加版本信息
3. 在模拟器或真机上运行 MAUI Android 应用
4. 观察应用输出: 检查更新 → 下载 → 校验 → 启动安装器

预期输出

```
Checking for updates...
Update v2.0.0 found.
Downloading... 45% 1.2MB/s
Downloading... 100%
SHA256 verified.
Installer launched – complete install on device.
```

下一步

- [GeneralUpdate.Maui.Android 组件文档](#): 完整 API 参考、配置说明和高级用法
- [MAUI Android 更新示例](#): 完整的 MAUI Android 更新示例项目

组件参考 Components

Component Introduction



1. What is GeneralUpdate?

Updates limitless, upgrades boundless.

GeneralUpdate is an open-source cross-platform application automatic update component based on .NET Standard 2.0 and licensed under the Apache license.

Github	Gitee	GitCode
		

一句话理解

把 Client（你的主程序）、Upgrade（升级进程）、Server（更新服务）和 Tools（打包工具）拆开，通过 manifest + IPC 加密契约串联成一个**全自动的更新闭环**。

2. What does GeneralUpdate offer?

Component Features

Feature	Supported	Notes
Resume Download	Yes	If an update fails, it will resume downloading the update package the next time it starts. (Effective by default when the component is referenced)
Version-by-Version Update	Yes	If the current client version is multiple versions behind the server, it will update one by one based on the release dates of the versions. (Effective by default when the component is referenced)
Binary Differential Update	Yes	Generates patch files by comparing new and old versions using differential algorithms. (Effective by default when the component is referenced)

Feature	Supported	Notes
Incremental Update	Yes	Updates only the files that have changed compared to the previous version and deletes files that do not exist in the current version. (Effective by default when the component is referenced)
Forced Update	Yes	Forces an update directly after opening the client.
Multi-Branch Update	Yes	Supports updating different content based on different branches of a product.
Latest Version Push	Yes	Implemented based on Signal R to push the latest version.
Multi-Language	Yes	Cross-platform, language-agnostic; can serve as an update "script" for apps written in any language. GeneralUpdate.Tools UI supports Chinese/English toggle.
Skip Update	Yes	Supports injecting a pop-up to let users decide whether to update this release; forced updates from the server side will override this.
Mutual Upgrade	Yes	The main program can update the upgrade program, and vice versa.
Blacklist	Yes	Skips files and file extensions listed in the blacklist during the update process.
OSS	Yes	Simplifies updates with an independent update mechanism. Only requires placing a <code>version.json</code> configuration file on the file server. The component will update and download based on the version information in the configuration file.
Rollback and Backup	Yes	Backs up local client files before updating, and rolls back if the client fails to start or crashes.
Driver Update	Supported	Cross-platform driver auto-update implemented based on the GeneralUpdate.DriverUpdate component, supporting Windows and Linux platforms. Provides complete functions including driver validation, backup, rollback, signature verification, and permission management.
Extension Management	Supported	VS Code-like extension system implemented based on the GeneralUpdate.Extension component, managing extension plugins such as Lua/Python. Supports extension directory management, remote query, download queue, version compatibility check, multi-platform support, dependency resolution, rollback mechanism, event notification, and other functions.
Custom Method List	Yes	Injects a collection of custom methods that execute before the update starts. If any exception occurs while executing the custom method list, it will be notified through exception subscription. (Recommended to check the current software environment before updating)
Multi-Protocol Auth	Yes	Supports HMAC-SHA256, Bearer Token, API Key, and HTTP Basic authentication schemes with custom extension.
Silent Update	Yes	Background version polling, silent download, and upgrade triggered on process exit — fully transparent to the user.

Feature	Supported	Notes
Concurrent Download	Yes	Multi-package concurrent downloads with resume support, SHA256 verification, and configurable concurrency.
File Tree Diff	Yes	Directory-level structural comparison between old and new versions, producing an incremental file manifest.
AOT	Yes	Supports AOT compilation and publishing.
Authentication	Supported	Identity authentication information (token) can be passed in HTTP requests for server resources.

3. What does GeneralUpdate support?

.NET Framework

Framework Name	Supported
.NET Core 2.0	Yes
.NET 5 ... to the latest version	Yes
.NET Framework 4.6.1	Yes

UI Framework

UI Framework Name	Supported	Contributor
WPF	Yes	JusterZhu
UWP	Not updatable in store mode	lindexi
MAUI	Currently supports only Android platform	JusterZhu
Avalonia	Yes	JusterZhu
WinUI	Yes	JusterZhu
Console	Yes	JusterZhu
WinForms	Yes	JusterZhu

Operating Systems

Operating System Name	Supported	Contributor
Windows	Yes	JusterZhu
Linux	Yes	JusterZhu
Android	Yes	JusterZhu
Kylin V10 (Feiteng S2500)	Yes	Wei
Kylin V10 (Feiteng FT-2000)	Yes	Yao Shengwei

Operating System Name	Supported	Contributor
Kylin V10 (x64)	Yes	Wei
Ubuntu	Yes	JusterZhu
Huawei EulerOS (Kunpeng)	Yes	Yao Shengwei
Loongson (Loongnix LoongArch)	Yes	Avalonia Chinese Community (Dong Bin Rabbitism)
Apple Mac (M1)	Yes	JusterZhu
UOS	Yes	JusterZhu

4. Repositories

Name	Description	Address
GeneralUpdate	Automatic Updates	GitHub Gitee GitCode
GeneralUpdate.Maui	MAUI Android Auto-Updates	GitHub Gitee GitCode
GeneralUpdate.Avalonia	Avalonia Android Auto-Updates	GitHub
GeneralUpdate.Tools	Update Patch Creation Tool	GitHub Gitee GitCode
GeneralUpdate-Samples	Usage Examples	GitHub Gitee GitCode

5. Unified Language

[图片缺失: client_server_upgrade.jpg]

Before starting to use GeneralUpdate, we need to understand some basic concepts in the system. Please download the GeneralUpdate-Samples repository and navigate to the `..\GeneralUpdate-Samples\src` directory for better understanding.

Name	Description	Explanation
Client	The client being updated.	If you want to update QQ, then QQ is the Client.
Upgrade	The upgrade program is an independent process. It should be placed in the same directory as the Client and should not be associated with any business or design during use (or coding); it must remain independently referenced.	QQ cannot update its own files during runtime, so the Upgrade handles this task.

Name	Description	Explanation
Packaget	Update Patch Package	Gamers often hear the concept of "patches," which are used to update vulnerabilities or content in a game. Similarly, here the update patch package contains files with content differences or new files from the old and new versions, and files that need to be deleted.
Server	Server Application	Manages version update information, patch packages, and version verification. (In the Samples, only a simple example is provided and does not fulfill these functions; developers need to implement this themselves or purchase the GeneralSpacestation service).
GeneralUpdate.Tools	Update Patch Creation Tool	A packaging tool provided by this open-source project for generating update patch packages (.zip file format).

更新闭环

典型的更新流程是：**Client** → **Server (版本检查)** → **下载补丁包** → **写 IPC 加密契约** → **启动 Upgrade** → **Upgrade 校验 Hash** → **解压覆盖** → **启动新版本 Client**。

Quick Start

- Quick Start: <https://www.justerzhu.cn/docs/quickstart/quikstart>
- Tutorial Video: <https://www.bilibili.com/video/BV1c8iyYZE7P>
- Official Website: <https://www.justerzhu.cn/>
- Help Documentation & Official Website Source Repository:
<https://github.com/GeneralLibrary/GeneralUpdate-Samples/tree/main/website/doc>

Version Number Standards

GeneralUpdate follows [Semantic Versioning](#) core principles (`MAJOR.MINOR.PATCH`), extended with a fourth `Revision` segment (`MAJOR.MINOR.PATCH.REVISION`) for `.NET System.Version` compatibility.

- Semantic Versioning: <https://semver.org/>
- NuGet Versioning Guidelines: <https://docs.microsoft.com/en-us/nuget/concepts/package-versioning>
- Assembly Versioning Guidelines: <https://docs.microsoft.com/en-us/dotnet/standard/assembly/versioning>

GeneralUpdate.Core

命名空间: GeneralUpdate.Core | 主要入口: GeneralUpdateBootstrap | NuGet 包: GeneralUpdate.Core

1. 组件简介

1.1 组件概述

GeneralUpdate.Core 是 GeneralUpdate 生态的更新执行核心引擎，负责客户端应用的全生命周期更新管理。它提供可编程的启动器、配置模型、事件通知系统、下载子系统（支持并发、断点续传、重试、校验、后处理管道）、差分补丁管道、版本回写、IPC 进程通信以及平台策略扩展。

核心能力：

能力	说明
多策略更新执行	内置标准 Client/Upgrade 更新、OSS 对象存储更新、静默后台轮询更新三种执行策略
配置驱动	通过 UpdateRequest 强类型配置或 SetSource 轻配置入口，配合 generalupdate.manifest.json 实现极简接入
下载子系统	可替换的下载来源、执行器、重试策略、后处理管道和批量编排器，默认支持并发下载、断点续传、SHA256 校验
差分补丁管道	文件级二进制差分（BSDIFF 4.0 / Streaming HDiff），目录级对比与批量补丁分发，支持并行处理
事件通知	7 种事件回调（版本发现、下载进度、完成、错误、异常等），支持批量事件监听器注册
扩展点体系	10 个可替换接口：生命周期钩子、状态上报、SSL 证书策略、HTTP 认证、下载来源/策略/执行器/管道/编排器、平台策略
版本清单体系	generalupdate.manifest.json 自动发现应用身份，更新后自动回写本地版本，无需业务代码维护版本号
IPC 进程通信	主程序与升级程序之间通过加密文件传递更新上下文，保障升级流程的原子性和安全性
SignalR 实时推送	基于 SignalR 的版本更新实时推送（UpgradeHubService），支持点对点 and 广播推送、自动重连、多事件订阅

解决的业务痛点：

- 桌面应用需要可靠的自动更新能力，但手写更新逻辑涉及版本对比、下载、校验、解压、文件替换、进程重启等多个复杂环节
- 大型应用的分发包体积大，全量更新带宽成本高，需要差分更新降低下载量
- 需要灵活的更新策略（静默后台、用户手动触发、OSS/CDN 分发）
- 升级程序版本独立演进，需要主程序和升级程序的版本协调
- 多产品线需要统一的更新框架，减少重复开发

业务使用场景：

- WPF / WinForms / Avalonia / WinUI 桌面应用的自动更新
- 企业内部工具的统一版本管理
- 通过 CDN / OSS 分发更新包的客户端应用
- 需要差分更新降低带宽消耗的大型客户端

Core 在生态中的定位

GeneralUpdate.Core 是整个生态的**核心引擎**。它不负责生成补丁（那是 Tools/Differential 的工作），也不负责崩溃后的恢复（那是 Bowl 的工作），而是负责**编排整个更新流程**：从版本检查、下载、校验到启动 Upgrade 进程完成文件替换。

1.2 环境与依赖

项目	说明
版本	10.5.0-beta.7
目标框架	netstandard2.0 ; net8.0 ; net10.0 （兼容 .NET Framework 4.6.1+ / .NET Core 2.0+ / .NET 5+ ; net8.0 + 支持 AOT/Trim）
依赖包	GeneralUpdate.Differential （差分算法）、System.Text.Json 、 Microsoft.AspNetCore.SignalR.Client
兼容性	Windows / Linux / macOS；支持 x86 / x64 / ARM64

2. 组件功能列表

功能名称	功能描述	类型	是否必填	备注限制
标准 Client 更新	主程序检查版本、下载更新包、启动升级程序替换文件	基础	必选	需要服务端版本检查 API
标准 Upgrade 更新	独立升级程序读取 IPC 上下文并执行文件替换、差分补丁、版本回写	基础	必选	由主程序启动，通过加密 IPC 传递上下文
OSS Client 更新	从 OSS/CDN 下载版本配置，对比后启动升级程序	基础	可选	版本配置文件托管在对象存储，不依赖服务端 API
OSS Upgrade 更新	OSS 模式下的升级程序，下载并解压资源包	基础	可选	配合 OssClient 使用
静默后台更新	后台轮询版本、静默下载、进程退出时触发升级	基础	可选	需设置 Option.Silent = true
差分补丁管道	文件级二进制差分生成与应用，目录级批量补丁分发	基础	可选	需要 Option.PatchEnabled = true
并发下载	多资源包并发下载，支持断点续传和 SHA256 校验	基础	可选	通过 Option.MaxConcurrency 控制
事件通知回调	版本发现、下载进度、完成、异常等 7 种事件	基础	可选	通过 AddListener* 方法注册
应用身份清单	generalupdate.manifest.json 自动发现与版本回写	拓展	推荐	由 GeneralUpdate.Tools 生成
下载来源扩展	自定义版本清单和下载资源来源	拓展	可选	实现 IDownloadSource
下载执行器扩展	自定义单文件下载实现（HTTP/FTP/SFTP 等）	拓展	可选	实现 IDownloadExecutor

功能名称	功能描述	类型	是否必填	备注限制
下载重试策略扩展	自定义重试、超时、熔断策略	拓展	可选	实现 <code>IDownloadPolicy</code>
下载后处理管道扩展	下载完成后自定义校验、解密、扫描等	拓展	可选	实现 <code>IDownloadPipeline</code>
批量下载编排扩展	完全替换批量下载并发控制逻辑	拓展	可选	实现 <code>IDownloadOrchestrator</code>
生命周期钩子	更新前、下载后、更新后、异常、启动前的业务逻辑注入	拓展	可选	实现 <code>IUpdateHooks</code>
状态上报扩展	更新状态上报到自有服务端	拓展	可选	实现 <code>IUpdateReporter</code>
HTTP 认证扩展	自定义 HTTP 请求认证头	拓展	可选	实现 <code>IHttpAuthProvider</code>
SSL 证书策略扩展	自定义 HTTPS 证书校验逻辑	拓展	可选	实现 <code>ISslValidationPolicy</code>
平台策略扩展	替换平台级文件操作或启动逻辑	拓展	可选	实现 <code>IStrategy</code>
SignalR 实时推送	服务端主动推送版本更新通知，客户端订阅接收，支持点对点和广播推送	拓展	可选	<code>UpgradeHubService</code> ，命名空间 <code>GeneralUpdate.Core.Hubs</code>
推送重连机制	断线自动重连（随机退避策略），连接生命周期管理	拓展	可选	<code>RandomRetryPolicy</code>
推送事件订阅	接收消息、在线状态、重连通知、关闭通知四种事件	拓展	可选	通过 <code>AddListener*</code> 方法注册
多协议认证	支持 HMAC-SHA256、Bearer Token、API Key、HTTP Basic 四种认证方案	基础	可选	通过 <code>AuthScheme</code> 枚举或 <code>HttpAuth()</code> 自定义
跨平台策略	内置 Windows / Linux / macOS 三种 OS 级更新策略，自动选择	基础	自动	根据运行时平台自动选择对应策略
文件树比对	新旧版本目录结构级差异对比，生成增量文件清单	基础	可选	<code>FileTree</code> / <code>FileTreeDiffer</code> / <code>FileTreeComparer</code>
AOT/Trim 兼容	<code>net8.0</code> + 目标框架支持 AOT 发布和裁剪，含源生成 JSON 序列化上下文	基础	可选	<code>JsonContext</code> 命名空间下 9 个序列化上下文

3. API 配置说明

3.1 配置字段（属性 Props）

`UpdateRequest` 配置属性：

字段名	数据类型	默认值	是否必填	枚举/取值范围	说明
<code>UpdateUrl</code>	<code>string</code>	—	是	有效绝对 URL	更新检查 API 地址

字段名	数据类型	默认值	是否必填	枚举/取值范围	说明
UpdateAppName	string	"Update.exe"	推荐	有效文件名	升级程序文件名，如与实际不同必须显式设置
MainAppName	string	—	推荐	有效文件名	主程序文件名，用于重启和识别
ClientVersion	string	—	推荐	SemVer 格式	当前主程序版本
AppSecretKey	string	—	推荐	—	应用密钥，用于服务端认证
InstallPath	string	AppDomain.CurrentDomain.BaseDirectory	可选	有效目录路径	应用安装根目录
ReportUrl	string	null	可选	有效绝对 URL	更新状态上报 API
UpdateLogUrl	string	null	可选	有效绝对 URL	更新日志页面地址
UpgradeClientVersion	string	—	可选	SemVer 格式	升级程序自身版本
ProductId	string	—	可选	—	产品标识，多产品时用于区分
UpdatePath	string	InstallPath	可选	有效目录路径	升级程序所在目录
Bowl	string	null	可选	有效文件名	更新前需关闭的辅助进程名
Scheme	string	null	可选	"Bearer" 等	认证方案（已废弃，推荐使用 AuthScheme ）
Token	string	null	可选	—	认证令牌
AuthScheme	AuthScheme?	null	可选	Hmac , Bearer , ApiKey , Basic	认证方案枚举，设置后自动选择对应 Provider
BasicUsername	string	null	可选	—	HTTP Basic 认证用户名（ AuthScheme = Basic 时必填）
BasicPassword	string	null	可选	—	HTTP Basic 认证密码（ AuthScheme = Basic 时必填）
Files	List	null	可选	—	更新时跳过的指定文件列表
Formats	List	null	可选	—	更新时跳过的扩展名列表
Directories	List	null	可选	—	更新时跳过的目录列表
DriverDirectory	string	null	可选	有效目录路径	驱动更新目录

Option 运行时选项：

字段名	数据类型	默认值	是否必填	枚举/取值范围	说明
Option.AppType	AppType	Client	是	Client(1), Upgrade(2), OssClient(3), OssUpgrade(4)	当前进程角色
Option.DiffMode	DiffMode	Serial	可选	Serial, Parallel	下载执行模式
Option.Encoding	Encoding	UTF8	可选	Encoding 实例	压缩包处理编码
Option.Format	Format	Zip	可选	Zip	更新包格式
Option.DownloadTimeout	int?	30	可选	正整数 (秒)	下载超时时间
Option.PatchEnabled	bool?	true	可选	true / false	是否启用差分补丁
Option.BackupEnabled	bool?	true	可选	true / false	更新前是否备份被替换文件
Option.Silent	bool	false	可选	true / false	是否启用静默轮询更新
Option.SilentPollIntervalMinutes	int	60	可选	正整数	静默模式轮询间隔 (分钟)
Option.LaunchClientAfterUpdate	bool	true	可选	true / false	升级后是否启动主程序
Option.MaxConcurrency	int	3	可选	1 ~ ProcessorCount × 2	下载最大并发数
Option.EnableResume	bool	true	可选	true / false	是否启用断点续传
Option.RetryCount	int	3	可选	非负整数	下载重试次数
Option.VerifyChecksum	bool	true	可选	true / false	是否校验下载文件 Hash
Option.RetryInterval	TimeSpan	1s	可选	TimeSpan 正值	下载重试间隔

3.2 实例方法

GeneralUpdateBootstrap:

方法名	入参明细	使用场景	注意事项
LaunchAsync()	无 (读取已配置的 Option 和 Config)	所有 Core 使用场景的最终入口	会根据 Option.AppType 自动选择执行策略
Cancel()	无	UI 中提供"取消更新"按钮	触发内部 CancellationTokenSource
SetConfig(UpdateRequest)	configInfo — 更新配置对象	主程序内显式配置更新参数	会调用 Validate() 检查关键字段

方法名	入参明细	使用场景	注意事项
<code>SetConfig(string)</code>	<code>filePath</code> — JSON 配置文件路径	从文件读取更新配置	支持相对路径和绝对路径；UTF-8 JSON 格式
<code>SetSource(...)</code>	<code>updateUrl</code> , <code>appSecretKey</code> , <code>reportUrl?</code> , <code>scheme?</code> , <code>token?</code> , <code>authScheme?</code> , <code>basicUsername?</code> , <code>basicPassword?</code> , <code>installPath?</code>	轻配置入口，配合 manifest 使用	只提供服务端入口和密钥，身份信息由 manifest 补齐；支持 HMAC / Bearer / Basic 多种认证方式
<code>SetOption(Option, T)</code>	<code>option</code> — 选项键 <code>value</code> — 选项值	设置运行时选项	传入 <code>null</code> 给可空选项会移除当前设置
<code>UseDiffPipeline(Action)</code>	<code>configure</code> — 差分管道配置委托	替换或调整差分补丁管道	未调用时使用默认配置
<code>AddListenerUpdateInfo(...)</code>	<code>EventHandler</code>	接收服务端版本信息	无更新时也会触发 (<code>Info.Code = 404</code>)
<code>AddListenerUpdatePrecheck(...)</code>	<code>Func</code>	下载前预检查	返回 <code>true</code> 表示跳过非强制更新
<code>AddListenerProgress(...)</code>	<code>EventHandler</code>	更新进度条、状态文本	同时包含下载进度和差分进度
<code>AddListenerMultiDownloadCompleted(...)</code>	<code>EventHandler</code>	标记单个资源下载完成	不要当做"全部下载完成"
<code>AddListenerMultiAllDownloadCompleted(...)</code>	<code>EventHandler</code>	全部下载完成后的后续处理	包含失败汇总 <code>FailedVersions</code>
<code>AddListenerMultiDownloadError(...)</code>	<code>EventHandler</code>	记录单个资源下载失败	整体成功仍以 <code>MultiAllDownloadCompleted</code> 为准
<code>AddListenerMultiDownloadStatistics(...)</code>	<code>EventHandler</code>	展示下载速度和剩余时间	新代码优先使用 <code>AddListenerProgress</code>
<code>AddListenerException(...)</code>	<code>EventHandler</code>	上报异常、展示错误信息	仅通知，不自动重试
<code>AddEventListener()</code>	泛型参数 — 监听器类型	批量注册事件监听器	<code>T</code> 必须实现 <code>IUpdateEventListener</code> ，推荐继承 <code>UpdateEventListenerBase</code>
<code>Hooks()</code>	泛型参数 — Hook 类型	注册生命周期钩子	<code>T</code> 必须实现 <code>IUpdateHooks</code> 且有无参构造函数
<code>UpdateReporter()</code>	泛型参数 — Reporter 类型	注册状态上报器	<code>T</code> 必须实现 <code>IUpdateReporter</code>

方法名	入参明细	使用场景	注意事项
<code>SslPolicy()</code>	泛型参数 — SSL 策略类型	自定义 HTTPS 证书校验	生产环境不建议无条件返回 <code>true</code>
<code>HttpAuth()</code>	泛型参数 — 认证提供器类型	自定义 HTTP 请求认证	<code>T</code> 必须实现 <code>IHttpAuthProvider</code>
<code>DownloadSource()</code>	泛型参数 — 下载来源类型	自定义版本清单来源	<code>T</code> 必须实现 <code>IDownloadSource</code>
<code>DownloadPolicy()</code>	泛型参数 — 下载策略类型	自定义下载重试/超时策略	<code>T</code> 必须实现 <code>IDownloadPolicy</code>
<code>DownloadExecutor()</code>	泛型参数 — 下载执行器类型	自定义单文件下载实现	<code>T</code> 必须实现 <code>IDownloadExecutor</code>
<code>DownloadPipeline()</code>	泛型参数 — 下载管道类型	自定义下载后处理	<code>T</code> 必须实现 <code>IDownloadPipeline</code>
<code>DownloadOrchestrator()</code>	泛型参数 — 下载编排器类型	完全替换批量下载逻辑	只有需要完整替换下载行为时才建议实现
<code>Strategy()</code>	泛型参数 — 策略类型	自定义平台级更新策略	<code>T</code> 必须实现 <code>IStrategy</code>

DiffPipelineBuilder:

方法名	入参明细	使用场景	注意事项
<code>UseDiffer(IBinaryDiffer)</code>	<code>differ</code> — 差分算法实例	替换文件级差分算法	可选 <code>BsdiffDiffer</code> / <code>StreamingHdiffDiffer</code> / 自定义
<code>UseCleanMatcher(ICleanMatcher)</code>	<code>matcher</code> — Clean 匹配器	自定义 Clean 阶段的文件匹配逻辑	默认 <code>DefaultCleanMatcher</code>
<code>UseDirtyMatcher(IDirtyMatcher)</code>	<code>matcher</code> — Dirty 匹配器	自定义 Dirty 阶段的补丁匹配逻辑	默认 <code>DefaultDirtyMatcher</code>
<code>WithParallelism(int)</code>	<code>degree</code> — 并行度	设置差分文件并行处理数	默认 2; 建议 1-8
<code>WithStopOnFirstError(bool)</code>	<code>stop</code> — 是否首次错误即停止	错误策略控制	默认 <code>false</code>
<code>WithProgress(IProgress)</code>	<code>progress</code> — 进度报告器	接入差分进度回调	可配合 <code>Progress</code> 使用
<code>Build()</code>	无	构建差分管道实例	一般在 <code>UseDiffPipeline</code> 回调内部调用

UpdateRequestBuilder:

方法名	入参明细	使用场景	注意事项
<code>Create()</code>	无	从 <code>update_config.json</code> 读取配置	文件不存在会抛出 <code>FileNotFoundException</code>
<code>SetUpdateUrl(string)</code>	<code>url</code>	设置更新地址	必须为绝对 URL

方法名	入参明细	使用场景	注意事项
SetUpgradeAppName(string)	name	设置升级程序文件名	—
SetMainAppName(string)	name	设置主程序文件名	—
SetClientVersion(string)	version	设置客户端版本	SemVer 格式
SetAppSecretKey(string)	key	设置应用密钥	—
SetInstallPath(string)	path	设置安装目录	—
SetProductId(string)	id	设置产品标识	—
SetReportUrl(string)	url	设置上报地址	—
SetUpdateLogUrl(string)	url	设置更新日志地址	—
SetUpgradeClientVersion(string)	version	设置升级程序版本	—
SetBowl(string)	bowl	设置 Bowl 进程名	—
SetDriverDirectory(string)	path	设置驱动目录	—
SetScheme(string)	scheme	设置认证方案	—
SetToken(string)	token	设置认证令牌	—
SetAuthScheme(AuthScheme)	authScheme — 认证方案枚举	设置认证方案 (Hmac / Bearer / ApiKey / Basic)	默认 Hmac
SetBasicUsername(string)	username	设置 Basic 认证用户名	需配合 SetAuthScheme(AuthScheme.Basic)
SetBasicPassword(string)	password	设置 Basic 认证密码	需配合 SetAuthScheme(AuthScheme.Basic)
SetFiles(List)	files	设置跳过文件列表	—
SetFormats(List)	formats	设置跳过扩展名列表	—
SetDirectories(List)	dirs	设置跳过目录列表	—
Build()	无	构建并校验配置对象	会执行 Validate()

UpgradeHubService:

方法名	入参明细	使用场景	注意事项
UpgradeHubService(string, string?, string?)	url — SignalR Hub 地址; token — 可选 ID4 认证令牌; appkey — 可选客户端唯一标识	创建推送服务实例	appkey 用于服务端定向推送, 推荐使用固定 GUID

方法名	入参明细	使用场景	注意事项
StartAsync()	无	建立 SignalR 长连接，开始接收推送	可重复调用（先 StopAsync 后重新 StartAsync ）
StopAsync()	无	优雅停止连接，保留重连能力	适合应用进入后台时调用
DisposeAsync()	无	彻底释放 Hub 及所有资源	释放后不可再复用
AddListenerReceive(Action)	receiveMessageCallback — 接收消息回调	订阅服务端推送的版本更新消息	消息内容为服务端推送的 JSON 字符串
AddListenerOnline(Action)	onlineMessageCallback — 状态回调	订阅在线/离线状态变化通知	—
AddListenerReconnected(Func?)	reconnectedCallback — 重连回调	订阅断线重连成功通知	参数为新的 connectionId （可能为 null）
AddListenerClosed(Func)	closeCallback — 关闭回调	订阅连接关闭通知	正常关闭时异常参数为 null

3.3 回调事件

事件名称	回调参数	触发时机	使用说明
AddListenerUpdateInfo	UpdateInfoEventArgs — Info.Code , Info.Body （ VersionEntry 列表）	标准 Client 策略完成版本对比后触发	无更新时 Code = 404 ；有更新时 Body 包含待下载的 VersionEntry 列表
AddListenerUpdatePrecheck	Func — 返回 true 跳过（非强制） , false 继续	UpdateInfo 事件之后、下载之前	用于磁盘空间检查、网络检测、用户确认弹窗；强制更新不进入跳过逻辑
AddListenerProgress	ProgressEventArgs — Progress （下载）或 DiffProgress （差分）	下载进度或差分进度更新时	同一参数中 Progress 和 DiffProgress 只有一个非空
AddListenerMultiDownloadCompleted	MultiDownloadCompletedEventArgs — Version , IsCompleted	单个资源包下载完成时	不要当作"全部资源下载完成"的判断依据
AddListenerMultiAllDownloadCompleted	MultiAllDownloadCompletedEventArgs — IsAllDownloadCompleted , FailedVersions	所有下载任务结束后触发一次	失败明细在 FailedVersions 中
AddListenerMultiDownloadError	MultiDownloadErrorEventArgs — Exception , Version	单个资源下载失败时	记录失败项用于展示或监控
AddListenerMultiDownloadStatistics	MultiDownloadStatisticsEventArgs — Speed , Remaining , BytesReceived	兼容旧下载统计/自定义下载实现	新代码建议使用 AddListenerProgress
AddListenerException	ExceptionEventArgs — Exception , Message	各策略捕获异常时	仅通知，不等同于自动重试

UpgradeHubService 推送事件：

事件名称	回调参数	触发时机	使用说明
AddListenerReceive	Action — 消息内容（JSON 字符串）	服务端推送版本更新消息时	消息格式由服务端决定，建议 JSON 格式
AddListenerOnline	Action — 状态描述	在线/离线状态变化时	用于 UI 状态展示
AddListenerReconnected	Func? — 新的 connectionId	断线重连成功后	可用于刷新客户端状态
AddListenerClosed	Func — 关闭原因（null 为正常关闭）	连接关闭时	用于记录日志和清理资源

4. 扩展示例（高阶用法）

4.1 组件可扩展能力总览

Core 通过 `AbstractBootstrap` 基类提供 10 个扩展注册方法，全部返回当前 bootstrap 实例，支持链式调用。所有注册的类型必须具有无参构造函数。

扩展接口	注册方法	影响范围
IUpdateHooks	Hooks()	更新生命周期前后置逻辑
IUpdateReporter	UpdateReporter()	更新状态上报
ISslValidationPolicy	SslPolicy()	HTTPS 证书校验
IHttpAuthProvider	HttpAuth()	HTTP 请求认证
IDownloadSource	DownloadSource()	版本清单和下载资源来源
IDownloadPolicy	DownloadPolicy()	下载重试、超时、熔断策略
IDownloadExecutor	DownloadExecutor()	单文件下载实现
IDownloadPipeline	DownloadPipeline()	下载后处理（校验、解密、扫描）
IDownloadOrchestrator	DownloadOrchestrator()	批量下载完整编排
IStrategy	Strategy()	自定义平台级更新策略

4.2 分场景示例

场景 1：自定义差分算法与并行度

【场景说明】大型项目希望使用 `StreamingHdiffDiffer` 获得更快的客户端补丁应用速度，并设置并行度为 4。

【示例代码】

```
using GeneralUpdate.Core;
using GeneralUpdate.Core.Differential;
using GeneralUpdate.Core.Pipeline;
using GeneralUpdate.Differential.Differ;

await new GeneralUpdateBootstrap()
    .SetConfig(request)
    .UseDiffPipeline(builder =>
    {
        builder
            .UseDiffer(new StreamingHdiffDiffer())
            .UseCleanMatcher(new DefaultCleanMatcher())
            .UseDirtyMatcher(new DefaultDirtyMatcher())
            .WithParallelism(4)
            .WithStopOnFirstError(true);
    })
    .SetOption(Option.PatchEnabled, true)
    .SetOption(Option.AppType, AppType.Client)
    .LaunchAsync();
```

【效果&注意事项】

- `StreamingHdiffDiffer` 默认使用 Deflate 压缩，客户端应用补丁更快
- `WithStopOnFirstError(true)` 表示任意补丁失败立即停止所有并行任务
- 并行度 4 适合多核 CPU + SSD 环境

场景 2：自定义生命周期钩子

【场景说明】在更新前检查磁盘空间、更新后写日志、Linux/macOS 下启动前赋予执行权限。

【示例代码】

```
using GeneralUpdate.Core.Hooks;

public sealed class ProductUpdateHooks : IUpdateHooks
{
    public Task<bool> OnBeforeUpdateAsync(HookContext ctx)
    {
        var drive = new DriveInfo(Path.GetPathRoot(ctx.InstallPath!));
        if (drive.AvailableFreeSpace < 500L * 1024 * 1024)
        {
            Console.WriteLine("Insufficient disk space for update.");
            return Task.FromResult(false); // 拒绝更新
        }
        return Task.FromResult(true);
    }

    public Task OnDownloadCompletedAsync(DownloadContext ctx)
    {
        Console.WriteLine($"Downloaded {ctx.AssetName}: {ctx.Success}");
        return Task.CompletedTask;
    }

    public Task OnAfterUpdateAsync(HookContext ctx)
    {
        File.AppendAllText(
            Path.Combine(ctx.InstallPath, "update-history.log"),
            $"{DateTimeOffset.Now:O} {ctx.CurrentVersion} -> {ctx.TargetVersion}{Environment.NewLine}");
        return Task.CompletedTask;
    }

    public Task OnUpdateErrorAsync(HookContext ctx, Exception ex)
    {
        File.AppendAllText(
            Path.Combine(ctx.InstallPath, "update-error.log"),
            $"{ex}{Environment.NewLine}");
        return Task.CompletedTask;
    }

    public Task OnBeforeStartAppAsync(HookContext ctx)
    {
        // Linux/macOS 下可用 UnixPermissionHooks
        return Task.CompletedTask;
    }
}
```

```
await new GeneralUpdateBootstrap()  
    .SetConfig(request)  
    .Hooks()  
    .SetOption(Option.AppType, AppType.Client)  
    .LaunchAsync();
```

【效果&注意事项】

- `OnBeforeUpdateAsync` 返回 `false` 会中断本次更新
- `OnBeforeStartAppAsync` 适合 Linux/macOS 的 `chmod +x` 操作
- 内置 `UnixPermissionHooks` 可直接使用

场景 3：自定义下载来源（私有服务/配置中心）

【场景说明】从企业内部配置中心获取下载资源列表，而不是调用标准版本检查 API。

【示例代码】

```

using GeneralUpdate.Core.Download.Abstractions;
using GeneralUpdate.Core.Download.Models;

public sealed class ConfigCenterDownloadSource : IDownloadSource
{
    public async Task ListAsync(Cancellation token = default)
    {
        // 从配置中心拉取资源列表...
        var assets = new[]
        {
            new DownloadAsset(
                Name: "MyApp-2.0.0.zip",
                Url: "https://cdn.internal.example.com/releases/MyApp-2.0.0.zip",
                Size: 50_000_000,
                SHA256: "abc123...",
                Version: "2.0.0")
        };

        return new DownloadSourceResult
        {
            Assets = assets,
            HasMainUpdate = true,
            HasUpgradeUpdate = false
        };
    }
}

await new GeneralUpdateBootstrap()
    .SetConfig(request)
    .DownloadSource()
    .SetOption(Option.AppType, AppType.Client)
    .LaunchAsync();

```

【效果&注意事项】

- 自定义 `IDownloadSource` 会完全替换默认的 HTTP 版本检查逻辑
- 需要同时注册 `DownloadOrchestrator()` 时, orchestrator 会接管完整下载流程

场景 4: 多协议 HTTP 认证

【场景说明】Core 内置四种认证方案, 通过 `AuthScheme` 枚举一键切换, 也可通过 `IHttpAuthProvider` 完全自定义。

方式一: 使用内置 `AuthScheme` 枚举 (推荐)

```
using GeneralUpdate.Core.Configuration;

// HMAC-SHA256 签名认证（默认）
var request = new UpdateRequest
{
    UpdateUrl = "https://update.example.com/api/upgrade/verification",
    AppSecretKey = "your-app-secret",
    AuthScheme = AuthScheme.Hmac
};

// Bearer Token 认证
var bearerRequest = new UpdateRequest
{
    UpdateUrl = "https://update.example.com/api/upgrade/verification",
    AppSecretKey = "your-app-secret",
    AuthScheme = AuthScheme.Bearer,
    Token = "your-jwt-token"
};

// API Key 认证
var apiKeyRequest = new UpdateRequest
{
    UpdateUrl = "https://update.example.com/api/upgrade/verification",
    AppSecretKey = "your-app-secret",
    AuthScheme = AuthScheme.ApiKey,
    Token = "your-api-key"
};

// HTTP Basic 认证
var basicRequest = new UpdateRequest
{
    UpdateUrl = "https://update.example.com/api/upgrade/verification",
    AppSecretKey = "your-app-secret",
    AuthScheme = AuthScheme.Basic,
    BasicUsername = "admin",
    BasicPassword = "password123"
};
```

方式二：自定义 IHttpAuthProvider（高级）

【场景说明】为 Core 发出的 HTTP 请求追加自定义认证逻辑（如从配置中心动态获取 Token）。

【示例代码】

```

using GeneralUpdate.Core.Security;

public sealed class JwtAuthProvider : IHttpAuthProvider
{
    private readonly string _token;

    public JwtAuthProvider()
    {
        // 从配置或环境变量读取 token
        _token = Environment.GetEnvironmentVariable("UPDATE_JWT_TOKEN") ?? "";
    }

    public Task ApplyAuthAsync(HttpRequestMessage request, CancellationToken token = default)
    {
        request.Headers.Authorization =
            new System.Net.Http.Headers.AuthenticationHeaderValue("Bearer", _token);
        return Task.CompletedTask;
    }
}

await new GeneralUpdateBootstrap()
    .SetConfig(request)
    .HttpAuth()
    .SetOption(Option.AppType, AppType.Client)
    .LaunchAsync();

```

【效果&注意事项】

- 内置 Provider: `HmacAuthProvider` (默认)、`BearerTokenAuthProvider`、`ApiKeyAuthProvider`、`BasicAuthProvider`
- `HttpAuthProviderFactory` 根据 `AuthScheme` 自动选择对应 Provider
- 自定义 Provider 需要在无参构造函数中自行读取配置
- HMAC 签名算法: `HMAC-SHA256(body|timestamp)`, 请求头 `X-Update-Timestamp` + `X-Update-Signature`

场景 5: 静默更新 + 进程退出触发升级

【场景说明】主程序启动后后台轮询更新, 下载完成后不打扰用户, 待进程退出时启动升级程序。

【示例代码】


```

using GeneralUpdate.Core;

// 主程序启动时
var bootstrap = new GeneralUpdateBootstrap()
    .SetSource(
        updateUrl: "https://update.example.com/api/upgrade/verification",
        appSecretKey: "your-app-secret")
    .SetOption(Option.AppType, AppType.Client)
    .SetOption(Option.Silent, true)
    .SetOption(Option.SilentPollIntervalMinutes, 30)
    .SetOption(Option.LaunchClientAfterUpdate, true)
    .AddListenerException( (_, e) =>
    {
        Console.WriteLine($"Update error: {e.Message}");
    });

await bootstrap.LaunchAsync();

// 应用退出时检查是否有准备好的更新
AppDomain.CurrentDomain.ProcessExit += ( _, _ ) =>
{
    if (bootstrap.SilentOrchestrator?.HasPreparedUpdate == true)
    {
        bootstrap.SilentOrchestrator.TryLaunchUpgrade();
    }
};

```

【效果&注意事项】

- 静默模式在 `AppType.Client` 下生效
- 文件替换发生在主程序退出之后
- 轮询间隔建议不低于 30 分钟

场景 6: SignalR 实时推送 + 常规更新联动

【场景说明】客户端同时使用 `UpgradeHubService` 接收服务端实时推送和 `GeneralUpdateBootstrap` 执行常规更新。服务端有新版时可立即推送通知，客户端无需等待轮询即可触发更新。

【示例代码】

```
using GeneralUpdate.Core;
using GeneralUpdate.Core.Configuration;
using GeneralUpdate.Core.Hubs;

// 1. 启动 SignalR 推送监听
var hub = new UpgradeHubService(
    "http://localhost:5000/UpgradeHub",
    token: null,
    appkey: "dfef5833-975e-4afb-88f1-6278ee9aef6");

hub.AddListenerReceive(async (message) =>
{
    Console.WriteLine($"收到实时推送: {message}");
    // 收到推送后可以立即触发更新检查
    // 或在 UI 中提示用户有新版本可用
});

hub.AddListenerOnline((info) =>
    Console.WriteLine($"在线状态: {info}"));

hub.AddListenerReconnected((connectionId) =>
{
    Console.WriteLine($"已重连, connectionId={connectionId}");
    return Task.CompletedTask;
});

hub.AddListenerClosed((exception) =>
{
    if (exception is not null)
        Console.WriteLine($"连接异常关闭: {exception.Message}");
    else
        Console.WriteLine("连接已正常关闭");
    return Task.CompletedTask;
});

await hub.StartAsync();

// 2. 常规更新流程
await new GeneralUpdateBootstrap()
    .SetSource(
        updateUrl: "https://update.example.com/api/upgrade/verification",
        appSecretKey: "your-app-secret")
    .SetOption(Option.AppType, AppType.Client)
    .AddListenerException(_, e) => Console.WriteLine(e.Exception))
```

```
.LaunchAsync();

// 3. 应用退出时清理
// await hub.StopAsync();
// await hub.DisposeAsync();
```

【效果&注意事项】

- `UpgradeHubService` 与 `GeneralUpdateBootstrap` 互补：推送做通知，Bootstrap 做实际更新
- `appkey` 推荐与 `AppSecretKey` 保持一致，便于服务端定向推送
- `StopAsync` 保留重连能力，适合应用进入后台时调用
- `DisposeAsync` 彻底释放，适合应用退出时调用

场景 7：DI 容器中注册 `UpgradeHubService`

【场景说明】在 Prism / Generic Host / ASP.NET Core 等 DI 容器中注册 `IUpgradeHubService`，管理推送服务生命周期。

【示例代码】

```
using GeneralUpdate.Core.Hubs;

// Prism 示例
protected override void RegisterTypes(IContainerRegistry containerRegistry)
{
    containerRegistry.Register<IUpgradeHubService, UpgradeHubService>();
}

// 在 ViewModel 中使用
public MainWindowViewModel(IUpgradeHubService hubService)
{
    hubService.AddListenerReceive((message) =>
    {
        Console.WriteLine($"收到推送: {message}");
    });
    _ = hubService.StartAsync();
}

// Generic Host / ASP.NET Core 示例
builder.Services.AddSingleton(sp =>
{
    var config = sp.GetRequiredService();
    return new UpgradeHubService(
        config["HubUrl"]!,
        appkey: config["AppSecretKey"]);
});
```

【效果&注意事项】

- DI 容器管理生命周期，避免手动 Dispose
- 可将配置从 `appsettings.json` 注入

场景 8：跨平台自适应策略

【场景说明】Core 根据运行时平台自动选择 `WindowsStrategy` / `LinuxStrategy` / `MacStrategy`，无需手动指定。也可通过 `PlatformType` 显式控制或自定义平台策略。

【示例代码】

```
using GeneralUpdate.Core;
using GeneralUpdate.Core.Configuration;
using GeneralUpdate.Core.Strategy;

// 方式一：自动检测（推荐）
// Core 会根据 RuntimeInformation 自动选择对应平台策略
await new GeneralUpdateBootstrap()
    .SetSource(
        updateUrl: "https://update.example.com/api/upgrade/verification",
        appSecretKey: "your-app-secret")
    .SetOption(Option.AppType, AppType.Client)
    .LaunchAsync();

// Windows → WindowsStrategy
// Linux   → LinuxStrategy（无 Bowl 支持）
// macOS   → MacStrategy

// 方式二：显式指定平台策略（高级）
await new GeneralUpdateBootstrap()
    .SetConfig(request)
    .Strategy()
    .SetOption(Option.AppType, AppType.Client)
    .LaunchAsync();
```

【效果&注意事项】

- 三种平台策略均继承 `AbstractStrategy`，共享 Hash → Compress → Patch 管道
- `WindowsStrategy` 额外支持 Bowl 辅助进程管理
- `LinuxStrategy` 建议配合 `UnixPermissionHooks` 使用以自动赋予执行权限
- 平台策略可被自定义 `IStrategy` 完全替换

5. 常规使用示例

5.1 快速入门示例（最简 demo）

最简配置：使用 manifest 自动发现身份信息，只需要配置服务端入口和密钥。

```
using GeneralUpdate.Core;

await new GeneralUpdateBootstrap()
    .SetSource(
        updateUrl: "https://update.example.com/api/upgrade/verification",
        appSecretKey: "your-app-secret")
    .SetOption(Option.AppType, AppType.Client)
    .LaunchAsync();
```

升级程序入口（Update.exe）：

```
await new GeneralUpdateBootstrap()
    .SetOption(Option.AppType, AppType.Upgrade)
    .AddListenerException((_, e) => Console.WriteLine(e.Exception))
    .LaunchAsync();
```

5.2 基础参数组合示例

```
using GeneralUpdate.Core;
using GeneralUpdate.Core.Configuration;

var request = new UpdateRequest
{
    UpdateUrl = "https://update.example.com/api/upgrade/verification",
    ReportUrl = "https://update.example.com/api/upgrade/report",
    UpdateAppName = "UpgradeSample.exe",
    MainAppName = "ClientSample.exe",
    InstallPath = AppDomain.CurrentDomain.BaseDirectory,
    ClientVersion = "1.0.0",
    AppSecretKey = "your-app-secret",
    ProductId = "your-product-id",
    Files = new List<string> { "appsettings.json" },
    Formats = new List<string> { ".log", ".tmp" },
    Directories = new List<string> { "logs", "cache" }
};

await new GeneralUpdateBootstrap()
    .SetConfig(request)
    .SetOption(Option.AppType, AppType.Client)
    .SetOption(Option.DiffMode, DiffMode.Parallel)
    .SetOption(Option.MaxConcurrency, 4)
    .SetOption(Option.PatchEnabled, true)
    .AddListenerProgress((_, e) =>
    {
        if (e.Progress != null)
            Console.WriteLine($"{e.Progress.AssetName}: {e.Progress.Percentage:F1}%");
    })
    .AddListenerException((_, e) => Console.WriteLine(e.Exception))
    .LaunchAsync();
```

5.2.1 SignalR 实时推送快速入门

```
using GeneralUpdate.Core.Hubs;

// 创建推送客户端
var hub = new UpgradeHubService(
    "http://localhost:5000/UpgradeHub",
    appkey: Guid.NewGuid().ToString());

// 订阅推送消息
hub.AddListenerReceive((message) =>
{
    Console.WriteLine($"收到更新推送: {message}");
});

// 建立连接
await hub.StartAsync();

Console.WriteLine("已连接, 等待服务端推送...");
Console.ReadLine();

// 停止连接 (保留重连能力)
await hub.StopAsync();

// 释放资源 (不可再复用)
await hub.DisposeAsync();
```

5.3 真实业务落地示例 (多参数联动)

完整 Client 端更新流程, 包含事件监听、差分管道、并发控制和状态上报:

```

using GeneralUpdate.Core;
using GeneralUpdate.Core.Configuration;
using GeneralUpdate.Core.Download;
using GeneralUpdate.Core.Download.Reporting;
using GeneralUpdate.Core.Event;
using GeneralUpdate.Core.Hooks;
using GeneralUpdate.Core.Pipeline;
using GeneralUpdate.Differential.Differ;

// 1. 构建配置
var request = new UpdateRequestBuilder()
    .SetUpdateUrl("https://update.mycompany.com/api/upgrade/verification")
    .SetReportUrl("https://update.mycompany.com/api/upgrade/report")
    .SetUpgradeAppName("MyApp.Upgrade.exe")
    .SetMainAppName("MyApp.exe")
    .SetClientVersion("1.0.0")
    .SetUpgradeClientVersion("1.0.0")
    .SetAppSecretKey("prod-secret-key")
    .SetProductId("my-product")
    .SetInstallPath(AppDomain.CurrentDomain.BaseDirectory)
    .SetScheme("Bearer")
    .SetToken(Environment.GetEnvironmentVariable("UPDATE_TOKEN") ?? "")
    .SetFiles(new List<string> { "appsettings.Development.json" })
    .SetFormats(new List<string> { ".log", ".tmp", ".pdb" })
    .SetDirectories(new List<string> { "logs", "cache", "temp" })
    .Build();

```

```

// 2. 注入业务钩子
public sealed class BusinessUpdateHooks : IUpdateHooks
{
    public Task<bool> OnBeforeUpdateAsync(HookContext ctx)
    {
        // 检查磁盘空间
        var drive = new DriveInfo(Path.GetPathRoot(ctx.InstallPath)!);
        if (drive.AvailableFreeSpace < 1024L * 1024 * 1024) // < 1GB
            return Task.FromResult(false);
        return Task.FromResult(true);
    }

    public Task OnDownloadCompletedAsync(DownloadContext ctx)
        => Task.CompletedTask;

    public Task OnAfterUpdateAsync(HookContext ctx)
    {

```



```

        // 写更新成功日志
        File.AppendAllText(
            Path.Combine(ctx.InstallPath, "update.log"),
            $"{DateTimeOffset.Now:O} Updated to {ctx.TargetVersion}{Environment.NewLine}");
        return Task.CompletedTask;
    }

    public Task OnUpdateErrorAsync(HookContext ctx, Exception ex)
    {
        // 上报到监控系统
        return Task.CompletedTask;
    }

    public Task OnBeforeStartAppAsync(HookContext ctx)
        => Task.CompletedTask;
}

// 3. 配置差分管道
Action configurePipeline = builder =>
{
    builder
        .UseDiffer(new StreamingHdiffDiffer())
        .WithParallelism(4)
        .WithStopOnFirstError(true)
        .WithProgress(new Progress(p =>
        {
            Console.WriteLine($"Patch: {p.Completed}/{p.Total} {p.CurrentFile} {p.Percentage}%");
        }));
};

// 4. 启动更新
var bootstrap = new GeneralUpdateBootstrap()
    .SetConfig(request)
    .SetOption(Option.AppType, AppType.Client)
    .SetOption(Option.DiffMode, DiffMode.Parallel)
    .SetOption(Option.MaxConcurrency, 4)
    .SetOption(Option.DownloadTimeout, 120)
    .SetOption(Option.PatchEnabled, true)
    .SetOption(Option.BackupEnabled, true)
    .SetOption(Option.VerifyChecksum, true)
    .SetOption(Option.RetryCount, 5)
    .SetOption(Option.RetryInterval, TimeSpan.FromSeconds(2))
    .Hooks()
    .UseDiffPipeline(configurePipeline)

```

```

.AddListenerUpdateInfo( (_, e) =>
{
    if (e.Info?.Code == "404")
    {
        Console.WriteLine("Already up to date.");
        return;
    }
    Console.WriteLine($"Found {e.Info?.Body?.Count ?? 0} version(s) to download.");
})
.AddListenerUpdatePrecheck(e =>
{
    var hasUpdate = (e.Info?.Body?.Count ?? 0) > 0;
    return !hasUpdate; // 有更新则继续 (返回 false = 不跳过)
})
.AddListenerProgress( (_, e) =>
{
    if (e.Progress != null)
    {
        var pct = e.Progress.Percentage;
        var status = e.Progress.Status;
        var name = e.Progress.AssetName;
        Console.WriteLine($"[Download] {name}: {pct:F1}% ({status})");
    }
    if (e.DiffProgress != null)
    {
        var d = e.DiffProgress;
        Console.WriteLine($"[Patch] {d.CurrentFile}: {d.Completed}/{d.Total} {d.Percentage}%");
    }
})
.AddListenerMultiAllDownloadCompleted( (_, e) =>
{
    if (e.IsAllDownloadCompleted)
        Console.WriteLine("All downloads completed successfully.");
    else
    {
        Console.WriteLine($"Download completed with {e.FailedVersions.Count} failure(s):");
        foreach (var (asset, error) in e.FailedVersions)
            Console.WriteLine($" - {asset.Name}: {error}");
    }
})
.AddListenerException( (_, e) =>
{
    Console.WriteLine($"Update error: {e.Message}");
    Console.WriteLine(e.Exception);
}

```

```
});
```

```
await bootstrap.LaunchAsync();
```

5.4 OSS 对象存储更新（零服务端部署）

【场景说明】OSS（Object Storage Service，对象存储服务）更新模式适用于**没有后端服务**的部署场景：将版本配置与更新包直接托管在对象存储（阿里云 OSS / AWS S3 / MinIO 等）上，客户端通过静态 `versions.json` 文件完成版本检查与更新，全程无需编写服务端 API。

工作流程：

对象存储 Bucket（阿里云 OSS / AWS S3 / MinIO）

└─ versions.json

└─ packet_20250102230201638_1.0.0.1.zip

← 版本配置（静态文件）

← 更新包

步骤	角色	说明
1	运维	将 <code>versions.json</code> 与更新包上传至对象存储 Bucket
2	<code>OssClient</code> （主程序）	启动时从 <code>UpdateUrl</code> 下载 <code>versions.json</code> ，按 <code>PubTime</code> 倒序取最新版本
3	<code>OssClient</code> （主程序）	最新版本与 <code>ClientVersion</code> 做 SemVer 2.0 对比，无新版本则结束检查
4	<code>OssClient</code> （主程序）	检测到新版本 → 启动升级程序 → 主程序自行退出
5	<code>OssUpgrade</code> （升级程序）	读取版本配置，下载所有高于当前版本的更新包并解压覆盖安装目录
6	<code>OssUpgrade</code> （升级程序）	回写 <code>generalupdate.manifest.json</code> 版本号 → 启动主程序 → 自行退出

两个进程如何配合：

OSS 模式需要**两个独立的可执行程序**，角色由各自代码中的 `AppType` 决定，而不是通过启动参数或 IPC 文件传递：

程序	项目代码	AppType	职责
主程序（如 <code>MyApp.exe</code> ）	业务应用	<code>AppType.OssClient</code>	检查版本、拉起升级程序、退出
升级程序（如 <code>UpgradeApp.exe</code> ）	独立小项目	<code>AppType.OssUpgrade</code>	下载更新包、解压安装、拉起主程序、退出

`OssClient` 检测到新版本后通过 `Process.Start` 直接启动升级程序，**不传任何参数**——升级程序靠自身代码中的 `AppType.OssUpgrade` 知道自己该执行升级，并依靠安装目录下的 `generalupdate.manifest.json` 获取主程序身份（见下文“安装目录布局”）。

versions.json 版本配置格式：

```
[
  {
    "PacketName": "packet_20250102230201638_1.0.0.1",
    "Hash": "ad1a85a9169ca0083ab54ba390e085c56b9059efc3ca8aa1ec9ed857683cc4b1",
    "Version": "1.0.0.1",
    "Url": "https://your-bucket.example.com/packages/packet_20250102230201638_1.0.0.1.zip",
    "PubTime": "2025-01-02T23:48:21"
  }
]
```

字段	类型	说明
PacketName	string	更新包名称，用于生成本地压缩文件名（ {PacketName}.zip ）
Hash	string	更新包 SHA256 哈希，下载完成后与本地文件比对校验
Version	string	版本号（SemVer 2.0 格式），用于版本对比
Url	string	更新包下载地址（对象存储公开或预签名 URL）
PubTime	DateTime	发布时间，OSS 模式按此字段倒序选取最新版本

配置文件命名

`versions.json` 下载后会以 `{MainAppName}_versions.json` 的名称保存在安装目录，升级程序从同一位置读取。主程序与升级程序必须使用一致的 `MainAppName` / `UpdateAppName` 配置。

配置文件命名

`versions.json` 下载后会以 `{MainAppName}_versions.json` 的名称保存在安装目录，升级程序从同一位置读取。主程序与升级程序必须使用一致的 `MainAppName` / `UpdateAppName` 配置。

安装目录布局（主程序 + 升级程序 + manifest）：

安装目录 (InstallPath)

- └─ MyApp.exe ← 主程序 (AppType.OssClient)
- └─ MyApp.dll / 资源文件...
- └─ generalupdate.manifest.json ← 应用身份清单 (见下方示例)
- └─ update/
 - └─ UpgradeApp.exe ← 升级程序 (AppType.OssUpgrade)

安装目录 (InstallPath)

- └─ MyApp.exe ← 主程序 (AppType.OssClient)
- └─ MyApp.dll / 资源文件...
- └─ generalupdate.manifest.json ← 应用身份清单 (见下方示例)
- └─ update/
 - └─ UpgradeApp.exe ← 升级程序 (AppType.OssUpgrade)

对应 `generalupdate.manifest.json` 示例（身份字段也可全部写在代码里，两者共存时 manifest 优先）：

```
{
  "mainAppName": "MyApp.exe",
  "clientVersion": "1.0.0.0",
  "appType": "OssClient",
  "updateAppName": "UpgradeApp.exe",
  "updatePath": "update/"
}
```

- 升级程序必须位于 `InstallPath` 或 `UpdatePath` 指定的目录，否则 `OssClient` 无法拉起它并抛出 `FileNotFoundException`

- 升级程序运行时会读取 `{MainAppName}_versions.json` (`OssClient` 下载后保存在 `InstallPath`)，并依靠 `generalupdate.manifest.json` 获得 `MainAppName` / `UpdateAppName` / `ClientVersion` 等身份
- 若升级程序放在 `update/` 子目录（其程序基目录是子目录）而 `manifest` 在父目录，升级程序代码中需通过 `SetSource(..., installPath: ...)` 显式指向父目录，例如 `Path.GetFullPath(Path.Combine(baseDir, ".."))`

OSS 模式 UpdateRequest 字段说明：

字段	是否必填	说明
UpdateUrl	必填	<code>versions.json</code> 的公开访问地址， <code>OssClient</code> 启动时据此下载版本配置
MainAppName	推荐	主程序可执行文件名；决定版本配置的本地文件名（ <code>{MainAppName}_versions.json</code> ），也是升级完成后拉起的目标
UpdateAppName	推荐	升级程序可执行文件名； <code>OssClient</code> 检测到新版本后拉起的进程，默认 <code>Update.exe</code>
ClientVersion	推荐	当前主程序版本号（SemVer 2.0 格式），用于版本对比与更新包筛选
InstallPath	可选	应用安装根目录，默认 <code>AppDomain.CurrentDomain.BaseDirectory</code> ； <code>versions.json</code> 的保存位置与更新包的解压目标目录
UpdatePath	可选	升级程序所在目录，默认 <code>InstallPath</code> ； <code>OssClient</code> 优先从此目录解析升级程序

OSS 模式无需配置的字段

OSS 模式没有服务端 API，以下与认证、上报、日志相关的字段**无需配置**（配置了也不会被使用）：`AppSecretKey`、`ReportUrl`、`UpdateLogUrl`、`Token`、`AuthScheme`、`BasicUsername`、`BasicPassword`、`Bowl`。

另外，OSS 模式还会用到以下 `Option` 运行时选项：

Option	默认值	说明
Option.AppType	—	必填，指定当前进程角色： <code>OssClient</code> （主程序）或 <code>OssUpgrade</code> （升级程序）
Option.Encoding	UTF8	更新包解压时的字符编码
Option.DownloadTimeout	60	更新包下载超时时间（秒）

主程序示例（`AppType.OssClient`）：

```

using GeneralUpdate.Core;
using GeneralUpdate.Core.Configuration;

var request = new UpdateRequest
{
    UpdateUrl = "https://your-bucket.example.com/packages/versions.json",
    UpdateAppName = "OSSUpgradeSample.exe",
    MainAppName = "OSSClientSample.exe",
    ClientVersion = "1.0.0.0",
    InstallPath = AppDomain.CurrentDomain.BaseDirectory
};

await new GeneralUpdateBootstrap()
    .SetConfig(request)
    .SetOption(Option.AppType, AppType.OssClient)
    .AddListenerException( (_, e) => Console.WriteLine(e.Exception))
    .LaunchAsync();

```

升级程序示例（ AppType.OssUpgrade ）：

```

using GeneralUpdate.Core;

// 升级程序无需 SetConfig: MainAppName / UpdateAppName / ClientVersion 等身份
// 字段由安装目录下的 generalupdate.manifest.json 自动发现（AppMetadataDiscoverer）。
// 若升级程序位于 update/ 子目录（manifest 在父目录），需显式指向父目录：
//
// var baseDir = AppDomain.CurrentDomain.BaseDirectory;
// await new GeneralUpdateBootstrap()
//     .SetSource("https://your-bucket.example.com/packages/versions.json", "",
//         installPath: Path.GetFullPath(Path.Combine(baseDir, "..")))
//     .SetOption(Option.AppType, AppType.OssUpgrade)
//     .LaunchAsync();

await new GeneralUpdateBootstrap()
    .SetOption(Option.AppType, AppType.OssUpgrade)
    .AddListenerException( (_, e) => Console.WriteLine(e.Exception))
    .LaunchAsync();

```

新手 5 步上手

1. **建主程序项目**：引入 NuGet 包 `GeneralUpdate.Core`，按上文"主程序示例"编写代码（`AppType.OssClient`），`UpdateAppName` 填你的升级程序文件名
2. **建升级程序项目**：独立小项目，按上文"升级程序示例"编写代码（`AppType.OssUpgrade`），编译产物文件名与 `UpdateAppName` 保持一致
3. **准备 manifest**：用 `GeneralUpdate.Tools` 生成 `generalupdate.manifest.json`（或参照上文示例手写），与主程序、升级程序一同放入安装目录
4. **发布更新包**：将应用文件打成 zip 上传对象存储；编写 `versions.json` 并上传（`Hash` 填该 zip 的 SHA256，`Url` 填 zip 的可访问地址）
5. **运行验证**：启动主程序，观察日志确认下载 `versions.json`、检测到新版本、拉起升级程序并完成安装

【效果&注意事项】

- `UpdateUrl` 直接指向 `versions.json` 的公开地址，更新包同样托管在对象存储上，无需 `Verification / Report API`
- `Bucket` 建议设为私有并使用预签名 URL；若设为公共读，请勿存放敏感内容
- `OSS` 模式不区分主程序与升级程序的更新包：`versions.json` 中所有高于当前版本的包都会被依次下载并应用
- 打包更新包时请勿包含组件内部依赖程序集（如 `System.Text.Json.dll`、`Microsoft.Bcl.AsyncInterfaces.dll` 等），也可通过 `Files / Formats / Directories` 跳过配置排除
- 版本号使用 SemVer 2.0 格式（如 `1.0.0.0`），否则版本对比会失败
- 与标准模式一致，`OSS` 模式同样支持 `generalupdate.manifest.json` 身份发现、`Hooks()` 生命周期钩子、`UpdateReporter()` 状态上报，以及 `DownloadSource()` / `DownloadOrchestrator()` 自定义下载来源与编排

6. 全局配置

`Core` 不依赖全局配置文件，而是通过 `generalupdate.manifest.json` 提供应用身份信息。下面是 `manifest` 的配置语法和优先级规则。

Manifest 配置语法

```
{
  "mainAppName": "ClientSample.exe",
  "clientVersion": "1.0.0",
  "appType": "Client",
  "updateAppName": "UpgradeSample.exe",
  "upgradeClientVersion": "1.0.0",
  "productId": "sample-product",
  "updatePath": "update/"
}
```

配置优先级规则

优先级越高，覆盖力越强。代码中的显式设置总是优先于 `manifest` 和默认值。

优先级	配置来源	说明
1（最高）	代码中 <code>SetConfig(UpdateRequest)</code> 或 <code>SetSource(...)</code> 显式设置的值	覆盖所有其他来源
2	<code>generalupdate.manifest.json</code> 中的字段	自动补齐代码中未显式设置的字段
3（最低）	组件内部默认值	<code>UpdateAppName = "Update.exe"</code> , <code>InstallPath = BaseDirectory</code> 等

最佳实践：用 manifest 存身份，用代码存 secret

把 `MainAppName`、`ClientVersion`、`UpdateAppName`、`ProductId` 等身份字段放在 manifest 中（由 Tools 自动生成），把 `UpdateUrl`、`AppSecretKey`、`Token` 等敏感信息写在代码里。身份属于项目结构，不应硬编码；secret 属于安全配置，不应落入版本控制。

Manifest 字段映射

JSON 字段	Core 字段	说明
<code>mainAppName</code>	<code>MainAppName</code>	主程序可执行文件名
<code>clientVersion</code>	<code>ClientVersion</code>	当前主程序版本
<code>appType</code>	<code>AppType</code>	当前进程角色（ <code>Client</code> , <code>Upgrade</code> , <code>OssClient</code> , <code>OssUpgrade</code> ）
<code>updateAppName</code>	<code>UpdateAppName</code>	升级程序文件名
<code>upgradeClientVersion</code>	<code>UpgradeClientVersion</code>	升级程序自身版本
<code>productId</code>	<code>ProductId</code>	产品标识
<code>updatePath</code>	<code>UpdatePath</code>	升级程序所在目录（相对 <code>InstallPath</code> ）

版本回写机制

更新成功后，Core 自动回写客户端版本到 `generalupdate.manifest.json`：

场景	回写字段
主程序更新完成	<code>ClientVersion</code>
升级程序自身更新完成	<code>UpgradeClientVersion</code>

日志配置

```
using GeneralUpdate.Core;

// 关闭日志（性能敏感场景）
GeneralTracer.SetTracingEnabled(false);

// 重新开启（排查问题）
GeneralTracer.SetTracingEnabled(true);

// 释放日志资源
GeneralTracer.Dispose();
```

AOT / Trim 兼容性

net8.0 和 net10.0 目标框架支持 AOT 发布和裁剪。Core 在 JsonContext 命名空间下提供了 9 个源生成 JSON 序列化上下文，覆盖所有配置和模型类型的序列化需求，避免反射导致的裁剪问题：

上下文类	覆盖类型
BaseResponseJsonContext	BaseResponseDTO 、 VersionRespDTO
VersionEntryJsonContext	VersionEntry 、 VersionIdentity
UpdateRequestJsonContext	UpdateRequest 、 UpdateConfiguration
ProcessContractJsonContext	ProcessContract
ManifestInfoJsonContext	ManifestInfo
OssVersionRecordJsonContext	OssVersionRecord
DownloadAssetJsonContext	DownloadAsset
BlackPolicyJsonContext	BlackPolicy
PushPayloadJsonContext	PushPayload

进程环境变量（IPC 辅助）

Environments 静态类通过 AES 加密临时文件在 Client 与 Upgrade 进程之间传递键值对环境变量，用于升级失败追踪等场景：

环境变量 Key	说明
UpgradeFail	记录上次升级失败的版本号，避免重复尝试失败版本

```
using GeneralUpdate.Core.Configuration;

// 设置环境变量（Client 端写入）
Environments.Set("UpgradeFail", "2.0.0");

// 读取环境变量（Upgrade 端读取）
var failedVersion = Environments.Get("UpgradeFail");
```

相关资源

- [GeneralUpdate 仓库](#)
- [Samples 示例代码](#)
- [GeneralUpdate.Tools](#)
- [入门实战手册](#)
- [组件文档](#)

GeneralUpdate.Bowl

命名空间: GeneralUpdate.Bowl | 主要入口: new Bowl().LaunchAsync(BowlContext, CancellationToken) | NuGet 包: GeneralUpdate.Bowl

1. 组件简介

1.1 组件概述

GeneralUpdate.Bowl 是升级完成后的启动守护组件。它不负责下载、解压或替换升级包，而是在新版本文件落地后、主程序即将启动或已启动时，监控目标进程是否在启动阶段崩溃。如果捕获到崩溃，它会生成 Dump 内存快照、写出失败报告 JSON、导出系统诊断信息，并在升级模式下自动将备份目录恢复回安装目录，避免用户一直停留在不可启动的新版本上。

核心能力：

能力	说明
进程崩溃监控	通过 ProcDump (Windows/Linux) 或 lldb (macOS) 附加到目标进程，捕获启动期末处理异常
Dump 内存快照	支持 Full / Mini / Heap 三种 Dump 类型，可按需选择文件大小和完整度
崩溃报告生成	自动生成包含监控参数和工具输出的 {version}_fail.json 报告文件
系统诊断导出	Windows 下自动导出驱动列表、系统信息和最近系统事件日志
自动回滚恢复	升级模式下将备份目录覆盖复制回安装目录，实现一键回退到旧版本
失败版本标记	写入 UpgradeFail 标记，Core 后续跳过该失败版本直到服务端提供更高版本
事件回调通知	OnCrash 回调允许上传诊断包、通知用户或记录审计信息
独立监控模式	Normal 模式只做崩溃捕获和报告输出，不自动恢复备份，适合通用进程监控

解决的业务痛点：

- 新版本升级后在启动阶段崩溃，用户无法使用应用且无法自行回退
- 开发者缺少崩溃现场信息（Dump、系统环境）来定位"升级后打不开"的问题
- 需要自动化回滚机制降低升级风险，避免人工介入

Bowl 不是更新组件，是守护组件

Bowl **不负责**下载、解压或替换升级包。它的唯一职责是：在新版本文件落地后、主程序启动时，监控目标进程是否在启动阶段崩溃。如果发现崩溃 → 生成诊断信息 → 可选地自动回滚。它应该放在升级流程的**最后一步**——在文件替换完成后才启动守护。

业务使用场景：

- 桌面应用升级后启动健康检查与自动回滚保护
- 通用进程启动崩溃监控与诊断信息采集
- CI/CD 冒烟测试失败后的自动诊断

1.2 环境与依赖

项目	说明
版本	10.5.0-beta.7
目标框架	netstandard2.0 （兼容 .NET Framework 4.6.1+ / .NET Core 2.0+ / .NET 5+）
依赖包	System.Collections.Immutable , System.Text.Json
内置工具	Windows: procdump.exe / procdump64.exe / procdump64a.exe ; Linux: procdump deb/rpm 包 + install.sh ; macOS: /usr/bin/lldb
兼容性	Windows （完整支持） / Linux （deb/rpm 发行版） / macOS （基础支持，受 SIP 和调试权限限制）

2. 组件功能列表

功能名称	功能描述	类型	是否必填	备注限制
升级模式监控	监控新版本启动崩溃，自动恢复备份、标记失败版本	基础	推荐	WorkModel = "Upgrade"
独立监控模式	仅捕获崩溃、生成报告，不自动恢复	基础	可选	WorkModel = "Normal"
Full Dump	完整内存快照，信息最完整	基础	可选	DumpType.Full ， 文件最大
Mini Dump	小型内存快照，生成更快	基础	可选	DumpType.Minor ， 生产环境推荐
Heap Dump	带堆信息的小型 Dump	基础	可选	DumpType.Heap ， 介于 Minor 和 Full 之间
崩溃报告 JSON	自动生成结构化崩溃报告文件	基础	自动	输出到 FailDirectory
系统诊断导出	Windows 下导出驱动/系统信息/事件日志	拓展	自动	Windows only
自动备份恢复	崩溃后将备份目录覆盖回安装目录	基础	可选	AutoRestore = true
失败版本标记	写入升级失败版本，Core 后续跳过	基础	自动	升级模式下生效
崩溃回调通知	检测到崩溃后触发业务回调	拓展	可选	OnCrash 回调函数
日志追踪	GeneralTracer 运行时诊断日志	拓展	可选	默认开启，可关闭

3. API 配置说明

3.1 配置字段（属性 Props）

BowlContext:

字段名	数据类型	默认值	是否必填	枚举/取值范围	说明
ProcessNameOrId	string	—	是	进程名或 PID	要监控的目标进程名称或进程 ID
DumpFileName	string	—	是	有效文件名	Dump 输出文件名，推荐 "{version}_fail.dmp"
FailFileName	string	—	是	有效文件名	崩溃报告 JSON 文件名，推荐 "{version}_fail.json"
TargetPath	string	—	是	有效目录路径	应用安装根目录，恢复备份时覆盖复制到这里
FailDirectory	string	—	是	有效目录路径	故障文件输出目录，推荐 "{TargetPath}/fail/{version}"
BackupDirectory	string	—	推荐	有效目录路径	升级前备份目录，AutoRestore 打开时必须存在
WorkModel	string	"Upgrade" (Normalize() 后)	可选	"Upgrade" / "Normal"	工作模式：升级回滚 / 独立监控
ExtendedField	string	null	可选	—	扩展字段，通常存储版本号，升级模式下写入 UpgradeFail
TimeoutMs	int	30000 (Normalize() 后)	可选	正整数（毫秒）	监控子进程超时时间，按应用启动耗时调整
DumpType	DumpType	DumpType.Full (Normalize() 后)	可选	Full(0) , Mini(1) , Heap(2)	Dump 捕获类型
AutoRestore	bool	false	可选	true / false	是否自动恢复备份，升级模式需显式设为 true
OnCrash	Func?	null	可选	—	崩溃事件回调，仅在检测到 Dump 后触发

DumpType 枚举:

枚举值	数值	Windows ProcDump 参数	特点
Full	0	-ma	完整内存快照，信息最完整，文件最大
Mini	1	-mm	小型快照，生成快、文件小，适合生产默认采集
Heap	2	-mh	带堆信息小型快照，介于 Mini 和 Full 之间

3.2 实例方法

Bowl:

方法名	入参明细	使用场景	注意事项
<code>LaunchAsync(BowlContext , CancellationToken)</code>	<code>context</code> — 执行上下文（建议先调用 <code>Normalize()</code> ）； <code>ct</code> — 取消令牌	启动崩溃监控守护流程	三阶段：准备监控 → 运行监控 → 检测到 Dump 则进入故障处理管线

BowlContext:

方法名	入参明细	使用场景	注意事项
<code>Normalize()</code>	无	应用默认值（ <code>WorkModel</code> → "Upgrade"， <code>TimeoutMs</code> → 30000， <code>DumpType</code> → Full）	返回新实例，不修改原实例

3.3 回调事件

事件名称	回调参数	触发时机	使用说明
<code>OnCrash</code>	<code>CrashInfo</code> — <code>DumpFilePath</code> ， <code>CrashReportPath</code> ， <code>Version</code> ， <code>ExitCode</code> ； <code>CancellationToken</code>	检测到 Dump 文件后触发	适合上传诊断包、通知用户"新版本已回退"、记录业务审计。回调异常被 Bowl 记录到追踪日志，不会阻止 <code>LaunchAsync</code> 返回

GeneralTracer 日志控制:

方法	说明
<code>GeneralTracer.SetTracingEnabled(false)</code>	关闭 Bowl 日志输出
<code>GeneralTracer.SetTracingEnabled(true)</code>	重新开启日志输出
<code>GeneralTracer.IsTracingEnabled()</code>	查询当前日志开关状态
<code>GeneralTracer.Dispose()</code>	释放文件监听器并清空 Trace listeners

4. 扩展示例（高阶用法）

4.1 组件可扩展能力总览

Bowl 的主要扩展点是 `BowlContext.OnCrash` 回调。内部策略接口（`IBowlStrategy`、`ICrashReporter`、`ISystemInfoProvider`）为 internal，如需新增平台支持或自定义报告逻辑，可在 GeneralUpdate 仓库贡献代码。

扩展点	类型	说明
<code>OnCrash</code> 回调	Func?	<code>BowlContext</code> 中配置，崩溃时触发
<code>GeneralTracer</code> 日志	静态类	可通过 <code>SetTracingEnabled</code> 开关日志

4.2 分场景示例

场景 1：升级模式监控 + 崩溃告警上传

【场景说明】桌面应用升级到新版本后，Bowl 监控启动崩溃；发生崩溃时自动回退，并上传诊断包到内部日志平台。

【示例代码】

```

using GeneralUpdate.Bowl;

var version = "2.0.0";
var installPath = AppDomain.CurrentDomain.BaseDirectory;

var context = new BowlContext
{
    ProcessNameOrId = "MyApp.exe",
    DumpFileName = $"{version}_fail.dmp",
    FailFileName = $"{version}_fail.json",
    TargetPath = installPath,
    FailDirectory = Path.Combine(installPath, "fail", version),
    BackupDirectory = Path.Combine(installPath, version),
    WorkModel = "Upgrade",
    ExtendedField = version,
    TimeoutMs = 60_000,    // 应用启动较慢, 给 60 秒
    DumpType = DumpType.Mini,
    AutoRestore = true,
    OnCrash = async (info, ct) =>
    {
        // 打包诊断文件
        var zipPath = Path.Combine(
            Path.GetDirectoryName(info.DumpFilePath)!,
            $"crash_{info.Version}_{DateTimeOffset.Now:yyyyMMddHHmmss}.zip");

        System.IO.Compression.ZipFile.CreateFromDirectory(
            Path.GetDirectoryName(info.DumpFilePath)!, zipPath);

        // 上传到日志平台
        using var client = new HttpClient();
        var content = new MultipartFormDataContent();
        content.Add(new StreamContent(File.OpenRead(zipPath)), "file", Path.GetFileName(zipPath));
        content.Add(new StringContent(info.Version), "version");
        content.Add(new StringContent(info.ExitCode.ToString()), "exitCode");

        await client.PostAsync("https://logs.example.com/api/crash", content, ct);

        // 通知用户
        Console.WriteLine($"Version {info.Version} crashed (exit code {info.ExitCode}).");
        Console.WriteLine($"Diagnostics uploaded. Previous version restored.");
    }
};

BowlResult result = await new Bowl().LaunchAsync(context);

```

```

if (result.DumpCaptured && result.Restored)
    Console.WriteLine("Crash detected and backup restored.");
else if (!result.DumpCaptured)
    Console.WriteLine("Process started successfully.");

```

【效果&注意事项】

- `TimeoutMs` 需要大于应用正常启动时间
- `DumpType.Mini` 生成更快，适合生产环境；疑难问题再切换到 `Full`
- 回调异常不会阻断恢复流程

场景 2：独立监控模式（非升级场景）

【场景说明】对 Worker 进程做通用启动崩溃监控，只采集诊断信息，不自动回滚。

【示例代码】

```

var context = new BowlContext
{
    ProcessNameOrId = "MyWorker.exe",
    DumpFileName = "startup_fail.dmp",
    FailFileName = "startup_fail.json",
    TargetPath = AppDomain.CurrentDomain.BaseDirectory,
    FailDirectory = Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "fail", "startup"),
    BackupDirectory = Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "backup"),
    WorkModel = "Normal",
    TimeoutMs = 15_000,
    DumpType = DumpType.Mini,
    AutoRestore = false,
    OnCrash = (info, ct) =>
    {
        Console.WriteLine($"Worker crashed: {info.DumpFilePath}");
        return Task.CompletedTask;
    }
};

BowlResult result = await new Bowl().LaunchAsync(context);

if (result.DumpCaptured)
{
    Console.WriteLine($"Dump captured at: {result.DumpFilePath}");
    Console.WriteLine($"Report at: {result.CrashReportPath}");
}

```

【效果&注意事项】

- `WorkModel = "Normal"` 不会恢复备份也不会标记 `UpgradeFail`

- 适合通用进程监控、CI 测试守护等非升级场景
-

5. 常规使用示例

5.1 快速入门示例（最简 demo）

```
using GeneralUpdate.Bowl;

var context = new BowlContext
{
    ProcessNameOrId = "MyApp.exe",
    DumpFileName = "fail.dmp",
    FailFileName = "fail.json",
    TargetPath = AppDomain.CurrentDomain.BaseDirectory,
    FailDirectory = Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "fail"),
    BackupDirectory = Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "backup"),
}.Normalize(); // 应用默认值

BowlResult result = await new Bowl().LaunchAsync(context);
Console.WriteLine($"Success: {result.Success}, Dump captured: {result.DumpCaptured}");
```

5.2 基础参数组合示例

```
var version = "2.0.0";
var installPath = @"C:\Program Files\MyApp";

var context = new BowlContext
{
    ProcessNameOrId = "MyApp.exe",
    DumpFileName = $"{version}_fail.dmp",
    FailFileName = $"{version}_fail.json",
    TargetPath = installPath,
    FailDirectory = Path.Combine(installPath, "fail", version),
    BackupDirectory = Path.Combine(installPath, version),
    WorkModel = "Upgrade",
    ExtendedField = version,
    TimeoutMs = 30_000,
    DumpType = DumpType.Full,
    AutoRestore = true,
    OnCrash = (info, ct) =>
    {
        Console.WriteLine($"Crash: {info.DumpFilePath}");
        return Task.CompletedTask;
    }
};

BowlResult result = await new Bowl().LaunchAsync(context);

if (result.DumpCaptured && result.Restored)
    Console.WriteLine("The upgraded version crashed and the backup was restored.");
```

5.3 真实业务落地示例

完整升级后 Bowl 守护流程，包含升级程序侧集成：

```

using GeneralUpdate.Bowl;

// --- 升级程序 (Update.exe) 侧 ---
// 1. 主程序已停止，升级程序完成文件替换
// 2. 备份旧版本到 BackupDirectory
// 3. 启动 Bowl 监控新版本

var version = "2.0.0";
var installPath = @"C:\Program Files\MyApp";

var bowlContext = new BowlContext
{
    ProcessNameOrId = "MyApp.exe",
    DumpFileName = $"{version}_fail.dmp",
    FailFileName = $"{version}_fail.json",
    TargetPath = installPath,
    FailDirectory = Path.Combine(installPath, "fail", version),
    BackupDirectory = Path.Combine(installPath, "backups", version),
    WorkModel = "Upgrade",
    ExtendedField = version,
    TimeoutMs = 45_000, // 应用冷启动约 30s, 留 15s buffer
    DumpType = DumpType.Mini,
    AutoRestore = true,
    OnCrash = async (info, ct) =>
    {
        try
        {
            // 上传诊断信息
            using var client = new HttpClient();
            var crashData = new
            {
                version = info.Version,
                exitCode = info.ExitCode,
                dumpPath = info.DumpFilePath,
                reportPath = info.CrashReportPath,
                timestamp = DateTimeOffset.UtcNow
            };
            await client.PostAsJsonAsync(
                "https://monitor.mycompany.com/api/crash-report",
                crashData, ct);
        }
        catch (Exception ex)
        {
            // 上报失败不影响恢复流程

```

```

        Console.WriteLine($"Failed to upload crash report: {ex.Message}");
    }
}
};

var result = await new Bowl().LaunchAsync(bowlContext);

if (result.Success)
{
    Console.WriteLine("New version started successfully.");
}
else if (result.DumpCaptured)
{
    Console.WriteLine($"New version crashed (exit code: {result.ExitCode}).");
    Console.WriteLine($"Backup restored: {result.Restored}");
    Console.WriteLine($"Dump: {result.DumpFilePath}");
    Console.WriteLine($"Report: {result.CrashReportPath}");
}
else
{
    Console.WriteLine($"Process exited abnormally (exit code: {result.ExitCode}), but no dump was captured.");
}
}

```

6. 全局配置

Bowl 不依赖全局配置文件。所有配置通过 `BowlContext` 传入。日志行为通过静态类 `GeneralTracer` 控制。

日志开关

```

// 性能敏感场景关闭日志
GeneralTracer.SetTracingEnabled(false);

var result = await new Bowl().LaunchAsync(context);

// 排查问题时重新开启
GeneralTracer.SetTracingEnabled(true);

```

输出文件结构

一次升级失败后，推荐按版本存放所有故障文件：

```
MyApp/
  fail/
    2.0.0/
      2.0.0_fail.dmp      # Dump 内存快照
      2.0.0_fail.json    # 崩溃报告 JSON
      driverInfo.txt      # Windows 驱动列表
      systeminfo.txt     # OS/硬件/内存信息
      systemlog.evtx     # Windows 系统事件日志
  Logs/
    generalupdate-trace 2026-01-01.log # Bowl 自身追踪日志
```

平台差异

平台	监控工具	诊断导出	注意事项
Windows	内置 ProcDump (procdump.exe / procdump64.exe / procdump64a.exe)	支持 driverInfo.txt 、 systeminfo.txt 、 systemlog.evtx	需要足够权限生成 Dump
Linux	内置 deb/rpm 包 + install.sh 安装 ProcDump	当前为 no-op	支持 Ubuntu/Debian/RHEL/CentOS /Fedora/ClearOS
macOS	/usr/bin/lldb	当前为 no-op	受 SIP、调试权限、签名策略影响，基础实现

相关资源

- [Bowl 示例代码](#)
- [GeneralUpdate 仓库](#)

GeneralUpdate.Differential

命名空间: GeneralUpdate.Differential | 主要入口: IBinaryDiffer、BsdiffDiffer、StreamingHdiffDiffer |
NuGet 包: GeneralUpdate.Differential

1. 组件简介

1.1 组件概述

GeneralUpdate.Differential 是 GeneralUpdate 的二进制差分组件, 专注解决"一个旧文件 + 一个补丁文件 = 一个新文件"的问题。它提供可替换的文件级差分算法 (BSDIFF 4.0 / Streaming HDiff)、补丁压缩抽象 (BZip2 / Deflate, 源码中预留 .NET 6+ Brotli) 和 BSDIFF 兼容补丁读写能力。

目录级对比、批量补丁生成、并行调度、删除文件处理和更新流程编排由 GeneralUpdate.Core 的 DiffPipeline 或 GeneralUpdate.Tools 承担。

核心能力:

能力	说明
文件级差分生成	CleanAsync(oldFile, newFile, patchFile) — 对比新旧文件生成 .patch 补丁
文件级差分应用	DirtyAsync(oldFile, newFile, patchFile) — 旧文件 + 补丁 → 新文件
可替换差分算法	BsdiffDiffer (BSDIFF 4.0, 后缀排序) 和 StreamingHdiffDiffer (块哈希索引)
可替换压缩格式	BZip2 (0x00)、Deflate (0x01), 源码中通过 #if NET6_0_OR_GREATER 条件编译预留 Brotli (0x02)
BSDIFF 兼容格式	写入 33 字节扩展头 (32 字节 BSDIFF40 + 1 字节压缩格式), 兼容 32 字节旧头
线程安全	内置 differ 和压缩提供器均支持并发调用

解决的业务痛点:

- 全量更新带宽成本高, 差分更新可将更新包从 GB 级降低到 MB 甚至 KB 级
- 不同文件类型和变化模式需要不同的差分策略 (细粒度匹配 vs 快速块匹配)
- 压缩算法的选择影响客户端解压速度和补丁体积的平衡

Differential 是底层库, 通常不直接使用

大多数场景下你不需要直接调用 IBinaryDiffer。目录级差分、批量补丁生成、并行调度和更新流程编排由 GeneralUpdate.Core 的 DiffPipeline 或 GeneralUpdate.Tools 承担。Differential 只解决一个原子问题: 一个旧文件 + 一个补丁文件 = 一个新文件。

业务使用场景:

- 大型桌面应用 (多 DLL、资源文件) 的增量更新
- 固件/驱动包的二进制差分分发
- 游戏资源热更新
- CI/CD 发布流水线中自动生成增量补丁包

1.2 环境与依赖

项目	说明
版本	10.5.0-beta.7
目标框架	netstandard2.0 （兼容 .NET Framework 4.6.1+ / .NET Core 2.0+ / .NET 5+）
依赖包	无外部依赖（纯 .NET BCL）
兼容性	所有支持 .NET Standard 2.0 的平台

2. 组件功能列表

功能名称	功能描述	类型	是否必填	备注限制
BSDIFF 4.0 差分生成	基于后缀排序的经典差分算法，补丁体积稳定	基础	可选	BsdiffDiffer，默认 BZip2 压缩
BSDIFF 4.0 补丁应用	将 BSDIFF 格式补丁应用到旧文件	基础	可选	支持 32/33 字节两种头部格式
Streaming HDiff 差分生成	基于 FNV-1a 块哈希索引的快速差分	基础	可选	StreamingHdiffDiffer，默认 Deflate 压缩
BZip2 压缩	补丁控制段/差异段/额外段的 BZip2 压缩	基础	可选	格式字节 0x00，BsdiffDiffer 默认
Deflate 压缩	补丁段的 Deflate 压缩，解压更快	基础	可选	格式字节 0x01，StreamingHdiffDiffer 默认
自定义差分算法	实现 IBinaryDiffer 接入自研算法	拓展	可选	需保证 Clean/Dirty 一致性
自定义压缩提供器	实现 ICompressionProvider 替换压缩方式	拓展	可选	新格式字节需配合扩展补丁读取逻辑

3. API 配置说明

3.1 配置字段（属性 Props）

Differential 本身是底层库，不提供配置类。所有参数通过构造函数传入。

BsdiffDiffer 构造参数：

字段名	数据类型	默认值	是否必填	枚举/取值范围	说明
compressionProvider	ICompressionProvider	BZip2CompressionProvider	可选	BZip2CompressionProvider / DeflateCompressionProvider	补丁压缩提供器

StreamingHdiffDiffer 构造参数：

字段名	数据类型	默认值	是否必填	枚举/取值范围	说明
compression Provider	ICompression Provider	DeflateCompress ionProvider	可选	BZip2CompressionProvider / DeflateCompressionProvider	补丁压缩提供器
blockSize	int	65536 (64 KB)	可选	正整数字节数	块大小，用于旧文件哈希索引
maxWindowSi ze	int	134217728 (128 MB)	可选	正整数字节数	参与计算的最大内存窗口

DeflateCompressionProvider 构造参数：

字段名	数据类型	默认值	是否必填	枚举/取值范围	说明
optimalLevel	bool	true	可选	true / false	true = CompressionLevel.Optimal , false = CompressionLevel.Fastest

ICompressionProvider 格式标识：

Provider	格式字节	可用性	说明
BZip2CompressionP rovider	0x00	完全可用	BSDIFF 旧补丁兼容，解压成本较高
DeflateCompressio nProvider	0x01	完全可用	解压速度更友好，适合客户端批量应用
BrotliCompression Provider	0x02	仅 .NET 6+ 编译（源码中为完整实现，通过 #if NET6_0_OR_GREATER 条件编译）	当前 netstandard2.0 包中不包含，生产不建议使用

3.2 实例方法

IBinaryDiffer：

方法名	入参明细	使用场景	注意事项
CleanAsync(string, string, string, CancellationToken)	oldFilePath — 旧文件路径； newFilePath — 新文件路径； patchFilePath — 补丁输出路径； cancellationToken	发布/构建阶段生成补丁	大文件取消不会立即响应，需等待当前文件处理完成
DirtyAsync(string, string, string, CancellationToken)	oldFilePath — 旧文件路径； newFilePath — 补丁还原后文件输出路径； patchFilePath — 补丁文件路径； cancellationToken	客户端升级阶段应用补丁	不会直接覆盖旧文件，结果写入 newFilePath

3.3 回调事件

Differential 不发布事件。进度报告和事件通知由 Core 的 DiffPipeline 通过 DiffProgress 和 EventManager 实现。

4. 扩展示例（高阶用法）

4.1 组件可扩展能力总览

扩展接口	说明
IBinaryDiffer	自定义文件级差分算法，可接入原生库或自研算法
ICompressionProvider	自定义补丁段压缩方式

4.2 分场景示例

场景 1：自定义差分算法

【场景说明】接入企业内部自研的高压缩率差分算法。

【示例代码】

```

using GeneralUpdate.Differential.Abstractions;

public sealed class HighRatioDiffer : IBinaryDiffer
{
    public Task CleanAsync(
        string oldFilePath,
        string newFilePath,
        string patchFilePath,
        CancellationToken cancellationToken = default)
    {
        // 调用自研算法生成补丁
        // NativeMethods.GeneratePatch(oldFilePath, newFilePath, patchFilePath);
        return Task.CompletedTask;
    }

    public Task DirtyAsync(
        string oldFilePath,
        string newFilePath,
        string patchFilePath,
        CancellationToken cancellationToken = default)
    {
        // 调用自研算法应用补丁
        // NativeMethods.ApplyPatch(oldFilePath, patchFilePath, newFilePath);
        return Task.CompletedTask;
    }
}

// 在 Core DiffPipeline 中使用
using GeneralUpdate.Core.Pipeline;

var pipeline = new DiffPipelineBuilder()
    .UseDiffer(new HighRatioDiffer())
    .WithParallelism(4)
    .Build();

await pipeline.CleanAsync(oldDir, newDir, patchDir);

```

【效果&注意事项】

- 必须保证 `CleanAsync` 产出的补丁能被同一算法的 `DirtyAsync` 正确应用
- 发布侧和客户端必须使用同一套 differ 实现

场景 2：自定义压缩提供器 + `BsdiffDiffer`

【场景说明】使用 `BsdiffDiffer` 算法 + Deflate 压缩，获得更快的客户端补丁应用速度。

【示例代码】

```
using GeneralUpdate.Differential.Abstractions;
using GeneralUpdate.Differential.Differ;

// BsdiffDiffer 的精确匹配 + Deflate 的快速解压
var differ = new BsdiffDiffer(
    new DeflateCompressionProvider(optimalLevel: false));

await differ.CleanAsync(oldFile, newFile, patchFile);
await differ.DirtyAsync(oldFile, outputFile, patchFile);
```

【效果&注意事项】

- `optimalLevel: false` 生成更快，适合开发/CI 环境
- `optimalLevel: true` 补丁体积更小，适合生产环境
- 生成和消费两侧都需要能识别 Deflate 格式（`0x01`）

场景 3：StreamingHdiffDiffer 参数调优

【场景说明】大型单文件（200MB+）的差分，调整窗口预算避免内存溢出。

【示例代码】

```
using GeneralUpdate.Differential.Abstractions;
using GeneralUpdate.Differential.Differ;

// 大文件场景：增大窗口，减小块大小以获得更精细匹配
var differ = new StreamingHdiffDiffer(
    compressionProvider: new DeflateCompressionProvider(optimalLevel: true),
    blockSize: 32 * 1024,           // 32 KB 块，更密集的哈希索引
    maxWindowSize: 256 * 1024 * 1024); // 256 MB，允许读入更大文件

await differ.CleanAsync(oldLargeFile, newLargeFile, patchFile);
```

【效果&注意事项】

- `blockSize` 越小，哈希索引越密集，匹配更精确但内存消耗更大
 - `maxWindowSize` 决定能参与计算的最大数据量，超出部分不参与匹配
 - 超大文件建议先在业务侧压测补丁体积和应用还原结果
-

5. 常规使用示例

5.1 快速入门示例（最简 demo）

```
using GeneralUpdate.Differential.Abstractions;
using GeneralUpdate.Differential.Differ;

IBinaryDiffer differ = new BsdiffDiffer();

var oldFile = @"D:\releases\1.0.0\app.dll";
var newFile = @"D:\releases\1.0.1\app.dll";
var patchFile = @"D:\patches\app.dll.patch";
var outputFile = @"D:\restore\app.dll";

// 生成补丁: oldFile + newFile → patchFile
await differ.CleanAsync(oldFile, newFile, patchFile);

// 应用补丁: oldFile + patchFile → outputFile
await differ.DirtyAsync(oldFile, outputFile, patchFile);

// 验证还原结果
var newHash = ComputeSha256(newFile);
var outputHash = ComputeSha256(outputFile);
Console.WriteLine(newHash == outputHash ? "Patch verified." : "MISMATCH!");
```

5.2 基础参数组合示例

```
using GeneralUpdate.Differential.Abstractions;
using GeneralUpdate.Differential.Differ;

// 方案 A: 经典 BSDIFF + BZip2 → 补丁体积最小
var differA = new BsdiffDiffer();

// 方案 B: 经典 BSDIFF + Deflate → 补丁体积小 + 应用更快
var differB = new BsdiffDiffer(new DeflateCompressionProvider(optimalLevel: true));

// 方案 C: Streaming HDiff + Deflate → 生成快 + 应用最快
var differC = new StreamingHdiffDiffer(
    new DeflateCompressionProvider(optimalLevel: true),
    blockSize: 64 * 1024,
    maxWindowSize: 128 * 1024 * 1024);

// 对同一组文件测试三种方案，选择最优
foreach (var differ in new IBinaryDiffer[] { differA, differB, differC })
{
    var sw = Stopwatch.StartNew();
    await differ.CleanAsync(oldFile, newFile, patchFile);
    sw.Stop();

    var patchSize = new FileInfo(patchFile).Length;
    Console.WriteLine($"{differ.GetType().Name}: {sw.ElapsedMilliseconds}ms, {patchSize} bytes");
}
```

5.3 真实业务落地示例（通过 Core DiffPipeline 使用）

大多数情况下不直接使用 Differential，而是通过 Core 的 `DiffPipeline` 做目录级差分：

```
using GeneralUpdate.Core;
using GeneralUpdate.Core.Pipeline;
using GeneralUpdate.Core.Models;
using GeneralUpdate.Differential.Differ;

// 构建端：对比新旧版本目录，生成补丁
var pipeline = new DiffPipelineBuilder()
    .UseDiffer(new StreamingHdiffDiffer())
    .WithParallelism(8)           // CI 构建机，高并行度
    .WithStopOnFirstError(true)
    .WithProgress(new Progress(p =>
    {
        Console.WriteLine($"[Build] {p.Completed}/{p.Total} {p.CurrentFile}");
    })))
    .Build();

await pipeline.CleanAsync(
    @"D:\builds\v1.0.0",
    @"D:\builds\v1.0.1",
    @"D:\patches\v1.0.0-to-v1.0.1");

// 客户端：通过 GeneralUpdateBootstrap 使用
await new GeneralUpdateBootstrap()
    .SetSource(
        updateUrl: "https://update.mycompany.com/api/upgrade/verification",
        appSecretKey: "prod-key")
    .SetOption(Option.AppType, AppType.Client)
    .SetOption(Option.PatchEnabled, true)
    .SetOption(Option.DiffMode, DiffMode.Parallel)
    .SetOption(Option.MaxConcurrency, 4)
    .UseDiffPipeline(builder => builder
        .UseDiffer(new StreamingHdiffDiffer())
        .WithParallelism(4))
    .LaunchAsync();
```

6. 算法选择指南

Clean 与 Dirty 语义

术语	方法	输入	输出	常用位置
Clean	CleanAsync	旧文件、新文件、补丁输出路径	.patch 补丁文件	构建/发布阶段
Dirty	DirtyAsync	旧文件、输出新文件路径、补丁路径	还原后的新文件	客户端升级阶段

算法对比

对比维度	BsdiffDiffer	StreamingHdiffDiffer
核心思路	经典 BSDIFF 4.0, 后缀排序 + 最长匹配	块级 FNV-1a 哈希索引 + 字节级扩展匹配
默认压缩	BZip2 (0x00)	Deflate (0x01)
补丁应用	自实现 BSDIFF Dirty	委托给 BsdiffDiffer (BSDIF 兼容)
补丁体积	更稳定, 通常更小	受文件变化分布影响大, 块命中差时可能接近原文件大小
客户端应用速度	BZip2 解压较慢	Deflate 解压更快 (约 1.5-5x)
生成内存	全量读入新旧文件	按 maxWindowSize 预算读入
兼容性	兼容旧 BSDIFF/BZip2 补丁	适合新项目

场景推荐

场景	推荐
补丁体积优先	BsdiffDiffer + BZip2
客户端应用速度优先	StreamingHdiffDiffer (默认 Deflate)
兼容旧补丁格式	BsdiffDiffer (32 字节旧头自动按 BZip2 处理)
大文件 (>500MB)	先压测, 可能需要 StreamingHdiffDiffer + 调大 maxWindowSize
目录级批量差分	通过 Core DiffPipeline, 配合 WithParallelism 提升吞吐
新项目	先用默认配置跑基准, 再根据体积和速度需求调整

并发模型

- 单个 IBinaryDiffer 实例线程安全 (当 ICompressionProvider 线程安全时)
- 真正的多线程差分在 Core DiffPipeline 层, 通过 WithParallelism(n) 控制
- BZip2 / Deflate 内置 provider 均为每次调用创建新流, 支持并发

相关资源

- [GeneralUpdate 仓库](#)
- [Samples 差分示例](#)
- [Core DiffPipeline 文档](#)

GeneralUpdate.DriveIution

命名空间: GeneralUpdate.DriveIution | 主要入口: GeneralDriveIution (静态类) | NuGet 包: GeneralUpdate.DriveIution

1. 组件简介

1.1 组件概述

GeneralUpdate.DriveIution 是面向驱动更新场景的跨平台组件。它把驱动更新中容易出错的步骤拆成统一流水线: 平台识别 → 权限检查 → 文件验证 (哈希/签名/兼容性) → 备份 → 安装 → 结果验证 → 失败后回滚, 并在 Windows、Linux、macOS 上分别调用系统原生工具完成驱动安装。

驱动更新不是普通应用文件替换。应用文件通常只需要下载、解压、覆盖并重启进程; 驱动更新会影响内核、设备节点、系统扩展或驱动仓库, 因此必须额外关注管理员权限、签名可信度、目标系统和 CPU 架构、安装命令返回值、是否需要系统重启以及失败后的恢复路径。

核心能力:

能力	说明
跨平台抽象	<code>GeneralDriveIution.Create()</code> 自动检测平台 (Windows/Linux/macOS) 创建对应实现
标准更新流水线	权限检查 → 文件验证 (存在/哈希/签名/兼容性) → 备份 → 安装 → 验证 → 失败回滚
文件验证	文件存在检查、SHA256/MD5 哈希校验、Authenticode/GPG/codesign 签名校验、OS/架构兼容性检查
备份与回滚	<code>RequireBackup</code> 默认为 <code>true</code> , 失败后自动回滚并保留备份路径供业务显式调用 <code>RollbackAsync</code>
Windows 安装	通过 <code>pnputil.exe /add-driver /install</code> 安装 INF 驱动包
Linux 安装	支持 <code>.ko</code> (<code>insmod / modprobe</code>)、 <code>.deb</code> (<code>dpkg -i</code>)、 <code>.rpm</code> (<code>rpm -ivh / dnf install</code>)
macOS 安装	支持 <code>.kext</code> (<code>kextload</code>)、 <code>.dext</code> (SystemExtensions)、 <code>.pkg</code> (<code>installer</code>)
批量更新	<code>BatchUpdateAsync</code> 支持 <code>BatchMode.Sequential</code> 和 <code>BatchMode.Parallel</code>
进度报告	通过 <code>IProgress</code> 上报各步骤的进度、状态和消息
日志追踪	<code>GeneralTracer</code> 默认控制台 + 按日期轮转文件输出

解决的业务痛点:

- 硬件厂商客户端需要随应用交付驱动更新, 但不同 OS 驱动安装方式差异大
- 驱动安装需要管理员权限、签名校验、架构兼容性检查等多重保障
- 驱动安装失败后需要可靠的回滚机制, 避免设备不可用

驱动更新 ≠ 普通文件替换

驱动更新会影响内核、设备节点、系统扩展或驱动仓库。**必须关注:** 管理员权限、签名可信度、目标系统和 CPU 架构、安装命令返回值、是否需要系统重启以及失败后的恢复路径。生产环境中**永远不要**跳过哈希和签名校验。

业务使用场景:

- 硬件厂商客户端: 网卡、采集卡、USB 设备、虚拟设备驱动随应用一起交付

- 企业/工业现场：批量扫描驱动包并按清单更新
- 安装器/维护工具：统一处理 Windows/Linux/macOS 驱动差异

1.2 环境与依赖

项目	说明
版本	10.5.0-beta.7
目标框架	net8.0 / net10.0 （多目标）
依赖包	Microsoft.Extensions.DependencyInjection、Microsoft.Extensions.Logging.Abstractions、Microsoft.Extensions.Options
兼容性	Windows（完整支持，需管理员权限）/ Linux（需 root/sudo）/ macOS（受 SIP 和系统扩展策略影响）

2. 组件功能列表

功能名称	功能描述	类型	是否必填	备注限制
单驱动快速更新	QuickUpdateAsync 使用默认策略快速更新单个驱动	基础	可选	推荐生产环境使用自定义策略
自定义策略更新	UpdateAsync 配合 UpdateStrategy 和 DrivelutionOptions	基础	推荐	可控制备份、重试、超时、重启策略
驱动验证	文件存在、SHA256 哈希、签名、OS/架构兼容性检查	基础	可选	ValidateAsync，可通过策略跳过部分校验
驱动备份	更新前备份驱动文件到指定路径	基础	自动	RequireBackup 默认为 true
驱动回滚	从备份路径恢复驱动	基础	可选	RollbackAsync(backupPath)
目录扫描	从目录扫描并解析驱动信息（.inf / .ko / .kext 等）	基础	可选	GetDriversFromDirectoryAsync
批量更新	按清单批量更新，支持顺序/并行模式	拓展	可选	BatchUpdateAsync
平台信息查询	获取当前 OS/架构/版本/是否支持	基础	可选	GetPlatformInfo()
重启行为控制	UpdateStrategy.RestartMode 设定重启意图	拓展	可选	更新流水线不会自动重启，需业务层调用 RestartHelper
DI 注册	AddDrivelution 扩展方法注册平台服务	拓展	可选	支持 Generic Host / ASP.NET Core
日志追踪	GeneralTracer 运行时诊断日志	拓展	可选	默认开启，可关闭

3. API 配置说明

3.1 配置字段（属性 Props）

DriverInfo:

字段名	数据类型	默认值	是否必填	枚举/取值范围	说明
Name	string	""	是	—	驱动名称
Version	string	""	推荐	SemVer 格式	驱动版本，扫描目录时会尽量从元数据读取
FilePath	string	""	是	有效文件路径	Windows: .inf ; Linux: .ko / .deb / .rpm ; macOS: .kext / .dext / .pkg
TargetOS	string	""	可选	"Windows", "Linux", "MacOS"	为空时不限制 OS
Architecture	string	""	可选	"x64" / "amd64" / "x86" / "arm64" / "arm"	支持常见别名归一化；为空不限制
HardwareId	string	""	可选	—	硬件 ID 或模块别名
Hash	string	""	可选	SHA256/MD5 哈希值	非空且 SkipHashValidation = false 时执行哈希校验
HashAlgorithm	string	"SHA256"	可选	"SHA256" / "MD5"	哈希算法
TrustedPublishers	List	new()	可选	—	可信发布者列表，非空且未跳过签名校验时验证签名
Description	string	""	可选	—	驱动描述
ReleaseDate	DateTime	—	可选	—	发布时间
Metadata	Dictionary	new()	可选	—	扩展元数据

UpdateStrategy:

字段名	数据类型	默认值	是否必填	枚举/取值范围	说明
RequireBackup	bool	true	可选	true / false	是否执行备份步骤
BackupPath	string	""	推荐	有效目录路径	备份根路径，流水线在该路径下生成 backup_{Name}_{yyyyMMddHHmmss}
RestartMode	RestartMode	Prompt	可选	None, Prompt, Delayed, Immediate	重启意图，流水线不会自动重启系统
SkipHashValidation	bool	false	可选	true / false	跳过哈希校验（仅调试/受控环境）

字段名	数据类型	默认值	是否必填	枚举/取值范围	说明
SkipSignatureValidation	bool	false	可选	true / false	跳过签名校验（仅调试/受控环境）
TimeoutSeconds	int	300	可选	正整数	单次更新超时，≤0 时使用 <code>DrivelutionOptions.DefaultTimeoutSeconds</code>
RetryCount	int	3	可选	正整数	重试次数（策略字段，实际重试来自 <code>DrivelutionOptions</code> ）
RetryIntervalSeconds	int	5	可选	正整数	重试间隔秒数
Mode	UpdateMode	Full	可选	Full, Incremental	更新模式
ForceUpdate	bool	false	可选	true / false	是否强制更新
Priority	int	0	可选	—	优先级

DrivelutionOptions:

字段名	数据类型	默认值	是否必填	说明
DefaultBackupPath	string	"./DriverBackups"	可选	默认备份路径
DefaultRetryCount	int	3	可选	默认重试次数
DefaultRetryIntervalSeconds	int	5	可选	默认重试间隔
DefaultTimeoutSeconds	int	300	可选	默认超时时间
DebugModeSkipSignature	bool	false	可选	调试模式跳过签名
DebugModeSkipHash	bool	false	可选	调试模式跳过哈希
ForceTerminateOnPermissionFailure	bool	true	可选	权限失败时立即终止
AutoCleanupBackups	bool	true	可选	自动清理旧备份
BackupsToKeep	int	5	可选	保留备份数量
UseExponentialBackoff	bool	false	可选	是否使用指数退避

3.2 实例方法

GeneralDrivelution（静态类）：

方法名	入参明细	使用场景	注意事项
Create(DrivelutionOptions?)	options — 全局选项	创建当前平台的驱动更新器	自动检测平台
Create(IServiceProvider)	serviceProvider — DI 容器	从 DI 容器解析	未注册时回退到自动平台创建

方法名	入参明细	使用场景	注意事项
QuickUpdateAsync(DriverInfo, UpdateStrategy?, IProgress?, CancellationToken)	driverInfo — 驱动信息; strategy — 可选策略 (null 时使用默认); progress — 进度报告; ct — 取消令牌	快速单驱动更新	使用安全默认策略
ValidateAsync(DriverInfo, CancellationToken)	driverInfo — 驱动信息; ct — 取消令牌	单独验证驱动文件	—
GetPlatformInfo()	无	查询当前平台信息	返回 OS/架构/版本/是否支持
GetDriversFromDirectoryAsync(string, string?, CancellationToken)	directoryPath — 目录路径; searchPattern — 搜索模式; ct — 取消令牌	扫描目录解析驱动信息	默认搜索模式与平台相关
BatchUpdateAsync(IEnumerable, UpdateStrategy, BatchMode, IProgress?, CancellationToken)	drivers — 驱动列表; strategy — 更新策略; mode — 顺序/并行; progress — 进度; ct — 取消令牌	批量更新多个驱动	并行模式下底层系统工具可能竞争资源

IGeneralDrivelution:

方法名	入参明细	使用场景	注意事项
UpdateAsync(...)	同 QuickUpdateAsync	执行完整更新流水线	—
ValidateAsync(...)	同 GeneralDrivelution.ValidateAsync	单独验证	—
BackupAsync(DriverInfo, string, CancellationToken)	driverInfo, backupPath, ct	单独备份驱动文件	—
RollbackAsync(string, CancellationToken)	backupPath — 备份路径; ct — 取消令牌	从备份恢复驱动	不同平台恢复逻辑不同
GetDriversFromDirectoryAsync(...)	同 GeneralDrivelution	扫描目录	—
BatchUpdateAsync(...)	同 GeneralDrivelution	批量更新	—

3.3 回调事件

Drivelution 通过 IProgress 报告进度，不提供独立的事件系统。

进度字段	类型	说明
CurrentStatus	UpdateStatus	当前状态 (Validating / BackingUp / Updating / Verifying / Succeeded / Failed / RolledBack)
StepName	string	当前步骤名称
Percentage	int	进度百分比 (0-100)
Message	string	进度消息
StepIndex	int	当前步骤索引
TotalSteps	int	总步骤数

4. 扩展示例（高阶用法）

4.1 组件可扩展能力总览

扩展接口	说明
IGeneralDrivelution	完整替换驱动更新器的所有行为
IDriverValidator	自定义文件验证逻辑
IDriverBackup	自定义备份/恢复策略
ICommandRunner	自定义系统命令执行器
INetworkDownloader	预留接口（网络下载）
BaseDriverUpdater	抽象基类，可继承创建新平台实现

4.2 分场景示例

场景 1：自定义策略 + 回滚处理

【场景说明】生产环境驱动更新，启用所有安全检查，失败时显式回滚。

【示例代码】

```

using GeneralUpdate.DriveLution;
using GeneralUpdate.DriveLution.Abstractions.Configuration;
using GeneralUpdate.DriveLution.Abstractions.Models;

var options = new DriveLutionOptions
{
    DefaultBackupPath = @"C:\DriverBackups",
    DefaultRetryCount = 3,
    DefaultRetryIntervalSeconds = 5,
    DefaultTimeoutSeconds = 600,
    UseExponentialBackoff = true,
    ForceTerminateOnPermissionFailure = true
};

var updater = GeneralDriveLution.Create(options);

var strategy = new UpdateStrategy
{
    RequireBackup = true,
    BackupPath = @"C:\DriverBackups\graphics",
    TimeoutSeconds = 600,
    RestartMode = RestartMode.Prompt,
    SkipHashValidation = false,
    SkipSignatureValidation = false
};

var driver = new DriverInfo
{
    Name = "Graphics Driver",
    Version = "2.1.0",
    FilePath = @"C:\Drivers\graphics.inf",
    TargetOS = "Windows",
    Architecture = "x64",
    Hash = "expected-sha256...",
    HashAlgorithm = "SHA256",
    TrustedPublishers = { "Contoso Hardware Inc." }
};

var progress = new Progress(p =>
{
    Console.WriteLine($"[{p.StepName}] {p.Percentage}%: {p.Message}");
});

var result = await updater.UpdateAsync(driver, strategy, progress);

```

```
if (!result.Success)
{
    Console.WriteLine($"Update failed: {result.Error?.Message}");

    if (result.BackupPath is not null)
    {
        Console.WriteLine("Rolling back...");
        await updater.RollbackAsync(result.BackupPath);
    }
}
else if (RestartHelper.IsRestartRequired(strategy.RestartMode))
{
    await RestartHelper.HandleRestartAsync(
        strategy.RestartMode,
        delaySeconds: 60,
        message: "Driver updated. Restart now?");
}
```

【效果&注意事项】

- 生产环境不要跳过哈希和签名校验
- 回滚后驱动恢复到安装前状态，但仍建议提示用户可能需要重启

场景 2：DI 容器集成

【场景说明】在 ASP.NET Core / Generic Host 应用中通过 DI 注册驱动更新服务。

【示例代码】

```

using GeneralUpdate.DriveLution.Core;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddDriveLution(options =>
{
    options.DefaultBackupPath = "./DriverBackups";
    options.DefaultTimeoutSeconds = 600;
    options.DefaultRetryCount = 3;
    options.AutoCleanupBackups = true;
    options.BackupsToKeep = 5;
});

var app = builder.Build();

// 在 Controller 或 Service 中使用
app.MapPost("/drivers/update", async (DriverInfo driver, IGeneralDriveLution updater) =>
{
    var result = await updater.UpdateAsync(driver, new UpdateStrategy());
    return result.Success ? Results.Ok(result) : Results.BadRequest(result.Error);
});

```

【效果&注意事项】

- `AddDriveLution` 自动注册平台对应的所有服务
- 支持通过 `IServiceProvider` 创建：`GeneralDriveLution.Create(serviceProvider)`

场景 3：批量并行扫描 + 顺序安装

【场景说明】大型项目先并行扫描验证所有驱动，再按风险分组顺序安装。

【示例代码】


```
using GeneralUpdate.DriveLution;
using GeneralUpdate.DriveLution.Abstractions.Models;

// 1. 并行扫描和验证所有驱动
var drivers = await GeneralDriveLution.GetDriversFromDirectoryAsync(@"C:\DriverPackages");

var compatibleDrivers = new List();
foreach (var driver in drivers)
{
    var valid = await GeneralDriveLution.ValidateAsync(driver);
    if (valid)
    {
        Console.WriteLine($"{driver.Name} v{driver.Version}: valid");
        compatibleDrivers.Add(driver);
    }
    else
    {
        Console.WriteLine($"{driver.Name} v{driver.Version}: INVALID, skipping");
    }
}

// 2. 按风险分组
var coreDrivers = compatibleDrivers
    .Where(d => d.Metadata.ContainsKey("RiskLevel") && d.Metadata["RiskLevel"] == "Core")
    .ToList();

var optionalDrivers = compatibleDrivers
    .Except(coreDrivers)
    .ToList();

// 3. 核心驱动顺序安装
if (coreDrivers.Any())
{
    var coreResult = await GeneralDriveLution.BatchUpdateAsync(
        coreDrivers,
        new UpdateStrategy { RequireBackup = true },
        BatchMode.Sequential); // 顺序安装, 降低风险

    Console.WriteLine($"Core drivers: {coreResult.SucceededCount}/{coreResult.SucceededCount + coreResult.FailedCount}");
}

// 4. 可选驱动并行安装
if (optionalDrivers.Any())
```

```
{
    var optResult = await GeneralDrivelution.BatchUpdateAsync(
        optionalDrivers,
        new UpdateStrategy { RequireBackup = true },
        BatchMode.Parallel);    // 互不依赖，并行安装

    Console.WriteLine($"Optional drivers: {optResult.SucceededCount}/{optResult.SucceededCount + optResult.FailedCount}");
}
```

【效果&注意事项】

- `BatchMode.Parallel` 不一定更快，底层系统工具可能竞争驱动仓库锁
- 核心驱动建议顺序安装，降低系统风险

5. 常规使用示例

5.1 快速入门示例（最简 demo）

```
using GeneralUpdate.Drivelution;
using GeneralUpdate.Drivelution.Abstractions.Models;

var driver = new DriverInfo
{
    Name = "MyDevice Driver",
    Version = "1.2.0",
    FilePath = @"C:\Drivers\mydevice.inf",
    TargetOS = "Windows",
    Architecture = "x64",
    Hash = "driver-file-sha256",
    HashAlgorithm = "SHA256",
    TrustedPublishers = { "Contoso Hardware" }
};

var result = await GeneralDrivelution.QuickUpdateAsync(driver);

if (result.Success)
    Console.WriteLine($"Driver updated successfully in {result.DurationMs}ms.");
else
{
    Console.WriteLine($"Update failed: {result.Error?.Message}");
    foreach (var log in result.StepLogs)
        Console.WriteLine($" {log}");
}
```

5.2 基础参数组合示例

```
using GeneralUpdate.DriveIution;
using GeneralUpdate.DriveIution.Abstractions.Configuration;
using GeneralUpdate.DriveIution.Abstractions.Models;

// 查询平台信息
var platform = GeneralDriveIution.GetPlatformInfo();
Console.WriteLine($"OS: {platform.OperatingSystem}, Arch: {platform.Architecture}");

// 扫描驱动目录
var drivers = await GeneralDriveIution.GetDriversFromDirectoryAsync(@"C:\Drivers");
Console.WriteLine($"Found {drivers.Count} driver(s).");

foreach (var d in drivers)
    Console.WriteLine($" {d.Name} v{d.Version} ({d.FilePath})");

// 创建更新器
var updater = GeneralDriveIution.Create(new DriveIutionOptions
{
    DefaultBackupPath = @"C:\DriverBackups",
    DefaultRetryCount = 3,
    DefaultTimeoutSeconds = 600
});

// 更新，带进度
var progress = new Progress(p =>
    Console.WriteLine($"{p.StepName}: {p.Percentage}%"));

var strategy = new UpdateStrategy
{
    RequireBackup = true,
    BackupPath = @"C:\DriverBackups\mydevice",
    TimeoutSeconds = 300
};

var result = await updater.UpdateAsync(drivers[0], strategy, progress);
Console.WriteLine($"Result: {(result.Success ? "Success" : "Failed")}");

if (result.Success && RestartHelper.IsRestartRequired(strategy.RestartMode))
    await RestartHelper.HandleRestartAsync(strategy.RestartMode, 60);
```

5.3 真实业务落地示例

完整驱动更新 workflow，覆盖扫描、验证、安装、回滚、重启：

```
using GeneralUpdate.DriveLution;
using GeneralUpdate.DriveLution.Abstractions.Configuration;
using GeneralUpdate.DriveLution.Abstractions.Models;

// 1. 全局配置
var options = new DriveLutionOptions
{
    DefaultBackupPath = @"C:\ProgramData\MyProduct\DriverBackups",
    DefaultRetryCount = 2,
    DefaultRetryIntervalSeconds = 10,
    DefaultTimeoutSeconds = 900,
    UseExponentialBackoff = true,
    ForceTerminateOnPermissionFailure = true,
    AutoCleanupBackups = true,
    BackupsToKeep = 3
};

// 2. 创建更新器
var updater = GeneralDriveLution.Create(options);

// 3. 扫描驱动目录
var platform = GeneralDriveLution.GetPlatformInfo();
Console.WriteLine($"Running on {platform.OperatingSystem} {platform.Architecture}");

var drivers = await GeneralDriveLution.GetDriversFromDirectoryAsync(
    @"C:\ProgramData\MyProduct\DriverPackages");

var compatible = drivers
    .Where(d => string.IsNullOrEmpty(d.TargetOS) || d.TargetOS == platform.OperatingSystem)
    .ToList();

Console.WriteLine($"Found {drivers.Count} driver(s), {compatible.Count} compatible.");

// 4. 逐驱动验证并更新
var results = new List<(DriverInfo Driver, UpdateResult Result)>();
foreach (var driver in compatible)
{
    // 先验证
    var valid = await GeneralDriveLution.ValidateAsync(driver);
    if (!valid)
    {
        Console.WriteLine($"[SKIP] {driver.Name}: validation failed");
        continue;
    }
}
```

```

// 再更新
var strategy = new UpdateStrategy
{
    RequireBackup = true,
    BackupPath = Path.Combine(options.DefaultBackupPath, driver.Name),
    TimeoutSeconds = 600,
    RestartMode = RestartMode.Prompt,
    SkipHashValidation = false,
    SkipSignatureValidation = false
};

var progress = new Progress(p =>
{
    if (p.Percentage % 25 == 0 || p.CurrentStatus == UpdateStatus.Succeeded)
        Console.WriteLine($"[{driver.Name}] {p.StepName} {p.Percentage}%: {p.Message}");
});

var result = await updater.UpdateAsync(driver, strategy, progress);
results.Add((driver, result));

if (!result.Success)
{
    Console.WriteLine($"[FAIL] {driver.Name}: {result.Error?.Message}");
    if (result.BackupPath != null)
    {
        Console.WriteLine($" Rolling back {driver.Name}...");
        await updater.RollbackAsync(result.BackupPath);
    }
}
else
{
    Console.WriteLine($"[OK] {driver.Name} v{driver.Version} ({result.DurationMs}ms)");
}
}

// 5. 汇总并处理重启
var succeeded = results.Count(r => r.Result.Success);
var failed = results.Count(r => !r.Result.Success);
Console.WriteLine($"Summary: {succeeded} succeeded, {failed} failed.");

if (succeeded > 0 && results.Any(r => RestartHelper.IsRestartRequired(r.Result.Status == UpdateStatus.Succeeded
    ? RestartMode.Prompt : RestartMode.None)))

```

```
{
    Console.WriteLine("Some drivers may require a restart.");
    var userAccepted = RestartHelper.PromptUserForRestart("Driver update completed. Restart now?");
    if (userAccepted)
        await RestartHelper.RestartSystemAsync();
}
```

6. 全局配置

平台差异速查

平台	驱动格式	安装命令	签名验证	权限要求
Windows	.inf	pnputil.exe /add-driver /install	Authenticode	管理员
Linux	.ko / .deb / .rpm	insmod / modprobe / dpkg -i / rpm -ivh	GPG (.sig / .asc)	root/sudo
macOS	.kext / .dext / .pkg	kextload / SystemExtensions / installer	codesign -v	root (SIP 和用户批准策略影响)

日志配置

```
// 性能敏感场景关闭日志
GeneralTracer.SetTracingEnabled(false);

// 排查问题时重新开启
GeneralTracer.SetTracingEnabled(true);
```

相关资源

- [驱动更新示例](#)
- [GeneralUpdate 仓库](#)
- [GeneralUpdate.Drivelution 组件文档](#)

GeneralUpdate.Extension

命名空间: GeneralUpdate.Extension | 主要入口: GeneralExtensionHost (实现 IExtensionHost) | NuGet 包: GeneralUpdate.Extension

1. 组件简介

1.1 组件概述

GeneralUpdate.Extension 是面向 .NET 应用的扩展管理组件，设计目标是让宿主程序具备类似 VS Code 的扩展生态能力：从远程服务查询扩展、下载扩展包、安装或更新到本地目录，并在这个过程中处理版本兼容、平台匹配、依赖扩展、SHA256 校验、失败回滚和事件通知。

它适合把主程序和可选能力拆开发布的场景，例如报表、认证、行业插件、客户定制模块、脚本执行器等。主程序只需要集成 `GeneralExtensionHost`，扩展包可以独立发布、独立更新。

核心能力：

能力	说明
扩展查询	通过服务端 API 分页查询可用扩展，支持名称、发布者、分类、平台等筛选条件
一键更新	<code>UpdateExtensionAsync</code> 串起查询→兼容性检查→平台检查→依赖递归安装→下载→SHA256 校验→安装→catalog 更新
安全安装	Zip Slip 路径穿越防护、安装前备份、失败自动回滚到旧版本
批量更新	<code>UpdateExtensionsAsync</code> 按顺序批量处理多个扩展，返回每个扩展的成功/失败结果
版本兼容性	<code>MinHostVersion ≤ HostVersion ≤ MaxHostVersion</code> 范围内才允许安装
平台匹配	<code>[Flags] TargetPlatform</code> 位运算判断扩展是否支持当前 OS
依赖解析	拓扑排序依赖树，检测循环依赖，递归安装未安装的依赖扩展
断点续传	下载支持 HTTP Range，已存在部分文件时从断点继续
本地 Catalog	每个扩展独立 <code>manifest.json</code> ，原子写入（ <code>.tmp</code> → 重命名），支持持久化和加载
生命周期钩子	安装前后、激活/停用前后、卸载前后的业务逻辑注入
自动更新策略	<code>SetGlobalAutoUpdate</code> / <code>SetAutoUpdate</code> 控制全局或单扩展的自动更新开关
DI 集成	<code>ExtensionHostBuilder</code> 注册默认服务，所有服务均可通过 DI 替换

解决的业务痛点：

- 主程序体积膨胀，需要把非核心功能拆成可独立更新的扩展
- 不同客户需要不同功能组合，扩展生态可以实现按需安装
- 扩展之间有依赖关系，需要自动管理依赖的安装和版本兼容性
- 需要统一的扩展管理框架减少重复开发

类比：VS Code 扩展市场

如果你的产品需要像 VS Code 那样的插件生态——远程查询扩展、一键安装、版本兼容检查、依赖自动安装、失败回滚——`GeneralUpdate.Extension` 提供了完整的底层能力。你只需要在此基础上搭一个"扩展市场"的 UI。

业务使用场景：

- IDE 类应用的插件市场
- 企业 ERP/CRM 的行业模块（报表模板、认证方式、数据导出等）
- 客户定制功能独立分发
- 脚本执行器/工具集的组件化发布

1.2 环境与依赖

项目	说明
版本	10.5.0-beta.7
目标框架	netstandard2.0 （兼容 .NET Framework 4.6.1+ / .NET Core 2.0+ / .NET 5+）
依赖包	Microsoft.Extensions.DependencyInjection、Microsoft.Extensions.Logging.Abstractions、Microsoft.Extensions.Options、Newtonsoft.Json、System.Net.Http、System.IO.Compression、System.IO.Compression.ZipFile
兼容性	所有支持 .NET Standard 2.0 的平台

2. 组件功能列表

功能名称	功能描述	类型	是否必填	备注限制
扩展查询	从服务端 API 分页查询扩展列表	基础	推荐	支持多条件筛选
扩展下载	从服务端下载扩展 ZIP 包，支持断点续传	基础	自动	通过 <code>DownloadExtensionAsync</code> 或一键更新自动触发
扩展安装	安全解压 ZIP 到本地目录，支持 Zip Slip 防护和回滚	基础	自动	仅接受 <code>.zip</code> 格式
一键更新	自动串起查询→兼容性→依赖→下载→校验→安装全流程	基础	推荐	<code>UpdateExtensionAsync</code>
批量更新	按顺序批量更新多个扩展	拓展	可选	<code>UpdateExtensionsAsync</code>
扩展卸载	从本地 catalog 移除并删除扩展目录	基础	可选	<code>UninstallExtensionAsync</code>
版本兼容性检查	宿主版本必须在扩展的 Min/Max 范围内	基础	自动	更新流程中自动检查

功能名称	功能描述	类型	是否必填	备注限制
平台匹配	自动识别当前 OS，匹配扩展支持的平台	基础	自动	PlatformMatcher 通过 RuntimeInformation 检测
依赖递归安装	发现未安装依赖时递归调用更新	基础	自动	依赖必须能被同一服务端查询和下载
循环依赖检测	拓扑排序时检测依赖环	基础	自动	DependencyResolver
SHA256 校验	下载后校验文件完整性	基础	自动	服务端 Hash 非空时校验
本地 Catalog 管理	每个扩展独立 manifest.json，原子写入	基础	自动	存储在扩展目录下
自动更新策略	全局/单扩展自动更新开关	拓展	可选	仅在内存中保存状态，不自动轮询
生命周期钩子	安装/激活/停用/卸载前后业务逻辑	拓展	可选	实现 IExtensionLifecycleHooks 或继承 DefaultExtensionLifecycleHooks
DI Builder	ExtensionHostBuilder 注册并替换所有服务	拓展	可选	支持自定义 IExtensionServiceFactory
下载队列管理	并发下载控制（默认 3）	拓展	可选	DownloadQueueManager

3. API 配置说明

3.1 配置字段（属性 Props）

ExtensionHostOptions：

字段名	数据类型	默认值	是否必填	枚举/取值范围	说明
ServerUrl	string	—	是	有效绝对 URL	扩展服务根地址，客户端调用 {ServerUrl}/Query 和 {ServerUrl}/Download/{extensionId}
Scheme	string	""	可选	"Bearer" 等	Authorization 认证方案，为空不设置认证头
Token	string	""	可选	—	Authorization token，需和 Scheme 同时非空才生效
HostVersion	string	—	推荐	SemVer 格式	宿主应用版本，用于兼容性判断
ExtensionsDirectory	string	—	是	有效目录路径	扩展包下载、安装和 .backup 目录所在位置
CatalogPath	string	null	可选	有效目录路径	本地扩展目录扫描路径，为空时使用 ExtensionsDirectory

ExtensionMetadata（本地模型）：

字段名	数据类型	默认值	是否必填	说明
Id	string	—	是	扩展唯一 ID，依赖、查询、更新、卸载都以它为关键标识
Name	string	null	推荐	扩展目录名和包名的稳定名称
DisplayName	string	null	可选	展示名称
Version	string	null	推荐	扩展版本，建议 1.2.3 格式
FileSize	long?	null	可选	包大小（字节）
Format	string	null	推荐	包格式，当前安装要求 .zip
Hash	string	null	推荐	SHA256，非空时更新流程校验下载文件
Publisher	string	null	可选	发布者
Categories	string	null	可选	逗号分隔分类
SupportedPlatforms	TargetPlatform	All	推荐	[Flags] 位标志：Windows(1)，Linux(2)，MacOS(4)，All(7)
MinHostVersion	string	null	可选	最低宿主版本
MaxHostVersion	string	null	可选	最高宿主版本
Dependencies	string	null	可选	逗号分隔的依赖扩展 ID
IsPreRelease	bool	false	可选	是否预发布
CustomProperties	string	null	可选	JSON 字符串形式的自定义属性

ExtensionQueryDTO（查询筛选）：

字段名	数据类型	默认值	是否必填	说明
Id	string?	null	可选	按 ID 精确查询
Name	string?	null	可选	按名称模糊匹配
Publisher	string?	null	可选	按发布者模糊匹配
Category	string?	null	可选	按分类筛选
Platform	TargetPlatform?	null	可选	按目标平台筛选
HostVersion	string?	null	可选	用于服务端兼容性判断
IsPreRelease	bool?	null	可选	是否包含预发布
Status	bool?	null	可选	按启用状态筛选
PageNumber	int	1	可选	页码（从 1 开始）
PageSize	int	10	可选	每页大小

3.2 实例方法

IExtensionHost：

方法名	入参明细	使用场景	注意事项
QueryExtensionsAsync(ExtensionQueryDTO)	query — 查询条件	搜索/浏览可用扩展	响应数据在 Body.Items 中
DownloadExtensionAsync(string, string)	extensionId — 扩展 ID; savePath — 保存路径	单独下载扩展包	支持 HTTP Range 断点续传
UpdateExtensionAsync(string)	extensionId — 扩展 ID	一键更新单个扩展（推荐入口）	串起查询→兼容性→依赖→下载→校验→安装全流程
InstallExtensionAsync(string, bool)	extensionPath — ZIP 包路径; rollbackOnFailure — 是否失败回滚	手动安装本地扩展包	仅接受 .zip 格式
UpdateExtensionsAsync(IEnumerable, CancellationToken)	extensionIds — 扩展 ID 列表; ct — 取消令牌	批量更新	按传入顺序逐个处理
UninstallExtensionAsync(string, CancellationToken)	extensionId — 扩展 ID; ct — 取消令牌	卸载扩展	移除 catalog 记录并删除扩展目录
ActivateExtensionAsync(string, CancellationToken)	extensionId ; ct	激活扩展	调用生命周期钩子
DeactivateExtensionAsync(string, CancellationToken)	extensionId ; ct	停用扩展	调用生命周期钩子
IsExtensionCompatible(ExtensionMetadata)	extension — 扩展元数据	检查扩展兼容性	基于 HostVersion 与 MinHostVersion / MaxHostVersion 比较
SetAutoUpdate(string, bool)	extensionId — 扩展 ID; autoUpdate — 是否自动更新	设置单扩展自动更新开关	仅内存状态，不自动后台轮询
SetGlobalAutoUpdate(bool)	enabled — 是否启用	设置全局自动更新默认值	仅内存状态

GeneralExtensionHost 附加方法：

方法名	入参明细	使用场景	注意事项
IsAutoUpdateEnabled(string)	extensionId — 扩展 ID	查询指定扩展的自动更新开关	单扩展设置优先于全局设置

ExtensionHostBuilder：

方法名	入参明细	使用场景	注意事项
ConfigureOptions(Action)	configure — 配置委托	通过 Lambda 配置选项	—
WithOptions(ExtensionHostOptions)	options — 选项对象	直接设置选项	—
ConfigureServices(Action)	configure — DI 注册委托	替换或添加服务	在 Build() 前调用
Build()	无	构建宿主实例	自动注册未覆盖的默认服务

3.3 回调事件

事件名称	回调参数	触发时机	使用说明
ExtensionUpdateStatusChanged	ExtensionUpdateEventArgs — ExtensionId, ExtensionName, Status, Progress (0-100), ErrorMessage	扩展更新流程各阶段	Status: Queued → Updating (下载进度) → UpdateSuccessful / UpdateFailed

ExtensionUpdateStatus 枚举：

值	说明
Queued (0)	已加入更新队列
Updating (1)	正在下载/更新中
UpdateSuccessful (2)	更新成功
UpdateFailed (3)	更新失败

4. 扩展示例（高阶用法）

4.1 组件可扩展能力总览

所有服务均可通过 ExtensionHostBuilder.ConfigureServices() 替换：

服务接口	默认实现	说明
IExtensionHttpClient	ExtensionHttpClient	HTTP 通信（查询/下载）
IVersionCompatibilityChecker	VersionCompatibilityChecker	版本兼容性检查
IDownloadQueueManager	DownloadQueueManager	下载队列管理
IPlatformMatcher	PlatformMatcher	平台检测
IPlatformServices	RuntimePlatformServices	运行时平台信息
IExtensionMetadataMapper	DefaultExtensionMetadataMapper	DTO→模型映射
IExtensionCatalog	ExtensionCatalog	本地扩展目录管理
IDependencyResolver	DependencyResolver	依赖解析
IExtensionLifecycleHooks	DefaultExtensionLifecycleHooks	生命周期钩子（所有方法 virtual）
IExtensionServiceFactory	ExtensionServiceFactory	服务工厂

4.2 分场景示例

场景 1：自定义生命周期钩子

【场景说明】在扩展安装前后执行自定义逻辑：安装前检查许可证、安装后初始化扩展数据库。

【示例代码】

```
using GeneralUpdate.Extension.Core;
using GeneralUpdate.Extension.Common.Models;

public sealed class LicensedLifecycleHooks : DefaultExtensionLifecycleHooks
{
    public override async Task<bool> OnBeforeInstallAsync(
        ExtensionMetadata extension,
        string? packagePath,
        CancellationToken cancellationToken = default)
    {
        // 检查许可证
        if (!LicenseManager.IsLicensed(extension.Id))
        {
            Console.WriteLine($"Extension '{extension.Id}' is not licensed.");
            return false; // 阻止安装
        }
        return true;
    }

    public override async Task OnAfterInstallAsync(
        ExtensionMetadata extension,
        CancellationToken cancellationToken = default)
    {
        // 初始化扩展数据库
        if (extension.CustomProperties != null)
        {
            var props = Newtonsoft.Json.JsonConvert
                .DeserializeObject<Dictionary<string, string>>(extension.CustomProperties);
            if (props?.ContainsKey("DbInitScript") == true)
            {
                await DatabaseInitializer.RunAsync(props["DbInitScript"], cancellationToken);
            }
        }
        Console.WriteLine($"Extension '{extension.DisplayName}' installed successfully.");
    }

    public override async Task<bool> OnBeforeUninstallAsync(
        ExtensionMetadata extension,
        CancellationToken cancellationToken = default)
    {
        // 检查是否有关联数据
        var hasData = await DataService.HasExtensionDataAsync(extension.Id, cancellationToken);
        if (hasData)
        {

```

```

        Console.WriteLine($"Extension '{extension.Id}' has associated data. Clean up first.");
        return false; // 阻止卸载
    }
    return true;
}
}

// 使用 Builder 注册
var host = new ExtensionHostBuilder()
    .WithOptions(options)
    .ConfigureServices(services =>
    {
        services.AddSingleton<IExtensionLifecycleHooks, LicensedLifecycleHooks>();
    })
    .Build();

```

【效果&注意事项】

- 返回 `false` 会阻止操作继续执行
- 所有钩子方法都是 `virtual`，只需覆写需要的部分

场景 2：自定义 HTTP 客户端 + 共享连接池

【场景说明】与主应用共享 `HttpClient` 连接池，避免 socket 耗尽；同时切换为 POST 查询。

【示例代码】

```

using GeneralUpdate.Extension.Communication;

// 共享主应用的 HttpClient
var sharedClient = new HttpClient(); // 或从 IHttpClientFactory 获取

var httpClient = new ExtensionHttpClient(
    serverUrl: "https://extensions.mycompany.com/Extension",
    scheme: "Bearer",
    token: "jwt-token",
    httpClient: sharedClient,
    ownsHttpClient: false) // 不拥有, 不 Dispose
{
    UsePostForQuery = true // 服务端要求 POST 查询
};

var host = new ExtensionHostBuilder()
    .WithOptions(options)
    .ConfigureServices(services =>
    {
        services.AddSingleton(httpClient);
    })
    .Build();

```

【效果&注意事项】

- `ownsHttpClient: false` 确保 Dispose 时不关闭共享连接
- `UsePostForQuery = true` 将默认 GET+JSON Body 改为 POST+JSON Body

场景 3：依赖解析 + 条件批量更新

【场景说明】用户选择安装一个扩展时，自动解析依赖并一起安装。

【示例代码】

```
using GeneralUpdate.Extension.Core;
using GeneralUpdate.Extension.Common.DTOs;
using GeneralUpdate.Extension.Common.Enums;

var host = new GeneralExtensionHost(options);

// 查询目标扩展
var response = await host.QueryExtensionsAsync(new ExtensionQueryDTO
{
    Id = "report-extension",
    PageSize = 1
});

if (response.Body?.Items.Any() != true)
{
    Console.WriteLine("Extension not found.");
    return;
}

var ext = response.Body.Items.First();

// 解析依赖
var catalog = host.ExtensionCatalog;
catalog.LoadInstalledExtensions();

var resolver = new GeneralUpdate.Extension.Dependencies.DependencyResolver(catalog);
var deps = resolver.ResolveDependencies(
    new ExtensionMetadata { Id = ext.Id, Dependencies = string.Join(",", ext.Dependencies ?? []) });

var missingDeps = resolver.GetMissingDependencies(
    new ExtensionMetadata { Id = ext.Id, Dependencies = string.Join(",", ext.Dependencies ?? []) });

Console.WriteLine($"Dependencies for {ext.DisplayName}: {deps.Count} total, {missingDeps.Count} missin
g.");

// 先安装缺失依赖
var updateOrder = new List<string>();
updateOrder.AddRange(missingDeps);
updateOrder.Add(ext.Id);

var results = await host.UpdateExtensionsAsync(updateOrder);
```



```
foreach (var (id, success) in results)
    Console.WriteLine($" {id}: {(success ? "OK" : "FAILED")}");
```

【效果&注意事项】

- `DependencyResolver.ResolveDependencies` 返回拓扑排序后的完整依赖列表
 - `GetMissingDependencies` 过滤出未安装在本地 catalog 中的依赖
 - 循环依赖会被检测并抛出异常
-

5. 常规使用示例

5.1 快速入门示例（最简 demo）

```
using GeneralUpdate.Extension.Core;
using GeneralUpdate.Extension.Common.DTOS;
using GeneralUpdate.Extension.Common.Models;

var options = new ExtensionHostOptions
{
    ServerUrl = "https://extensions.example.com/Extension",
    Scheme = "Bearer",
    Token = "your-token",
    HostVersion = "1.0.0",
    ExtensionsDirectory = "./extensions"
};

var host = new GeneralExtensionHost(options);

host.ExtensionUpdateStatusChanged += (sender, e) =>
{
    Console.WriteLine($"[{e.Status}] {e.ExtensionId}: {e.Progress}% {e.ErrorMessage}");
};

// 查询可用扩展
var response = await host.QueryExtensionsAsync(new ExtensionQueryDTO
{
    Platform = TargetPlatform.Windows,
    PageNumber = 1,
    PageSize = 20
});

if (response.Body != null)
{
    foreach (var ext in response.Body.Items)
        Console.WriteLine($"{ext.DisplayName} v{ext.Version} [{ext.Id}]");
}

// 更新指定扩展
var success = await host.UpdateExtensionAsync("report-extension");
Console.WriteLine(success ? "Extension updated." : "Update failed.");
```

5.2 基础参数组合示例

```
using GeneralUpdate.Extension.Core;
using GeneralUpdate.Extension.Common.DTOs;
using GeneralUpdate.Extension.Common.Enums;

var host = new GeneralExtensionHost(new ExtensionHostOptions
{
    ServerUrl = "https://extensions.mycompany.com/Extension",
    Scheme = "Bearer",
    Token = Environment.GetEnvironmentVariable("EXTENSION_TOKEN") ?? "",
    HostVersion = "2.0.0",
    ExtensionsDirectory = Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "extensions")
});

// 事件监听
host.ExtensionUpdateStatusChanged += (_, e) =>
{
    switch (e.Status)
    {
        case ExtensionUpdateStatus.Queued:
            Console.WriteLine($"{e.ExtensionId}: queued");
            break;
        case ExtensionUpdateStatus.Updating:
            Console.WriteLine($"{e.ExtensionId}: downloading... {e.Progress}%");
            break;
        case ExtensionUpdateStatus.UpdateSuccessful:
            Console.WriteLine($"{e.ExtensionName ?? e.ExtensionId}: updated successfully");
            break;
        case ExtensionUpdateStatus.UpdateFailed:
            Console.WriteLine($"{e.ExtensionId}: failed - {e.ErrorMessage}");
            break;
    }
};

// 安装本地扩展包
var installed = await host.InstallExtensionAsync(
    "./downloads/report-extension_1.0.0.zip",
    rollbackOnFailure: true);
Console.WriteLine(installed ? "Installed." : "Installation failed.");

// 查询已安装扩展
host.ExtensionCatalog.LoadInstalledExtensions();
var installedExts = host.ExtensionCatalog.GetInstalledExtensions();
```

```
foreach (var ext in installedExts)
{
    var compat = host.IsExtensionCompatible(ext);
    Console.WriteLine($"{ext.DisplayName} v{ext.Version} - compatible: {compat}");
}

// 配置自动更新策略
host.SetGlobalAutoUpdate(true);
host.SetAutoUpdate("large-extension", false); // 大型扩展关闭自动更新

var checkResult = host.IsAutoUpdateEnabled("large-extension");
Console.WriteLine($"Auto-update for large-extension: {checkResult}");
```

5.3 真实业务落地示例

包含异常处理、依赖管理、兼容性检查的完整 workflow:

```
using GeneralUpdate.Extension.Core;
using GeneralUpdate.Extension.Common.DTOs;
using GeneralUpdate.Extension.Common.Enums;
using GeneralUpdate.Extension.Common.Models;

// 1. 初始化
var options = new ExtensionHostOptions
{
    ServerUrl = "https://extensions.mycompany.com/Extension",
    Scheme = "Bearer",
    Token = Configuration.GetExtensionToken(),
    HostVersion = AppInfo.CurrentVersion.ToString(),
    ExtensionsDirectory = Path.Combine(AppInfo.DataDirectory, "extensions")
};

// 2. 使用 Builder 注册自定义服务
var host = new ExtensionHostBuilder()
    .WithOptions(options)
    .ConfigureServices(services =>
    {
        services.AddSingleton<IExtensionLifecycleHooks, AuditLifecycleHooks>();
    })
    .Build();

host.ExtensionUpdateStatusChanged += OnExtensionStatusChanged;

// 3. 加载本地已安装扩展
host.ExtensionCatalog.LoadInstalledExtensions();
var installed = host.ExtensionCatalog.GetInstalledExtensions();
Console.WriteLine($"Loaded {installed.Count} installed extension(s).");

// 4. 查询服务端可用扩展
HttpResponseDTO<PagedResultDTO>? response = null;
try
{
    response = await host.QueryExtensionsAsync(new ExtensionQueryDTO
    {
        Platform = TargetPlatform.Windows | TargetPlatform.Linux,
        HostVersion = options.HostVersion,
        Status = true,
        PageNumber = 1,
        PageSize = 100
    });
}
}
```

```

catch (HttpRequestException ex)
{
    Console.WriteLine($"Failed to query extensions: {ex.Message}");
    return;
}

if (response?.Body == null)
{
    Console.WriteLine($"Server returned: {response?.Code} {response?.Message}");
    return;
}

// 5. 筛选可更新的扩展
var toUpdate = new List<string>();
foreach (var ext in response.Body.Items)
{
    var installedExt = host.ExtensionCatalog.GetInstalledExtensionById(ext.Id);
    if (installedExt == null)
    {
        Console.WriteLine($"[NEW] {ext.DisplayName} v{ext.Version}");
        continue; // 新扩展, 不自动安装
    }

    if (!host.IsExtensionCompatible(new ExtensionMetadata
    {
        MinHostVersion = ext.MinHostVersion,
        MaxHostVersion = ext.MaxHostVersion
    }))
    {
        Console.WriteLine($"[INCOMPATIBLE] {ext.DisplayName}: requires host {ext.MinHostVersion}-{ext.MaxH
ostVersion}");
        continue;
    }

    if (Version.TryParse(ext.Version, out var remoteVer) &&
        Version.TryParse(installedExt.Version, out var localVer) &&
        remoteVer > localVer)
    {
        if (host.IsAutoUpdateEnabled(ext.Id))
        {
            Console.WriteLine($"[UPDATE] {ext.DisplayName}: {installedExt.Version} → {ext.Version}");
            toUpdate.Add(ext.Id);
        }
        else

```

```

        {
            Console.WriteLine($"[SKIP] {ext.DisplayName}: auto-update disabled");
        }
    }
}

// 6. 执行批量更新
if (toUpdate.Any())
{
    Console.WriteLine($"Updating {toUpdate.Count} extension(s)...");
    var results = await host.UpdateExtensionsAsync(toUpdate);

    var succeeded = results.Count(r => r.Value);
    var failed = results.Count(r => !r.Value);

    Console.WriteLine($"Done: {succeeded} succeeded, {failed} failed.");
    foreach (var (id, success) in results.Where(r => !r.Value))
        Console.WriteLine($"  Failed: {id}");
}
else
{
    Console.WriteLine("All extensions up to date.");
}

// 事件处理
void OnExtensionStatusChanged(object? sender, ExtensionUpdateEventArgs e)
{
    switch (e.Status)
    {
        case ExtensionUpdateStatus.Queued:
            break;

        case ExtensionUpdateStatus.Updating:
            UpdateProgressUI(e.ExtensionId, e.Progress);
            break;

        case ExtensionUpdateStatus.UpdateSuccessful:
            Log.Info($"Extension '{e.ExtensionName ?? e.ExtensionId}' updated.");
            RefreshUI();
            break;

        case ExtensionUpdateStatus.UpdateFailed:
            Log.Error($"Extension '{e.ExtensionId}' update failed: {e.ErrorMessage}");
            NotifyUser($"Failed to update {e.ExtensionName ?? e.ExtensionId}");
            break;
    }
}
}

```

6. 全局配置

服务器 API 契约

查询接口：

```
GET {ServerUrl}/Query
Content-Type: application/json
Authorization: {Scheme} {Token}

Body: ExtensionQueryDTO (JSON)
```

注意：当前实现使用 GET + JSON Body，非标准 HTTP 风格。经过代理/网关时可能需要调整为 POST 或 query string。

下载接口：

```
GET {ServerUrl}/Download/{extensionId}
Authorization: {Scheme} {Token}
Range: bytes={existingLength}-
```

服务端应支持 HTTP Range 以启用断点续传。

扩展包结构与 Tools 关系

角色	说明
Extension 组件	消费侧：下载、校验、解压、安装、回滚、登记 manifest
Tools / CI/CD	生产侧：编译扩展、生成元数据、计算 SHA256、压缩为 ZIP、上传服务端

推荐包名格式： {Name}_{Version}.zip

推荐 ZIP 内容：

```
report-extension_1.0.0.zip
├─ manifest.json      # 推荐放入包内
├─ extension.dll       # 扩展主体程序集
├─ extension.deps.json # .NET 依赖描述
├─ README.md
├─ CHANGELOG.md
└─ LICENSE.txt
```

自动更新策略优先级

单扩展设置 > 全局设置 > 默认值 (false)

平台兼容性速查

枚举值	数值	说明
TargetPlatform.None	0	不匹配任何平台
TargetPlatform.Windows	1	Windows
TargetPlatform.Linux	2	Linux
TargetPlatform.MacOS	4	macOS
TargetPlatform.All	7	所有平台 (Windows Linux MacOS)

相关资源

- [扩展管理示例](#)
- [GeneralUpdate 仓库](#)

GeneralUpdate.Avalonia.Android

命名空间: GeneralUpdate.Avalonia.Android | 主要入口: GeneralUpdateBootstrap.CreateDefault() | NuGet 包: GeneralUpdate.Avalonia.Android

1. 组件简介

1.1 组件概述

GeneralUpdate.Avalonia.Android 是面向 Avalonia 应用的 Android 平台自动更新组件，专为 Android APK 的更新场景设计。它提供版本对比、可断点续传的 APK 下载、SHA256 完整性校验、以及 Android Package Installer 自动启动的全流程编排能力。

Android 平台的更新与桌面应用不同：APK 安装必须通过 Android 系统安装器完成，下载过程可能被用户中断或切换网络，安装前需要检查 "安装未知应用" 权限。该组件封装了这些平台差异，让 Avalonia 开发者只需提供更新包信息即可完成更新。

核心能力：

能力	说明
版本对比	内置 SystemVersionComparer，通过 IVersionComparer 接口支持自定义版本策略
断点续传下载	HttpResumableApkDownloader 支持 HTTP Range 请求头，中断后可续传
SHA256 完整性校验	下载完成后自动计算文件 SHA256，与服务端声明的哈希值比对，防止文件损坏或篡改
APK 安装编排	通过 AndroidApkInstaller 调用系统 Package Installer，自动处理 FileProvider URI
HTTP 传输配置	HttpDownloadOptions 支持 SSL/TLS 证书策略、代理、超时、重试和多种认证方案
多协议认证	支持 HMAC-SHA256、Bearer Token、API Key、HTTP Basic 四种认证方案，支持全局和单包粒度
事件通知	版本发现、下载进度、完成、失败等事件通知
状态快照	GetSnapshot() 随时获取当前更新状态、失败原因和消息
UI 线程调度	IUpdateEventDispatcher 支持将事件调度到 UI 线程（如 Avalonia 的 SynchronizationContext）
渐进式下载	临时文件 .part 和 Sidecar .json 机制，记录已完成字节数，支持进程级断点续传

解决的业务痛点：

- Avalonia 应用发布 Android 版本后，需要可靠的内置更新能力，但 Android APK 安装必须经由系统安装器
- 下载大 APK 包时可能被用户中断、网络切换，需要断点续传能力降低重复下载流量
- 需要对下载的 APK 进行完整性校验，防止传输损坏或被篡改
- 需要灵活的认证方案对接不同服务端的鉴权要求

Android 更新 ≠ 桌面文件替换

Android 平台不能像桌面端那样直接替换文件。APK 必须通过 Android Package Installer 安装，系统会处理签名验证、权限授予和应用替换。本组件封装了这些平台差异，但 **APK 签名、版本号递增等仍需要你在构建流水线中保证。**

业务使用场景：

- Avalonia 跨平台应用的 Android APK 自动更新
- 企业内部 Android 应用的静默或引导式更新
- 需要通过 CDN / OSS 分发 APK 包的场景

1.2 环境与依赖

项目	说明
NuGet 包	GeneralUpdate.Avalonia.Android
目标框架	net10.0-android（最低 Android API 26+）
依赖包	Xamarin.AndroidX.Core（FileProvider 支持）
兼容性	Android 8.0 (API 26) 及以上版本

2. 组件功能列表

功能名称	功能描述	类型	是否必填	备注限制
版本对比检查	比对当前版本与目标版本，判断是否有更新可用	基础	必选	ValidateAsync，支持自定义版本比较器
APK 断点续传下载	从服务端下载 APK 包，支持 HTTP Range 断点续传	基础	必选	DownloadAndVerifyAsync
SHA256 校验	下载完成后自动计算 SHA256 与服务端哈希比对	基础	自动	通过 IHashValidator 实现
APK 安装触发	调用 Android Package Installer 安装 APK	基础	必选	LaunchInstallerAsync
HTTP 传输配置	SSL 证书校验、代理、超时、重试、认证	拓展	可选	HttpDownloadOptions
多协议认证	HMAC-SHA256 / Bearer / API Key / Basic	拓展	可选	全局或单包粒度配置
进度通知	下载进度（速度、字节数、百分比）	基础	可选	AddListenerDownloadProgressChanged
完成通知	版本发现、下载完成、安装完成事件	基础	可选	AddListenerUpdateCompleted
失败通知	失败原因、异常信息、失败包信息	基础	可选	AddListenerUpdateFailed
验证通知	发现可用更新时触发	基础	可选	AddListenerValidate
状态快照	随时获取当前更新状态	拓展	可选	GetSnapshot()
UI 线程调度	自定义事件调度策略	拓展	可选	实现 IUpdateEventDispatcher

功能名称	功能描述	类型	是否必填	备注限制
自定义下载器	自定义下载实现	拓展	可选	实现 <code>IUpdateDownloader</code>
自定义哈希验证	自定义哈希算法	拓展	可选	实现 <code>IHashValidator</code>
自定义版本比较	自定义版本对比逻辑	拓展	可选	实现 <code>IVersionComparer</code>
自定义 APK 安装器	自定义安装实现	拓展	可选	实现 <code>IApkInstaller</code>
自定义文件存储	自定义文件读写实现	拓展	可选	实现 <code>IFileStorage</code>
自定义 SSL 策略	自定义 HTTPS 证书校验	拓展	可选	实现 <code>ISslValidationPolicy</code>
自定义 HTTP 认证	自定义 HTTP 请求认证	拓展	可选	实现 <code>IHttpAuthProvider</code>

3. API 配置说明

3.1 配置字段（属性 Props）

AndroidUpdateOptions:

字段名	数据类型	默认值	是否必填	说明
<code>DownloadDirectoryPath</code>	<code>string</code>	<code>""</code> （自动使用 <code>CacheDir/update</code> 或 <code>TempPath/update</code> ）	可选	下载文件存放目录，为空时自动选择
<code>FileProviderAuthority</code>	<code>string</code>	<code>""</code>	是	Android FileProvider authority，需与 AndroidManifest 配置一致
<code>TemporaryFileExtension</code>	<code>string</code>	<code>".part"</code>	可选	临时下载文件扩展名
<code>SidecarExtension</code>	<code>string</code>	<code>".json"</code>	可选	断点续传元数据文件扩展名
<code>DownloadBufferSize</code>	<code>int</code>	<code>65536</code> （64 KB）	可选	下载缓冲区大小（字节）
<code>SpeedSmoothingWindowSeconds</code>	<code>int</code>	<code>4</code>	可选	下载速度平滑窗口（秒）

HttpDownloadOptions:

字段名	数据类型	默认值	是否必填	说明
<code>SslValidationPolicy</code>	<code>ISslValidationPolicy?</code>	<code>null</code>	可选	自定义 SSL 证书验证策略； <code>null</code> 使用系统默认
<code>RequestTimeout</code>	<code>TimeSpan</code>	<code>30s</code>	可选	单次 HTTP 请求超时

字段名	数据类型	默认值	是否必填	说明
DownloadTimeout	TimeSpan	10min	可选	整体下载操作超时
Proxy	IWebProxy?	null	可选	HTTP 代理
UseProxy	bool	false	可选	是否启用配置的代理
MaxRetryAttempts	int	3	可选	最大重试次数（3 表示初始 1 次 + 2 次重试）
RetryBaseDelay	TimeSpan	1s	可选	指数退避基本延迟：baseDelay * 2^attempt
AuthProvider	IHttpAuthProvider?	null	可选	全局 HTTP 认证提供者，单包认证优先级更高

UpdatePackageInfo:

字段名	数据类型	默认值	是否必填	说明
Version	string	—	是	目标版本号
DownloadUrl	string	—	是	APK 下载地址
Sha256	string	—	是	APK 的 SHA256 哈希值，用于完整性校验
FileSize	long	0	可选	APK 文件大小（字节）
FileName	string?	null	可选	下载后的文件名
IsForced	bool	false	可选	是否强制更新
VersionName	string?	null	可选	版本展示名称
Description	string?	null	可选	版本描述
PublishTime	DateTimeOffset?	null	可选	发布时间
AuthScheme	AuthScheme?	null	可选	单包认证方案，优先于全局配置
AuthToken	string?	null	可选	Bearer 或 API Key 认证令牌
AuthSecretKey	string?	null	可选	HMAC-SHA256 签名密钥
BasicUsername	string?	null	可选	HTTP Basic 用户名
BasicPassword	string?	null	可选	HTTP Basic 密码

UpdateState 枚举:

枚举值	数值	说明
None	0	初始状态
Checking	1	正在检查更新
UpdateAvailable	2	发现可用更新
Downloading	3	正在下载
Verifying	4	正在校验
ReadyToInstall	5	准备安装
Installing	6	正在安装
Completed	7	更新完成

枚举值	数值	说明
Failed	8	更新失败
Canceled	9	已取消

AuthScheme 枚举：

枚举值	说明
Hmac	HMAC-SHA256 签名认证
Bearer	Bearer Token 认证
ApiKey	API Key 认证
Basic	HTTP Basic 认证

3.2 实例方法

GeneralUpdateBootstrap（静态工厂）：

方法名	入参明细	使用场景	注意事项
CreateDefault(AndroidUpdateOptions, ...)	options — 更新选项；可选参数：contextProvider, activityProvider, httpClient, versionComparer, eventDispatcher, logger, httpOptions	创建默认的 Android 更新引导实例	所有可选参数为 null 时使用内置默认实现

IAndroidBootstrap：

方法名	入参明细	使用场景	注意事项
ValidateAsync(UpdatePackageInfo, string, CancellationToken)	packageInfo — 更新包信息；currentVersion — 当前版本；ct — 取消令牌	检查是否有可用的更新	返回 UpdateCheckResult，包含 UpdateFound 和 TargetVersion
DownloadAndVerifyAsync(UpdatePackageInfo, CancellationToken)	packageInfo — 更新包信息；ct — 取消令牌	下载 APK 并校验 SHA256	返回 UpdateOperationResult，成功时 FilePath 为 APK 路径
LaunchInstallerAsync(UpdatePackageInfo, string, CancellationToken)	packageInfo — 更新包信息；apkFilePath — APK 文件路径；ct — 取消令牌	启动 Android Package Installer	需要 FileProvider authority 配置正确
GetSnapshot()	无	获取当前更新状态快照	返回 UpdateStateSnapshot(State, FailureReason, Message)
Dispose()	无	释放下载器等资源	—

3.3 回调事件

事件名称	回调参数	触发时机	使用说明
AddListenerValidate	ValidateEventArgs — PackageInfo, CurrentVersion	版本对比完成后，发现可用更新时触发	可用于 UI 展示新版本信息

事件名称	回调参数	触发时机	使用说明
AddListenerDownloadProgressChanged	DownloadProgressChangedEventArgs — ProgressPercentage , DownloadSpeedBytesPerSecond , DownloadedBytes , RemainingBytes , TotalBytes , PackageInfo , StatusDescription	下载进度更新时	可用于进度条展示
AddListenerUpdateCompleted	UpdateCompletedEventArgs — Result (UpdateOperationResult)	下载完成或安装完成时	可用于 UI 切换
AddListenerUpdateFailed	UpdateFailedEventArgs — Result (UpdateOperationResult)	任何阶段失败时	包含失败原因和异常信息

4. 扩展示例（高阶用法）

4.1 组件可扩展能力总览

所有服务均可通过 `CreateDefault` 的可选参数替换：

接口	默认实现	说明
<code>IVersionComparer</code>	<code>SystemVersionComparer</code>	版本对比策略
<code>IUpdateDownloader</code>	<code>HttpResumableApkDownloader</code>	APK 下载实现
<code>IHashValidator</code>	<code>Sha256HashValidator</code>	SHA256 哈希校验
<code>IApkInstaller</code>	<code>AndroidApkInstaller</code>	APK 安装触发
<code>IFileStorage</code>	<code>PhysicalFileStorage</code>	文件存储操作
<code>IUpdateEventDispatcher</code>	<code>ImmediateEventDispatcher</code>	事件调度器（直接调用）
<code>IUpdateLogger</code>	<code>NoOpUpdateLogger</code>	日志记录器
<code>IAndroidContextProvider</code>	<code>DefaultAndroidContextProvider</code>	Android Context 提供器
<code>IAndroidActivityProvider</code>	<code>NullAndroidActivityProvider</code>	Android Activity 提供器
<code>ISslValidationPolicy</code>	通过 <code>HttpDownloadOptions</code> 配置	SSL 证书验证策略
<code>IHttpAuthProvider</code>	通过 <code>HttpDownloadOptions</code> 或单包配置	HTTP 认证提供器

4.2 分场景示例

场景 1：自定义版本比较策略

【场景说明】应用使用 `year.month.day.build` 格式版本号，默认的 `System.Version` 解析会失败，需要自定义版本比较器。

【示例代码】

```

using GeneralUpdate.Avalonia.Android.Abstractions;

public sealed class CustomDateVersionComparer : IVersionComparer
{
    public bool TryCompare(string currentVersion, string targetVersion,
        out int compareResult, out string? errorMessage)
    {
        try
        {
            var current = ParseVersion(currentVersion);
            var target = ParseVersion(targetVersion);
            compareResult = target.CompareTo(current);
            errorMessage = null;
            return true;
        }
        catch (Exception ex)
        {
            compareResult = 0;
            errorMessage = ex.Message;
            return false;
        }
    }

    private static DateTime ParseVersion(string version) =>
        DateTime.ParseExact(version, "yyyy.MM.dd.fff", null);
}

// 使用
var bootstrap = GeneralUpdateBootstrap.CreateDefault(
    options,
    versionComparer: new CustomDateVersionComparer());

```

【效果&注意事项】

- 返回的 `compareResult` 遵循 `target.CompareTo(current)` 语义：正数表示有新版本
- 实现失败时返回 `false` 和错误信息，引导跳过更新检查

场景 2：自定义事件调度器（Avalonia UI 线程调度）

【场景说明】在 Avalonia 应用中，事件回调默认在后台线程触发，更新 UI 需要调度到 UI 线程。


```
using GeneralUpdate.Avalonia.Android.Abstractions;

public sealed class AvaloniaEventDispatcher : IUpdateEventDispatcher
{
    public void Dispatch(Action callback)
    {
        if (Avalonia.Threading.Dispatcher.UIThread.CheckAccess())
        {
            callback();
        }
        else
        {
            Avalonia.Threading.Dispatcher.UIThread.Post(callback);
        }
    }
}

// 使用
var bootstrap = GeneralUpdateBootstrap.CreateDefault(
    options,
    eventDispatcher: new AvaloniaEventDispatcher());
```

场景 3：自定义下载器（私有文件服务器）

【场景说明】企业内部使用私有文件服务器，需要在下载时注入自定义认证头。

```
using GeneralUpdate.Avalonia.Android.Abstractions;
using GeneralUpdate.Avalonia.Android.Models;

public sealed class EnterpriseDownloader : IUpdateDownloader
{
    private readonly HttpClient _httpClient;
    private readonly string _enterpriseToken;

    public EnterpriseDownloader(HttpClient httpClient, string enterpriseToken)
    {
        _httpClient = httpClient;
        _enterpriseToken = enterpriseToken;
    }

    public async Task DownloadAsync(
        UpdatePackageInfo packageInfo,
        Action? progressCallback,
        CancellationToken cancellationToken = default)
    {
        var request = new HttpRequestMessage(HttpMethod.Get, packageInfo.DownloadUrl);
        request.Headers.Add("X-Enterprise-Token", _enterpriseToken);

        // 自行实现下载和进度报告...
        // 返回 DownloadResult
    }
}
```

5. 常规使用示例

5.1 快速入门示例（最简 demo）

```
using GeneralUpdate.Avalonia.Android;
using GeneralUpdate.Avalonia.Android.Models;

var options = new AndroidUpdateOptions
{
    FileProviderAuthority = "com.myapp.fileprovider"
};

var bootstrap = GeneralUpdateBootstrap.CreateDefault(options);

// 1. 检查更新
var updatePackage = new UpdatePackageInfo
{
    Version = "2.0.0",
    DownloadUrl = "https://update.example.com/app-v2.0.0.apk",
    Sha256 = "expected-sha256-hash",
    FileSize = 50_000_000,
    FileName = "app_update_2.0.0.apk"
};

var checkResult = await bootstrap.ValidateAsync(
    updatePackage, currentVersion: "1.0.0");

if (!checkResult.UpdateFound)
{
    Console.WriteLine("Already up to date.");
    return;
}

// 2. 下载并校验
var downloadResult = await bootstrap.DownloadAndVerifyAsync(updatePackage);
if (!downloadResult.Success)
{
    Console.WriteLine($"Download failed: {downloadResult.Message}");
    return;
}

// 3. 安装
var installResult = await bootstrap.LaunchInstallerAsync(
    updatePackage, downloadResult.FilePath!);
```

```
if (installResult.Success)
    Console.WriteLine("Installer launched.");
```

5.2 基础参数组合示例

```
using GeneralUpdate.Avalonia.Android;
using GeneralUpdate.Avalonia.Android.Abstractions;
using GeneralUpdate.Avalonia.Android.Models;
using GeneralUpdate.Avalonia.Android.Services;

var options = new AndroidUpdateOptions
{
    DownloadDirectoryPath = Path.Combine(
        Android.App.Application.Context.CacheDir!.AbsolutePath, "myapp-updates"),
    FileProviderAuthority = "com.myapp.fileprovider",
    TemporaryFileExtension = ".downloading",
    DownloadBufferSize = 128 * 1024 // 128 KB buffer
};

// 带 HTTP 配置的启动
var httpOptions = new HttpDownloadOptions
{
    RequestTimeout = TimeSpan.FromSeconds(60),
    DownloadTimeout = TimeSpan.FromMinutes(30),
    MaxRetryAttempts = 5,
    RetryBaseDelay = TimeSpan.FromSeconds(2),
    // 开发环境使用自签名证书
    SslValidationPolicy = new AllowAllSslValidationPolicy()
};

var bootstrap = GeneralUpdateBootstrap.CreateDefault(
    options,
    httpOptions);

// 事件监听
bootstrap.AddListenerDownloadProgressChanged += (_, e) =>
{
    Console.WriteLine(
        $"Download: {e.ProgressPercentage:F1}% " +
        $"({FormatSize(e.DownloadedBytes)}/{FormatSize(e.TotalBytes)}) " +
        $"({FormatSpeed(e.DownloadSpeedBytesPerSecond)})");
};

bootstrap.AddListenerUpdateFailed += (_, e) =>
{
    Console.WriteLine($"Failed: {e.Result.Message} (reason: {e.Result.FailureReason})");
};
```

```

bootstrap.AddListenerUpdateCompleted += (_, e) =>
{
    Console.WriteLine($"Completed: {e.Result.State}");
};

// 执行更新
var package = new UpdatePackageInfo
{
    Version = "2.0.0",
    DownloadUrl = "https://update.example.com/app-v2.0.0.apk",
    Sha256 = "expected-sha256-hash",
    FileSize = 50_000_000,
    FileName = "app-v2.0.0.apk"
};

var checkResult = await bootstrap.ValidateAsync(package, "1.0.0");
if (!checkResult.UpdateFound) return;

var downloadResult = await bootstrap.DownloadAndVerifyAsync(package);
if (!downloadResult.Success) return;

await bootstrap.LaunchInstallerAsync(package, downloadResult.FilePath!);

static string FormatSize(long bytes) => bytes switch
{
    < 1024 => $"{bytes} B",
    < 1048576 => $"{bytes / 1024.0:F1} KB",
    _ => $"{bytes / 1048576.0:F1} MB"
};

static string FormatSpeed(double bytesPerSecond) =>
    $"{bytesPerSecond / 1048576.0:F1} MB/s";

```

5.3 真实业务落地示例

完整更新工作流，包含 HTTP 认证、断点续传、权限检查和进度 UI：

```

using GeneralUpdate.Avalonia.Android;
using GeneralUpdate.Avalonia.Android.Enums;
using GeneralUpdate.Avalonia.Android.Models;

public sealed class AppUpdateService
{
    private IAndroidBootstrap? _bootstrap;
    private bool _disposed;

    public event EventHandler<double>? ProgressChanged;
    public event EventHandler<string>? StatusChanged;

    public void Initialize(string fileProviderAuthority, string serverToken)
    {
        var httpOptions = new HttpDownloadOptions
        {
            MaxRetryAttempts = 3,
            RetryBaseDelay = TimeSpan.FromSeconds(1),
            AuthProvider = new Services.BearerTokenAuthProvider(serverToken)
        };

        _bootstrap = GeneralUpdateBootstrap.CreateDefault(
            new AndroidUpdateOptions
            {
                FileProviderAuthority = fileProviderAuthority
            },
            httpOptions: httpOptions);

        WireEvents();
    }

    public async Task<bool> UpdateAsync(
        UpdatePackageInfo package, string currentVersion,
        CancellationToken ct = default)
    {
        if (_bootstrap == null)
            throw new InvalidOperationException("Not initialized.");

        // Phase 1: 版本检查
        StatusChanged?.Invoke(this, "Checking for updates...");
        var checkResult = await _bootstrap.ValidateAsync(package, currentVersion, ct);
        if (!checkResult.UpdateFound)
        {
            StatusChanged?.Invoke(this, "Already up to date.");
        }
    }
}

```

```

        return true;
    }

    StatusChanged?.Invoke(this, $"Update v{package.Version} found.");

    // Phase 2: 下载并校验
    StatusChanged?.Invoke(this, "Downloading...");
    var downloadResult = await _bootstrap.DownloadAndVerifyAsync(package, ct);
    if (!downloadResult.Success)
    {
        StatusChanged?.Invoke(this,
            $"Download failed: {downloadResult.Message}");
        return false;
    }

    StatusChanged?.Invoke(this, "Download complete, verifying...");

    // Phase 3: 检查安装权限 (Android 8+)
    if (Android.OS.Build.VERSION.SdkInt >= Android.OS.BuildVersionCodes.O)
    {
        var ctx = Android.App.Application.Context;
        var pm = ctx.PackageManager;
        if (pm != null && !pm.CanRequestPackageInstalls())
        {
            StatusChanged?.Invoke(this,
                "Please allow install from unknown apps.");
            var intent = new Android.Content.Intent(
                Android.Provider.Settings.ActionManageUnknownAppSources)
                .SetData(Android.Net.Uri.Parse("package:" + ctx.PackageName));
            intent.AddFlags(Android.Content.ActivityFlags.NewTask);
            ctx.StartActivity(intent);
            return false;
        }
    }

    // Phase 4: 启动安装
    StatusChanged?.Invoke(this, "Installing...");
    var installResult = await _bootstrap.LaunchInstallerAsync(
        package, downloadResult.FilePath!, ct);

    if (installResult.Success)
    {
        StatusChanged?.Invoke(this, "Installer launched.");
        return true;
    }

```



```

    }

    StatusChanged?.Invoke(this,
        $"Install failed: {installResult.Message}");
    return false;
}

private void WireEvents()
{
    if (_bootstrap == null) return;

    _bootstrap.AddListenerDownloadProgressChanged += (_, e) =>
    {
        ProgressChanged?.Invoke(this, e.ProgressPercentage);
    };

    _bootstrap.AddListenerUpdateFailed += (_, e) =>
    {
        StatusChanged?.Invoke(this,
            $"Failed: {e.Result.Message} (reason: {e.Result.FailureReason})");
    };
}

public void Dispose()
{
    if (_disposed) return;
    _bootstrap?.Dispose();
    _disposed = true;
}
}

```

6. 全局配置

认证方案

Avalonia 组件支持四种认证方案，可在 `UpdatePackageInfo` 单包粒度配置：

认证方案	AuthScheme	所需字段
HMAC-SHA256	Hmac	AuthSecretKey
Bearer Token	Bearer	AuthToken
API Key	ApiKey	AuthToken
HTTP Basic	Basic	BasicUsername + BasicPassword

也可通过 `HttpDownloadOptions.AuthProvider` 设置全局认证提供器。单包配置优先于全局配置。

SSL 策略

策略	类名	使用场景
系统默认（推荐）	null	生产环境
允许所有（仅开发）	AllowAllSslValidationPolicy	自签名证书开发环境
自定义	实现 ISslValidationPolicy	私有 CA 或证书固定

断点续传机制

下载器使用临时文件（.part）和元数据文件（.json）实现断点续传：

```
CacheDir/update/
├─ app-v2.0.0.apk.part      # 部分下载的临时文件
├─ app-v2.0.0.apk.json     # 续传元数据（URL、ETag、预期大小、已下载字节数）
└─ app-v2.0.0.apk         # 下载完成后的最终文件
```

- 中断后重新下载同一 URL 时，自动读取 .json 元数据判断服务端是否支持 Range
- 支持 Range 时从已下载位置继续（Range: bytes={existingLength}-）
- 不支持 Range 时重新下载
- 下载完成后 .part 重命名为最终文件名，删除 .json 元数据

AndroidManifest 配置

使用 FileProvider 分享 APK 文件给 Package Installer，需要在 AndroidManifest.xml 中配置：

```
<application>
  <provider
    android:name="androidx.core.content.FileProvider"
    android:authorities="${applicationId}.fileprovider"
    android:exported="false"
    android:grantUriPermissions="true">
    <meta-data
      android:name="android.support.FILE_PROVIDER_PATHS"
      android:resource="@xml/file_paths" />
  </provider>
</application>
```

file_paths.xml（在 Resources/xml/ 目录下）：

```
<?xml version="1.0" encoding="utf-8"?>
<paths>
  <cache-path
    name="cache"
    path="update/" ></cache-path>
</paths>
```

平台差异

项目	说明
最低 API	26 (Android 8.0)
安装权限	Android 8.0+ 需要 <code>CanRequestPackageInstalls()</code> 检查
FileProvider	必须通过 FileProvider 向系统安装器传递 APK 文件 URI

相关资源

- [GeneralUpdate.Avalonia 仓库](#)
- [Android 更新示例](#)
- [Semantic Versioning](#)
- [FileProvider 文档](#)

GeneralUpdate.Maui.Android

命名空间: `GeneralUpdate.Maui.Android` | 主要入口: `GeneralUpdateBootstrap.CreateDefault()` / `AddGeneralUpdateMauiAndroid()` | NuGet 包: `GeneralUpdate.Maui.Android`

1. 组件简介

1.1 组件概述

GeneralUpdate.Maui.Android 是面向 .NET MAUI 应用的 Android 平台自动更新组件，为 Android APK 提供版本发现、可恢复下载、SHA256 完整性校验以及系统 Package Installer 触发的一站式更新编排。

与桌面应用的文件替换式更新不同，Android APK 更新必须经由系统 Package Installer 完成。该组件封装了这一平台差异，并提供两阶段 API：`ValidateAsync` 做版本检查，`ExecuteUpdateAsync` 串起下载→校验→安装的完整流程。

核心能力：

能力	说明
版本对比检查	内置 <code>System.Version</code> 解析，自动判断当前版本是否低于目标版本
可恢复 APK 下载	<code>HttpRangeDownloader</code> 基于 HTTP Range 请求头实现断点续传，支持进度报告
SHA256 完整性校验	下载后自动计算文件 SHA256 与服务端哈希比对，损坏文件自动删除
系统安装器触发	<code>AndroidApkInstaller</code> 通过 <code>FileProvider</code> + <code>Intent</code> 启动系统 Package Installer
DI 容器集成	<code>AddGeneralUpdateMauiAndroid()</code> 扩展方法一键注册所有服务到 <code>IServiceCollection</code>
HTTP 传输配置	<code>HttpDownloadOptions</code> 支持 SSL 证书策略、代理、超时、认证，支持指数退避重试
多协议认证	支持 HMAC-SHA256、Bearer Token、API Key、HTTP Basic 四种认证方案
事件通知	版本发现、下载进度、完成阶段、失败原因等全方位事件
并发安全	<code>ExecuteUpdateAsync</code> 内置 <code>Interlocked</code> 并发保护，重复调用直接返回 <code>AlreadyInProgress</code>
临时文件管理	下载使用 <code>.downloading</code> 临时扩展名，完成后原子替换为目标文件

解决的业务痛点：

- MAUI Android 应用需要内置自动更新能力，但 APK 安装必须绕过 Google Play 通过系统安装器完成
- 大 APK 下载过程中网络不稳定，需要断点续传减少用户流量消耗
- 需要确保下载的 APK 未被篡改或损坏，保障更新安全
- 需要与 MAUI 的 DI 容器深度集成，简化接入代码

MAUI 更新 ≠ 桌面文件替换

Android 平台无法像桌面端那样直接替换可执行文件。APK 必须通过 Android Package Installer 安装，系统会处理签名验证和应用替换。MAUI 组件封装了这一差异，但 **APK 签名、版本号递增** 仍然需要你在构建流水线中保证。

业务使用场景：

- MAUI Android 应用的自动更新（企业内部分发、Beta 测试、旁加载）

- 需要通过 CDN / OSS 分发 APK 包的应用
- 需要 DI 容器管理的 MAUI 应用

1.2 环境与依赖

项目	说明
NuGet 包	GeneralUpdate.Maui.Android
目标框架	net10.0 + net10.0-android （多目标; Android 最低 API 21）
依赖包	Microsoft.Extensions.DependencyInjection.Abstractions、Microsoft.Extensions.Http、Xamarin.AndroidX.Core
兼容性	Android 5.0 (API 21) 及以上版本

2. 组件功能列表

功能名称	功能描述	类型	是否必填	备注限制
版本对比检查	对比当前版本与目标版本，判断是否有更新可用	基础	必选	ValidateAsync，基于 System.Version 解析
更新执行	串起下载→校验→安装全流程	基础	必选	ExecuteUpdateAsync，并发保护
APK 可恢复下载	HTTP Range 断点续传 + 进度报告	基础	自动	HttpRangeDownloader
SHA256 校验	下载后自动校验，损坏文件自动删除	基础	自动	通过 IHashValidator，支持进度回调
APK 安装触发	FileProvider + Intent 启动系统安装器	基础	自动	AndroidApkInstaller，自动检查安装权限
DI 容器集成	一键注册所有更新服务	拓展	推荐	AddGeneralUpdateMauiAndroid()
HTTP 传输配置	SSL、代理、超时、重试、认证	拓展	可选	HttpDownloadOptions
多协议认证	HMAC-SHA256 / Bearer / API Key / Basic	拓展	可选	全局或单包粒度配置
进度通知	下载字节数、速度、百分比、剩余时间	基础	可选	IProgress 回调
完成通知	各完成阶段（下载完成、校验完成、安装触发、工作流完成）	基础	可选	AddListenerUpdateCompleted
失败通知	失败原因分类、异常信息	基础	可选	AddListenerUpdateFailed
验证通知	发现可用更新时触发	基础	可选	AddListenerValidate

功能名称	功能描述	类型	是否必填	备注限制
状态查询	实时查询当前更新状态	基础	可选	CurrentState 属性
并发保护	防止重复执行更新操作	基础	自动	Interlocked 原子操作
损坏文件自动清理	校验失败自动删除损坏包	拓展	可选	UpdateOptions.DeleteCorruptedPackageOnFailure
进度报告间隔控制	自定义进度回调频率	拓展	可选	UpdateOptions.ProgressReportInterval
自定义下载器	自定义下载实现	拓展	可选	实现 IUpdateDownloader
自定义哈希验证	自定义哈希实现	拓展	可选	实现 IHashValidator
自定义安装器	自定义安装逻辑	拓展	可选	实现 IApkInstaller
自定义存储提供者	自定义文件路径和存储操作	拓展	可选	实现 IUpdateStorageProvider
自定义日志	自定义日志输出	拓展	可选	实现 IUpdateLogger
自定义 SSL 策略	自定义 HTTPS 证书校验	拓展	可选	实现 ISslValidationPolicy
自定义 HTTP 认证	自定义 HTTP 请求认证	拓展	可选	实现 IHttpAuthProvider

3. API 配置说明

3.1 配置字段（属性 Props）

UpdateOptions:

字段名	数据类型	默认值	是否必填	说明
CurrentVersion	string	""	是	当前应用版本号
DownloadDirectory	string?	null	可选	下载目录，为空时自动选择
TemporaryFileExtension	string	".downloading"	可选	临时下载文件扩展名
DeleteCorruptedPackageOnFailure	bool	true	可选	校验失败时自动删除损坏包
ProgressReportInterval	TimeSpan	500ms	可选	进度报告间隔

字段名	数据类型	默认值	是否必填	说明
InstallOptions	AndroidInstallOptions	new()	可选	安装选项

AndroidInstallOptions：

字段名	数据类型	默认值	是否必填	说明
FileProviderAuthority	string	""	是	FileProvider authority，需与 AndroidManifest 一致
MimeType	string	"application/vnd.android.package-archive"	可选	APK 文件的 MIME 类型

HttpDownloadOptions：

字段名	数据类型	默认值	是否必填	说明
SslValidationPolicy	ISslValidationPolicy?	null	可选	自定义 SSL 证书验证策略
DownloadTimeout	TimeSpan	10min	可选	整体下载超时
Proxy	IWebProxy?	null	可选	HTTP 代理
UseProxy	bool	false	可选	是否启用代理
AuthProvider	IHttpAuthProvider?	null	可选	全局 HTTP 认证提供器

UpdatePackageInfo：

字段名	数据类型	默认值	是否必填	说明
Version	string	""	是	目标版本号
DownloadUrl	string	""	是	APK 下载地址
Sha256	string	""	是	SHA256 哈希值
PackageSize	long?	null	可选	包大小（字节）
ApkFileName	string?	null	可选	APK 文件名
ForceUpdate	bool	false	可选	是否强制更新
VersionName	string?	null	可选	展示名称
ReleaseNotes	string?	null	可选	发布说明
PublishTime	DateTimeOffset?	null	可选	发布时间
AuthScheme	AuthScheme?	null	可选	认证方案
AuthToken	string?	null	可选	认证令牌
AuthSecretKey	string?	null	可选	HMAC 密钥
BasicUsername	string?	null	可选	Basic 用户名
BasicPassword	string?	null	可选	Basic 密码

UpdateState 枚举:

枚举值	说明
None	初始/空闲状态
Checking	正在检查更新
UpdateAvailable	发现可用更新
Downloading	正在下载
Verifying	正在校验
ReadyToInstall	准备安装
Installing	正在安装
Completed	更新完成
Failed	更新失败
Canceled	已取消

UpdateFailureReason 枚举:

枚举值	说明
Unknown	未知错误
InvalidInput	输入参数无效
AlreadyInProgress	已有更新操作正在进行
Network	网络错误
Download	下载错误
FileAccess	文件访问错误
IntegrityCheckFailed	完整性校验失败
InstallPermissionDenied	安装权限被拒绝
Installation	安装过程错误
Canceled	用户取消
NoUpdateAvailable	无可用更新

UpdateCompletionStage 枚举:

枚举值	说明
DownloadCompleted	下载完成
VerificationCompleted	校验完成
InstallationTriggered	安装器已启动
WorkflowCompleted	工作流完成

3.2 实例方法

GeneralUpdateBootstrap（静态工厂）：

方法名	入参明细	使用场景	注意事项
CreateDefault(HttpClient?, IUpdateLogger?, HttpDownloadOptions?)	httpClient — 可选外部 HttpClient; logger — 可选日志器; httpOptions — 可选 HTTP 配置	创建默认的 Android 更新引导实例	httpOptions 生效时 httpClient 参数被忽略
AddGeneralUpdateMauiAndroid(IServiceCollection, HttpClient?)	services — DI 容器; httpClient — 可选外部 HttpClient	在 MAUI DI 容器中注册所有更新服务	返回 IServiceCollection 支持链式调用

IAndroidBootstrap：

方法名	入参明细	使用场景	注意事项
ValidateAsync(UpdatePackageInfo, UpdateOptions, CancellationToken)	packageInfo — 更新包信息; options — 更新选项; ct — 取消令牌	检查是否有可用的更新	返回 UpdateCheckResult，包含 IsUpdateAvailable
ExecuteUpdateAsync(UpdatePackageInfo, UpdateOptions, CancellationToken)	packageInfo — 更新包信息; options — 更新选项; ct — 取消令牌	执行完整更新：下载→校验→安装	内置并发保护，重复调用返回 AlreadyInProgress
Dispose()	无	释放下载器等资源	—
CurrentState	属性（ UpdateState ）	查询当前更新状态	线程安全（ Volatile.Read ）

3.3 回调事件

事件名称	回调参数	触发时机	使用说明
AddListener Validate	ValidateEventArgs — PackageInfo	版本对比完成，发现可用更新时	可用于 UI 展示新版本信息
AddListener DownloadProgressChanged	DownloadProgressChangedEventArgs — PackageInfo, Statistics (DownloadProgressInfo: ProgressPercentage, DownloadedBytes, TotalBytes, BytesPerSecond, RemainingBytes), StatusDescription	下载进度更新时	通过 IProgress 回调
AddListener UpdateCompleted	UpdateCompletedEventArgs — PackageInfo, Stage (UpdateCompletionStage), PackagePath	各阶段完成时	分为下载完成、校验完成、安装触发、工作流完成四个阶段
AddListener UpdateFailed	UpdateFailedEventArgs — Reason, Message, Exception, PackageInfo	任何阶段失败时	包含结构化的失败原因

4. 扩展示例（高阶用法）

4.1 组件可扩展能力总览

所有服务可通过 `AddGeneralUpdateMauiAndroid()` 注册后，通过 DI 容器替换：

接口	默认实现	说明
<code>IUpdateDownloader</code>	<code>HttpRangeDownloader</code>	可恢复 APK 下载
<code>IHashValidator</code>	<code>Sha256Validator</code>	SHA256 哈希校验
<code>IApkInstaller</code>	<code>AndroidApkInstaller</code>	APK 安装触发
<code>IUpdateStorageProvider</code>	<code>UpdateFileStore</code>	文件路径和存储操作
<code>IUpdateLogger</code>	<code>NullUpdateLogger</code>	日志记录器
<code>IUpdateBootstrap</code>	<code>AndroidBootstrap</code>	更新编排（通过 DI 注册）
<code>ISslValidationPolicy</code>	通过 <code>HttpDownloadOptions</code> 配置	SSL 证书验证策略
<code>IHttpAuthProvider</code>	通过 <code>HttpDownloadOptions</code> 或单包配置	HTTP 认证提供者

4.2 分场景示例

场景 1：DI 容器集成

【场景说明】在 MAUI 应用中将更新服务注册到 DI 容器，便于管理和测试。

```
using GeneralUpdate.Maui.Android.Abstractions;
using GeneralUpdate.Maui.Android.Models;
using GeneralUpdate.Maui.Android.Services;
using Microsoft.Extensions.Logging;

public static class MauiProgram
{
    public static MauiApp CreateMauiApp()
    {
        var builder = MauiApp.CreateBuilder();
        builder
            .UseMauiApp()
            .ConfigureFonts(fonts => { });

        // 注册更新服务
        builder.Services.AddGeneralUpdateMauiAndroid();

        // 注册应用服务
        builder.Services.AddSingleton();
        builder.Services.AddSingleton();

        return builder.Build();
    }
}

// 在 ViewModel 中使用
public class MainViewModel
{
    private readonly IAndroidBootstrap _bootstrap;

    public MainViewModel(IAndroidBootstrap bootstrap)
    {
        _bootstrap = bootstrap;
        WireEvents();
    }

    public async Task CheckAndUpdateAsync()
    {
        var package = new UpdatePackageInfo
        {
            Version = "2.0.0",
            DownloadUrl = "https://example.com/app-v2.apk",
            Sha256 = "expected-sha256"
        };
    }
}
```

```

var options = new UpdateOptions
{
    CurrentVersion = "1.0.0",
    InstallOptions = new AndroidInstallOptions
    {
        FileProviderAuthority = "com.myapp.fileprovider"
    }
};

var result = await _bootstrap.ExecuteUpdateAsync(package, options, CancellationToken.None);
Console.WriteLine(result.IsSuccess ? "Update succeeded!" : $"Failed: {result.Message}");
}

private void WireEvents()
{
    _bootstrap.AddListenerDownloadProgressChanged += (_, args) =>
    {
        Console.WriteLine($"Progress: {args.Statistics.ProgressPercentage:F1}%");
    };
}
}

```

场景 2：自定义 HTTP 认证

【场景说明】企业内部部分发的 APK 需要 Bearer Token 认证。

```
using GeneralUpdate.Maui.Android.Abstractions;
using GeneralUpdate.Maui.Android.Models;
using GeneralUpdate.Maui.Android.Services;

// 方式一：使用全局认证提供者
var httpOptions = new HttpDownloadOptions
{
    AuthProvider = new Services.BearerTokenAuthProvider("your-token-here")
};

var bootstrap = GeneralUpdateBootstrap.CreateDefault(
    httpClient: null,
    logger: null,
    httpOptions: httpOptions);

// 方式二：使用单包认证
var package = new UpdatePackageInfo
{
    Version = "2.0.0",
    DownloadUrl = "https://example.com/app-v2.apk",
    Sha256 = "expected-sha256",
    AuthScheme = Enums.AuthScheme.Bearer,
    AuthToken = "your-token-here"
};
```

场景 3：非容器环境下的直接使用

【场景说明】简单应用不依赖 DI 容器，直接使用静态工厂创建。

```
using GeneralUpdate.Maui.Android;
using GeneralUpdate.Maui.Android.Models;

// 直接创建
var bootstrap = GeneralUpdateBootstrap.CreateDefault();

// 配置更新选项
var options = new UpdateOptions
{
    CurrentVersion = "1.0.0",
    InstallOptions = new AndroidInstallOptions
    {
        FileProviderAuthority = "com.myapp.fileprovider"
    }
};

var package = new UpdatePackageInfo
{
    Version = "2.0.0",
    DownloadUrl = "https://example.com/app-v2.apk",
    Sha256 = "expected-sha256",
    PackageSize = 50_000_000,
    ApkFileName = "app-v2.apk"
};

// 执行更新
var result = await bootstrap.ExecuteUpdateAsync(package, options, CancellationToken.None);
```

5. 常规使用示例

5.1 快速入门示例（最简 demo）

```
using GeneralUpdate.Maui.Android;
using GeneralUpdate.Maui.Android.Models;

// 创建实例
var bootstrap = GeneralUpdateBootstrap.CreateDefault();

// 更新包信息（通常从服务端 API 获取）
var package = new UpdatePackageInfo
{
    Version = "2.0.0",
    DownloadUrl = "https://update.example.com/app-v2.0.0.apk",
    Sha256 = "a1b2c3d4e5f6..." // 服务端返回的 SHA256
};

// 更新选项
var options = new UpdateOptions
{
    CurrentVersion = "1.0.0",
    InstallOptions = new AndroidInstallOptions
    {
        FileProviderAuthority = "com.myapp.fileprovider"
    }
};

// 检查更新
var checkResult = await bootstrap.ValidateAsync(package, options, CancellationToken.None);
if (!checkResult.IsUpdateAvailable)
{
    Console.WriteLine("Already up to date.");
    return;
}

// 执行更新（下载 → 校验 → 安装）
var result = await bootstrap.ExecuteUpdateAsync(package, options, CancellationToken.None);
Console.WriteLine(result.IsSuccess ? "Update completed." : $"Update failed: {result.Message}");
```

5.2 基础参数组合示例

```
using GeneralUpdate.Maui.Android;
using GeneralUpdate.Maui.Android.Enums;
using GeneralUpdate.Maui.Android.Models;

var httpOptions = new HttpDownloadOptions
{
    DownloadTimeout = TimeSpan.FromMinutes(30),
    AuthProvider = new GeneralUpdate.Maui.Android.Services.BearerTokenAuthProvider("token")
};

var bootstrap = GeneralUpdateBootstrap.CreateDefault(
    httpClient: null,
    logger: null,
    httpOptions: httpOptions);

// 事件监听
bootstrap.AddListenerDownloadProgressChanged += (_, e) =>
{
    var s = e.Statistics;
    Console.WriteLine($"Progress: {s.ProgressPercentage:F1}% | " +
        $"{FormatSize(s.DownloadedBytes)}/{FormatSize(s.TotalBytes)} | " +
        $"{FormatSpeed(s.BytesPerSecond)}");
};

bootstrap.AddListenerUpdateCompleted += (_, e) =>
{
    Console.WriteLine($"Stage completed: {e.Stage}");
};

bootstrap.AddListenerUpdateFailed += (_, e) =>
{
    Console.WriteLine($"Failed: {e.Reason} - {e.Message}");
};

var options = new UpdateOptions
{
    CurrentVersion = "1.0.0",
    TemporaryFileExtension = ".downloading",
    DeleteCorruptedPackageOnFailure = true,
    ProgressReportInterval = TimeSpan.FromMilliseconds(250),
    InstallOptions = new AndroidInstallOptions
    {
```



```

        FileProviderAuthority = "com.myapp.fileprovider"
    }
};

var package = new UpdatePackageInfo
{
    Version = "2.0.0",
    DownloadUrl = "https://cdn.example.com/app-v2.0.0.apk",
    Sha256 = "expected-sha256",
    PackageSize = 50_000_000,
    ApkFileName = "app-v2.0.0.apk",
    ForceUpdate = false,
    ReleaseNotes = "Bug fixes and performance improvements"
};

var result = await bootstrap.ExecuteUpdateAsync(package, options, CancellationToken.None);

if (!result.IsSuccess)
{
    Console.WriteLine($"Update failed: {result.FailureReason}");
}

// 辅助方法
static string FormatSize(long bytes) => bytes switch
{
    < 1024 => $"{bytes} B",
    < 1048576 => $"{bytes / 1024.0:F1} KB",
    _ => $"{bytes / 1048576.0:F1} MB"
};

static string FormatSpeed(double bytesPerSecond) =>
    $"{bytesPerSecond / 1048576.0:F1} MB/s";

```

5.3 真实业务落地示例

完整 MAUI Android 更新工作流，包含服务端 API 对接、事件处理和错误分类：

```

using GeneralUpdate.Maui.Android;
using GeneralUpdate.Maui.Android.Abstractions;
using GeneralUpdate.Maui.Android.Enums;
using GeneralUpdate.Maui.Android.Models;
using System.Net.Http.Json;

public sealed class MauiUpdateService : IDisposable
{
    private readonly HttpClient _httpClient;
    private IAndroidBootstrap? _bootstrap;
    private readonly string _fileProviderAuthority;
    private bool _disposed;

    public MauiUpdateService(HttpClient httpClient, string fileProviderAuthority)
    {
        _httpClient = httpClient;
        _fileProviderAuthority = fileProviderAuthority;
    }

    // 初始化（接收 HttpDownloadOptions 实现可选认证）
    public void Initialize(HttpDownloadOptions? httpOptions = null)
    {
        _bootstrap = GeneralUpdateBootstrap.CreateDefault(
            httpClient: null,
            logger: null,
            httpOptions: httpOptions);
        WireEvents();
    }

    // 从服务端获取更新包信息并执行更新
    public async Task CheckAndUpdateAsync(
        string serverUrl, string currentVersion,
        CancellationToken ct = default)
    {
        if (_bootstrap == null)
            throw new InvalidOperationException("Call Initialize() first.");

        try
        {
            // 1. 从服务端获取更新包信息
            var package = await FetchPackageFromServerAsync(serverUrl, currentVersion, ct);
            if (package == null)
                return UpdateResult.NoUpdate();
        }
    }
}

```

```

// 2. 检查版本
var options = new UpdateOptions
{
    CurrentVersion = currentVersion,
    InstallOptions = new AndroidInstallOptions
    {
        FileProviderAuthority = _fileProviderAuthority
    }
};

var checkResult = await _bootstrap.ValidateAsync(package, options, ct);
if (!checkResult.IsUpdateAvailable)
    return UpdateResult.NoUpdate();

// 3. 执行更新
var execResult = await _bootstrap.ExecuteUpdateAsync(package, options, ct);

return execResult.IsSuccess
    ? UpdateResult.Success()
    : UpdateResult.Failure(execResult.FailureReason, execResult.Message);
}
catch (OperationCanceledException)
{
    return UpdateResult.Canceled();
}
catch (HttpRequestException ex)
{
    return UpdateResult.Failure(UpdateFailureReason.Network, ex.Message);
}
}

private async Task<UpdatePackageInfo?> FetchPackageFromServerAsync(
    string serverUrl, string currentVersion, CancellationToken ct)
{
    // 调用服务端版本检查 API
    var request = new
    {
        Version = currentVersion,
        AppType = 1,
        Platform = 4,
        ProductId = "your-product-id"
    };

    var response = await _httpClient.PostAsJsonAsync(

```

```

        $"{serverUrl}/Upgrade/Verification", request, ct);

response.EnsureSuccessStatusCode();

// 解析响应体
var body = await response.Content.ReadFromJsonAsync(ct);
// ... 映射到 UpdatePackageInfo

return null; // 实际开发中替换为真实映射逻辑
}

private void WireEvents()
{
    if (_bootstrap == null) return;

    _bootstrap.AddListenerDownloadProgressChanged += (_, e) =>
    {
        var s = e.Statistics;
        OnProgressChanged?.Invoke(this,
            (s.ProgressPercentage, s.BytesPerSecond));
    };

    _bootstrap.AddListenerUpdateFailed += (_, e) =>
    {
        OnStatusChanged?.Invoke(this,
            $"Failed [{e.Reason}]: {e.Message}");
    };

    _bootstrap.AddListenerUpdateCompleted += (_, e) =>
    {
        OnStatusChanged?.Invoke(this, $"Stage: {e.Stage}");
    };
}

// 公开事件
public event EventHandler<(double Percentage, double Speed)>? OnProgressChanged;
public event EventHandler<string>? OnStatusChanged;

public void Dispose()
{
    if (_disposed) return;
    _bootstrap?.Dispose();
    _disposed = true;
}

```

```
}

// 结果模型
public sealed record UpdateResult
{
    public bool IsSuccess { get; init; }
    public bool IsCanceled { get; init; }
    public UpdateFailureReason FailureReason { get; init; }
    public string? Message { get; init; }

    public static UpdateResult Success() => new() { IsSuccess = true };
    public static UpdateResult NoUpdate() => new() { IsSuccess = true, Message = "No update available." };
    public static UpdateResult Canceled() => new() { IsCanceled = true, Message = "Canceled." };
    public static UpdateResult Failure(UpdateFailureReason reason, string message) =>
        new() { FailureReason = reason, Message = message };
}

// 注意：此为示例结构，实际需要替换为服务端返回的格式
public sealed record ServerResponse
{
    public int Code { get; init; }
    public List? Body { get; init; }
}

public sealed record PackageEntry
{
    public string Version { get; init; } = "";
    public string Url { get; init; } = "";
    public string Hash { get; init; } = "";
    public long Size { get; init; }
}
```

6. 全局配置

DI 注册方式

```
// 方式一：使用默认 HttpClient
services.AddGeneralUpdateMauiAndroid();

// 方式二：外部传入 HttpClient（共享连接池）
services.AddGeneralUpdateMauiAndroid(myHttpClient);

// 方式三：IHttpClientFactory 注入（通过 AddHttpClient）
services.AddHttpClient<IUpdateDownloader, HttpRangeDownloader>(client =>
{
    client.Timeout = TimeSpan.FromMinutes(10);
});
services.AddSingleton<IAndroidBootstrap, AndroidBootstrap>();
// 其他服务同理...
```

认证方案

MAUI 组件支持四种认证方案，可在 `UpdatePackageInfo` 单包粒度配置：

认证方案	AuthScheme	所需字段
HMAC-SHA256	Hmac	AuthSecretKey
Bearer Token	Bearer	AuthToken
API Key	ApiKey	AuthToken
HTTP Basic	Basic	BasicUsername + BasicPassword

SSL 策略

策略	类名	使用场景
系统默认（推荐）	null	生产环境
允许所有（仅开发）	AllowAllSslValidationPolicy	自签名证书开发环境
自定义	实现 ISslValidationPolicy	私有 CA

AndroidManifest 配置

```
<application>
  <provider
    android:name="androidx.core.content.FileProvider"
    android:authorities="${applicationId}.fileprovider"
    android:exported="false"
    android:grantUriPermissions="true">
    <meta-data
      android:name="android.support.FILE_PROVIDER_PATHS"
      android:resource="@xml/file_paths" />
  </provider>
</application>
```

file_paths.xml （ Platforms/Android/Resources/xml/file_paths.xml ）：

```
<?xml version="1.0" encoding="utf-8"?>
<paths>
  <cache-path name="cache" path="update/" ></cache-path>
</paths>
```

平台差异

项目	说明
最低 API	21 (Android 5.0)
安装权限	Android 8.0+ 需要 <code>CanRequestPackageInstalls()</code> 检查
FileProvider	必须通过 FileProvider 向系统安装器传递 APK 文件 URI

相关资源

- [GeneralUpdate.Maui 仓库](#)
- [MAUI Android 更新示例](#)
- [FileProvider 文档](#)

GeneralUpdate.Core — 执行流程详解

目标读者： 第一次接触 GeneralUpdate.Core 的开发者

阅读完你将理解：

- 更新系统的整体架构和分层设计
 - 从 App 启动到更新完成的完整执行链路
 - Client 和 Upgrade 两个进程各自的职责
 - Chain（差分）和 Full（全量）包的选择逻辑和执行差异
 - 中间件管道的设计和工作原理
 - Chain 失败时自动 Full 回退的容错机制
 - IPC 通信如何跨进程传递更新任务
 - Silent Mode 延迟更新的设计意图
-

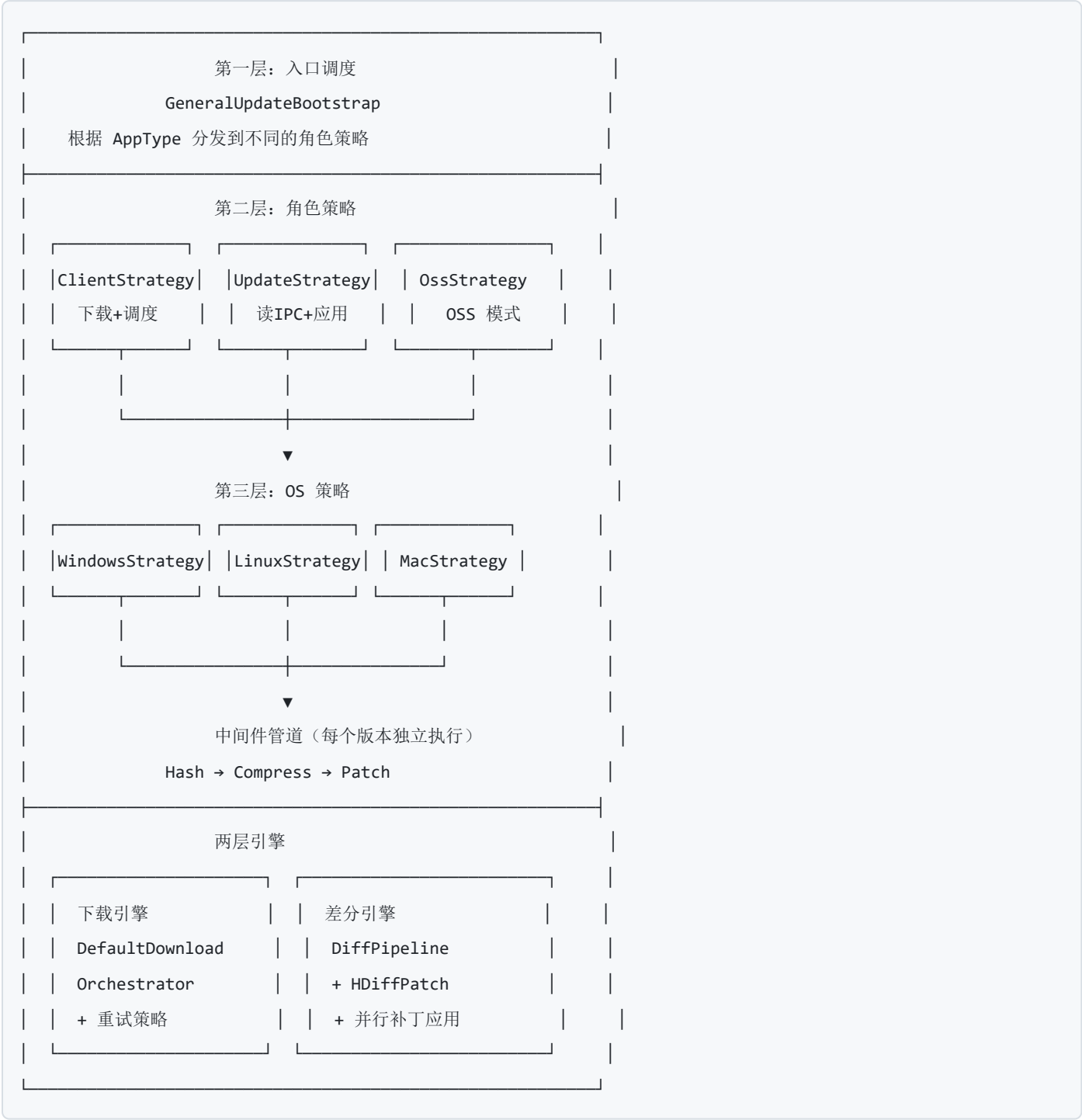
目录

1. 架构总览
 2. 入口：Bootstrap 的双重身份设计
 3. ClientStrategy：完整的更新流程
 4. DownloadPlanBuilder：下载计划与 Chain/Full 决策
 5. 下载引擎：DefaultDownloadOrchestrator
 6. 中间件管道：Hash → Compress → Patch
 7. DiffPipeline：差分引擎内部揭秘
 8. Chain→Full 回退机制
 9. IPC 通信协议
 10. UpdateStrategy：Upgrade 进程的执行流程
 11. OSS 模式：对象存储更新执行流程
 12. Silent Mode：延迟升级机制
 13. OS 策略的平台差异
 14. 错误恢复全景
 15. 关键代码路径索引
-

1. 架构总览

1.1 三层架构

GeneralUpdate.Core 采用**三层调度 + 两层引擎**的设计：



1.2 核心设计原则

原则	说明
Client 统一下载	所有包（Client + Upgrade + Chain + Full 的 ZIP）都在 Client 进程中一次性下载到 %TEMP%/main_temp/
Upgrade 只应用	Upgrade 进程不做任何网络请求，通过加密 IPC 文件接收版本信息，只运行中间件管道
链式回退	Chain（差分）包应用失败时，自动用已预下载的 Full 包原地重试，不需要第二次服务端请求
中间件管道	每个版本独立走一次 Hash → Compress → Patch 管道，单一职责、可测试、可替换

1.3 两种包类型

类型	PackageType	内容	应用方式
Chain (差分)	1	.patch 二进制差分文件 + 新增文件 + 删除清单	解压到临时 PatchPath → 安装在目录的旧文件 + .patch 通过 HDiffPatch 合成新文件
Full (全量)	2	完整的应用文件	直接解压覆盖安装目录，跳过 PatchMiddleware

1.4 两种进程角色

进程	AppType	入口 Strategy	职责
主程序 (如 MyApp.exe)	Client	ClientStrategy	服务端版本校验、一次性下载所有包、原地升级自身 (Upgrade 包)、写 IPC 文件、拉起 Upgrade 进程、退出
升级程序 (如 Updater.exe)	Upgrade	UpdateStrategy	读 IPC 文件获取版本信息、运行管道升级主程序文件、写回 manifest、拉起主程序、退出
主程序 (OSS 模式)	OssClient	OssStrategy	下载 versions.json、SemVer 对比、拉起升级程序、退出 (无 IPC)
升级程序 (OSS 模式)	OssUpgrade	OssStrategy	读 versions.json、下载并解压更新包、写回 manifest、拉起主程序、退出 (无 IPC)

2. 入口：Bootstrap 的双重身份设计

GeneralUpdateBootstrap 是整个更新库的入口点。它的构造函数在任何其他方法调用之前就会尝试读取 IPC 文件。

2.1 构造函数即 IPC 探测

```
public GeneralUpdateBootstrap()
{
    InitializeFromEnvironment(); // 读取加密 IPC 文件
}
```

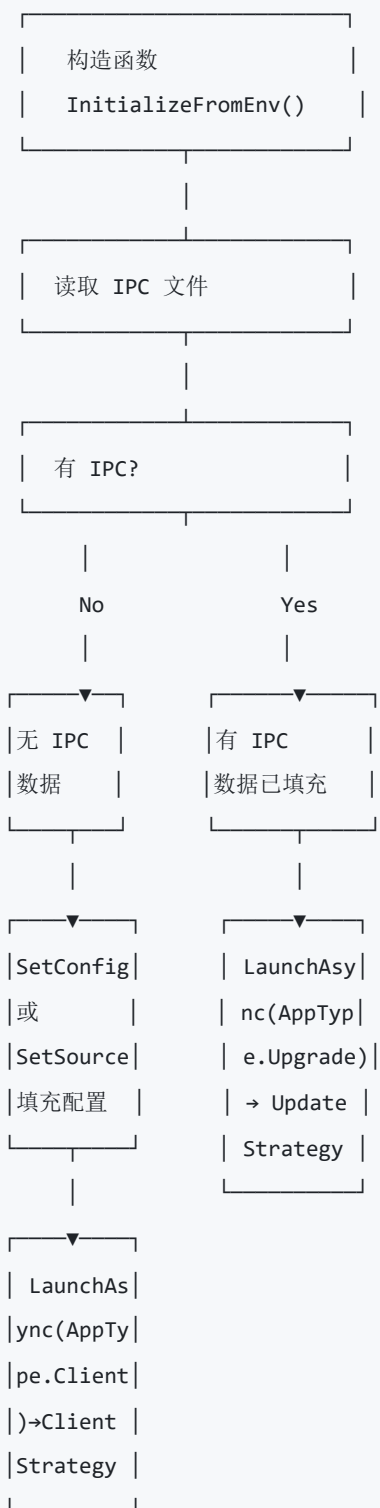
InitializeFromEnvironment() 做的事情：

```
void InitializeFromEnvironment()
{
    var provider = new EncryptedFileProcessContractProvider();
    var contract = provider.Receive(); // 读 %TEMP%/GeneralUpdate/ipc/process_info.enc

    if (contract == null) return; // 没有 IPC 文件 → 这不是一个 Upgrade 进程

    // 读到 IPC 文件 → 说明是 Upgrade 进程
    // 用 IPC 中的信息填充内部配置
    _configInfo.UpdateAppName = contract.AppName;
    _configInfo.InstallPath = contract.InstallPath;
    _configInfo.UpdateVersions = contract.UpdateVersions;
    // ... 其他字段
}
```

2.2 LaunchAsync 分发



2.3 关键理解

同一个 `GeneralUpdateBootstrap` 类型，在两种进程中有完全不同的执行路径：

Client 进程:

```
new GeneralUpdateBootstrap()  
  → InitializeFromEnvironment() → 没有 IPC 文件 → _configInfo 为空  
.SetConfig(request)           → _configInfo 从用户代码设置  
.LaunchAsync(AppType.Client)   → ClientStrategy
```

Upgrade 进程:

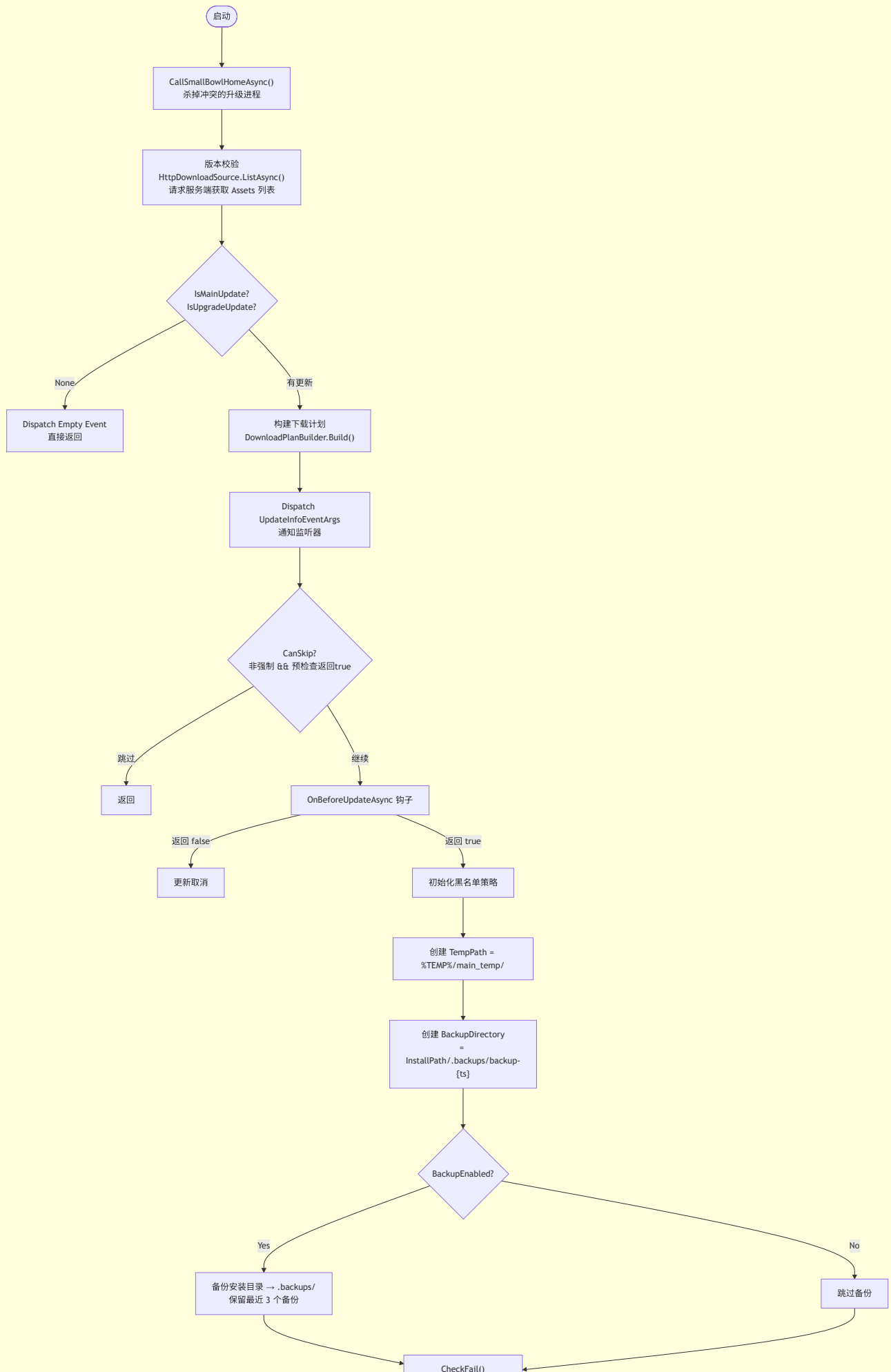
```
new GeneralUpdateBootstrap()  
  → InitializeFromEnvironment() → 读到 IPC 文件 → _configInfo 已填充  
.LaunchAsync(AppType.Upgrade) → UpdateStrategy (直接用构造函数中填充的配置)
```

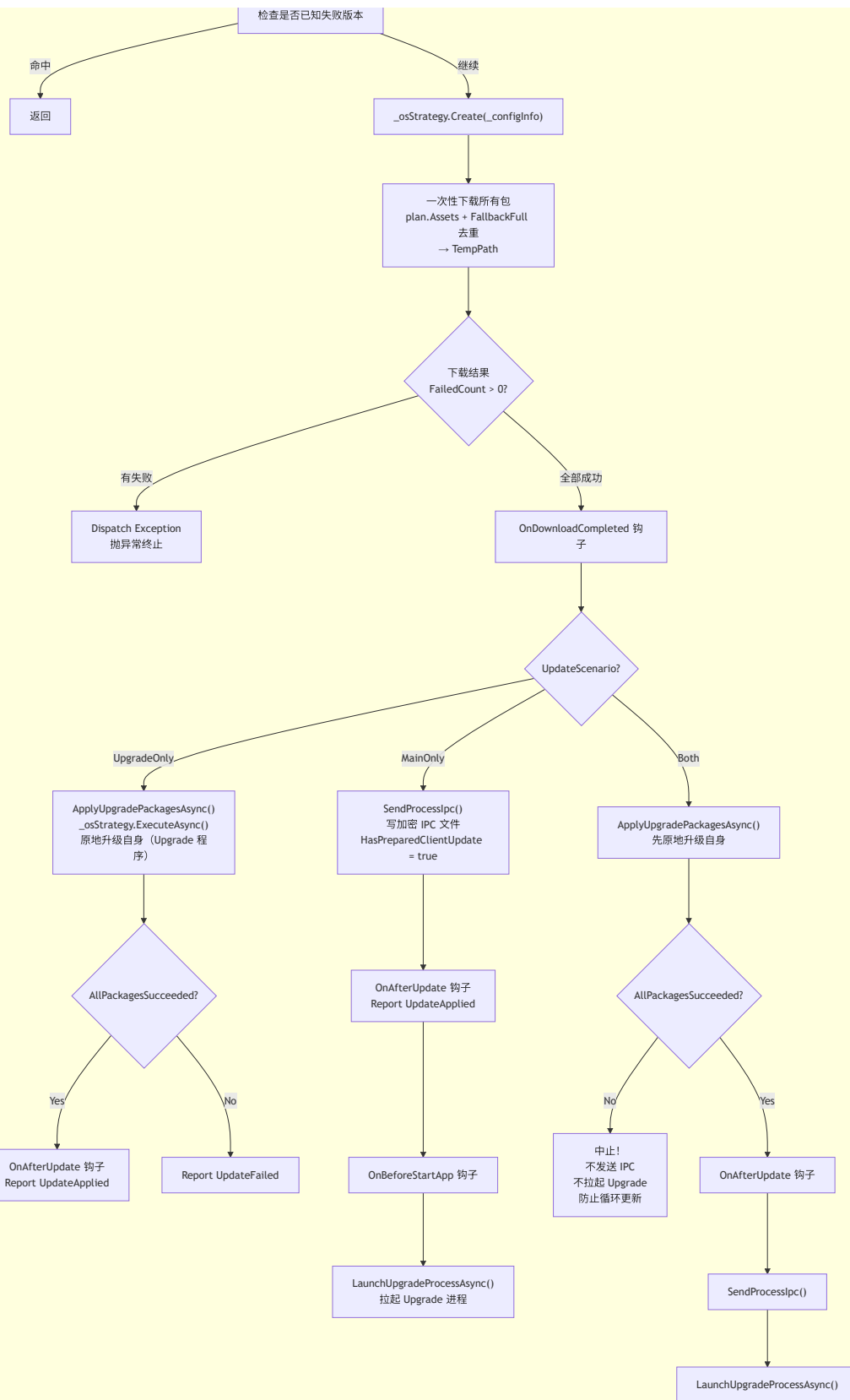
这意味着 Upgrade 进程不需要任何命令行参数或配置文件。所有信息通过 IPC 文件传递。

3. ClientStrategy: 完整的更新流程

ClientStrategy 是最复杂的角色策略, 负责从版本校验到拉起升级进程的完整链路。

3.1 全流程总图





3.2 步骤详解

Step 1: 清理冲突进程 (CallSmallBowlHomeAsync)

```

// ClientStrategy.cs:985-1002
// 在开始更新前，杀掉正在运行的升级进程 (Bowl)
// 防止它们持有文件锁，导致后续备份或替换失败

```



```
private async Task CallSmallBowlHomeAsync(string processName)
{
    var processes = Process.GetProcessesByName(processName);
    foreach (var process in processes)
        await GracefulExit.ShutdownAsync(process);
}
```

Step 2: 版本校验

```
// 构造下载源（默认 HTTP）
var downloadSource = new HttpDownloadSource(
    _configInfo.UpdateUrl,      // 服务端 URL
    _configInfo.ClientVersion,  // 主程序当前版本
    _configInfo.UpgradeClientVersion, // 升级程序当前版本
    _configInfo.AppSecretKey,   // 应用密钥
    GetPlatform(),              // 平台类型
    // ... 其他参数
);

// 请求服务端获取可用更新
var sourceResult = await downloadSource.ListAsync();
```

服务端返回 `List`，每个 Asset 包含：

- `Name`：包名，也是 ZIP 文件名（如 `1.0.1`）
- `Version`：版本号
- `Url`：下载地址
- `SHA256`：哈希校验值
- `Size`：文件大小
- `PackageType`：1=Chain, 2=Full
- `AppType`：1=Client, 2=Upgrade
- `IsFreeze`：是否冻结（冻结包不参与更新）
- `IsForcibly`：是否强制更新
- `MinClientVersion`：最低兼容版本
- `FallbackFullName/Url/Hash/Version`：对应的回退 full 包信息

Step 3: 场景判定

```
var scenario = (_configInfo.IsMainUpdate, _configInfo.IsUpgradeUpdate) switch
{
    (false, false) => UpdateScenario.None,          // 不需要更新
    (false, true)  => UpdateScenario.UpgradeOnly,    // 只需更新升级程序
    (true, false)  => UpdateScenario.MainOnly,       // 只需更新主程序
    (true, true)   => UpdateScenario.Both,          // 两者都要更新
};
```

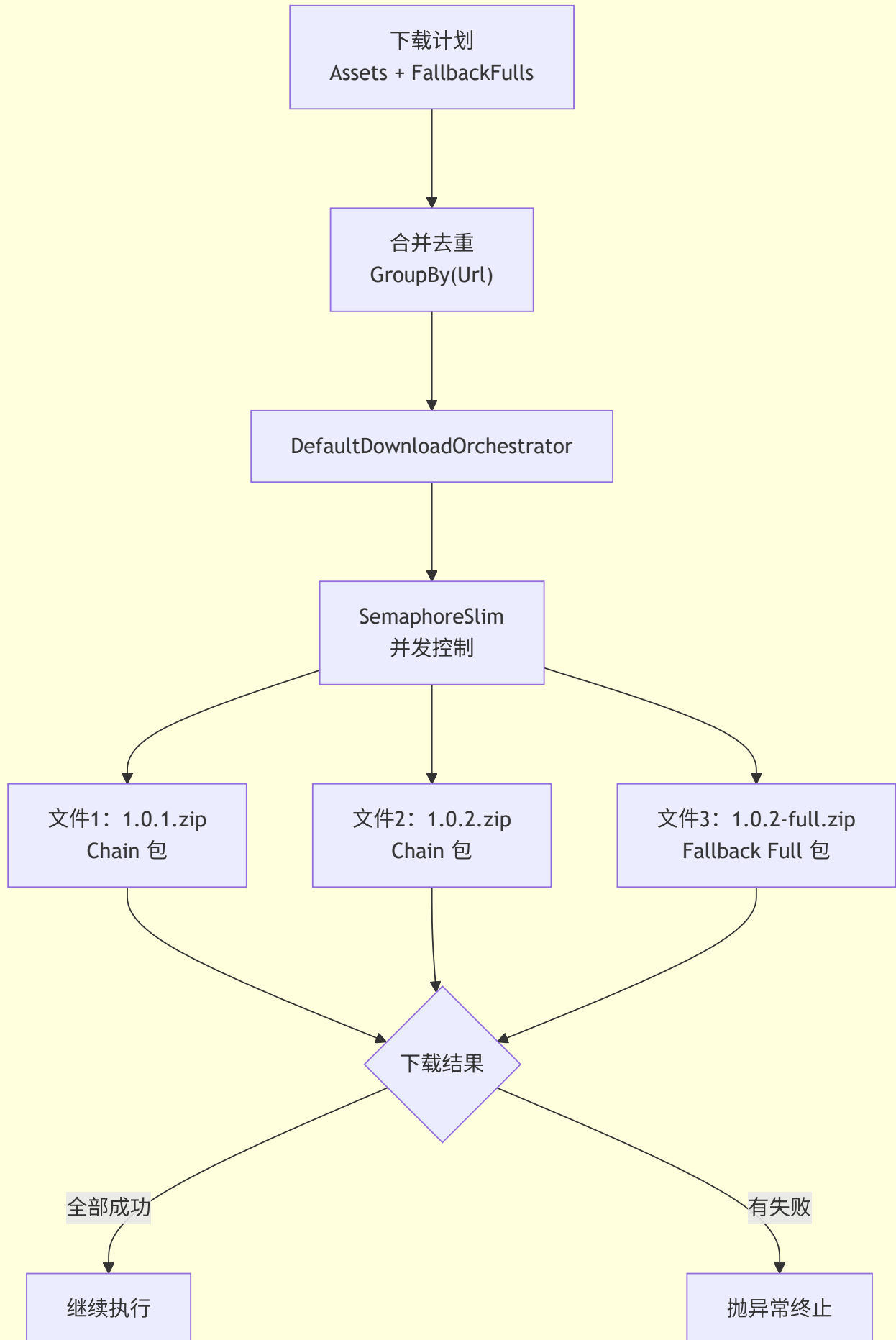
Step 4: 备份

```
// 备份安装目录到 .backups/
StorageManager.Backup(_configInfo.InstallPath, _configInfo.BackupDirectory, blacklist);
// 清理旧备份，只保留最近 3 个
StorageManager.CleanBackup(_configInfo.InstallPath, keepVersions: 3);
```

黑名单配置默认排除：.backups、.git、.svn、bin、obj、node_modules 等目录。

Step 5: 一次性下载所有包

下载阶段



```
// ClientStrategy.cs:589-597
// 合并链包和回退包，去重后一次性下载
var allAssets = plan.Assets.Concat(plan.FallbackFulls)
    .GroupBy(a => a.Url)
    .Select(g => g.First())
    .ToList();

var mergedPlan = new DownloadPlan(allAssets, plan.IsForcibly);
var downloadReport = await ExecuteDownloadAsync(mergedPlan);

if (downloadReport.FailedCount > 0)
    throw new InvalidOperationException("下载失败！");
```

关键点：Chain 包和对应的 Fallback Full 包**同时下载**，不存在"先下 chain 失败再下 full"的两阶段重试。这样设计是为了更好的用户体验——下载失败可以立即重试整个批次，而不是一次一次试。

Step 6: 场景分发

三种场景的执行路径：

UpgradeOnly

ApplyUpgradePackagesAsync()

↓

_osStrategy.Create(_configInfo)

↓

_osStrategy.ExecuteAsync()

↓

OS 策略对每个 upgrade 版本: Hash → Compress → Patch

↓

AllPackagesSucceeded → WriteBackUpgradeVersion()

MainOnly

SendProcessIpc()

↓

写 AES 加密 IPC 文件到 %TEMP%/GeneralUpdate/ipc/process_info.enc

↓

LaunchUpgradeProcessAsync()

↓

拉起 Upgrade 进程 → 当前 Client 进程继续运行或退出

Both

ApplyUpgradePackagesAsync() — 先升级自身

↓ 失败则中止

SendProcessIpc() — 再写 IPC

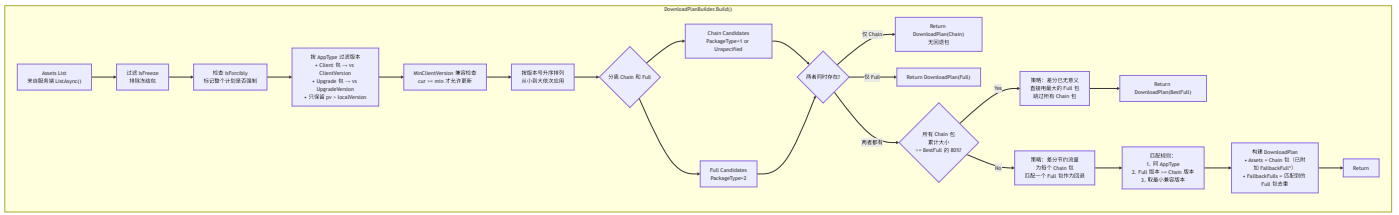
↓

LaunchUpgradeProcessAsync() — 最后拉起 Upgrade 进程

4. DownloadPlanBuilder: 下载计划与 Chain/Full 决策

`DownloadPlanBuilder` 是一个静态工具类, 负责从服务端返回的 Asset 列表中构建出最终要下载哪些包。

4.1 完整决策流程



4.2 换包阈值逻辑详解

80% 閾值的核心代码:

```
// DownloadPlanBuilder.cs:174-196
if (chainCandidates.Count > 0 && fullCandidates.Count > 0)
{
    // 取版本最大的 Full 包作为参考
    var bestFull = fullCandidates.OrderByDescending(v => v).First();

    // 计算同 AppType 的 Chain 包总大小
    long chainTotal = chainCandidates
        .Where(a => a.AppType == bestFull.AppType)
        .Sum(a => a.Size);

    // 如果 Chain 总和 >= Full 的 80%
    if (chainTotal >= (long)(bestFull.Size * 0.8))
    {
        // 直接用 Full 包，下载量更少、更可靠
        return new DownloadPlan(new[] { bestFull }, isForcibly);
    }
}
```

为什么要有这个阈值?

场景	Chain 大小	Full 大小	比例	决策	原因
只改了少量代码	1 MB	50 MB	2%	Chain	差分节省 98% 流量
大量文件变更	42 MB	50 MB	84%	Full	差分只省 16%，不值得增加复杂度和风险
新增较多文件	35 MB	50 MB	70%	Chain	省 30%，值得

4.3 回退包匹配规则

当 Chain 总大小 < 80% Full 大小时, 走 Chain + FallbackFull 模式:

```
// DownloadPlanBuilder.cs:204-237
var chainWithFallback = chainCandidates.Select(chain =>
{
    // 找到同 AppType、版本 >= chain 版本的最小 Full 包
    var match = fullCandidates
        .Where(f => f.AppType == chain.AppType)
        .OrderBy(f => f.Version)           // 最小版本优先
        .FirstOrDefault(f => f.Version >= chain.Version);

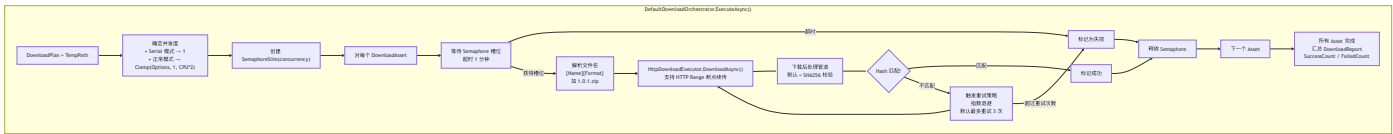
    if (match != null)
    {
        // 给这个 Chain 包附上 FallbackFull 元信息
        return chain with
        {
            FallbackFullName      = match.Name,
            FallbackFullUrl       = match.Url,
            FallbackFullHash      = match.SHA256,
            FallbackFullVersion   = match.Version
        };
    }
    return chain; // 没有匹配的 Full 包，那就没有回退能力
});
```

5. 下载引擎：DefaultDownloadOrchestrator

5.1 架构



5.2 执行逻辑



5.3 关键设计点

断点续传： 使用 HTTP Range 请求头，如果下载中断，只下载缺失的部分。

重试策略：

- 默认最多重试 3 次
- 间隔 = `RetryInterval * 2^(N-1)` （指数退避）
- 默认初始间隔 1 秒

并发度控制：

- `DiffMode.Serial` → 强制串行
- 正常模式 → `Clamp(MaxConcurrency, 1, Environment.ProcessorCount * 2)`

6. 中间件管道：Hash → Compress → Patch

中间件管道是 GeneralUpdate.Core 的核心处理模式。每个版本独立走一次完整的管道。

6.1 管道构建

管道由 OS 策略构建：

```
// WindowsStrategy.cs
protected override PipelineBuilder BuildPipeline(PipelineContext context)
{
    var needsPatch = context.Get<bool>("PatchEnabled")
        && context.Get<int>("PackageType") != (int)PackageType.Full;

    return new PipelineBuilder(context)
        .UseMiddleware() // 1. 完整性校验
        .UseMiddleware() // 2. 解压
        .UseMiddlewareIf( // 3. 差分应用（仅 Chain 包）
            needsPatch: needsPatch);
}
```

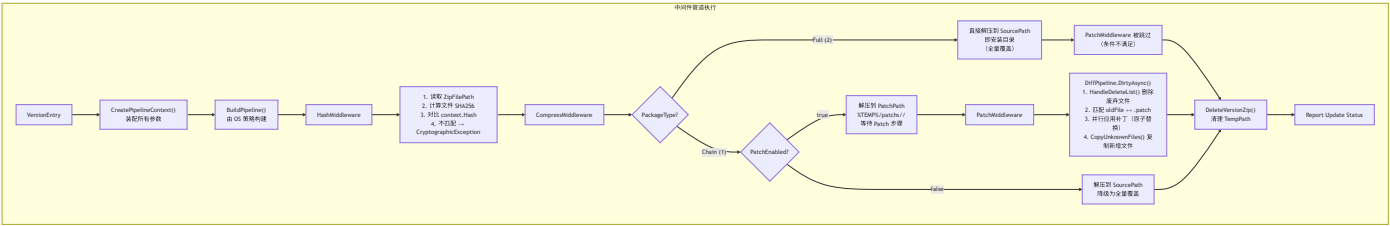
6.2 PipelineContext：中间件之间的数据契约

`PipelineContext` 是一个 `ConcurrentDictionary`，所有中间件通过它交换数据：

Key	写入者	消费者	说明
<code>ZipFilePath</code>	<code>AbstractStrategy</code>	<code>HashMiddleware</code> , <code>CompressMiddleware</code>	ZIP 完整路径 <code>TempPath/{name}.zip</code>
<code>Hash</code>	<code>AbstractStrategy</code>	<code>HashMiddleware</code>	预期的 SHA256

Key	写入者	消费者	说明
Format	AbstractStrategy	CompressMiddleware	压缩格式（当前仅 Zip）
Encoding	AbstractStrategy	CompressMiddleware	文件编码
SourcePath	AbstractStrategy	CompressMiddleware, PatchMiddleware	安装目录
PatchPath	AbstractStrategy	CompressMiddleware, PatchMiddleware	Chain 包临时解压目录
PatchEnabled	AbstractStrategy	OS 策略（BuildPipeline 时）	是否启用差分
PackageType	AbstractStrategy	OS 策略, CompressMiddleware	1=Chain, 2=Full
DiffPipeline	AbstractStrategy	PatchMiddleware	并行差分引擎

6.3 Chain 和 Full 的执行差异



6.4 各中间件详细实现

HashMiddleware

```
public async Task InvokeAsync(PipelineContext context)
{
    var zipPath = context.Get<string>("ZipFilePath");
    var expectedHash = context.Get<string>("Hash");

    var actualHash = ComputeSHA256(zipPath); // 计算 ZIP 文件 SHA256

    if (!string.Equals(actualHash, expectedHash, StringComparison.OrdinalIgnoreCase))
        throw new CryptographicException("文件哈希不匹配！");
}
```

CompressMiddleware

```
public async Task InvokeAsync(PipelineContext context)
{
    var packageType = context.Get<int>("PackageType");
    var zipPath = context.Get<string>("ZipFilePath");
    var sourcePath = context.Get<string>("SourcePath");
    var patchPath = context.Get<string>("PatchPath");
    var patchEnabled = context.Get<bool>("PatchEnabled");
    var format = context.Get("Format");
    var encoding = context.Get("Encoding");

    if (packageType == (int)PackageType.Full)
    {
        // Full 包 → 直接解压到安装目录
        CompressProvider.Decompress(zipPath, sourcePath, format, encoding);
    }
    else if (patchEnabled)
    {
        // Chain 包 + 启用补丁 → 解压到 PatchPath
        CompressProvider.Decompress(zipPath, patchPath, format, encoding);
    }
    else
    {
        // Chain 包 + 禁用补丁 → 降级为全量解压
        CompressProvider.Decompress(zipPath, sourcePath, format, encoding);
    }
}
```

PatchMiddleware

```
public async Task InvokeAsync(PipelineContext context)
{
    var sourcePath = context.Get<string>("SourcePath");
    var patchPath = context.Get<string>("PatchPath");
    var diffPipeline = context.Get("DiffPipeline");

    // 调用差分引擎应用补丁
    await diffPipeline.DirtyAsync(sourcePath, patchPath,
        progress: new DiffProgressReporter(this),
        cancellationToken: CancellationToken.None);
}
```

6.5 管道执行的主循环

OS 策略（AbstractStrategy）负责遍历所有版本并运行管道：

```
// AbstractStrategy.cs:149-287
public async Task ExecuteAsync()
{
    foreach (var version in _configinfo.UpdateVersions)
    {
        var context = CreatePipelineContext(version, patchPath);
        var pipelineBuilder = BuildPipeline(context);
        await pipelineBuilder.Build(); // 跑中间件管道
        DeleteVersionZip(version);    // 删除已处理的 ZIP
    }
}
```

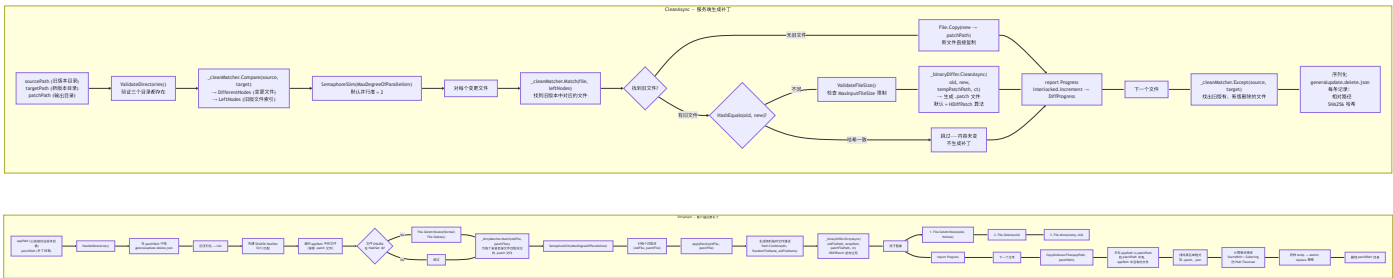
注意： 一个版本的失败不影响其他版本的处理。但 AllPackagesSucceeded 标记会告诉调用方是否有任何版本失败。

7. DiffPipeline：差分引擎内部揭秘

DiffPipeline 是 GeneralUpdate 的差分引擎，提供两个操作模式：

- **CleanAsync**：服务端用——比较新旧版本目录，生成 .patch 文件
- **DirtyAsync**：客户端用——应用 .patch 文件到已安装目录

7.1 主流程



7.2 原子替换策略 (ApplyPatch)

```
private async Task ApplyPatch(string appFilePath, string patchFilePath, CancellationToken ct)
{
    // 1. 生成随机临时文件名
    var tempPath = Path.Combine(
        Path.GetDirectoryName(appFilePath)!,
        $"{Path.GetRandomFileName()}_{Path.GetFileName(appFilePath)}");

    // 2. 应用 HDiffPatch → 写入临时文件
    await _binaryDiffer.DirtyAsync(appFilePath, tempPath, patchFilePath, ct);

    // 3. 原子替换
    if (File.Exists(tempPath))
    {
        File.SetAttributes(appFilePath, FileAttributes.Normal); // 处理只读文件
        File.Delete(appFilePath);                                // 删除原文件
        File.Move(tempPath, appFilePath);                        // 临时文件→正式位置
    }
}
```

为什么需要原子替换？

✗ 直接覆写：

写入到一半进程崩溃 → 文件半损坏 → 程序无法启动

✓ 临时文件策略：

1. 完全写入临时文件（如果崩溃，原文件完好无损）
2. 删除原文件
3. Move 临时文件到正式位置

如果第 2 步和第 3 步之间崩溃 → 文件可能丢失

但这对后续更新是安全的——下次更新会重新下载

7.3 删除文件识别策略

删除不是按文件名匹配，而是按文件内容的 SHA256 哈希匹配：

```
// 从 generalupdate.delete.json 读取被删除文件的哈希列表
// 扫描当前安装目录，计算每个文件的 SHA256
// 如果匹配 → 删除

var deleteHashes = new HashSet<string>(/* 从 JSON 读取 */);
foreach (var file in appFiles)
{
    var fileHash = hashAlgorithm.ComputeHash(file.FullName);
    if (deleteHashes.Contains(fileHash))
        File.Delete(file.FullName);
}
```

好处：即使文件被重命名，只要内容没变就不会误删。

7.4 并行控制

```
using var semaphore = new SemaphoreSlim(_options.MaxDegreeOfParallelism); // 默认=2

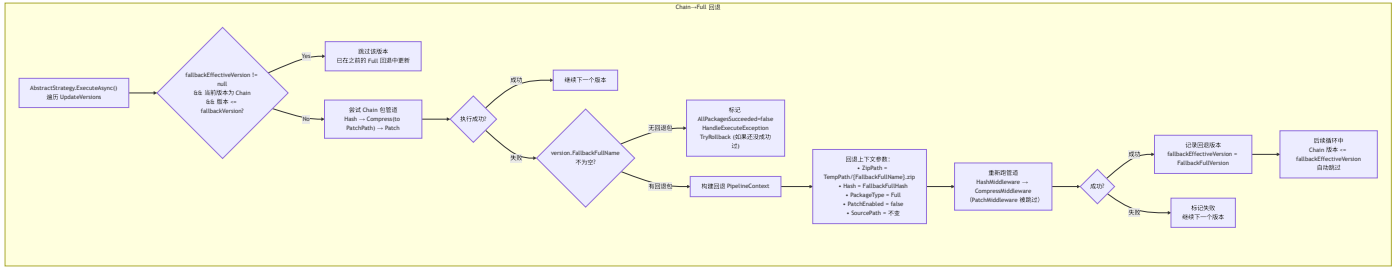
var tasks = matchedPairs.Select(pair => Task.Run(async () =>
{
    await semaphore.WaitAsync(ct);
    try
    {
        await ApplyPatch(...);
    }
    catch (Exception ex) when (!_options.StopOnFirstError)
    {
        // StopOnFirstError=false(默认) → 单个文件失败不阻断整体
        // 通过 Progress 报告错误，继续处理其他文件
    }
    finally
    {
        semaphore.Release();
    }
}));

await Task.WhenAll(tasks);
```

8. Chain→Full 回退机制

这是 GeneralUpdate.Core 的最高级容错机制，在 `AbstractStrategy.ExecuteAsync()` 中实现。

8.1 完整流程



8.2 跳过后继 Chain 包的版本跟踪

```
// AbstractStrategy.cs:168-185
SemVersion? fallbackEffectiveVersion = null;

foreach (var version in _configinfo.UpdateVersions)
{
    // 如果之前已经 Full 回退到某个版本
    // 跳过所有 <= 该版本的 Chain 包
    if (fallbackEffectiveVersion != null
        && version.PackageType == (int)PackageType.Chain
        && versionSv <= fallbackEffectiveVersion)
    {
        continue; // 已覆盖，跳过
    }

    try
    {
        await pipelineBuilder.Build(); // Chain 尝试
    }
    catch when (version.PackageType == Chain && FallbackFullName != null)
    {
        // Chain 失败 -> 用 Full 重试
        await fallbackBuilder.Build();
        fallbackEffectiveVersion = ffv; // 记录回退版本
    }
}
```

8.3 边界场景示例

场景：3 个 Chain 包，都回退到同一个 Full 包

版本列表: 1.0.1(chain), 1.0.2(chain), 1.0.3(chain)
Full 包: 1.0.3-full (作为所有 chain 的 FallbackFull)

执行:

1. 1.0.1 chain → 失败 → 回退到 1.0.3-full → fallbackEffectiveVersion = 1.0.3
2. 1.0.2 chain → 跳过 (1.0.2 <= 1.0.3)
3. 1.0.3 chain → 跳过 (1.0.3 <= 1.0.3)

场景: 部分 Chain 包失败, 每个有不同 Full 包

版本列表: 1.0.1(chain, fallback=1.0.1-full), 1.0.2(chain, fallback=1.0.2-full)

执行:

1. 1.0.1 chain → 成功 → 继续
2. 1.0.2 chain → 失败 → 回退到 1.0.2-full → fallbackEffectiveVersion = 1.0.2

场景: Chain 包成功, 不需要回退

版本列表: 1.0.1(chain), 1.0.2(chain)

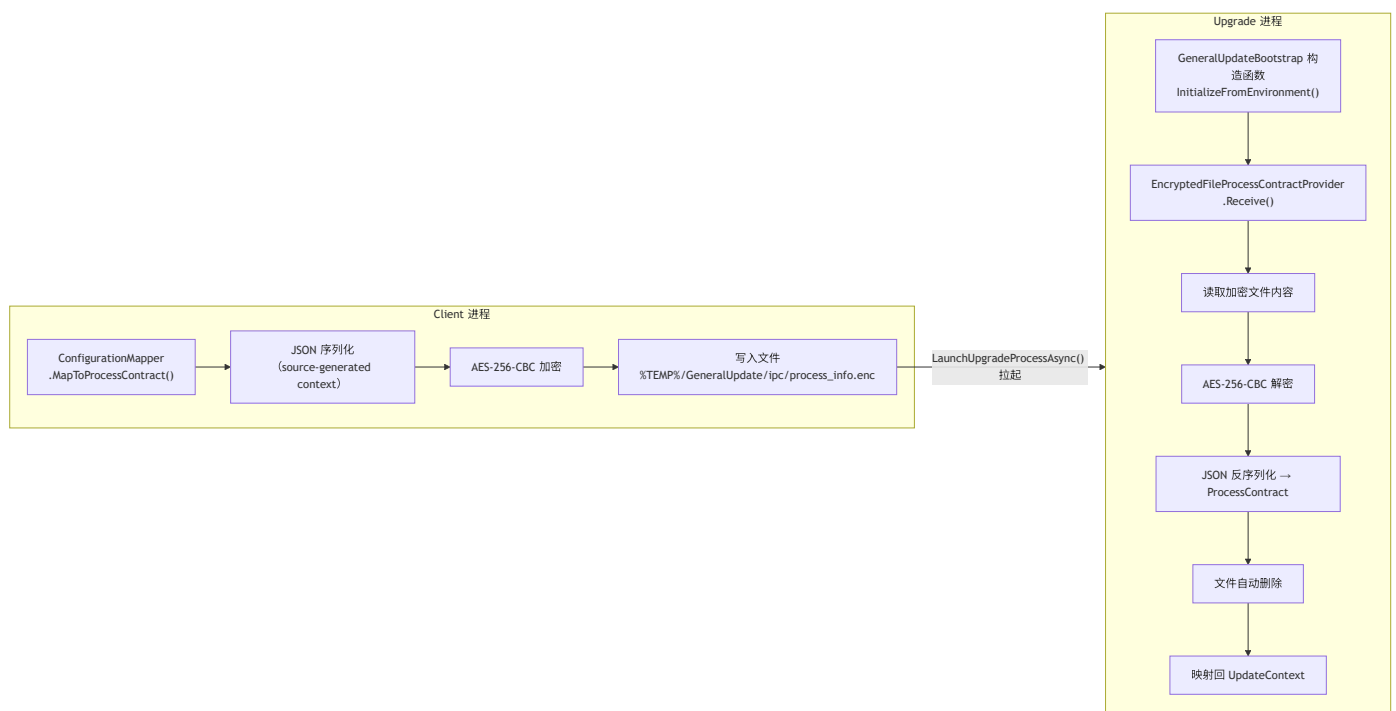
执行:

1. 1.0.1 chain → 成功
 2. 1.0.2 chain → 成功
- fallbackEffectiveVersion 始终为 null, 无需跳过

9. IPC 通信协议

IPC (Inter-Process Communication) 是 Client 进程和 Upgrade 进程之间的数据传输机制。

9.1 通信流程



9.2 加密细节

```
// IpcEncryption.cs
// 使用 AES-256-CBC 加密
// 密钥: SHA256("GeneralUpdate.ProcessContract.IPC.v1") → 32 字节
// IV: 固定 16 字节 (起始 0x47, 其余 0x00)
// 文件路径: %TEMP%/GeneralUpdate/ipc/process_info.enc

public static void EncryptToFile(byte[] plainBytes, string filePath, byte[] key, byte[] iv)
{
    using var aes = Aes.Create();
    aes.Key = key;
    aes.IV = iv;
    using var encryptor = aes.CreateEncryptor();
    var cipher = encryptor.TransformFinalBlock(plainBytes, 0, plainBytes.Length);

    // FileShare.Read 允许接收方在写入完成前开始读取
    using var fs = new FileStream(filePath, FileMode.Create, FileAccess.Write, FileShare.Read);
    fs.Write(cipher, 0, cipher.Length);
}
```

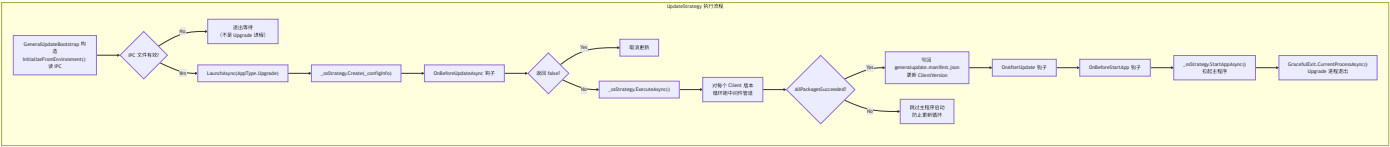
9.3 ProcessContract 字段映射

ProcessContract	来源	含义	消费方
UpdateAppName	_configInfo.MainAppName	更新后要启动的程序的名称	OS 策略的 StartAppAsync
InstallPath	_configInfo.InstallPath	安装目录	管道的 SourcePath
CurrentVersion	_configInfo.ClientVersion	当前主程序版本	日志/上报
LastVersion	_configInfo.LastVersion	目标版本	日志/上报
UpdateVersions	clientVersions 列表	要应用的包列表	管道的输入
TempPath	main_temp 目录	ZIP 文件所在目录	管道的 ZipFilePath
UpdatePath	_configInfo.UpdatePath	Upgrade 程序所在目录	路径解析
Encoding	_configInfo.Encoding	文件编码	CompressMiddleware
CompressFormat	_configInfo.Format	压缩格式	CompressMiddleware
DownloadTimeOut	_configInfo.DownloadTimeOut	下载超时	Orchestrator (预留)
BackupDirectory	_configInfo.BackupDirectory	.backups 目录	日志/上报
ReportUrl	_configInfo.ReportUrl	状态上报 URL	UpdateStrategy
LaunchClientAfterUpdate	_configInfo.LaunchClientAfterUpdate	更新后是否自动启动主程序	UpdateStrategy
ReportType	1 或 2	1=主动轮询 2=推送通知	上报区分

10. UpdateStrategy: Upgrade 进程的执行流程

UpdateStrategy 是 Upgrade 进程的策略。与 ClientStrategy 最大的区别是：它不做任何网络请求。

10.1 完整流程



10.2 为什么 Upgrade 进程不做网络请求?

```
// UpdateStrategy.cs 的核心逻辑
public async Task ExecuteAsync()
{
    // 没有调用 IDownloadSource.ListAsync()
    // 没有调用 IDownloadOrchestrator.ExecuteAsync()
    // 所有数据已经在 _configInfo 中, 来自 IPC 文件

    // 直接跑管道
    _osStrategy.Create(_configInfo);
    await _osStrategy.ExecuteAsync();

    // 检查结果
    if ((_osStrategy as AbstractStrategy)?.AllPackagesSucceeded == true)
    {
        // 写回 manifest
        ManifestInfo.TryUpdateVersion(installPath, clientVersion: latestVersion);

        // 启动主程序
        await _osStrategy.StartAppAsync();
    }

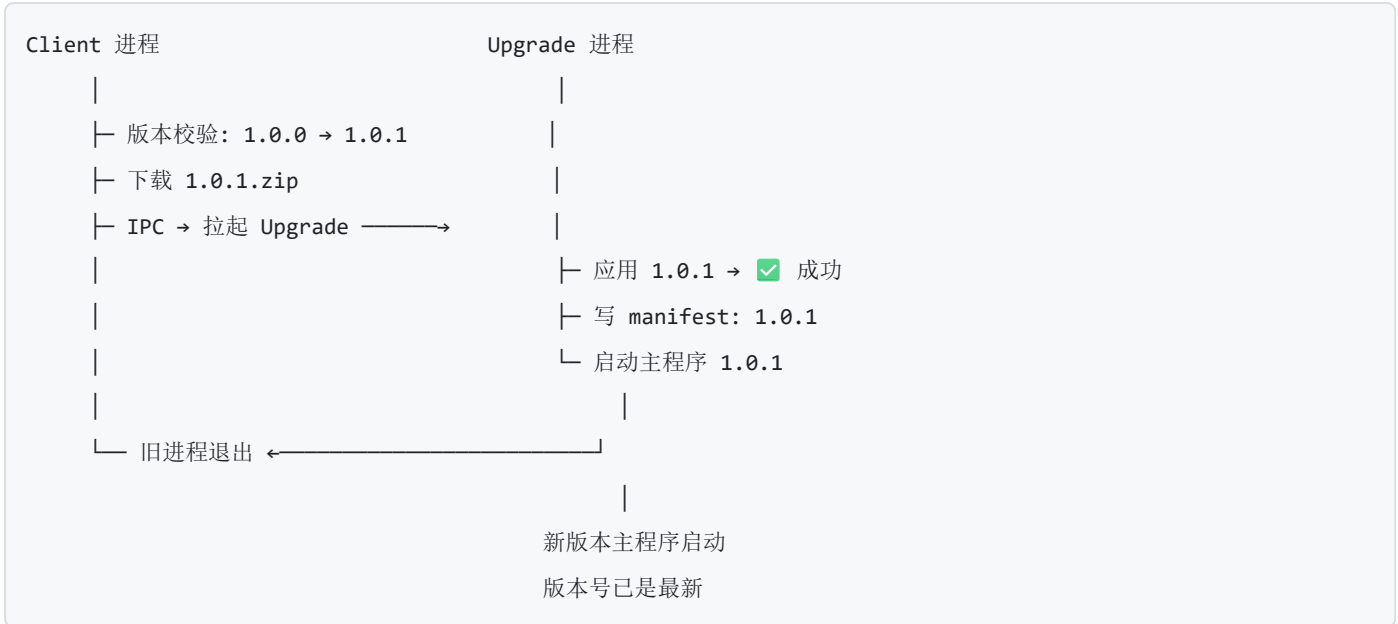
    // 如果失败 → 不写 manifest、不启动主程序
    // 下次 Client 启动时会重新检测到更新
}
```

设计意图：Upgrade 进程是一个“短暂的一次性进程”。

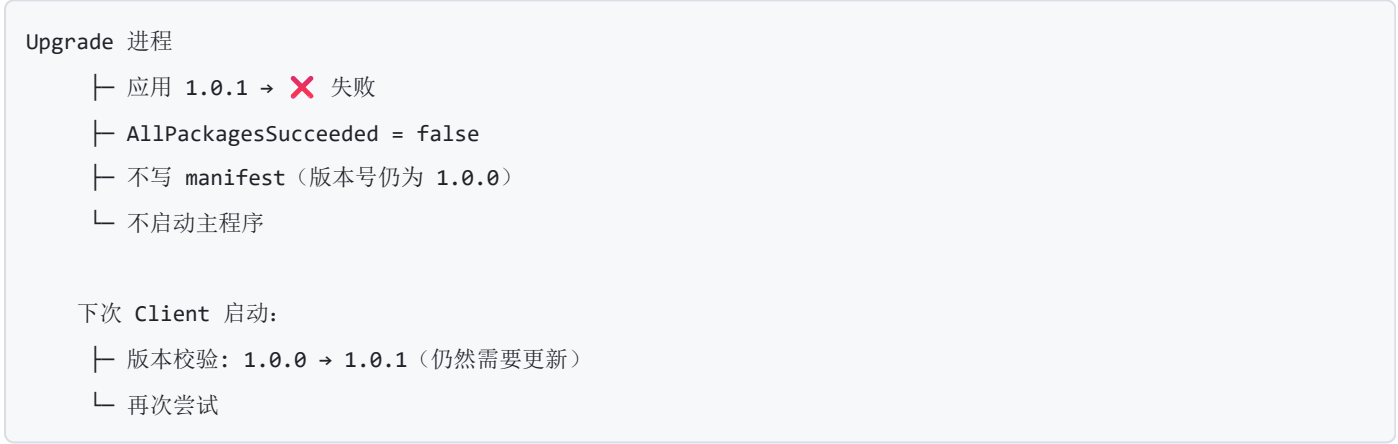
- 它不需要网络能力（减小二进制体积）
- 它不需要知道服务端地址（安全：攻击面更小）
- 它只需要读本地文件并执行文件操作

10.3 防止更新循环

AllPackagesSucceeded 是一个重要的安全闸门：



如果 Upgrade 进程失败:

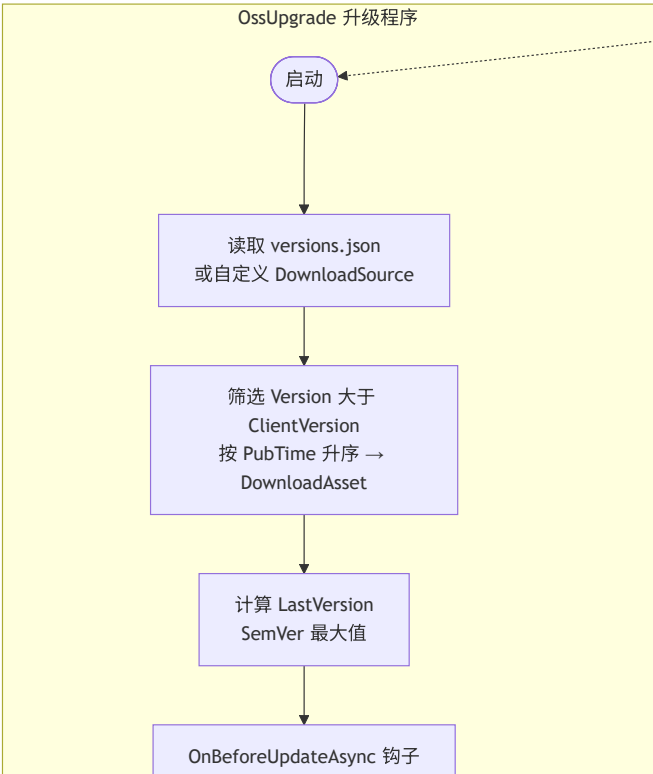
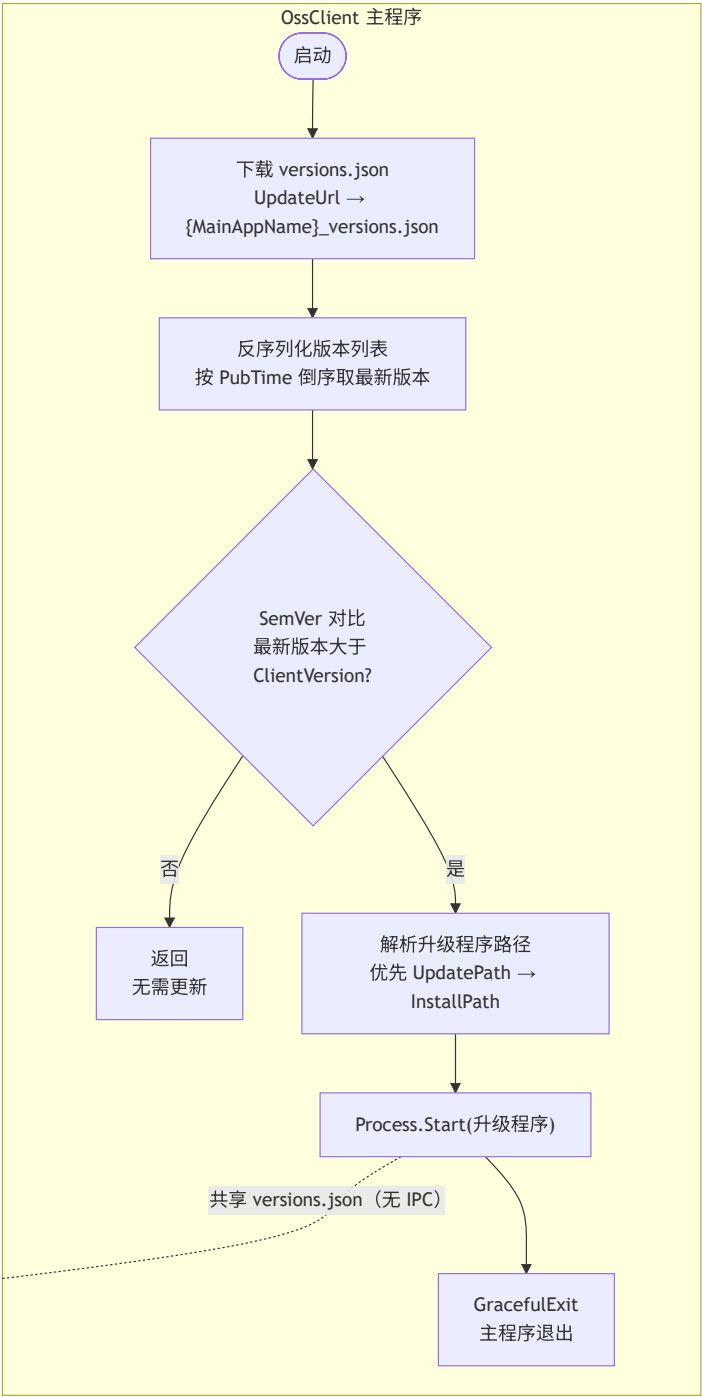


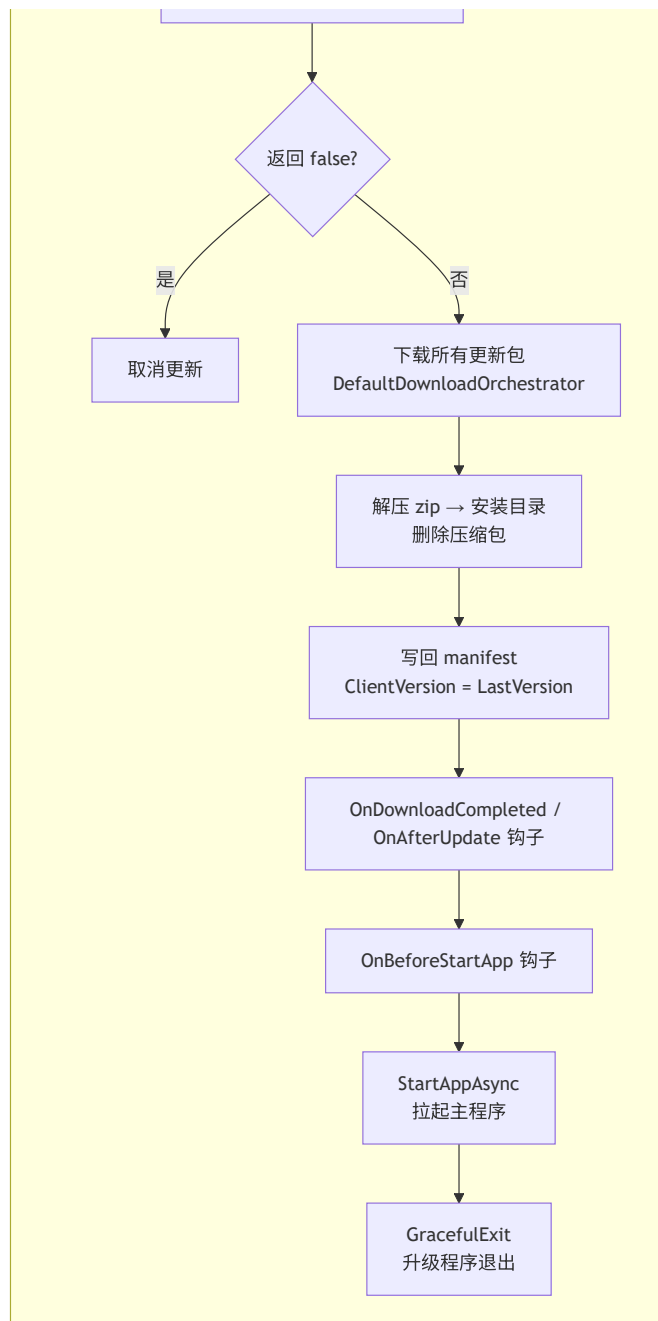
11. OSS 模式：对象存储更新执行流程

OssStrategy 是 OSS（Object Storage Service，对象存储服务）模式的核心策略，适用于**没有后端服务**的部署场景：版本配置与更新包直接托管在对象存储（阿里云 OSS / AWS S3 / MinIO 等）上，客户端通过静态 versions.json 文件完成版本检查与更新。

与标准模式不同，OSS 模式**不写 IPC 文件、不请求 Verification / Report API**。Client 与 Upgrade 之间通过安装目录下的 versions.json 文件传递更新上下文，因此 OssClient 与 OssUpgrade 必须使用一致的 MainAppName / UpdateAppName 配置。

11.1 完整流程





11.2 与标准模式的关键差异

维度	标准 Client / Upgrade	OSS 模式
服务端 API	需要 Verification / Report API	不需要，纯静态文件
进程间通信	加密 IPC 文件	安装目录下的 <code>versions.json</code> 文件
包类型	Chain (差分) + Full (全量)	仅 Full (zip 全量包)
更新管道	Hash → Compress → Patch	Hash 校验 + 解压 (无差分 Patch)
备份 / 回滚	有 (<code>.backups/</code>)	无
黑名单机制	有 (CheckFail)	无
版本对比	服务端返回 Assets 列表	本地解析 <code>versions.json</code> + SemVer 对比
防止更新循环	<code>AllPackagesSucceeded</code> 闸门	更新成功后回写 manifest 的 <code>ClientVersion</code>

11.3 OssClient：版本检查与拉起升级

【说明】OssClient 只负责"检查 + 拉起"，**不下载更新包**。版本配置下载失败或文件不存在时直接返回，不抛异常、不阻塞主程序启动。

步骤	行为	关键实现
1	下载版本配置	UpdateUrl → 保存为 {MainAppName}_versions.json （InstallPath 下）
2	解析并取最新	JsonContext.OssVersionRecordJsonContext 反序列化，按 PubTime 倒序
3	版本对比	SemVer 2.0: latest.Version > ClientVersion 才继续
4	解析升级程序路径	优先 UpdatePath，回退 InstallPath；UpdateAppName 默认 Update.exe
5	拉起升级程序	Process.Start(appPath)， 不传任何参数 ；升级程序靠自身代码的 AppType.OssUpgrade 和 manifest 身份执行升级
6	退出主程序	GracefulExit.CurrentProcessAsync()

11.4 OssUpgrade：下载、解压与应用

【说明】OssUpgrade 完成实际的下载与安装，全程不依赖服务端。

步骤	行为	关键实现
1	读取版本配置	{MainAppName}_versions.json （Client 已下载）或自定义 DownloadSource
2	筛选更新包	过滤 Version > ClientVersion，按 PubTime 升序转为 DownloadAsset （SHA256 = Hash）
3	计算目标版本	所有可解析版本号的 SemVer 最大值 → LastVersion
4	前置钩子	OnBeforeUpdateAsync，返回 false 可取消更新
5	下载更新包	默认 DefaultDownloadOrchestrator（超时默认 60s），或自定义 DownloadOrchestrator
6	解压安装	Format.Zip 解压到 InstallPath，完成后删除压缩包
7	回写版本	ManifestInfo.TryUpdateVersion 更新 manifest 的 ClientVersion，防止无限更新循环
8	收尾钩子	OnDownloadCompleted → OnAfterUpdate → OnBeforeStartApp
9	拉起主程序	StartAppAsync()
10	退出	GracefulExit.CurrentProcessAsync()；异常时触发 OnUpdateErrorAsync + 上报 UpdateFailed

11.5 注意事项

- OSS 模式**没有备份 / 回滚机制**，更新包解压后直接覆盖安装目录，建议发布前充分测试
- OSS 模式**不区分主程序与升级程序的更新包**：versions.json 中所有高于当前版本的包都会被依次下载并应用
- 打包更新包时请勿包含组件内部依赖程序集（如 System.Text.Json.dll、Microsoft.Bcl.AsyncInterfaces.dll 等）
- 版本号必须使用 SemVer 2.0 格式（如 1.0.0.0），否则版本对比会静默失败
- Bucket 建议设为私有并使用预签名 URL；OssDownloadSource 原生支持预签名 URL 场景
- 与标准模式一致，OSS 模式同样支持 Hooks() 生命周期钩子、UpdateReporter() 状态上报、DownloadSource() / DownloadOrchestrator() 自定义下载来源与编排

12. Silent Mode：延迟升级机制

Silent Mode 是标准更新流程的一种变体，唯一的区别是启动升级进程的时机从"立即"变成了"进程退出时"。

12.1 标准模式 vs 静默模式

标准模式：
[下载 → IPC → 立起 Upgrade → 退出] → Upgrade 运行 → 启动主程序

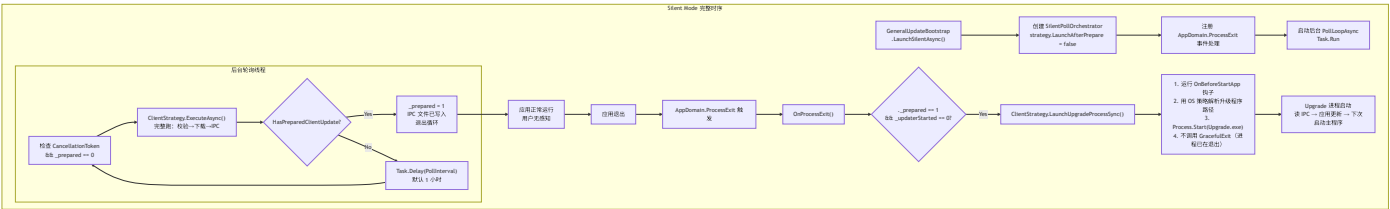
静默模式：
[下载 → IPC → 用户继续工作 → 进程退出 → 拉起 Upgrade] → Upgrade 运行 → 启动主程序

静默模式的流程是相同的，只是被拆成了两段：

PollLoopAsync (后台线程)：
ClientStrategy.ExecuteAsync() → HasPreparedClientUpdate = true → IPC 已写入 → 循环退出
(此时用户完全无感知，继续正常使用应用)

AppDomain.ProcessExit (进程退出时)：
SilentPollOrchestrator.OnProcessExit → LaunchUpgradeProcessSync()
→ Upgrade 进程启动 → 应用更新 → 下次启动时版本已更新

12.2 完整流程图



12.3 TryLaunchUpgrade — 兜底方法

```
// SilentPollOrchestrator.cs:187-210
// 如果 AppDomain.ProcessExit 不可靠 (如 Ctrl+C)
// 可以手动调用这个方法拉起升级进程
public bool TryLaunchUpgrade()
{
    if (_prepared != 1 || Interlocked.Exchange(ref _updaterStarted, 1) == 1)
        return false;

    _strategy.LaunchUpgradeProcessSync();
    return true;
}
```


13. OS 策略的平台差异

方面	Windows	Linux	macOS
管道构建	Hash→Compress→Patch(IfNeeded)	同上	同上
启动主程序	拉起应用 + 可选 Bowl 守护进程 → GracefulExit.CurrentProcessAsync()	只拉起应用 → GracefulExit	只拉起应用，额外 File.Exists 验证
路径大小写	不敏感	敏感	不敏感 (APFS 默认)
Bowl 支持	✔ 支持崩溃守护	✗	✗

Bowl 是一个 Windows-only 的崩溃守护进程。当主程序意外退出时，Bowl 可以检测到并重新拉起。在更新流程中，Bowl 会被 ClientStrategy 的 CallSmallBowlHomeAsync() 先杀死以避免文件锁。

14. 错误恢复全景

错误场景	捕获位置	处理方式	后果
下载失败	ClientStrategy.DownloadAndApplyAsync()	检查 FailedCount > 0，抛异常	冒泡到 ExecuteAsync() catch → 触发错误钩子 + 上报失败
Chain 包管道失败 (有 FallbackFull)	AbstractStrategy.ExecuteAsync() catch when	重建 PipelineContext，设置 PackageType=Full，重新跑 Hash→Compress	最终更新成功，fallbackEffectiveVersion 记录回退版本
Chain 包管道失败 (无 FallbackFull)	AbstractStrategy.ExecuteAsync() catch	AllPackagesSucceeded=false，触发 HandleExecuteException，如果还没成功过则 TryRollback()	该版本失败，继续下一个版本
Fallback Full 也失败	AbstractStrategy.ExecuteAsync() catch (内层 try)	AllPackagesSucceeded=false，继续下一个版本	该版本失败
Upgrade 包失败 (Both 场景)	ClientStrategy.cs Both 分支	中止 IPC 发送 + Upgrade 进程启动	防止 Upgrade 进程拿到失效的 TempPath
Upgrade 进程管道失败	UpdateStrategy.ExecuteAsync()	AllPackagesSucceeded=false，跳过 manifest 写回，跳过主程序启动	下次 Client 启动重新检测到更新
Rollback 失败	AbstractStrategy.TryRollback()	只打日志，不阻断	安装目录可能处于不一致状态
ZIP 哈希不匹配	HashMiddleware	CryptographicException	该版本管道立即失败，触发 Chain→Full 回退或版本失败
文件被锁定	IpcEncryption.DecryptFromFile()	捕获 IOException，返回 null	IPC 文件未就绪，Upgrade 进程等待或退出

Rollback 的逻辑

```
// AbstractStrategy.cs:504-532
private void TryRollback()
{
    // 只在"当前批次还没有任何版本成功"时调用
    // 如果某个版本已经成功应用并覆盖了文件，
    // 回退会撤销有效的工作，造成跨版本的降级

    if (!_appliedAnyVersion)
    {
        // 尝试从 .backups/ 恢复
        StorageManager.Restore(backupDir, _configInfo.InstallPath);
    }
}
```

15. 关键代码路径索引

步骤	文件	关键行
入口分发	Bootstrap/GeneralUpdateBootstrap.cs	LaunchAsync() @L125
Upgrade 路径 IPC 读取	Bootstrap/GeneralUpdateBootstrap.cs	InitializeFromEnvironment() @L357
Client 角色完整 workflow	Strategy/ClientStrategy.cs	ExecuteStandardWorkflowAsync() @L396
场景判定	Strategy/ClientStrategy.cs	Scenario switch @L452-458
一次性下载	Strategy/ClientStrategy.cs	DownloadAndApplyAsync() @L589-597
原地升级 Upgrade 包	Strategy/ClientStrategy.cs	ApplyUpgradePackagesAsync() @L723-737
IPC 写文件	Strategy/ClientStrategy.cs	SendProcessIpc() @L751-765
拉起 Upgrade 进程	Strategy/ClientStrategy.cs	LaunchUpgradeProcessAsync() @L777-789
下载计划构建	Download/DownloadPlanBuilder.cs	Build() @L111
Chain vs Full 阈值决策	Download/DownloadPlanBuilder.cs	@L174-196
回退包匹配	Download/DownloadPlanBuilder.cs	@L200-238
默认下载编排	Download/Orchestrators/DefaultDownloadOrchestrator.cs	ExecuteAsync()
并行管道循环 + Chain→Full 回退	Strategy/AbstractStrategy.cs	ExecuteAsync() @L149-287

步骤	文件	关键行
管道上下文装配	Strategy/AbstractStrategy.cs	CreatePipelineContext() @L322-346
Windows 管道构建	Strategy/WindowsStrategy.cs	BuildPipeline()
Hash 校验中间件	Pipeline/HashMiddleware.cs	InvokeAsync()
解压中间件 (chain/full 分支)	Pipeline/CompressMiddleware.cs	InvokeAsync()
补丁应用中间件	Pipeline/PatchMiddleware.cs	InvokeAsync()
服务端差分生成	Pipeline/DiffPipeline.cs	CleanAsync()
客户端差分支应用	Pipeline/DiffPipeline.cs	DirtyAsync()
原子替换	Pipeline/DiffPipeline.cs	ApplyPatch()
删除文件处理	Pipeline/DiffPipeline.cs	HandleDeleteList()
新增文件复制	Pipeline/DiffPipeline.cs	CopyUnknownFiles()
加密写文件	Ipc/IpcEncryption.cs	EncryptToFile()
解密读文件	Ipc/IpcEncryption.cs	DecryptFromFile()
IPC Provider	Ipc/IProcessInfoProvider.cs	EncryptedFileProcessContractProvider
ProcessContract 定义	Configuration/ProcessContract.cs	全部字段
Configuration 映射	Configuration/ConfigurationMapper.cs	MapToProcessContract()
静默模式轮询	Silent/SilentPollOrchestrator.cs	PollLoopAsync() @L116
静默模式退出时拉起	Silent/SilentPollOrchestrator.cs	OnProcessExit() @L156
Upgrade 角色	Strategy/UpdateStrategy.cs	ExecuteAsync()
管道上下文数据契约	Pipeline/PipelineContext.cs	ConcurrentDictionary
OS 策略解析	Bootstrap/OsStrategyResolver.cs	Resolve() / GetPlatform()
事件管理器	Event/EventManager.cs	Dispatch() / AddListener()
差分进度桥接	Pipeline/DiffProgressReporter.cs	Report()

这份文档涵盖了 GeneralUpdate.Core 的完整执行流程。如果你发现任何不准确或需要补充的地方，请提交 Issue 或 PR。

GeneralUpdate.Bowl — 执行流程详解

目标读者： 需要理解 Bowl 崩溃守护机制的开发者

阅读完你将理解：

- Bowl 在 GeneralUpdate 升级闭环中的定位与职责
 - 从 LaunchAsync 到崩溃处理完成的完整执行链路
 - ProcDump 进程监控的双阶段设计（Prepare → Run）
 - Dump 文件的检测逻辑与崩溃判定规则
 - 崩溃发生后的五步修复管线（报告 → 诊断 → 回滚 → 清理 → 回调）
 - Upgrade 模式与 Normal 模式的行为差异
 - 跨平台策略（Windows / Linux / macOS）的适配机制
 - 优雅降级与容错设计
-

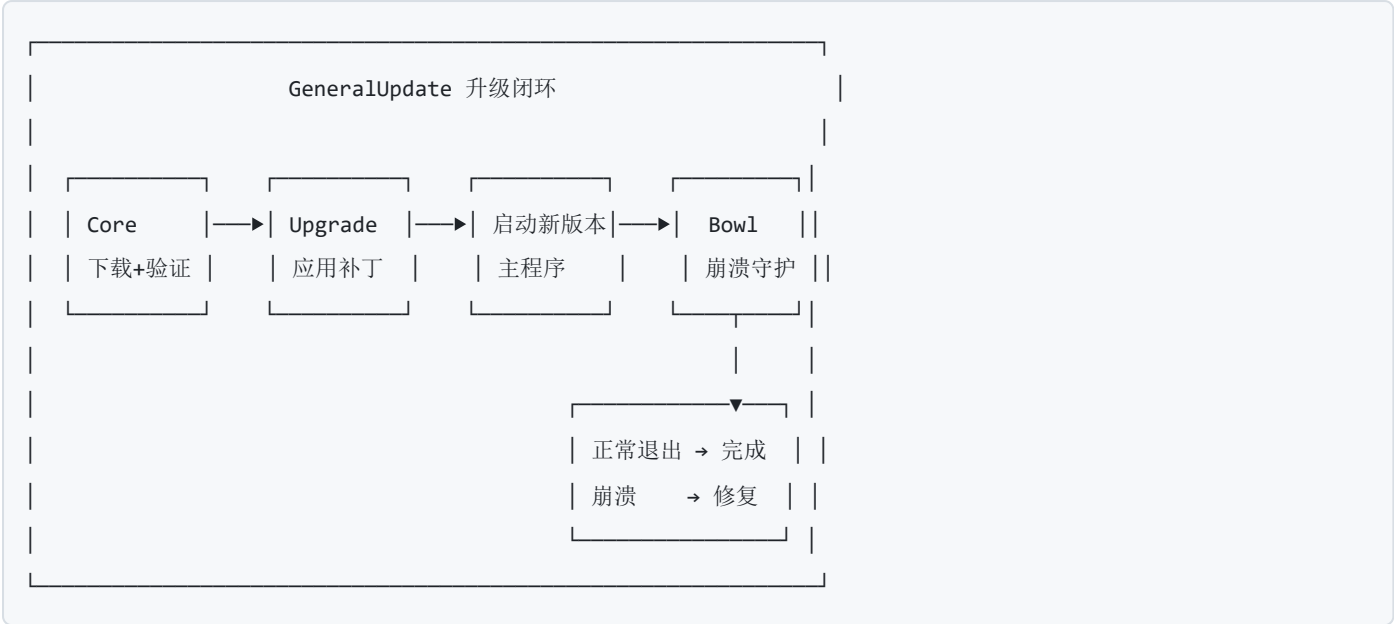
目录

1. 架构总览
 2. 入口：BowlBootstrap 的依赖注入设计
 3. BowlContext：不可变执行上下文
 4. LaunchAsync：监控主流程详解
 5. Phase 1：Strategy.Prepare — 平台策略准备
 6. Phase 2：ProcessRunner — 子进程运行与超时控制
 7. Phase 3：FindDumpFile — Dump 检测与崩溃判定
 8. Phase 4：HandleCrashAsync — 崩溃修复管线
 9. Upgrade vs Normal：两种工作模式对比
 10. 平台策略适配：Windows / Linux / macOS
 11. 容错与优雅降级
 12. 与 GeneralUpdate.Core 的集成
 13. 关键代码路径索引
-

1. 架构总览

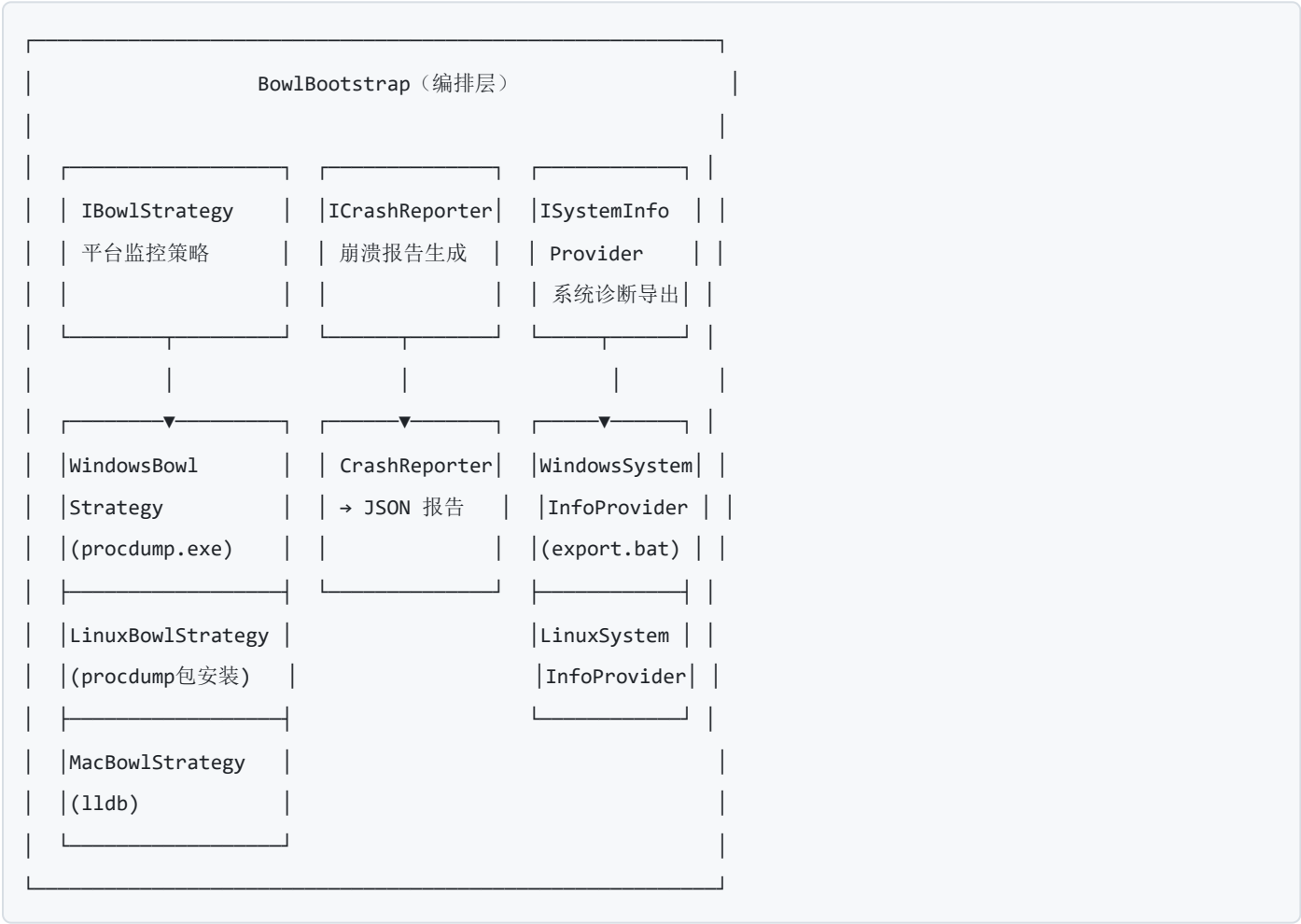
1.1 Bowl 在升级闭环中的位置

Bowl 是 GeneralUpdate **升级闭环的最后一道防线**。它不参与下载、解压或文件替换——这些是 Core 的职责。Bowl 的唯一任务是：在新版本文件落地后、主程序启动时，监控目标进程是否在启动阶段崩溃。



1.2 三层依赖架构

Bowl 采用策略 + 报告 + 诊断的三层依赖注入设计：



1.3 核心设计原则

原则	说明
只监控不更新	Bowl 不下载、不解压、不替换文件，只监控进程崩溃状态

原则	说明
策略模式	<code>IBowlStrategy</code> 封装平台差异，Windows 用 <code>ProcDump</code> ，Linux 用 <code>ProcDump</code> ，macOS 用 <code>lldb</code>
优雅降级	任何步骤失败都不影响后续步骤执行——报告生成失败仍会尝试回滚，回滚失败仍会触发回调
崩溃判定 = Dump 存在	崩溃的唯一判定标准是 Dump 文件是否存在。有 Dump = 崩溃，无 Dump = 正常退出
不可变上下文	<code>BowlContext</code> 是 <code>readonly record struct</code> ，通过 <code>Normalize()</code> 产生标准化副本

1.4 两种工作模式

模式	WorkModel	行为
Upgrade	"Upgrade"	监控新版本启动 → 崩溃时自动回滚备份 → 标记失败版本 → 写环境变量
Normal	"Normal"	独立监控模式 → 崩溃时生成报告和诊断 → 不自动回滚 → 不标记失败版本

2. 入口：BowlBootstrap 的依赖注入设计

`BowlBootstrap` 提供两个构造函数：无参构造自动装配默认实现，三参构造支持 DI 注入。

2.1 双构造函数

```
// 开箱即用：自动检测平台策略 + 默认报告器/诊断器
public BowlBootstrap()
{
    : this(
        StrategyFactory.Create(),           // 根据 RuntimeInformation 自动选择
        new CrashReporter(),               // JSON 崩溃报告
        SystemInfoProviderFactory.Create()) // 平台系统诊断
}

// DI 友好：所有依赖可替换、可 Mock
internal BowlBootstrap(
    IBowlStrategy strategy,
    ICrashReporter crashReporter,
    ISystemInfoProvider systemInfoProvider)
{
    _strategy = strategy;
    _crashReporter = crashReporter;
    _systemInfoProvider = systemInfoProvider;
}
```

2.2 策略工厂决策树



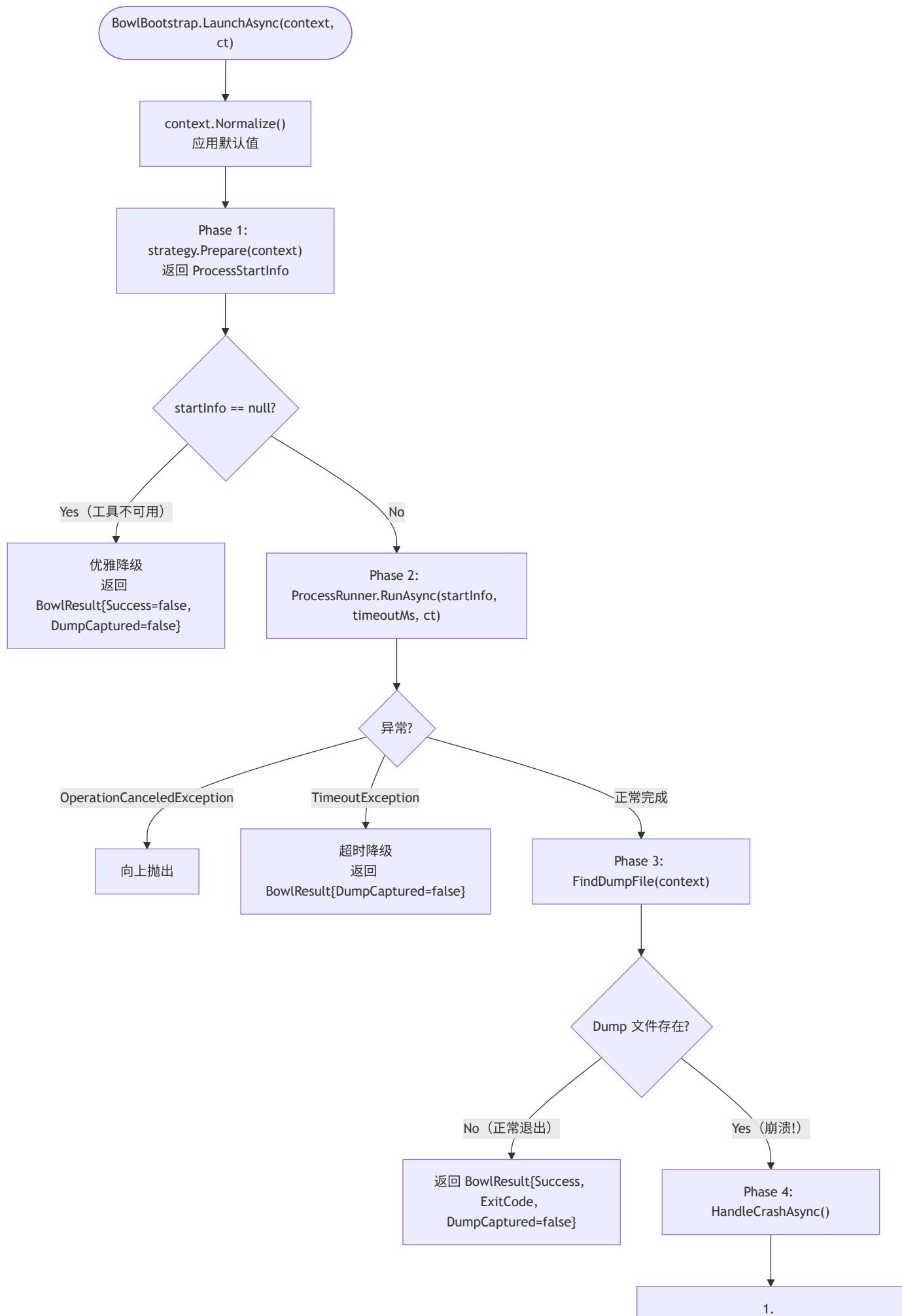
3.2 Normalize() 默认值应用

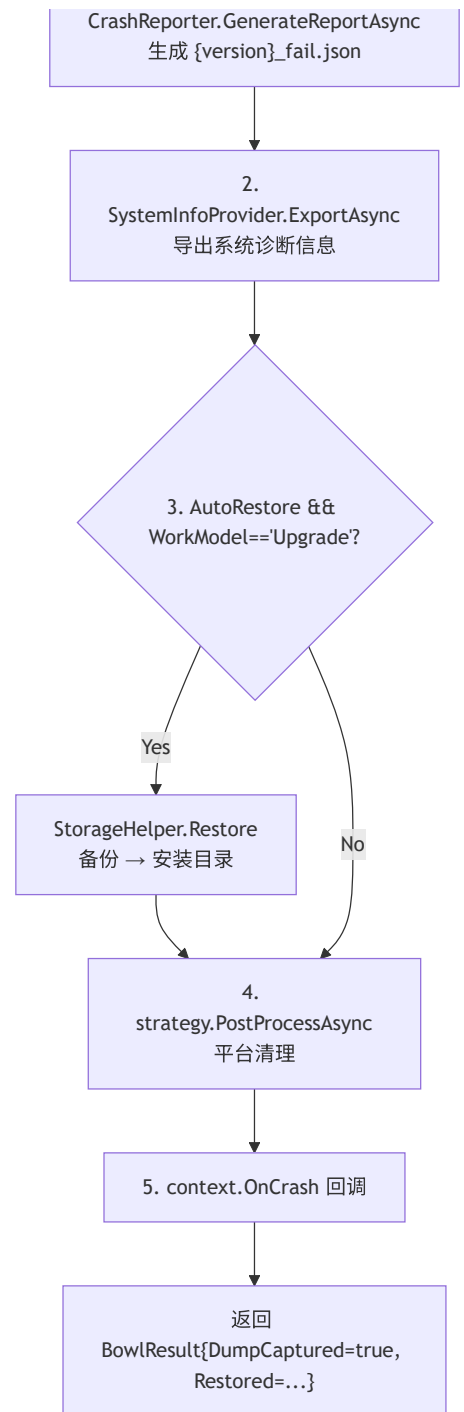
```
context = context.Normalize();
// 等效于:
// if (string.IsNullOrEmpty(context.WorkModel)) context = context with { WorkModel = "Upgrade" };
// if (context.TimeoutMs <= 0) context = context with { TimeoutMs = 30000 };
// if (context.DumpType == default) context = context with { DumpType = DumpType.Full };
// if (string.IsNullOrEmpty(context.FailDirectory))
//     context = context with { FailDirectory = Path.Combine(context.TargetPath, "fails") };
// if (string.IsNullOrEmpty(context.DumpFileName))
//     context = context with { DumpFileName = $"{context.ProcessNameOrId}.dmp" };
```

4. LaunchAsync: 监控主流程详解

`LaunchAsync` 是 Bowl 的唯一公开方法，封装了从策略准备到崩溃修复的完整链路。

4.1 全流程总图





4.2 BowlResult 返回值

```
public readonly record struct BowlResult
{
    public bool Success { get; init; } // 进程是否正常退出
    public int ExitCode { get; init; } // 进程退出码
    public bool DumpCaptured { get; init; } // 是否捕获到 Dump
    public string? DumpFilePath { get; init; } // Dump 文件路径
    public string? CrashReportPath { get; init; } // 崩溃报告路径
    public bool Restored { get; init; } // 是否执行了回滚
}
```

5. Phase 1: Strategy.Prepare — 平台策略准备

`Prepare` 方法的职责是：根据平台差异，构造启动 ProcDump（或 lldb）子进程所需的 `ProcessStartInfo`。如果工具不可用，返回 `null` 表示优雅降级。

5.1 WindowsBowlStrategy

```
// Windows 策略：选择正确的 procdump 架构版本
public ProcessStartInfo? Prepare(BowlContext context)
{
    var procDumpPath = GetProcDumpPath(); // 根据进程位数选择 procdump.exe / procdump64.exe
    if (!File.Exists(procDumpPath))
        return null; // 工具缺失 → 优雅降级

    return new ProcessStartInfo
    {
        FileName = procDumpPath,
        Arguments = $"-e -ma -accepteula {context.ProcessNameOrId} {dumpOutputPath}",
        // -e: 仅捕获未处理异常
        // -ma: Full Dump
        // -accepteula: 自动接受 EULA
    };
}
```

5.2 LinuxBowlStrategy

```
// Linux 策略：先检测 procdump 是否可用，不可用则尝试自动安装
public ProcessStartInfo? Prepare(BowlContext context)
{
    if (!IsProcDumpAvailable())
    {
        var installed = TryInstallProcDump(); // 识别发行版，执行 install.sh 或包管理器
        if (!installed) return null; // 安装失败 → 优雅降级
    }

    return new ProcessStartInfo
    {
        FileName = "procdump",
        Arguments = $"-e -ma {context.ProcessNameOrId} {dumpOutputPath}"
    };
}
```

5.3 MacBowlStrategy

macOS 策略使用系统内置的 `lldb`，但由于 SIP（系统完整性保护）限制，需要额外的调试权限。通常仅在开发/调试环境中可用。

6. Phase 2: ProcessRunner — 子进程运行与超时控制

`ProcessRunner` 是一个异步包装器，负责启动子进程、收集输出、等待退出或超时。

6.1 执行模型

```
ProcessRunner.RunAsync(startInfo, timeoutMs, ct)
|
|— Process.Start(startInfo)
|   RedirectStandardOutput = true
|   RedirectStandardError = true
|
|
|— 并发执行：
|   Task 1: process.WaitForExitAsync(ct) → 等待进程退出
|   Task 2: Task.Delay(timeoutMs, ct)    → 超时计时器
|
|
|— 收集 stdout/stderr 行 → List<string> OutputLines
|
|
|— 返回 ProcessExitResult { ExitCode, OutputLines }
```

6.2 异常处理

场景	行为
ct 被取消	抛出 <code>OperationCanceledException</code> （向上传播）
超时	抛出 <code>TimeoutException</code> （ <code>BowlBootstrap</code> 捕获后优雅降级）
进程正常退出	返回 <code>ExitCode</code> 和输出行

7. Phase 3: FindDumpFile — Dump 检测与崩溃判定

崩溃判定的逻辑极其简单：

```
private static string? FindDumpFile(BowlContext context)
{
    var path = Path.Combine(context.FailDirectory, context.DumpFileName);
    return File.Exists(path) ? path : null;
}
```

核心规则：有 Dump 文件 = 崩溃，无 Dump 文件 = 正常退出。

这个设计利用了 `ProcDump` 的 `-e` 参数行为——`ProcDump` 只在目标进程发生未处理异常时才生成 `Dump` 文件。如果进程正常退出，`ProcDump` 不会生成任何文件，自然也不会触发崩溃处理流程。

8. Phase 4: HandleCrashAsync — 崩溃修复管线

这是 Bowl 最核心的部分。当 Dump 文件被检测到时，启动五步修复管线：



8.1 步骤 1：生成崩溃报告

```
// CrashReporter.GenerateReportAsync
// 输出: {FailDirectory}/{version}_fail.json
var crashReportPath = await _crashReporter.GenerateReportAsync(
    context, exitResult.OutputLines, ct);
```

报告内容包含：

- 监控参数快照（进程名、版本、超时等）
- ProcDump 输出行
- 时间戳

容错设计： 报告生成失败不会中断后续步骤。异常被捕获并记录日志，管线继续执行。

8.2 步骤 2：导出系统诊断

```
await _systemInfoProvider.ExportAsync(context.FailDirectory, ct);
```

Windows： 运行内置的 `export.bat`，收集：

- 驱动列表（`driverquery`）
- 系统信息（`systeminfo`）
- 最近的系统事件日志

Linux/macOS： 收集 `dmesg`、`journalctl` 等系统日志。

容错设计： 诊断导出失败不中断后续步骤。

8.3 步骤 3：自动回滚（仅 Upgrade 模式）

```
if (context.AutoRestore && context.WorkModel == "Upgrade")
{
    StorageHelper.Restore(context.BackupDirectory, context.TargetPath);
    restored = true;
}
```

`StorageHelper.Restore` 将备份目录的内容覆盖复制到安装目录，实现一键回退到旧版本。

前置条件：

- `AutoRestore = true`（默认值）

- `WorkModel = "Upgrade"` (Normal 模式不自动回滚)
- 备份目录必须存在 (Core 在更新前已创建)

容错设计： 回滚失败不中断后续步骤——即使回滚失败，仍然会触发 OnCrash 回调。

8.4 步骤 4：平台清理

```
await _strategy.PostProcessAsync(context, exitResult, ct);
```

平台特定的后处理：

- **Windows：** 清理临时 ProcDump 文件
- **Linux：** 终止残留的 procdump 进程
- **macOS：** 清理 lldb 会话文件

8.5 步骤 5：回调通知

```
if (context.OnCrash != null)
{
    var crashInfo = new CrashInfo
    {
        DumpFilePath = dumpPath,
        CrashReportPath = crashReportPath,
        Version = context.ExtendedField,
        ExitCode = exitResult.ExitCode,
    };
    await context.OnCrash(crashInfo, ct);
}
```

业务方可以在回调中：

- 上传 Dump 和报告到崩溃收集服务
- 记录审计日志
- 通知用户
- 触发告警

9. Upgrade vs Normal：两种工作模式对比

维度	Upgrade 模式	Normal 模式
使用场景	升级后启动健康检查	通用进程崩溃监控
崩溃报告	✔ 生成	✔ 生成
系统诊断	✔ 导出	✔ 导出
自动回滚	✔ <code>StorageHelper.Restore</code>	✘ 跳过
失败版本标记	✔ 写 <code>UpgradeFail</code>	✘ 不标记
环境变量	✔ 设 <code>GU_UPGRADE_FAIL</code>	✘ 不设

维度	Upgrade 模式	Normal 模式
OnCrash 回调	✅ 触发	✅ 触发

9.1 与 Core 的联动 (Upgrade 模式)

Core 升级流程

|

└─ 备份安装目录 → .backups/backup-{ts}

└─ 下载 + 应用更新

└─ 写 IPC 文件

└─ 拉起 Upgrade 进程

|

└─ 启动 Bowl (Upgrade 模式)

|

└─ 监控新版本主程序启动

|

└─ 正常启动 → 无 Dump → 返回 Success → 升级完成 ✅

|

└─ 启动崩溃 → 有 Dump → 进入修复管线

└─ 回滚到 .backups/

└─ 写 {version}_fail.json

└─ 设环境变量 GU_UPGRADE_FAIL={version}

|

└─ Core 下次启动读取环境变量

→ CheckFail() 命中

→ 跳过该版本

→ 等待服务端提供更高版本

10. 平台策略适配: Windows / Linux / macOS

10.1 策略对比

维度	Windows	Linux	macOS
监控工具	procdump.exe / procdump64.exe	procdump (deb/rpm)	/usr/bin/lldb
工具获取	内置在 NuGet 包中	自动检测 + install.sh 安装	系统内置
Dump 参数	-e -ma	-e -ma	lldb 脚本
架构选择	根据进程位数自动选择 x86/x64/ARM64	不区分	不区分
权限要求	管理员权限 (调试权限)	root (ptrace 权限)	SIP 授权
系统诊断	export.bat : 驱动/系统信息/事件日志	dmesg / journalctl	sysdiagnose
支持级别	完整支持	完整支持 (需网络安装工具)	基础支持 (受 SIP 限制)

10.2 StrategyFactory 自动检测

```
public static IBowlStrategy Create()
{
    if (RuntimeInformation.IsOSPlatform(OSPlatform.Windows))
        return new WindowsBowlStrategy();
    if (RuntimeInformation.IsOSPlatform(OSPlatform.Linux))
        return new LinuxBowlStrategy();
    if (RuntimeInformation.IsOSPlatform(OSPlatform.OSX))
        return new MacBowlStrategy();
    throw new PlatformNotSupportedException();
}
```

11. 容错与优雅降级

Bowl 的设计哲学是：**任何单一组件失败都不应阻止其他组件的执行。**

11.1 降级层级

Level 0: 策略无法准备工具

- 返回 `BowlResult{Success=false, DumpCaptured=false}`
- 不崩溃、不抛异常
- 调用方自行决定后续行为

Level 1: 子进程超时

- 返回 `BowlResult{DumpCaptured=false}`
- 视为"未捕获到崩溃"
- 不做回滚处理

Level 2: 报告生成失败

- 记录日志
- 继续执行后续步骤

Level 3: 系统诊断失败

- 记录日志
- 继续执行后续步骤

Level 4: 自动回滚失败

- 记录日志
- 继续执行回调

Level 5: OnCrash 回调异常

- 记录日志
- 不影响 `BowlResult` 返回

11.2 异常传播策略

异常类型	行为
OperationCanceledException	唯一向上传播的异常 —— 调用方取消必须被尊重
TimeoutException	被 BowlBootstrap 捕获，优雅降级
其他所有异常	被各步骤独立捕获，记录日志后继续执行

12. 与 GeneralUpdate.Core 的集成

12.1 调用时机

Bowl 应在 Core 完成文件替换之后、启动新版本主程序之前被调用：

```
// Core ClientStrategy 中的典型调用模式
public async Task LaunchAsync()
{
    // ... Core 完成下载、验证、应用更新 ...

    // 启动 Bowl 守护新版本
    var bowlContext = new BowlContext
    {
        ProcessNameOrId = "MyApp",
        TargetPath = installPath,
        BackupDirectory = backupPath,
        WorkModel = "Upgrade",
        ExtendedField = newVersion,
        AutoRestore = true,
        OnCrash = async (info, ct) =>
        {
            // 上传崩溃诊断包
            await UploadCrashReportAsync(info, ct);
        }
    };

    var bowl = new BowlBootstrap();
    var result = await bowl.LaunchAsync(bowlContext);

    if (!result.Success && result.DumpCaptured)
    {
        // 崩溃已发生，回滚已执行
        // Core 下次启动时会通过 UpgradeFail 标记跳过此版本
    }
}
```

12.2 与 Core 共享的状态

状态通道	写入方	读取方	用途
{version}_fail.json	Bowl	Core + 业务	崩溃报告持久化
环境变量 GU_UPGRADE_FAIL	Bowl	Core	失败版本跳过
.backups/ 目录	Core (创建)	Bowl (回滚)	备份与恢复

13. 关键代码路径索引

组件	文件	关键方法
入口编排	BowlBootstrap.cs	LaunchAsync() → HandleCrashAsync()
执行上下文	BowlContext.cs	Normalize()
崩溃报告	Internal/CrashReporter.cs	GenerateReportAsync()
系统诊断	Internal/WindowsSystemInfoProvider.cs	ExportAsync()
文件回滚	FileSystem/StorageHelper.cs	Restore()
Windows 策略	Strategies/WindowsBowlStrategy.cs	Prepare() / PostProcessAsync()
Linux 策略	Strategies/LinuxBowlStrategy.cs	Prepare() / PostProcessAsync()
Mac 策略	Strategies/MacBowlStrategy.cs	Prepare() / PostProcessAsync()
进程运行器	Strategies/ProcessRunner.cs	RunAsync()
策略工厂	Strategies/StrategyFactory.cs	Create()
崩溃 DTO	Internal/Crash.cs	—
Dump 类型	DumpType.cs	Full / Mini / Heap
日志追踪	Tracer/GeneralTracer.cs	Info() / Warn() / Error()

GeneralUpdate.Drivelution — 执行流程详解

目标读者： 需要理解 Drivelution 驱动更新引擎内部机制的开发者

阅读完你将理解：

- Drivelution 的跨平台抽象架构与工厂模式设计
 - `BaseDriverUpdater` 模板方法如何编排统一流水线
 - 平台检测 → 权限检查 → 验证 → 备份 → 安装 → 验证 → 回滚的完整链路
 - `IPipelineStep` 的可组合流水线设计
 - 重试策略 (RetryPolicy) 与超时控制机制
 - Windows (pnputil) 、Linux (insmod/dpkg/rpm) 、macOS (kextload/installer) 的平台差异
 - 批量更新 (BatchUpdateAsync) 的顺序/并行执行模式
 - 异常到结构化 `ErrorInfo` 的映射机制
-

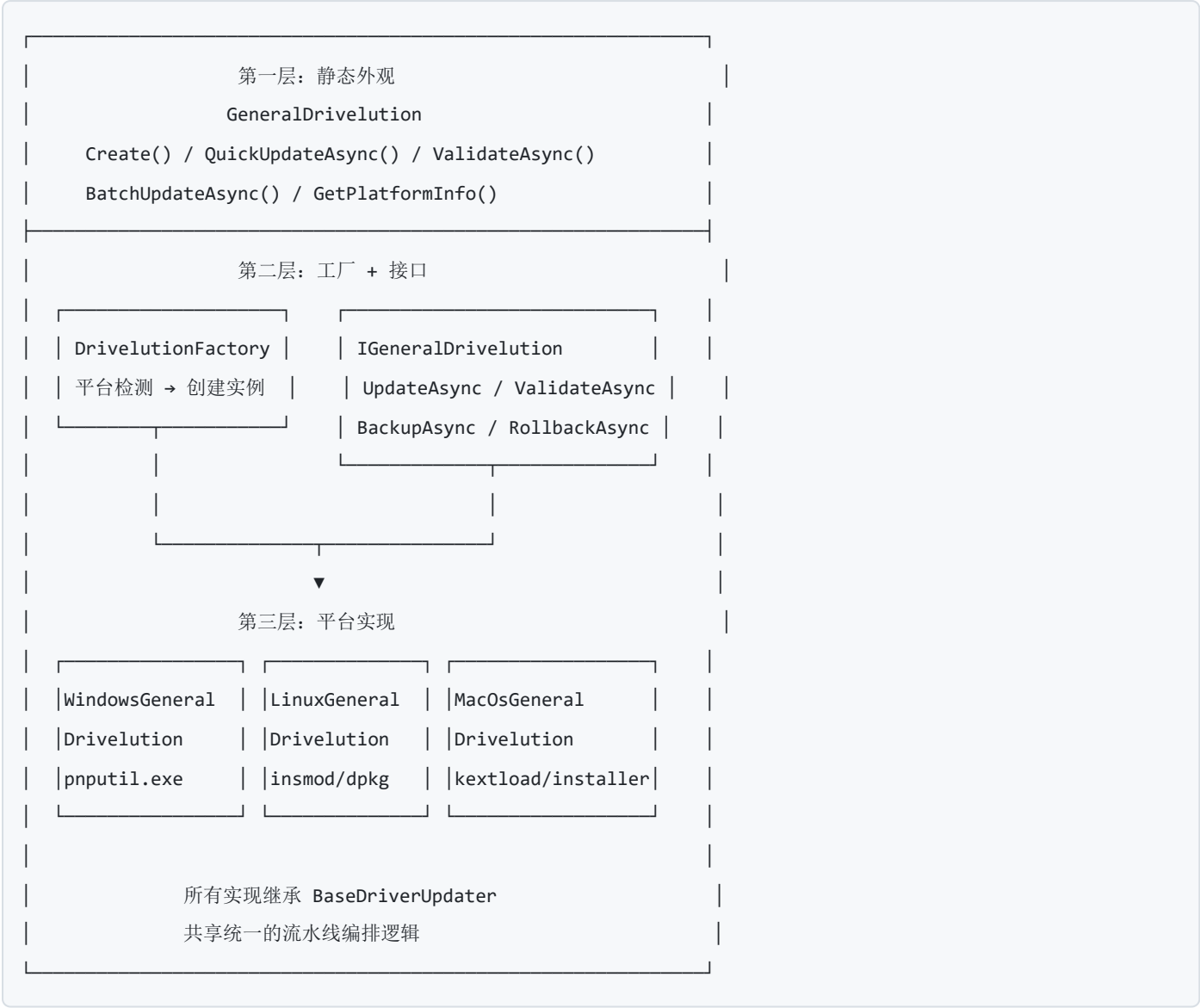
目录

1. 架构总览
 2. 入口: `GeneralDrivelution` 静态外观
 3. 工厂: `DrivelutionFactory` 平台检测
 4. `BaseDriverUpdater`: 模板方法流水线
 5. `IPipelineStep`: 可组合流水线步骤
 6. 重试与超时: `RetryPolicy` 与 `CancellationToken`
 7. 平台安装实现: Windows / Linux / macOS
 8. 回滚机制: `TryRollbackAsync`
 9. 批量更新: `BatchUpdateAsync`
 10. 异常映射: `MapExceptionToErrorInfo`
 11. 关键代码路径索引
-

1. 架构总览

1.1 三层抽象设计

Drivelution 采用外观 → 模板方法 → 平台实现的三层抽象：



1.2 核心设计原则

原则	说明
模板方法模式	BaseDriverUpdater.UpdateAsync() 定义流水线骨架，子类只需实现 InstallCoreAsync()
策略模式	UpdateStrategy 控制备份、重试、超时、重启等行为
工厂模式	DrivelutionFactory.Create() 自动检测 OS 并创建对应实现
流水线模式	IPipelineStep 可组合、可替换、可跳过 (ShouldExecute)
Bag 上下文	PipelineContext.Bag (Dictionary) 在步骤间共享中间数据

1.3 统一流水线

Windows:	[CheckPermissions] → Validate → Backup → Install → Verify
Linux:	[CheckSudo] → Validate → Backup → Install → Verify
macOS:	[CheckPermissions] → Validate → Backup → Install → Verify

每个步骤都有：条件判断 (ShouldExecute) → 执行 (ExecuteAsync) → 结果判断 → 失败回滚。

2. 入口：GeneralDrivelution 静态外观

`GeneralDrivelution` 是一个**静态外观类**，提供所有公开 API。它委托给工厂创建平台实现。

2.1 API 全景

```
public static class GeneralDrivelution
{
    // 工厂方法
    static IGeneralDrivelution Create(DrivelutionOptions? options = null);
    static IGeneralDrivelution Create(IServiceProvider serviceProvider); // DI 支持

    // 便捷方法
    static Task QuickUpdateAsync(DriverInfo, UpdateStrategy?, IProgress?, CT);
    static Task<bool> ValidateAsync(DriverInfo, CT);
    static Task<List> GetDriversFromDirectoryAsync(string path, string? pattern, CT);
    static Task BatchUpdateAsync(IEnumerable, UpdateStrategy, BatchMode, IProgress?, CT);
    static PlatformInfo GetPlatformInfo();
}
```

2.2 QuickUpdateAsync 默认策略

```
public static async Task QuickUpdateAsync(
    DriverInfo driverInfo,
    UpdateStrategy? strategy = null, ...)
{
    strategy ??= new UpdateStrategy
    {
        RequireBackup = true,           // 默认开启备份
        RetryCount = 3,                 // 默认重试 3 次
        RetryIntervalSeconds = 5        // 默认间隔 5 秒
    };

    var updater = Create();             // 自动检测平台
    return await updater.UpdateAsync(driverInfo, strategy, progress, ct);
}
```

3. 工厂：DrivelutionFactory 平台检测

```
public static IGeneralDrivelution Create(DrivelutionOptions? options = null)
{
    if (RuntimeInformation.IsOSPlatform(OSPlatform.Windows))
        return new WindowsGeneralDrivelution(options);

    if (RuntimeInformation.IsOSPlatform(OSPlatform.Linux))
        return new LinuxGeneralDrivelution(options);

    if (RuntimeInformation.IsOSPlatform(OSPlatform.OSX))
        return new MacOSGeneralDrivelution(options);

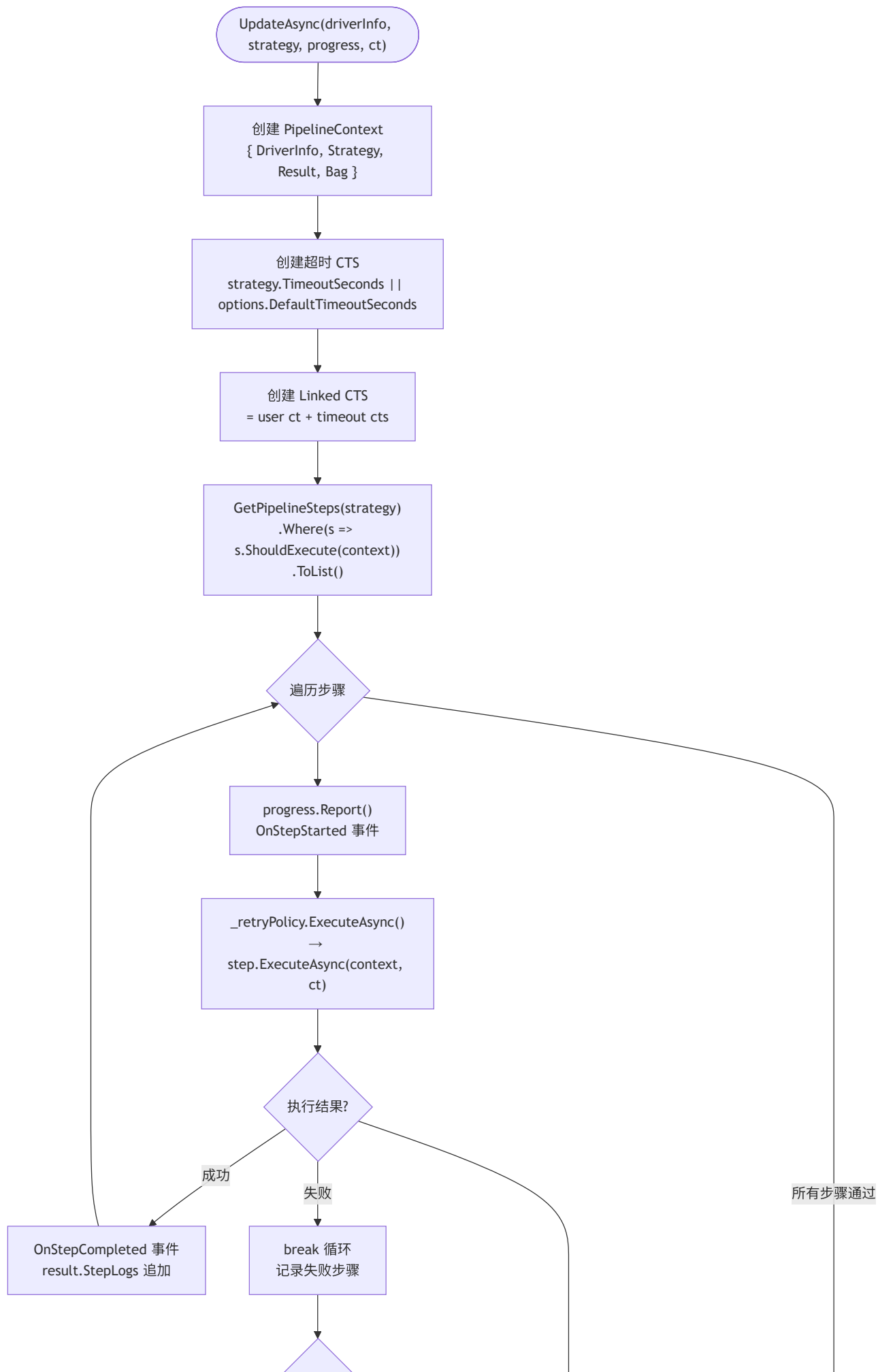
    throw new PlatformNotSupportedException("Current platform is not supported.");
}
```

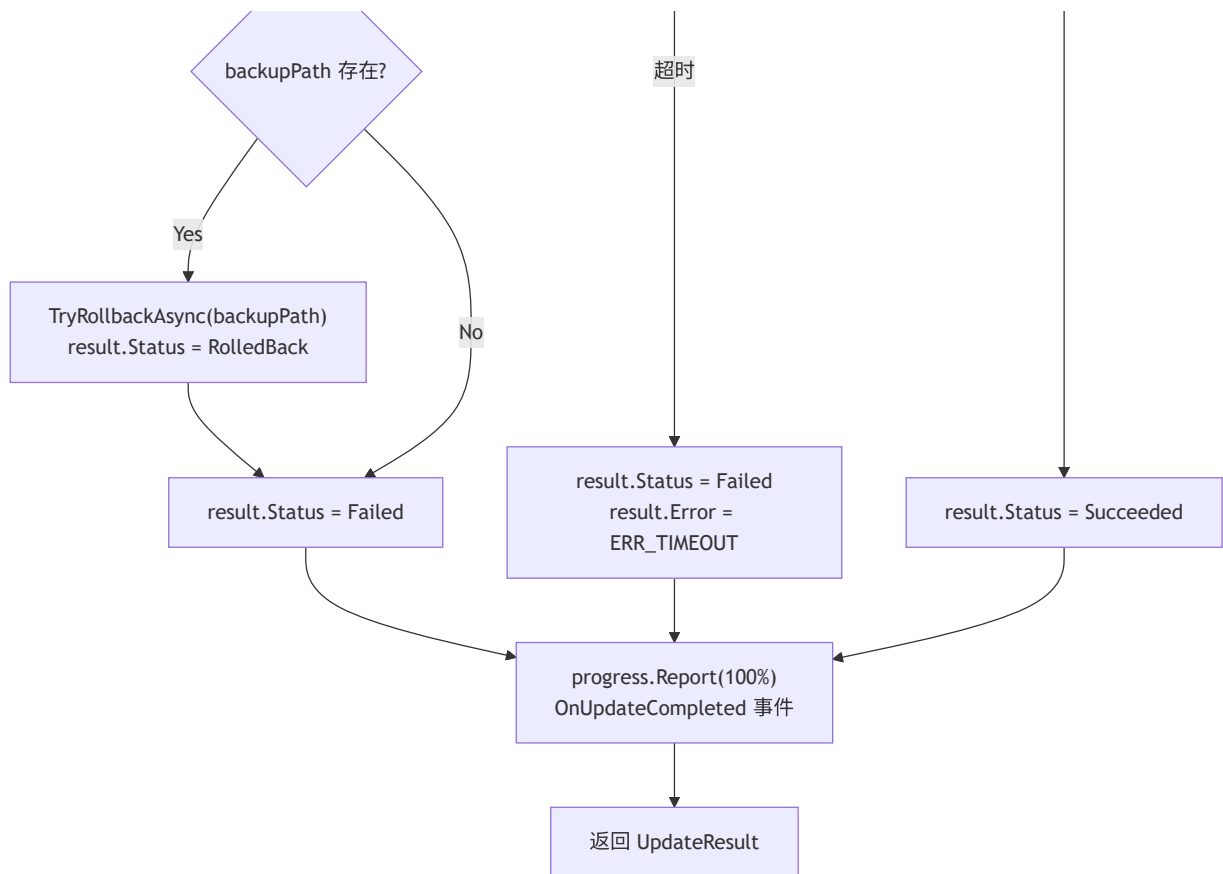
工厂同时提供 `IsPlatformSupported()`、`GetCurrentPlatform()` 等查询方法。

4. BaseDriverUpdater: 模板方法流水线

`BaseDriverUpdater` 是 Drivelution 的核心。它实现了 `IGeneralDrivelution` 接口，定义了完整的更新流水线模板。

4.1 UpdateAsync 全流程





4.2 模板方法 Hook

子类需要实现的核心抽象方法：

```
// 唯一必须实现的抽象方法—平台特定的安装逻辑
protected abstract Task InstallCoreAsync(
    DriverInfo driverInfo,
    UpdateStrategy strategy,
    CancellationToken cancellationToken);
```

可选的虚方法覆盖：

虚方法	默认行为	覆盖场景
<code>GetPipelineSteps(strategy)</code>	[Validate, Backup, Install, Verify]	Windows 插入 CheckPermissions, Linux 插入 CheckSudo
<code>VerifyInstallationAsync(driverInfo, ct)</code>	return true	平台特定的安装后验证
<code>GetDefaultSearchPattern()</code>	"*.*"	Windows 返回 "*.inf", Linux 返回 "*.ko"
<code>ParseDriverFromFile(filePath)</code>	基础解析	平台特定的驱动文件解析 (INF / modinfo)

5. IPipelineStep: 可组合流水线步骤

5.1 步骤接口

```
public interface IPipelineStep
{
    string StepName { get; }
    bool ShouldExecute(PipelineContext context);
    Task ExecuteAsync(PipelineContext context, CancellationToken ct);
}

public class PipelineResult
{
    public bool Success { get; set; }
    public string? ErrorMessage { get; set; }
    public Exception? Exception { get; set; }
}
```

5.2 内置步骤 (DefaultPipelineSteps)

```
public static class DefaultPipelineSteps
{
    // Validate: 哈希校验 + 签名校验 + 兼容性检查
    public static IPipelineStep CreateValidateStep(IDriverValidator validator);

    // Backup: 将当前驱动文件备份到指定路径
    public static IPipelineStep CreateBackupStep(IDriverBackup backup);

    // Install: 委托给 InstallCoreAsync (平台特定)
    public static IPipelineStep CreateInstallStep(Func<DriverInfo, UpdateStrategy, CT, Task> installCore);

    // Verify: 安装后验证 (可覆盖)
    public static IPipelineStep CreateVerifyStep(Func<DriverInfo, CT, Task<bool>> verify);
}
```

5.3 DelegateStep: 轻量自定义步骤

```
public class DelegateStep : IPipelineStep
{
    // 通过委托快速创建自定义步骤, 无需新建类
    public DelegateStep(string name, Func<PipelineContext, bool> shouldExecute,
        Func<PipelineContext, CT, Task> execute);
}
```

6. 重试与超时: RetryPolicy 与 CancellationToken

6.1 重试策略

```
public class RetryPolicy
{
    public int MaxRetries { get; init; }          // 最大重试次数（默认 3）
    public int RetryIntervalMs { get; init; }     // 重试间隔（默认 5000ms）
    public bool UseExponentialBackoff { get; init; } // 指数退避

    public async Task ExecuteAsync(
        Func<CancellationToken, Task> action,
        CancellationToken ct)
    {
        for (int attempt = 0; attempt <= MaxRetries; attempt++)
        {
            var result = await action(ct);
            if (result.Success) return result;

            if (attempt < MaxRetries)
            {
                var delay = UseExponentialBackoff
                    ? RetryIntervalMs * Math.Pow(2, attempt)
                    : RetryIntervalMs;
                await Task.Delay((int)delay, ct);
            }
        }
        // 所有重试耗尽，返回最后一次失败结果
    }
}
```

6.2 超时控制

```
// 双重 CancellationToken 联动
var timeoutSeconds = strategy.TimeoutSeconds > 0
    ? strategy.TimeoutSeconds
    : _options.DefaultTimeoutSeconds;

using var timeoutCts = new CancellationTokenSource(TimeSpan.FromSeconds(timeoutSeconds));
using var linkedCts = CancellationTokenSource.CreateLinkedTokenSource(
    cancellationTokentoken, timeoutCts.Token);
```

超时后:

- `linkedCts.Token` 被取消

- 当前步骤的 `ExecuteAsync` 收到 `OperationCanceledException`
- `BaseDriverUpdater` 捕获后返回 `UpdateStatus.Failed` + `ErrorType.Timeout`

7. 平台安装实现：Windows / Linux / macOS

7.1 Windows: pnputil

```
// WindowsGeneralDriverLution.InstallCoreAsync
protected override async Task InstallCoreAsync(DriverInfo driverInfo, ...)
{
    // pnputil /add-driver <inf文件> /install
    var args = $"/add-driver \"{driverInfo.FilePath}\" /install";
    var result = await CommandRunner.RunAsync("pnputil.exe", args, ct);

    if (result.ExitCode != 0)
        throw new DriverInstallationException(
            $"pnputil failed with exit code {result.ExitCode}: {result.Stderr}");
}
```

Windows 平台额外步骤：

- **CheckPermissions:** 检查是否以管理员权限运行
- **INF 解析:** 从 `.inf` 文件中提取驱动名称、版本、签名信息

7.2 Linux: 多种包格式

```
// LinuxGeneralDriverLution.InstallCoreAsync
protected override async Task InstallCoreAsync(DriverInfo driverInfo, ...)
{
    var ext = Path.GetExtension(driverInfo.FilePath).ToLowerInvariant();

    switch (ext)
    {
        case ".ko": // 内核模块
            await CommandRunner.RunAsync("insmod", driverInfo.FilePath, ct);
            break;
        case ".deb": // Debian 包
            await CommandRunner.RunAsync("dpkg", $"-i {driverInfo.FilePath}", ct);
            break;
        case ".rpm": // RPM 包
            // 优先 dnf, 回退 rpm
            await CommandRunner.RunAsync("dnf", $"install -y {driverInfo.FilePath}", ct);
            break;
    }
}
```

Linux 平台额外步骤：

- **CheckSudo**：检查是否有 root 权限
- **modinfo 解析**：从 `.ko` 文件中提取模块信息

7.3 macOS: kext/dext/pkg

```
// MacOSGeneralDriverLution.InstallCoreAsync
protected override async Task InstallCoreAsync(DriverInfo driverInfo, ...)
{
    var ext = Path.GetExtension(driverInfo.FilePath).ToLowerInvariant();

    switch (ext)
    {
        case ".kext": // 内核扩展
            await CommandRunner.RunAsync("kextload", driverInfo.FilePath, ct);
            break;
        case ".dext": // 系统扩展 (DriverKit)
            await CommandRunner.RunAsync("systemextensionsctl", $"install ...", ct);
            break;
        case ".pkg": // 安装包
            await CommandRunner.RunAsync("installer", $"-pkg {driverInfo.FilePath} -target /", ct);
            break;
    }
}
```

7.4 CommandRunner: 安全的进程执行

```
// 使用 ArgumentList 避免 shell 注入
public async Task RunAsync(string fileName, string arguments, CT ct)
{
    var psi = new ProcessStartInfo
    {
        FileName = fileName,
        RedirectStandardOutput = true,
        RedirectStandardError = true,
        UseShellExecute = false,
    };
    // 使用 ArgumentList 而不是 Arguments 字符串拼接
    foreach (var arg in ParseArguments(arguments))
        psi.ArgumentList.Add(arg);
    // ...
}
```

8. 回滚机制：TryRollbackAsync

8.1 触发条件

回滚在以下条件同时满足时触发：

1. 流水线中某步骤执行失败
2. `PipelineContext.Bag["BackupPath"]` 存在（Backup 步骤已成功写入）
3. 或 `result.BackupPath` 不为空

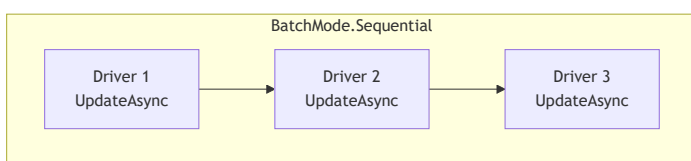
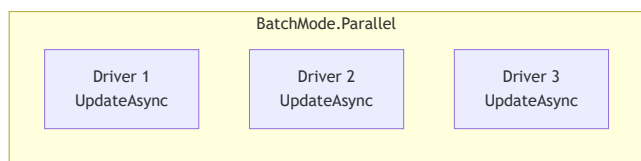
8.2 执行流程

```
TryRollbackAsync(backupPath, CancellationToken.None)
|
|— 检查备份目录是否存在
|   └─ 不存在 → 返回 false
|
|— 检查备份目录内容
|
|— 返回 true/false
|
|— result.RolledBack = true
    result.Status = UpdateStatus.RolledBack
```

注意： 回滚使用 `CancellationToken.None`，确保即使原始操作已超时，回滚仍有机会执行。

9. 批量更新：BatchUpdateAsync

9.1 两种执行模式



9.2 顺序模式实现

```
// BaseDriverUpdater.BatchUpdateAsync (Sequential)
for (int i = 0; i < driverList.Count; i++)
{
    progress?.Report(new UpdateProgress { /* 批次进度 */ });
    var updateResult = await UpdateAsync(driver, strategy, cancellationTokens: ct);
    result.Results.Add(new DriverUpdateEntry { DriverInfo = driver, Success = updateResult.Success, Result
= updateResult });
}
```

9.3 并行模式实现

```
// BaseDriverUpdater.BatchUpdateAsync (Parallel)
var tasks = driverList.Select(async (driver, index) =>
{
    var updateResult = await UpdateAsync(driver, strategy, cancellationTokens: ct);
    Interlocked.Increment(ref completed);
    return new DriverUpdateEntry { ... };
}).ToList();

var entries = await Task.WhenAll(tasks);
```

9.4 聚合结果

```
result.SucceededCount = result.Results.Count(r => r.Success);
result.FailedCount = result.Results.Count(r => !r.Success);
result.AllSucceeded = result.FailedCount == 0;
result.Duration = DateTime.UtcNow - startTime;
```

10. 异常映射：MapExceptionToErrorInfo

Drivelution 将异常类型映射为结构化的 ErrorInfo：

异常类型	ErrorType	Code	CanRetry	建议解决方案
DriverPermissionException	PermissionDenied	ERR_PERM	false	以管理员/root 权限重启应用
DriverValidationException	HashValidationFailed	ERR_VALID	false	检查驱动文件完整性
DriverInstallationException (CanRetry)	InstallationFailed	ERR_INSTALL_RETRY	true	验证驱动兼容性后重试
DriverInstallationException (!CanRetry)	InstallationFailed	ERR_INSTALL	false	验证驱动兼容性
DriverBackupException	BackupFailed	ERR_BACKUP	false	检查磁盘空间和权限
DriverRollbackException	RollbackFailed	ERR_ROLLBACK	false	检查备份目录完整性
OperationCanceledException	Timeout	ERR_TIMEOUT	true	增加 TimeoutSeconds
其他	Unknown	ERR_UNKNOWN	false	检查日志

11. 关键代码路径索引

组件	文件	关键方法
静态外观	GeneralDrivelution.cs	Create() / QuickUpdateAsync() / BatchUpdateAsync()
工厂	Core/DrivelutionFactory.cs	Create()
模板方法	Core/Pipeline/BaseDriverUpdater.cs	UpdateAsync() / GetPipelineSteps() / TryRollbackAsync()
流水线步骤接口	Core/Pipeline/IPipelineStep.cs	ShouldExecute() / ExecuteAsync()
内置步骤	Core/Pipeline/DefaultPipelineSteps.cs	CreateValidateStep() / CreateBackupStep() / CreateInstallStep()
流水线上下文	Core/Pipeline/PipelineContext.cs	DriverInfo / Strategy / Bag
重试策略	Core/Pipeline/RetryPolicy.cs	ExecuteAsync()
Windows 实现	Windows/Implementation/WindowsGeneralDrivelution.cs	InstallCoreAsync()
Linux 实现	Linux/Implementation/LinuxGeneralDrivelution.cs	InstallCoreAsync()
macOS 实现	MacOS/Implementation/MacOsGeneralDrivelution.cs	InstallCoreAsync()
命令执行器	Core/Execution/CommandRunner.cs	RunAsync()
兼容性检查	Core/Utilities/CompatibilityChecker.cs	GetCurrentOS() / GetCurrentArchitecture()
驱动信息	Abstractions/Models/DriverInfo.cs	—
更新策略	Abstractions/Models/UpdateStrategy.cs	—
更新结果	Abstractions/Models/UpdateResult.cs	—

GeneralUpdate.Extension — 执行流程详解

目标读者： 需要理解 Extension 扩展管理引擎内部机制的开发者

阅读完你将理解：

- `GeneralExtensionHost` 的 DI 注入架构与遗留兼容模式
- `UpdateExtensionAsync` 的完整九阶段执行链路
- 依赖解析器（`DependencyResolver`）的拓扑排序与循环检测机制
- 版本兼容性检查与平台匹配的判定逻辑
- `InstallExtensionAsync` 的备份→清理→解压→原子写入流程
- Zip Slip 路径穿越防护的实现细节
- Extension Catalog 的原子写入与崩溃安全设计
- 下载队列管理器（`DownloadQueueManager`）的并发控制
- 生命周期钩子（`IExtensionLifecycleHooks`）的 8 个事件注入点

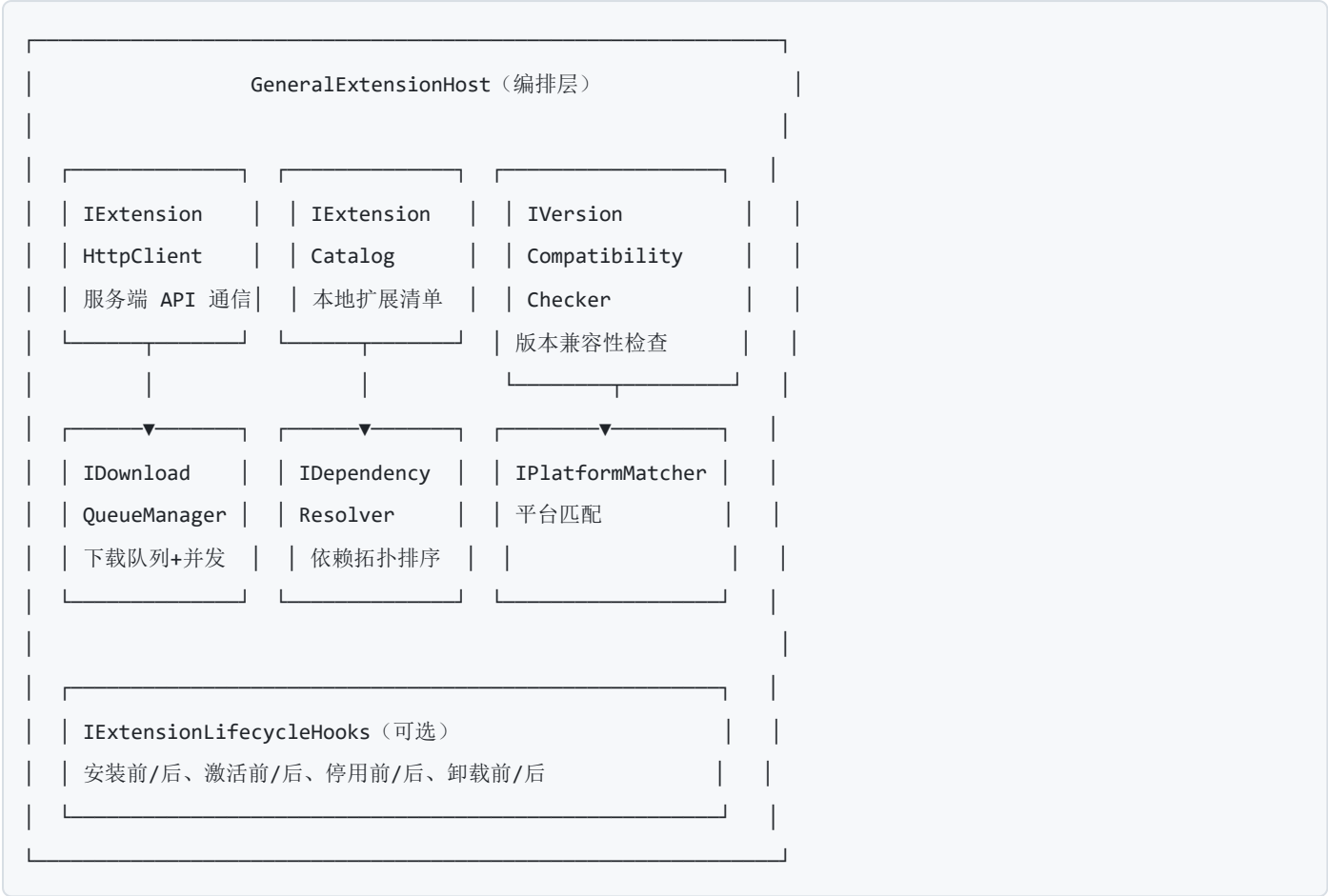
目录

1. 架构总览
2. 入口：`GeneralExtensionHost` 的双构造函数设计
3. `ExtensionHostBuilder`：DI Builder 模式
4. `UpdateExtensionAsync`：一键更新完整流程
5. 依赖解析：`DependencyResolver` 深度解析
6. 兼容性检查：版本 + 平台双校验
7. 下载：`DownloadQueueManager` 并发控制
8. 安装：`InstallExtensionAsync` 安全保障
9. Catalog：原子写入与崩溃安全
10. 生命周期钩子：8 个事件注入点
11. 关键代码路径索引

1. 架构总览

1.1 六层服务架构

Extension 采用**依赖注入 + Builder 模式**，所有服务均可替换：



1.2 核心设计原则

原则	说明
全 DI 可替换	每个服务都有接口，通过构造函数注入，方便单元测试和自定义
Builder 模式	<code>ExtensionHostBuilder</code> 提供流畅的配置 API，支持 <code>ConfigureServices</code>
遗留兼容	保留无参构造函数，内部自动创建默认实现，老代码无需修改
原子写入	<code>Catalog</code> 的 <code>manifest.json</code> 先写 <code>.tmp</code> 再重命名，崩溃时不会损坏
Zip Slip 防护	<code>SafeExtractZipAsync</code> 验证每个条目的目标路径不超出安装目录
递归依赖安装	缺失依赖自动递归调用 <code>UpdateExtensionAsync</code> ，确保依赖链完整

2. 入口：GeneralExtensionHost 的双构造函数设计

2.1 DI 构造函数（推荐）

```
public GeneralExtensionHost(  
    ExtensionHostOptions options,  
    IExtensionHttpClient httpClient,           // HTTP 通信  
    IExtensionCatalog catalog,               // 本地清单  
    IVersionCompatibilityChecker compatibilityChecker, // 版本兼容  
    IDownloadQueueManager downloadQueue,      // 下载队列  
    IDependencyResolver dependencyResolver,   // 依赖解析  
    IPlatformMatcher platformMatcher,        // 平台匹配  
    IExtensionLifecycleHooks? lifecycleHooks = null, // 生命周期钩子  
    IExtensionMetadataMapper? metadataMapper = null) // DTO 映射器
```

2.2 遗留构造函数（向后兼容）

```
public GeneralExtensionHost(ExtensionHostOptions options)  
{  
    // 自动创建默认实现  
    _httpClient = new ExtensionHttpClient(options.ServerUrl, options.Scheme, options.Token);  
    ExtensionCatalog = new ExtensionCatalog(options.CatalogPath ?? options.ExtensionsDirectory);  
    _compatibilityChecker = new VersionCompatibilityChecker();  
    _downloadQueue = new DownloadQueueManager();  
    _dependencyResolver = new DependencyResolver(ExtensionCatalog);  
    _platformMatcher = new PlatformMatcher();  
    // ...  
}
```

2.3 初始化流程

构造函数

- |
- |— 解析 `HostVersion`（宿主程序版本）
- |— 解析 `ExtensionsDirectory`（扩展安装根目录）
- |— 创建 `BackupDirectory = ExtensionsDirectory/.backup`
- |
- |— 注入所有服务依赖
- |
- |— 订阅 `DownloadQueue.DownloadStatusChanged`
 - 转发为 `ExtensionUpdateStatusChanged` 事件
- |
- |— 设置 `DownloadQueue.DownloadHandler`
 - 委托给 `_httpClient.DownloadExtensionAsync`
- |
- |— `Directory.CreateDirectory(ExtensionsDirectory)`
- |— `Directory.CreateDirectory(BackupDirectory)`
- |
- |— `ExtensionCatalog.LoadInstalledExtensions()`
 - 从 `manifest.json` 文件加载已安装扩展列表

3. ExtensionHostBuilder: DI Builder 模式

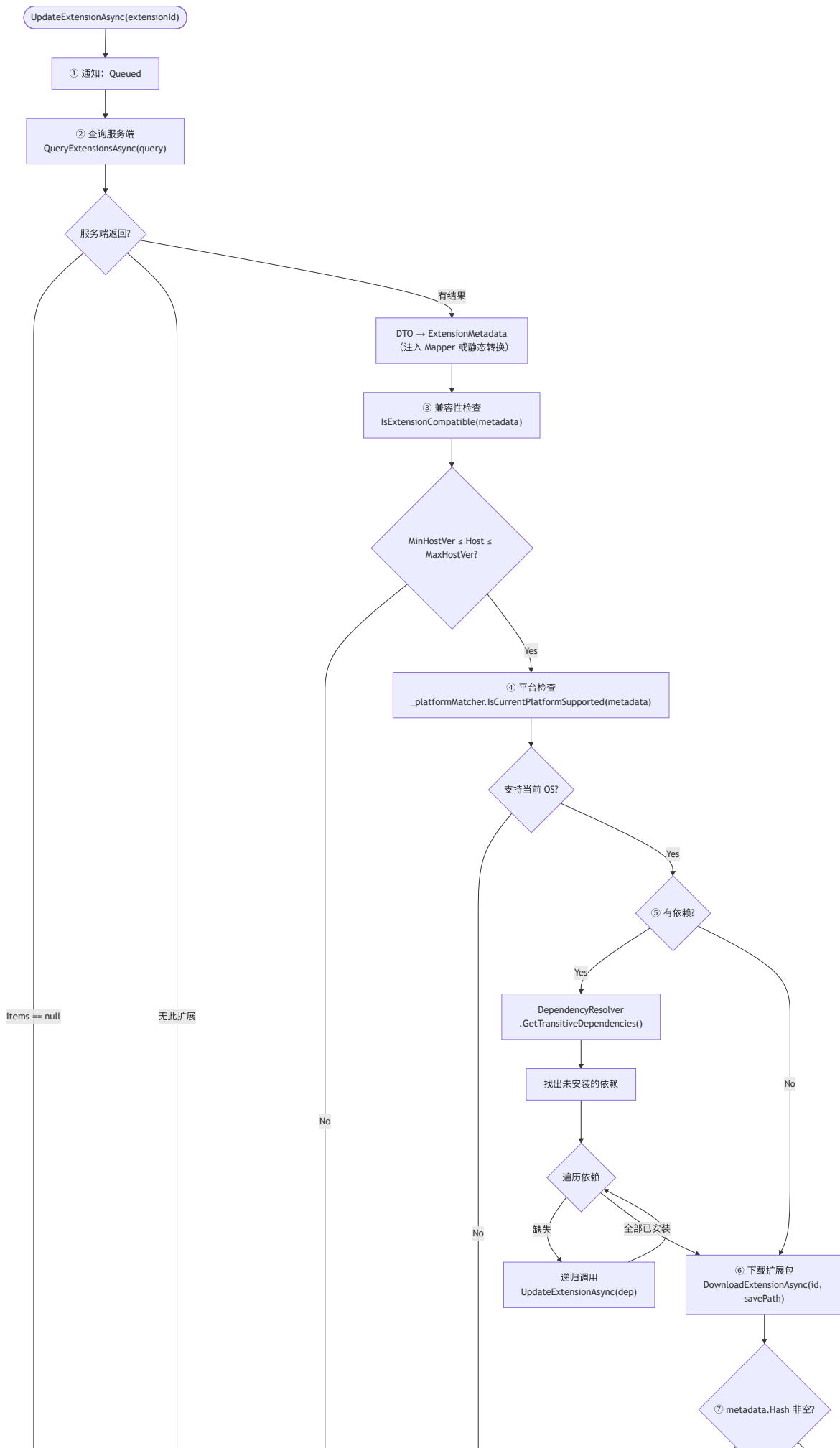
```
var host = new ExtensionHostBuilder()
    .ConfigureOptions(options =>
    {
        options.HostVersion = "2.0.0";
        options.ExtensionsDirectory = "./extensions";
        options.ServerUrl = "https://api.example.com";
    })
    .ConfigureServices(services =>
    {
        services.AddSingleton<IExtensionHttpClient, CustomHttpClient>(); // 替换 HTTP 客户端
        services.AddSingleton<IExtensionLifecycleHooks, MyLifecycleHooks>(); // 注入生命周期钩子
    })
    .Build();
```

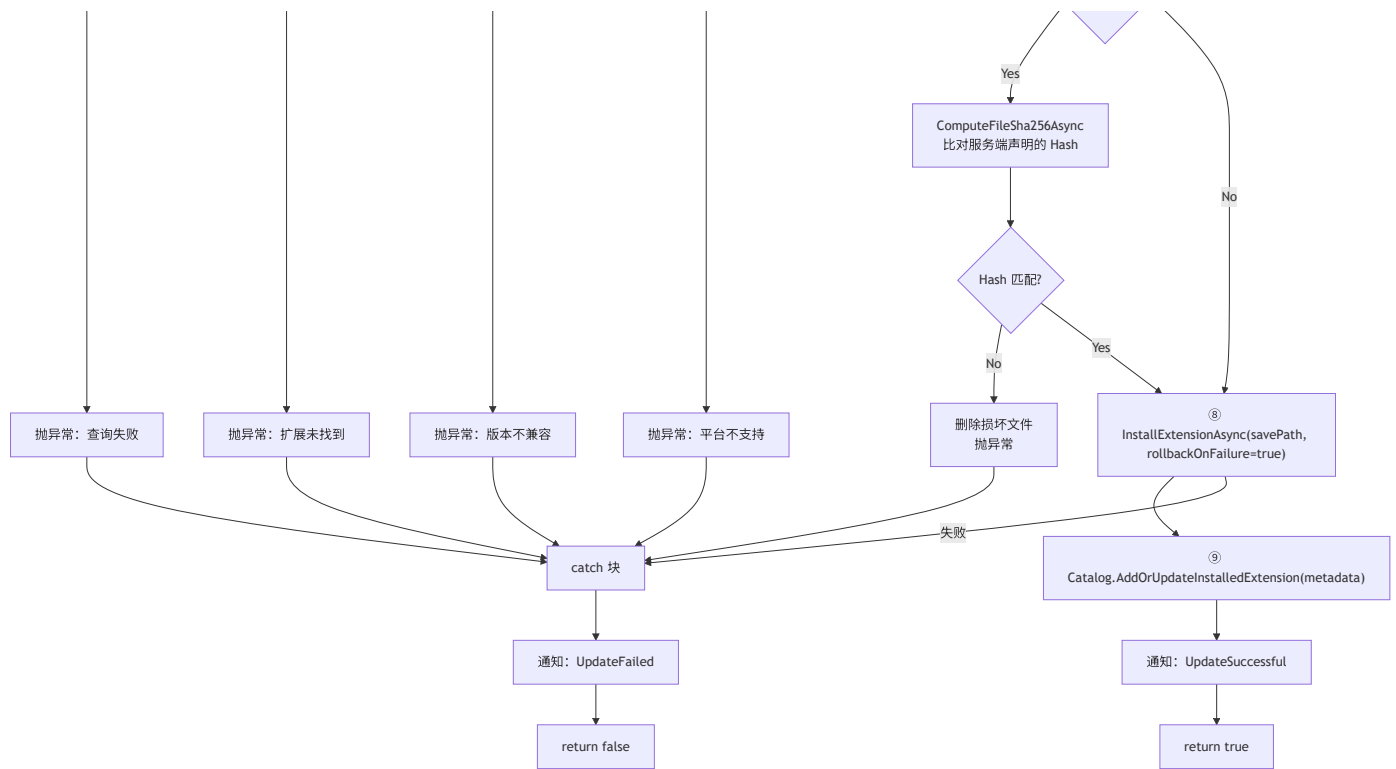
Builder 内部维护 `ServiceCollection`，在 `Build()` 时创建 `IServiceProvider` 并解析所有依赖。

4. UpdateExtensionAsync：一键更新完整流程

这是 Extension 最核心的方法。它串起查询 → 兼容性 → 平台 → 依赖递归 → 下载 → 哈希校验 → 安全安装 → Catalog 更新 → 事件通知的全流程。

4.1 全流程总图





4.2 步骤详解

步骤 ②：查询并映射

```

var response = await QueryExtensionsAsync(query); // HTTP GET /api/extensions?id=xxx
var serverExtension = response.Body.Items.FirstOrDefault(); // 从分页结果中筛选
var metadata = _metadataMapper?.ToMetadata(serverExtension) // DTO → 领域模型
                ?? ToMetadata(serverExtension); // 回退静态方法

```

步骤 ③：版本兼容性检查

```

public bool IsExtensionCompatible(ExtensionMetadata extension)
{
    return _compatibilityChecker.IsCompatible(extension, _hostVersion);
    // 内部逻辑:
    // MinHostVersion ≤ HostVersion ≤ MaxHostVersion
    // 使用 SemVer 2.0 比较
}

```

步骤 ④：平台匹配

```

public bool IsCurrentPlatformSupported(ExtensionMetadata metadata)
{
    var currentPlatform = GetCurrentPlatformFlags(); // Windows=1, Linux=2, macOS=4, Android=8...
    return (metadata.SupportedPlatforms & currentPlatform) != 0;
}

```

5. 依赖解析：DependencyResolver 深度解析

5.1 拓扑排序

```
public List<string> GetTransitiveDependencies(List<string> directDependencies)
{
    // 1. 构建依赖图（邻接表）
    // 2. Kahn 算法拓扑排序
    // 3. 检测循环依赖
    // 4. 返回安装顺序（被依赖的优先安装）
}
```

5.2 依赖安装决策树

```
DependencyResolver.GetTransitiveDependencies(deps)
|
├─ 展开传递依赖（A 依赖 B, B 依赖 C → 返回 [C, B, A]）
|
├─ 检测循环依赖（A → B → A）
|   └─ 检测到环 → 抛出异常
|
└─ 返回拓扑排序列表
```

调用方：

```
var missingDeps = sortedDeps.Where(d => Catalog.GetInstalledExtensionById(d) == null);

foreach (var dep in sortedDeps)
{
    if (missingDeps.Contains(dep))
        await UpdateExtensionAsync(dep); // 递归安装缺失的依赖
}
```

关键设计： 依赖安装失败会抛出异常，导致父扩展的更新流程也终止——这保证了依赖完整性，避免安装了扩展但缺少依赖的情况。

6. 兼容性检查：版本 + 平台双校验

6.1 版本兼容性（SemVer 2.0）

条件	说明
HostVersion < MinHostVersion	✗ 宿主太旧，扩展需要更高版本
HostVersion > MaxHostVersion	✗ 宿主太新，扩展尚未适配
MinHostVersion ≤ HostVersion ≤ MaxHostVersion	✓ 兼容

6.2 平台匹配 (位标志)

```
[Flags]
public enum TargetPlatform
{
    Windows = 1,
    Linux   = 2,
    macOS   = 4,
    Android = 8,
    iOS     = 16,
    All     = Windows | Linux | macOS | Android | iOS
}
```

`PlatformMatcher` 通过 `RuntimeInformation.IsOSPlatform()` 自动检测当前 OS，然后与扩展的 `SupportedPlatforms` 做位与运算。

7. 下载: DownloadQueueManager 并发控制

7.1 架构

```
DownloadQueueManager
|
|— SemaphoreSlim(3) // 默认最多 3 个并发下载
|
|— DownloadHandler 委托
|   → _httpClient.DownloadExtensionAsync(id, path, progress, ct)
|
|— DownloadStatusChanged 事件
|   → 转发到 GeneralExtensionHost.ExtensionUpdateStatusChanged
```

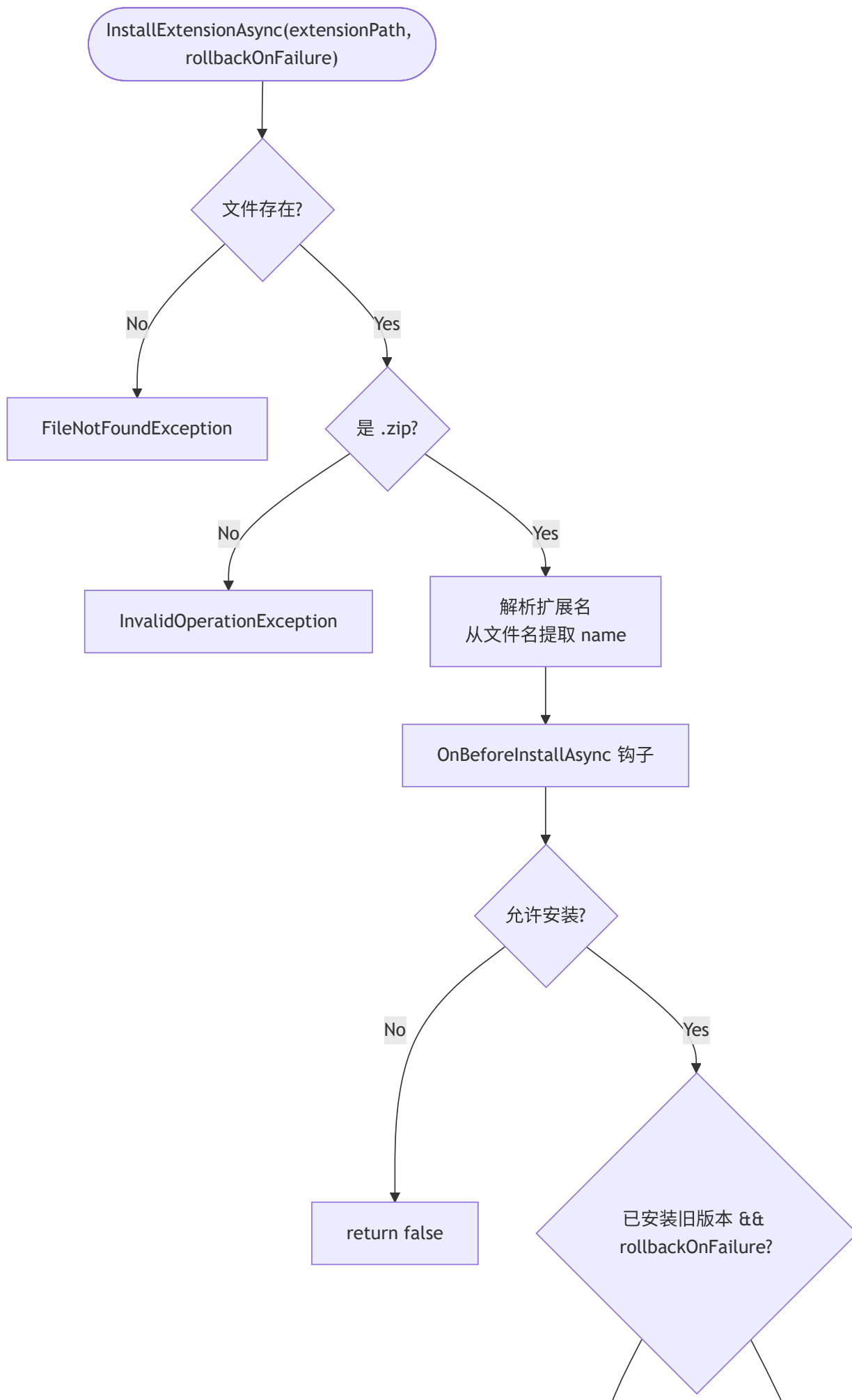
7.2 下载流程

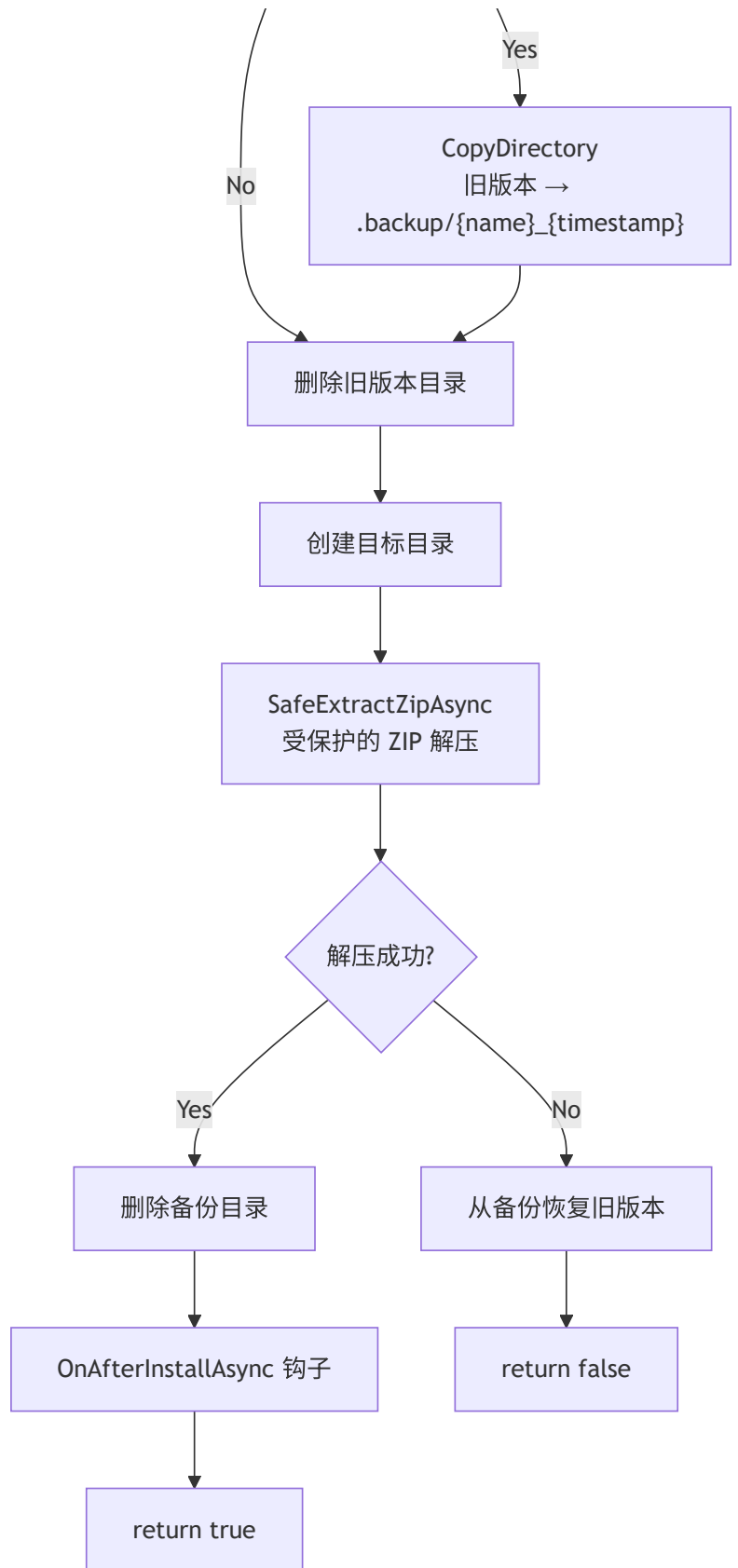
```
public async Task<bool> DownloadExtensionAsync(string extensionId, string savePath)
{
    var progress = new Progress<int>(p =>
    {
        ExtensionUpdateStatusChanged?.Invoke(this, new ExtensionUpdateEventArgs
        {
            ExtensionId = extensionId,
            Status = ExtensionUpdateStatus.Updating,
            Progress = p
        });
    });

    return await _httpClient.DownloadExtensionAsync(extensionId, savePath, progress);
}
```

8. 安装: InstallExtensionAsync 安全保障

8.1 安装流程





8.2 Zip Slip 防护

```
private static async Task SafeExtractZipAsync(string zipPath, string destinationDir)
{
    var fullDestDir = Path.GetFullPath(destinationDir)
        .TrimEnd(Path.DirectorySeparatorChar, Path.AltDirectorySeparatorChar);
```

```

using var archive = ZipFile.OpenRead(zipPath);
foreach (var entry in archive.Entries)
{
    var destinationPath = Path.GetFullPath(Path.Combine(fullDestDir, entry.FullName));

    // 核心防护：验证解析后的路径仍在目标目录内
    if (!destinationPath.StartsWith(fullDestDir + Path.DirectorySeparatorChar)
        && destinationPath != fullDestDir)
    {
        // 检测到路径穿越攻击 → 跳过此条目
        continue;
    }

    if (string.IsNullOrEmpty(entry.Name))
        Directory.CreateDirectory(destinationPath); // 目录条目
    else
        entry.ExtractToFile(destinationPath, overwrite: true); // 文件条目
}
}

```

攻击示例： 恶意 ZIP 包含条目 `../../../etc/malicious.dll` → `destinationPath` 解析为 `C:\etc\malicious.dll` → `StartsWith(fullDestDir)` 为 `false` → 被跳过。

9. Catalog：原子写入与崩溃安全

9.1 原子写入策略

```

// ExtensionCatalog.AddOrUpdateInstalledExtension
public void AddOrUpdateInstalledExtension(ExtensionMetadata metadata)
{
    // 1. 更新内存中的字典
    _installedExtensions[metadata.Id] = metadata;

    // 2. 先写临时文件
    var tempPath = manifestPath + ".tmp";
    var json = JsonConvert.SerializeObject(_installedExtensions.Values);
    File.WriteAllText(tempPath, json);

    // 3. 原子重命名（崩溃安全）
    File.Move(tempPath, manifestPath, overwrite: true);
}

```

为什么安全： 如果在 `File.WriteAllText(tempPath)` 期间崩溃 → `manifest.json` 未被修改 → 下次启动加载的是旧但完整的数据。如果在 `File.Move` 期间崩溃 → 文件系统保证重命名是原子的。

9.2 每个扩展独立 manifest

每个扩展目录下都有独立的 `manifest.json`：

```
extensions/
├─ demo-extension/
│   └─ manifest.json    ← 此扩展的元数据
│   └─ ... (扩展文件)
├─ report-module/
│   └─ manifest.json
│   └─ ...
└─ .backup/             ← 备份目录
```

10. 生命周期钩子：8 个事件注入点

```
public interface IExtensionLifecycleHooks
{
    // 安装
    Task<bool> OnBeforeInstallAsync(ExtensionMetadata metadata, string packagePath);
    Task OnAfterInstallAsync(ExtensionMetadata metadata);

    // 激活
    Task OnBeforeActivateAsync(string extensionId, CancellationTokens ct);
    Task OnAfterActivateAsync(string extensionId, CancellationTokens ct);

    // 停用
    Task OnBeforeDeactivateAsync(string extensionId, CancellationTokens ct);
    Task OnAfterDeactivateAsync(string extensionId, CancellationTokens ct);

    // 卸载
    Task<bool> OnBeforeUninstallAsync(ExtensionMetadata metadata, CancellationTokens ct);
    Task OnAfterUninstallAsync(string extensionId, CancellationTokens ct);
}
```

10.1 钩子返回值语义

钩子	返回 false 的行为
OnBeforeInstallAsync	取消安装，返回 false
OnBeforeUninstallAsync	取消卸载，返回 false
其他 After 钩子	仅通知，返回值不影响流程

10.2 典型使用场景

```
public class MyLifecycleHooks : IExtensionLifecycleHooks
{
    public async Task<bool> OnBeforeInstallAsync(ExtensionMetadata metadata, string packagePath)
    {
        // 检查许可证是否有效
        if (!await LicenseService.ValidateAsync(metadata.Id))
            return false; // 阻止安装
        return true;
    }

    public async Task OnAfterInstallAsync(ExtensionMetadata metadata)
    {
        // 记录审计日志
        await AuditLog.WriteAsync($"Extension installed: {metadata.Id}");
    }
}
```

11. 关键代码路径索引

组件	文件	关键方法
扩展主机 (编排)	Core/GeneralExtensionHost.cs	UpdateExtensionAsync() / InstallExtensionAsync() / SafeExtractZipAsync()
主机接口	Core/IExtensionHost.cs	—
Builder	Core/ExtensionHostBuilder.cs	ConfigureOptions() / ConfigureServices() / Build()
工厂	Core/ExtensionServiceFactory.cs	—
生命周期钩子	Core/IExtensionLifecycleHooks.cs	8 个钩子方法
Catalog	Catalog/ExtensionCatalog.cs	AddOrUpdateInstalledExtension() / LoadInstalledExtensions()
下载队列	Download/DownloadQueueManager.cs	SemaphoreSlim 并发控制
依赖解析	Dependencies/DependencyResolver.cs	GetTransitiveDependencies()
平台匹配	Compatibility/PlatformMatcher.cs	IsCurrentPlatformSupported()
版本兼容检查	Compatibility/VersionCompatibilityChecker.cs	IsCompatible()
HTTP 通信	Communication/ExtensionHttpClient.cs	QueryExtensionsAsync() / DownloadExtensionAsync()
扩展元数据	Common/Models/ExtensionMetadata.cs	—

组件	文件	关键方法
日志追踪	GeneralTracer.cs	—

GeneralUpdate.Avalonia.Android — 执行流程详解

目标读者： 需要在 Avalonia Android 应用中集成自动更新的开发者

阅读完你将理解：

- `AndroidBootstrap` 的显式三步 API 设计意图 (Validate → DownloadAndVerify → LaunchInstaller)
- `HttpResumableApkDownloader` 的断点续传机制：Range 请求 + Sidecar 元数据 + 原子重命名
- SHA256 校验的安全上下限与文件大小双重验证
- Android APK 安装的 FileProvider URI 授权流程
- `SemaphoreSlim(1,1)` 并发保护与线程安全设计
- `IUpdateEventDispatcher` 的 UI 线程调度机制
- 多协议认证架构 (HMAC / Bearer / API Key / Basic)

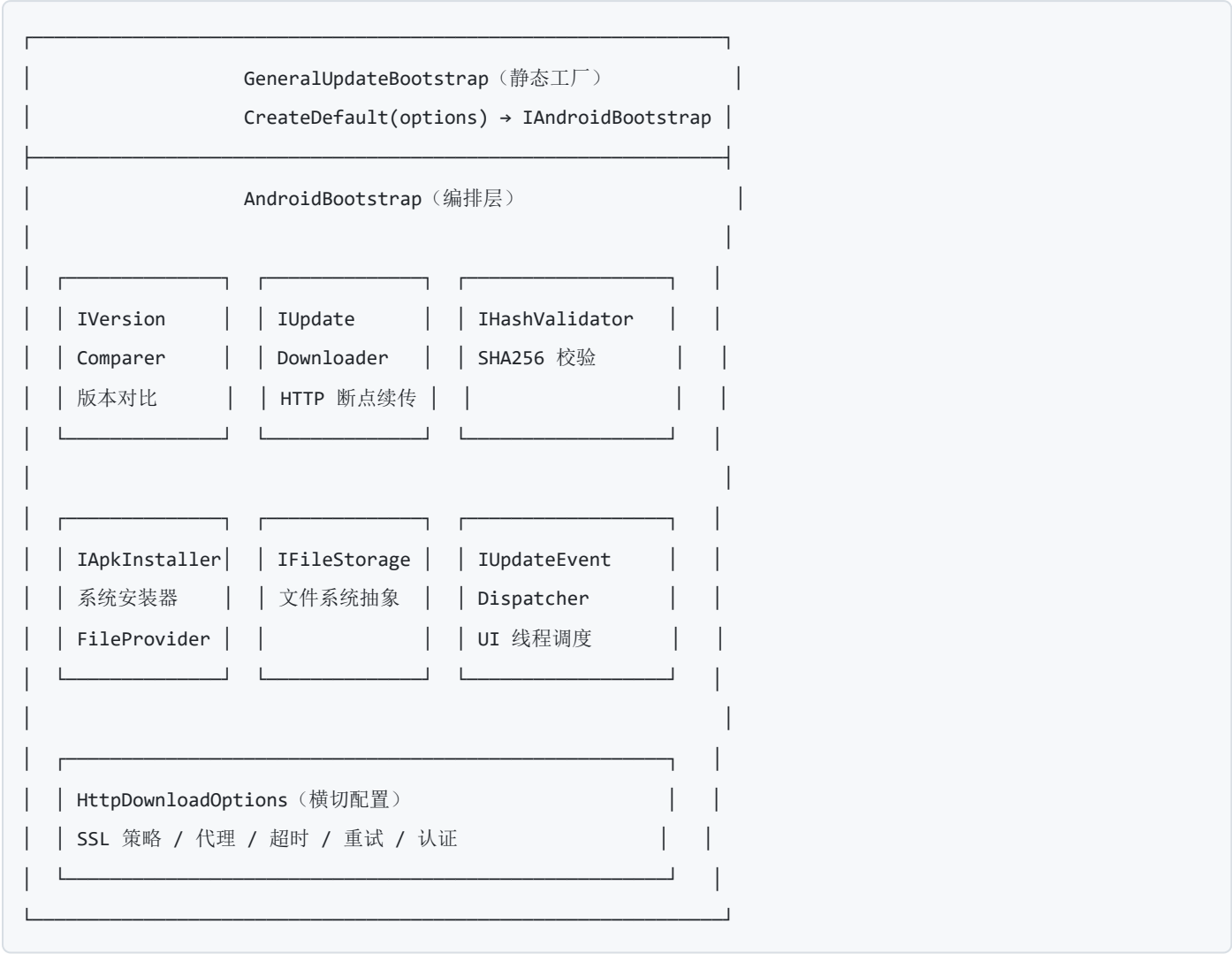
目录

1. 架构总览
 2. 入口：GeneralUpdateBootstrap 工厂
 3. AndroidBootstrap：三步显式 API
 4. Step 1: ValidateAsync — 版本校验
 5. Step 2: DownloadAndVerifyAsync — 下载与校验
 6. 断点续传：HttpResumableApkDownloader 深度解析
 7. 哈希校验与文件大小验证
 8. Step 3: LaunchInstallerAsync — APK 安装触发
 9. 并发安全与线程模型
 10. 事件调度与 UI 线程
 11. 多协议认证架构
 12. 关键代码路径索引
-

1. 架构总览

1.1 六服务可替换架构

Avalonia.Android 采用**全接口抽象 + 工厂装配**的设计，每个核心能力都有接口，可独立替换：



2. 入口: GeneralUpdateBootstrap 工厂

```
public static class GeneralUpdateBootstrap
{
    public static IAndroidBootstrap CreateDefault(AndroidUpdateOptions? options = null)
    {
        options ??= new AndroidUpdateOptions();
        var downloadDir = ResolveDownloadDirectory(options);

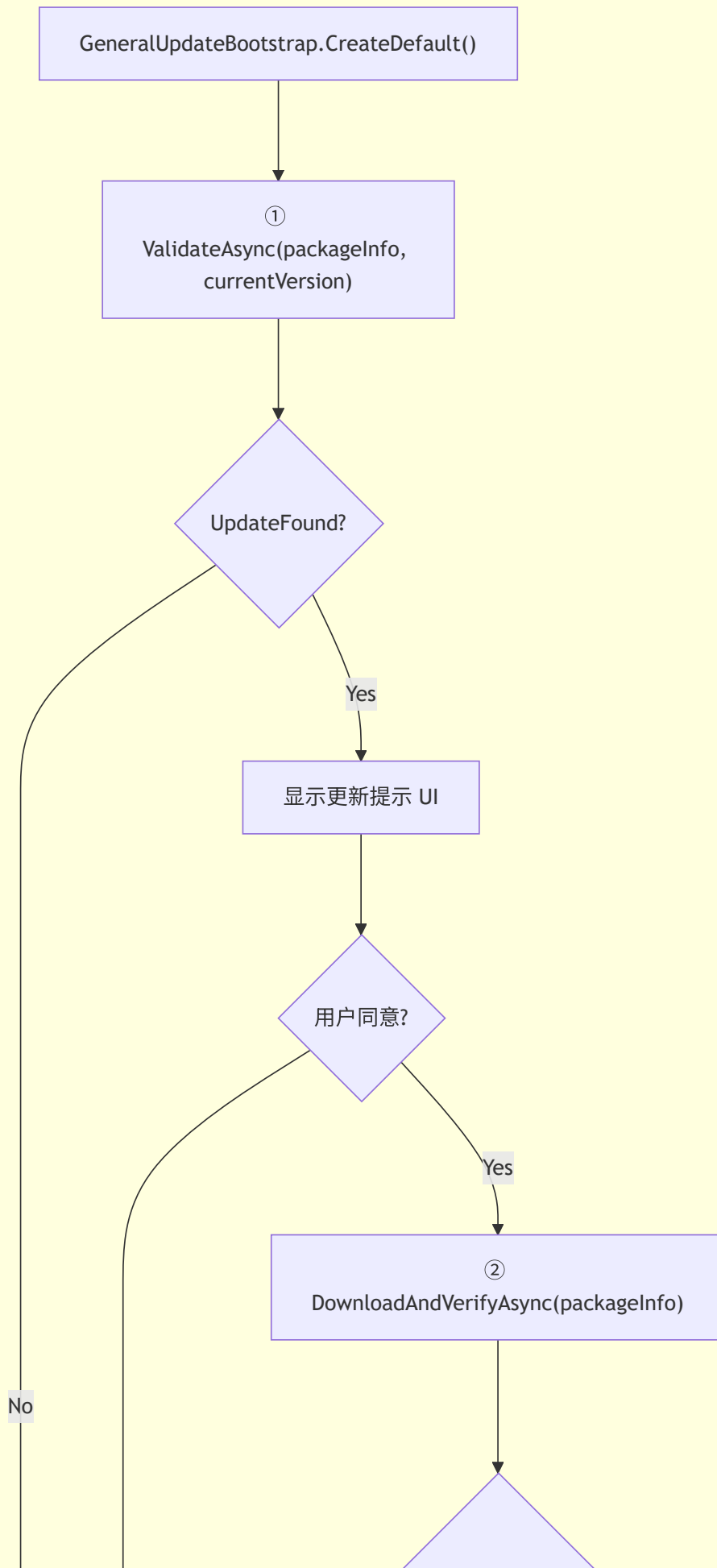
        return new AndroidBootstrap(
            versionComparer: new SystemVersionComparer(),
            downloader: new HttpResumableApkDownloader(options.HttpOptions),
            hashValidator: new Sha256HashValidator(),
            apkInstaller: new AndroidApkInstaller(),
            fileStorage: new PhysicalFileStorage(),
            eventDispatcher: new ImmediateEventDispatcher(),
            logger: new NoOpUpdateLogger()
        );
    }

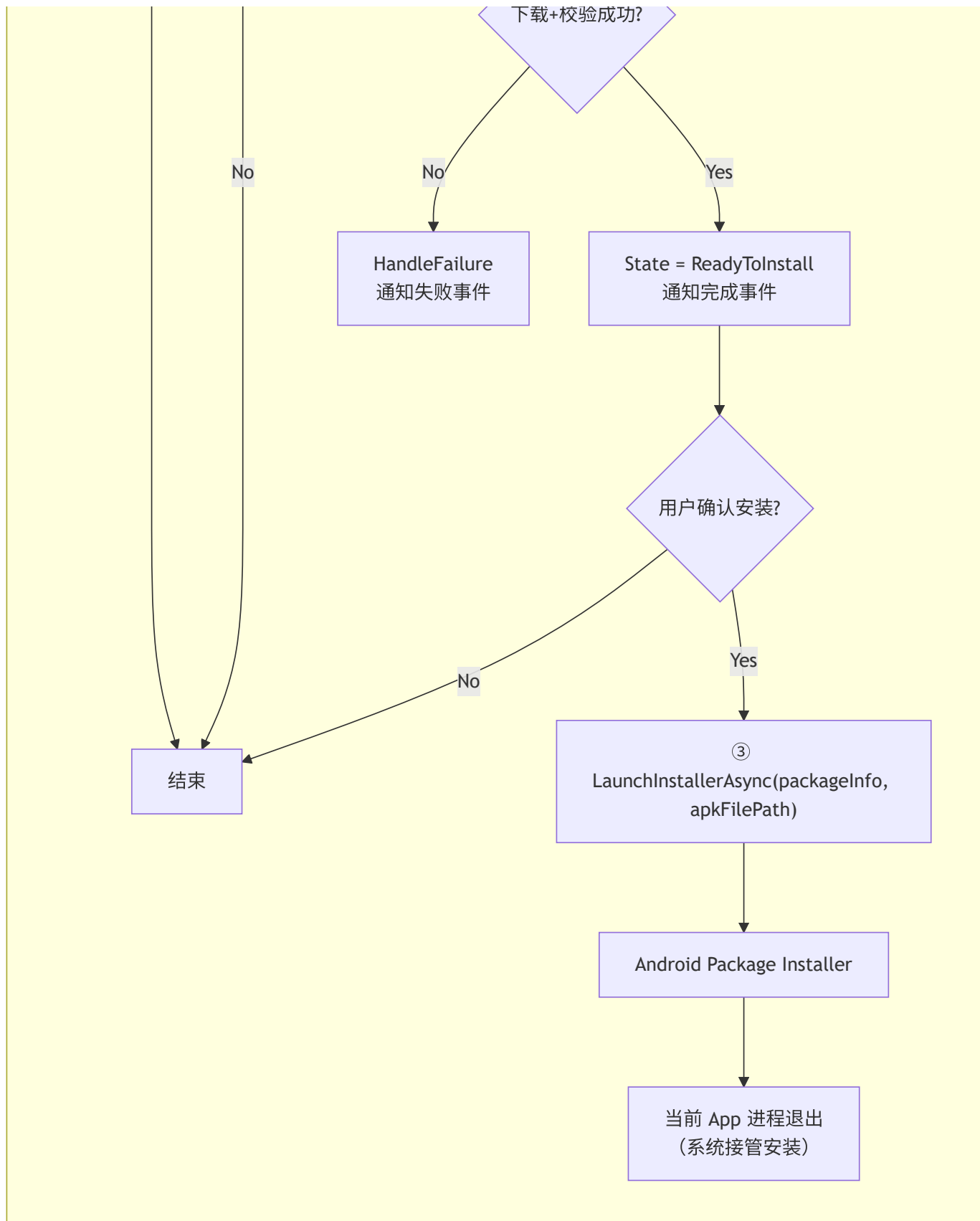
    private static string ResolveDownloadDirectory(AndroidUpdateOptions options)
    {
        // 1. 优先使用用户指定的路径
        // 2. 回退到 Android 缓存目录
        // 3. 确保目录存在
    }
}
```

3. AndroidBootstrap: 三步显式 API

3.1 完整生命周期

Avalonia App 调用流程





3.2 状态机

None → Checking → UpdateAvailable → Downloading → Verifying → ReadyToInstall → Installing → Completed

↓
Failed

↓
Failed

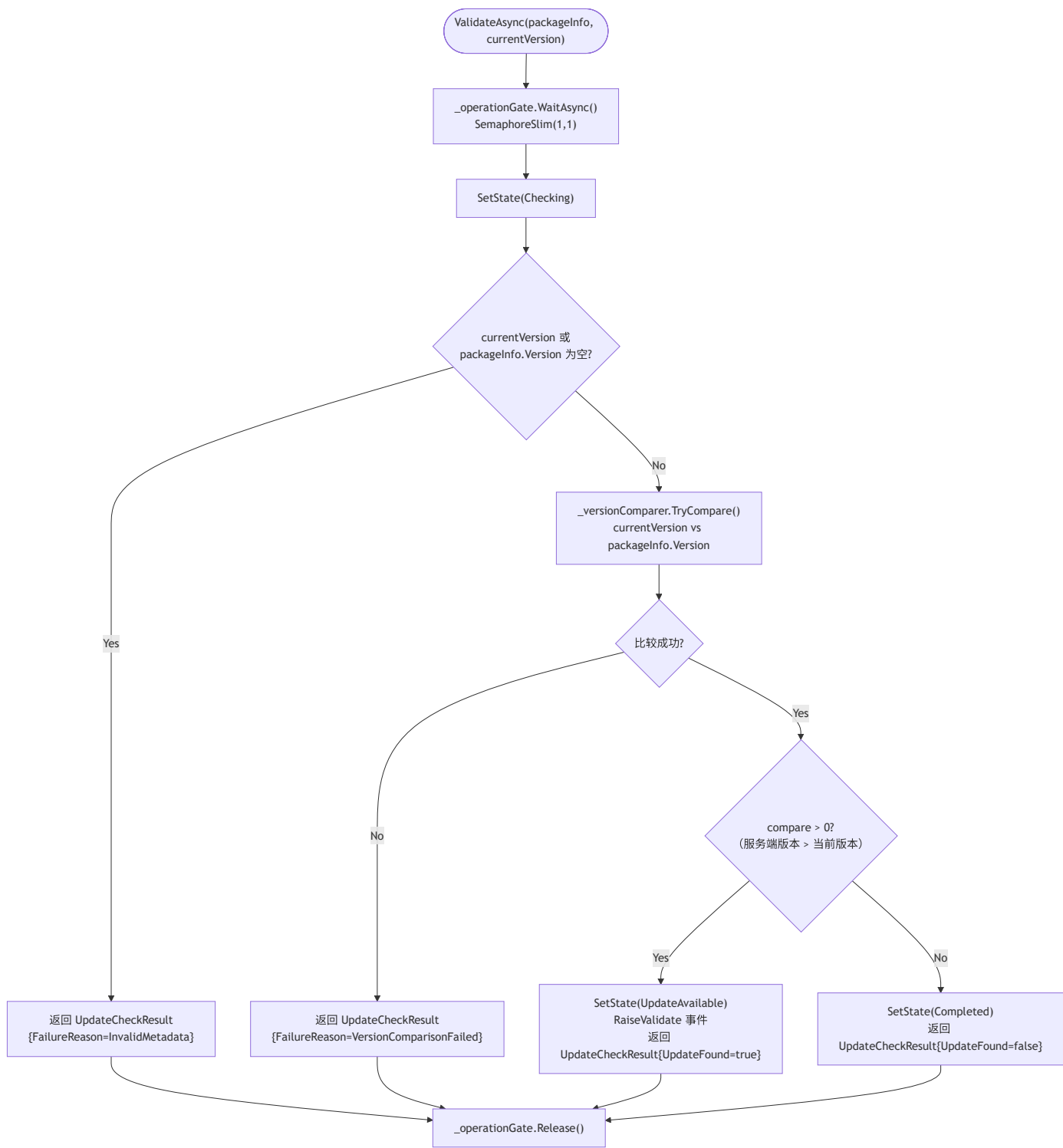
↓
Canceled

↓
Canceled

状态通过 `SetState()` 线程安全地更新，可通过 `GetSnapshot()` 随时查询：

```
public UpdateStateSnapshot GetSnapshot()
{
    lock (_sync)
    {
        return _snapshot; // { State, FailureReason, Message }
    }
}
```

4. Step 1: ValidateAsync — 版本校验



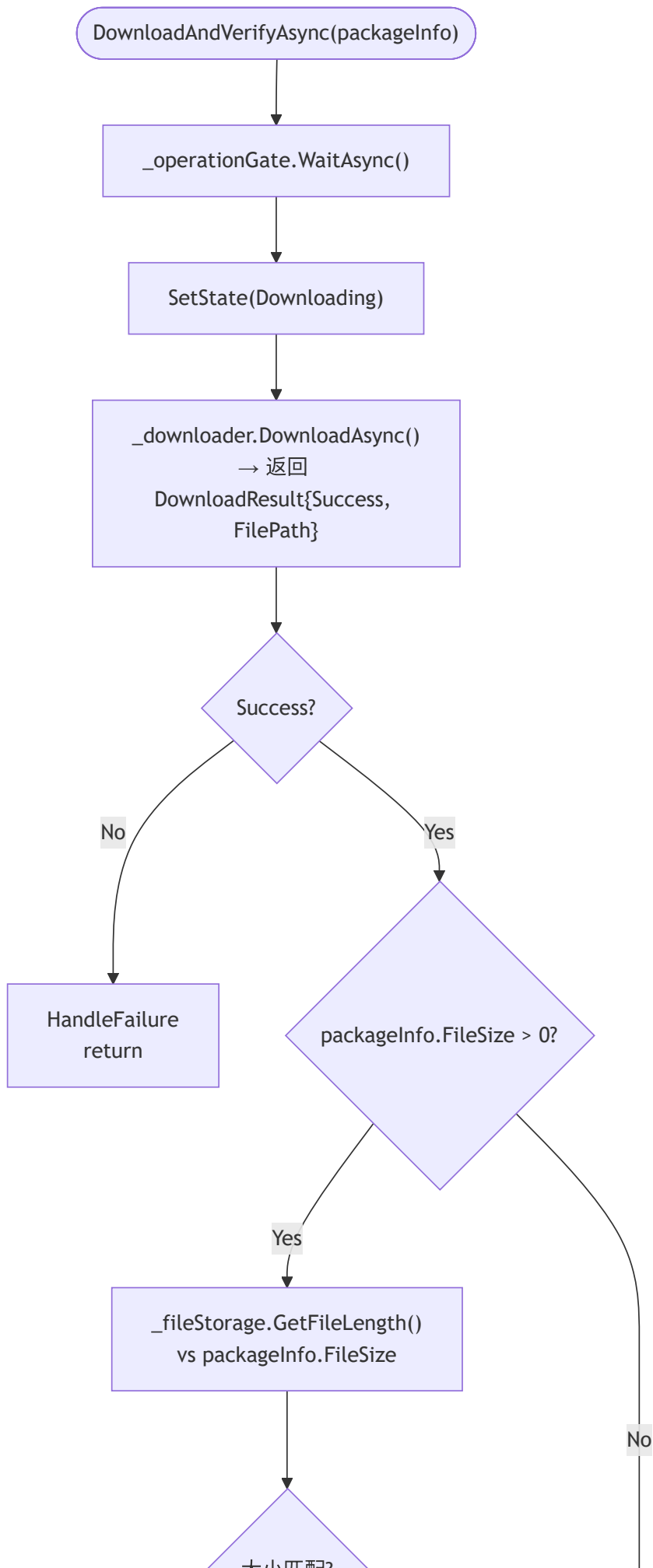
4.1 版本比较器接口

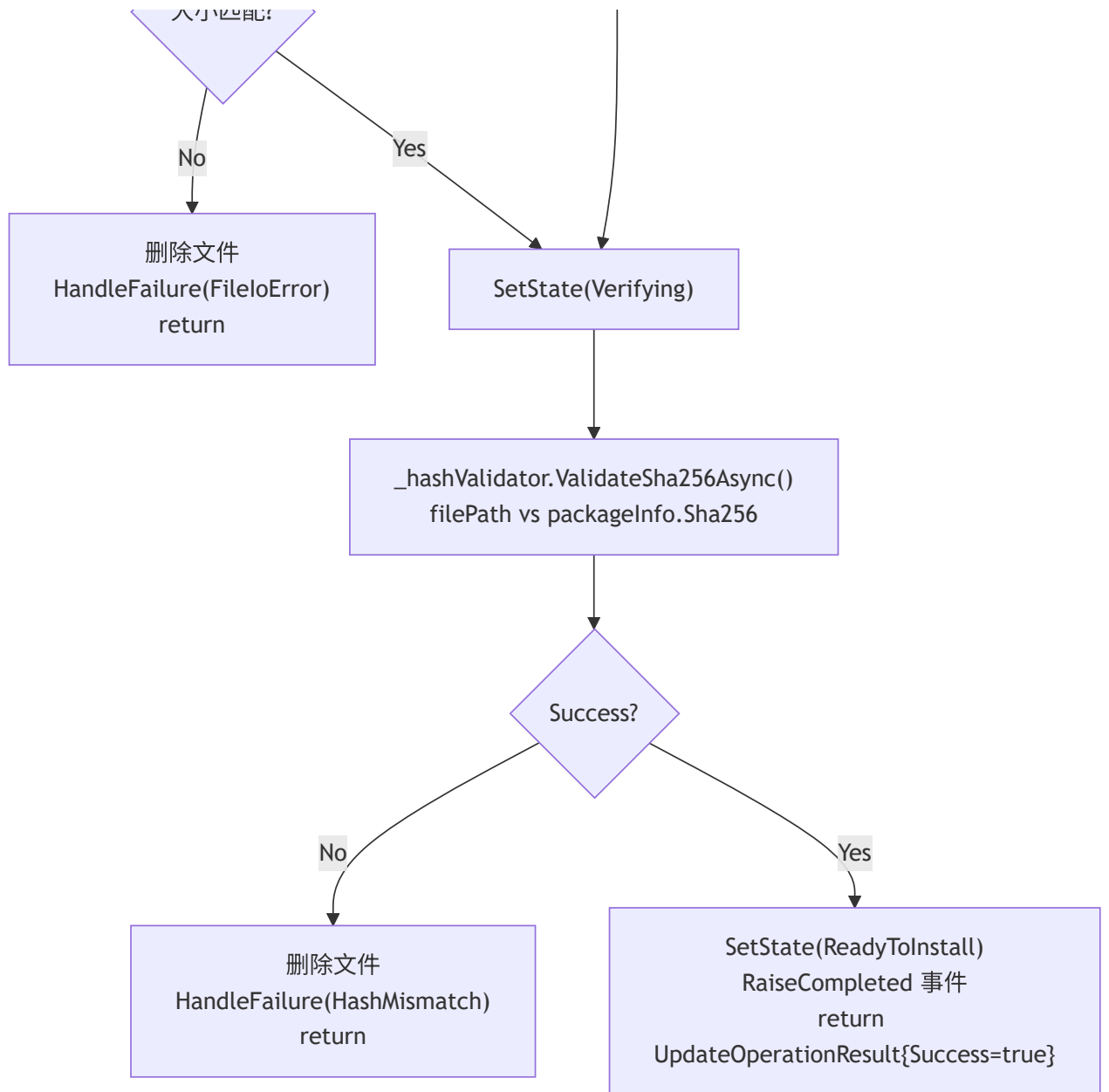
```
public interface IVersionComparer
{
    bool TryCompare(string currentVersion, string targetVersion,
                    out int compare, out string? error);

    // compare > 0: targetVersion 更大（有更新）
    // compare = 0: 相同
    // compare < 0: targetVersion 更小（降级，通常不应发生）
}
```

默认实现 `SystemVersionComparer` 使用 `System.Version` 类进行语义化比较。

5. Step 2: DownloadAndVerifyAsync — 下载与校验





6. 断点续传：HttpResumableApkDownloader 深度解析

6.1 核心机制

下载流程

|

└─ Phase 1: HEAD 请求

|

探测服务端能力: Accept-Ranges, ETag, Content-Length

|

└─ Phase 2: 检测已有部分下载

|

检查 {filename}.part + {filename}.json (sidecar)

|

sidecar 记录: URL, SHA256, 已下载字节数, ETag, LastModified

|

└─ Phase 3: 续传一致性验证

|

URL 是否相同? SHA256 是否相同? 文件大小是否匹配? ETag 是否匹配?

|

如果任何一项变化 → 删除旧临时文件, 从头下载

|

└─ Phase 4: GET with Range

|

从头下载: GET {url}

|

续传: GET {url} + Range: bytes={downloaded}-

|

流式写入 {filename}.part, 实时更新 sidecar

|

└─ Phase 5: 原子重命名

|

下载完成 → {filename}.part → {filename}.apk

|

删除 sidecar

|

└─ Progress 报告

已下载字节数、总大小、速度 (bytes/s)、百分比

6.2 Sidecar 元数据

{

"url": "https://cdn.example.com/app-v2.0.apk",

"sha256": "a1b2c3d4...",

"downloadedBytes": 15728640,

"totalBytes": 52428800,

"etag": "\"abc123\"",

"lastModified": "2026-06-01T12:00:00Z"

}

6.3 速度计量

```
// SpeedCalculator 基于滑动窗口计算下载速度
public class SpeedCalculator
{
    // 记录最近 N 秒的字节数
    // 计算 bytes/sec, 格式化为可读字符串 (KB/s, MB/s)
}
```

7. 哈希校验与文件大小验证

7.1 双重验证

DownloadAndVerifyAsync 验证顺序:

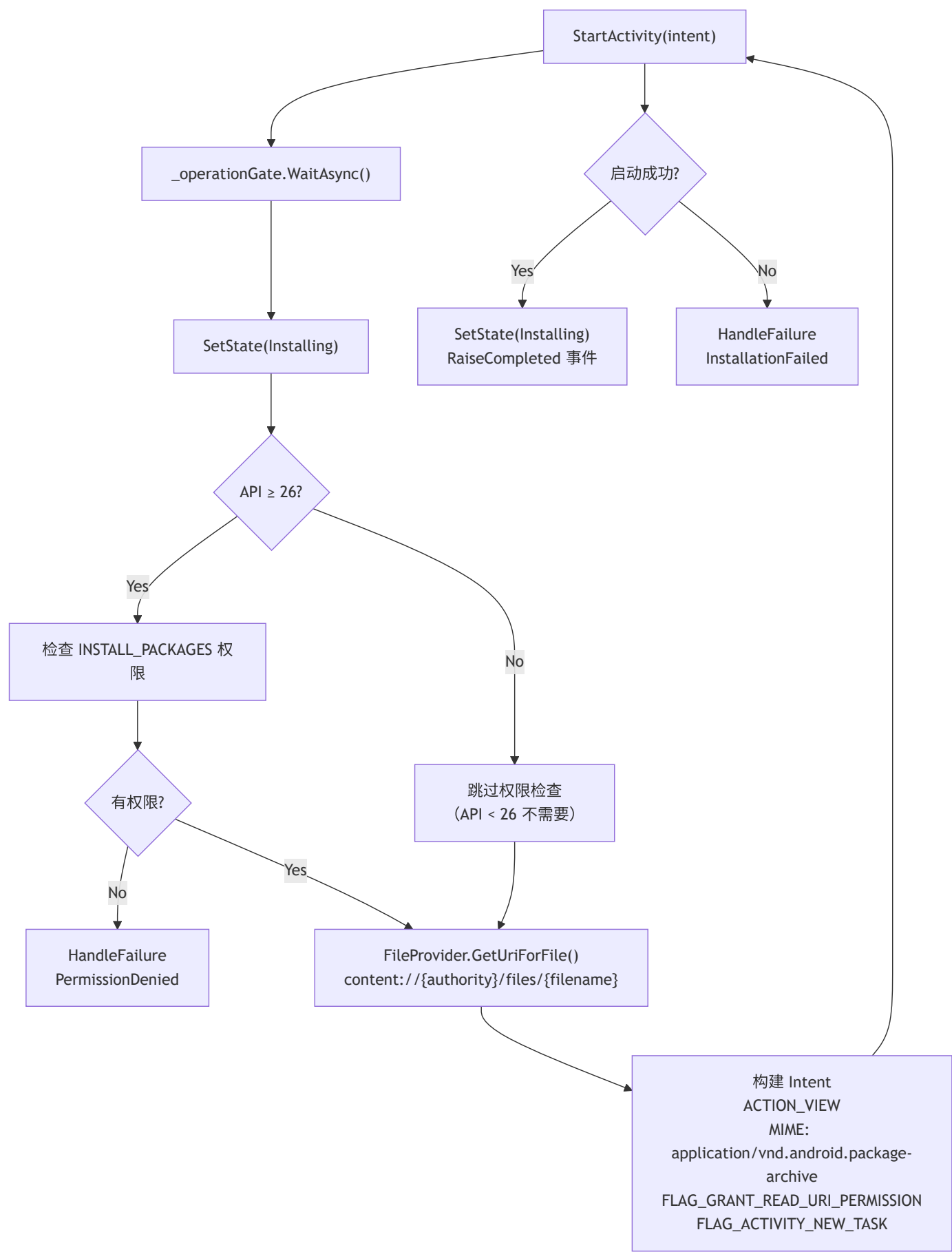
1. 文件大小验证 (如果 metadata 提供了 FileSize)
 - 不匹配 → 删除文件, 返回 `FileIoError`
2. SHA256 验证 (如果 metadata 提供了 Sha256)
 - 不匹配 → 删除文件, 返回 `HashMismatch`

7.2 Sha256HashValidator

```
public class Sha256HashValidator : IHashValidator
{
    public async Task ValidateSha256Async(
        string filePath, string expectedHash, CancellationToken ct)
    {
        var actualHash = await ComputeSha256Async(filePath, ct);
        if (!string.Equals(actualHash, expectedHash, StringComparison.OrdinalIgnoreCase))
        {
            return new UpdateOperationResult
            {
                Success = false,
                FailureReason = UpdateFailureReason.HashMismatch,
                Message = $"Hash mismatch. Expected: {expectedHash}, Actual: {actualHash}"
            };
        }
        return new UpdateOperationResult { Success = true };
    }
}
```

8. Step 3: LaunchInstallerAsync — APK 安装触发

8.1 Android 安装流程



8.2 FileProvider 配置

在 `AndroidManifest.xml` 中：

```
<application>
    <provider
        android:name="androidx.core.content.FileProvider"
        android:authorities="${applicationId}.fileprovider"
        android:exported="false"
        android:grantUriPermissions="true">
        <meta-data
            android:name="android.support.FILE_PROVIDER_PATHS"
            android:resource="@xml/file_paths" />
    </provider>
</application>
```

在 `xml/file_paths.xml` 中：

```
<paths>
    <cache-path name="updates" path="." ></cache-path>
    <external-files-path name="external_updates" path="." ></external-files-path>
</paths>
```

9. 并发安全与线程模型

9.1 SemaphoreSlim 操作门

```
private readonly SemaphoreSlim _operationGate = new(1, 1);

public async Task ValidateAsync(...)
{
    await _operationGate.WaitAsync(cancellationToken);
    try
    {
        // 临界区：版本检查
    }
    finally
    {
        _operationGate.Release();
    }
}
```

设计意图：

- 三步 API 各自独立获取和释放锁

- 调用方在步骤之间可以做 UI 操作（显示对话框等）
- 但在任一步骤内部，不会有并发操作
- `SemaphoreSlim(1,1)` 确保同一时间只有一个操作在执行

9.2 状态快照的线程安全

```
private readonly object _sync = new();
private UpdateStateSnapshot _snapshot;

private void SetState(UpdateState state, UpdateFailureReason reason, string? message)
{
    lock (_sync)
    {
        _snapshot = new UpdateStateSnapshot(state, reason, message);
    }
}
```

10. 事件调度与 UI 线程

10.1 IUpdateEventDispatcher

```
public interface IUpdateEventDispatcher
{
    void Dispatch(Action action);
}
```

默认实现 `ImmediateEventDispatcher` 直接执行委托。调用方可替换为 Avalonia 的 UI 线程调度：

```
public class AvaloniaEventDispatcher : IUpdateEventDispatcher
{
    public void Dispatch(Action action)
    {
        Avalonia.Threading.Dispatcher.UIThread.Post(action);
    }
}
```

10.2 事件体系

事件	EventArgs	触发时机
<code>AddListenerValidate</code>	<code>ValidateEventArgs</code>	发现可用更新
<code>AddListenerDownloadProgressChanged</code>	<code>DownloadProgressChangedEventArgs</code>	下载进度（速度、字节、百分比）
<code>AddListenerUpdateCompleted</code>	<code>UpdateCompletedEventArgs</code>	下载+校验完成 / 安装触发
<code>AddListenerUpdateFailed</code>	<code>UpdateFailedEventArgs</code>	任何步骤失败

11. 多协议认证架构

11.1 认证方案

```
public interface IHttpAuthProvider
{
    void ApplyAuth(HttpRequestMessage request);
}
```

支持四种方案：

方案	实现	典型 Header
HMAC-SHA256	HmacAuthProvider	Authorization: HMAC-SHA256 {signature}
Bearer Token	BearerAuthProvider	Authorization: Bearer {token}
API Key	ApiKeyAuthProvider	X-API-Key: {key}
HTTP Basic	BasicAuthProvider	Authorization: Basic {base64}

11.2 全局 vs 单包粒度

```
// 全局认证：所有下载请求共用
var options = new AndroidUpdateOptions
{
    HttpOptions = new HttpDownloadOptions
    {
        AuthProvider = new BearerAuthProvider("global-token")
    }
};

// 单包认证：特定包的下载使用不同凭据
var packageInfo = new UpdatePackageInfo
{
    AuthScheme = AuthScheme.Bearer,
    AuthToken = "per-package-token"
};

// 单包认证优先级 > 全局认证
```

12. 关键代码路径索引

组件	文件	关键方法
静态工厂	GeneralUpdateBootstrap.cs	CreateDefault()
编排器	Services/AndroidBootstrap.cs	ValidateAsync() / DownloadAndVerifyAsync() / LaunchInstallerAsync()

组件	文件	关键方法
编排器接口	Abstractions/IAndroidBootstrap.cs	—
断点续传下载器	Services/HttpResumableApkDownloader.cs	DownloadAsync() / HEAD 探测 / Range 请求
SHA256 校验	Services/Sha256HashValidator.cs	ValidateSha256Async()
APK 安装器	Services/AndroidApkInstaller.cs	LaunchInstallAsync() / FileProvider URI
版本比较器	Services/SystemVersionComparer.cs	TryCompare()
文件存储	Services/PhysicalFileStorage.cs	GetFileLength() / DeleteFile()
事件调度	Services/ImmediateEventDispatcher.cs	Dispatch()
速度计量	Utilities/SpeedCalculator.cs	滑动窗口速度计算
下载选项	Models/HttpDownloadOptions.cs	SSL / 代理 / 超时 / 重试 / 认证
更新选项	Models/AndroidUpdateOptions.cs	DownloadDirectory / FileProviderAuthority
认证接口	Abstractions/IHttpAuthProvider.cs	ApplyAuth()
SSL 策略	Abstractions/ISslValidationPolicy.cs	—

GeneralUpdate.Maui.Android — 执行流程详解

目标读者： 需要在 .NET MAUI Android 应用中集成自动更新的开发者

阅读完你将理解：

- `AndroidBootstrap` 的合并式两步 API 设计意图 (`ValidateAsync` → `ExecuteUpdateAsync`)
 - `HttpRangeDownloader` 的可恢复下载机制：Range 请求 + 临时文件 + 原子替换
 - `ExecuteUpdateAsync` 内部的 Interlocked 并发保护与状态原子转换
 - SHA256 校验失败后自动清理损坏文件的策略
 - Android Package Installer 的 FileProvider + Intent 触发流程
 - `AddGeneralUpdateMauiAndroid()` DI 注册扩展的设计
 - 与 Avalonia.Android 在 API 风格和 DI 策略上的核心差异
 - 异常到 `UpdateFailureReason` 的分类映射
-

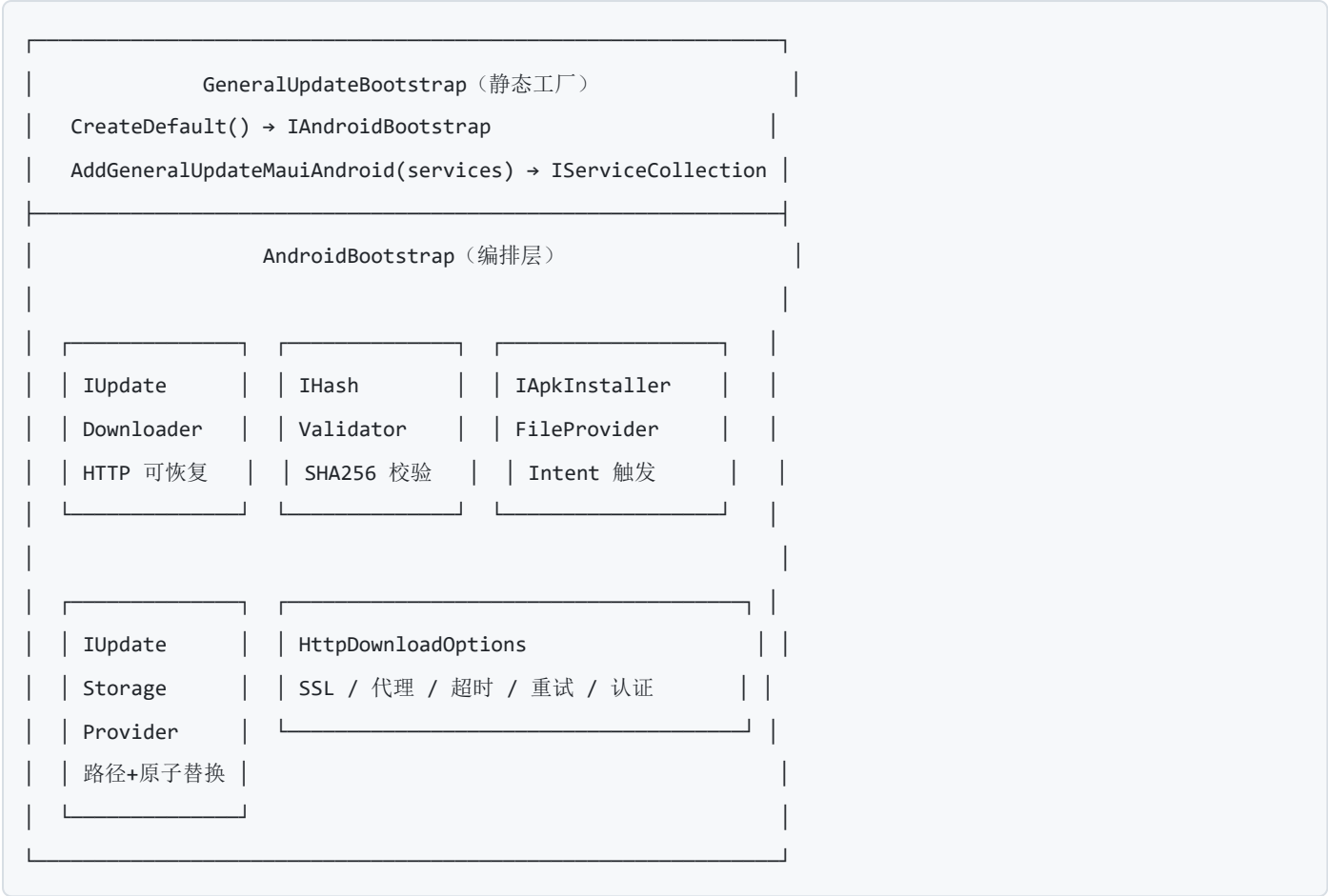
目录

1. 架构总览
 2. 入口：DI 优先的工厂设计
 3. `AndroidBootstrap`：合并式两步 API
 4. Step 1: `ValidateAsync` — 版本校验
 5. Step 2: `ExecuteUpdateAsync` — 原子执行完整更新
 6. 可恢复下载：`HttpRangeDownloader` 深度解析
 7. SHA256 校验与损坏文件清理
 8. APK 安装：平台守卫的 Installer
 9. 并发安全：Interlocked 原子操作
 10. `SafeInvoke`：防御式事件触发
 11. 异常映射：`MapFailureReason` 分类机制
 12. 与 `Avalonia.Android` 的设计对比
 13. 关键代码路径索引
-

1. 架构总览

1.1 五服务 DI 优先架构

Maui.Android 采用 **DI 优先 + 手动装配并存** 的设计：



1.2 与 Avalonia.Android 的核心差异

维度	Maui.Android	Avalonia.Android
API 风格	两步合并式 (Validate + ExecuteUpdate)	三步显式 (Validate + DownloadAndVerify + LaunchInstaller)
DI 策略	DI 优先, AddGeneralUpdateMauiAndroid()	手动装配优先, CreateDefault()
并发保护	Interlocked 原子操作 + 状态原子转换	SemaphoreSlim(1,1) 操作门
事件安全	SafeInvoke 迭代委托列表	IUpdateEventDispatcher 调度
平台守卫	#if ANDROID 编译时守卫	运行时平台检查
下载临时文件	.downloading 扩展名	.part + .json sidecar
进度报告	IProgress	EventHandler
完成阶段	4 阶段事件 (Download/Verify/Install/Workflow)	2 事件 (Completed/Failed)

2. 入口：DI 优先的工厂设计

2.1 DI 注册扩展

```
public static class GeneralUpdateBootstrap
{
    // DI 优先：一键注册所有服务
    public static IServiceCollection AddGeneralUpdateMauiAndroid(
        this IServiceCollection services,
        HttpClient? httpClient = null)
    {
        services.AddSingleton(sp =>
            new HttpRangeDownloader(httpClient ?? new HttpClient()));
        services.AddSingleton<IHashValidator, Sha256Validator>();
        services.AddSingleton<IApkInstaller, AndroidApkInstaller>();
        services.AddSingleton<IUpdateStorageProvider, UpdateFileStore>();
        services.AddSingleton<IUpdateLogger, DefaultUpdateLogger>();
        services.AddSingleton<IAndroidBootstrap, AndroidBootstrap>();
        return services;
    }

    // 手动装配：非 DI 场景
    public static IAndroidBootstrap CreateDefault(
        HttpClient? httpClient = null,
        IUpdateLogger? logger = null,
        HttpDownloadOptions? httpOptions = null)
    {
        var client = httpClient ?? new HttpClient();
        return new AndroidBootstrap(
            new HttpRangeDownloader(client),
            new Sha256Validator(),
            new AndroidApkInstaller(),
            new UpdateFileStore(),
            logger);
    }
}
```

2.2 MAUI 应用中的典型注册

```
// MauiProgram.cs
public static MauiApp CreateMauiApp()
{
    var builder = MauiApp.CreateBuilder();
    builder.Services.AddGeneralUpdateMauiAndroid();
    // ...
    return builder.Build();
}

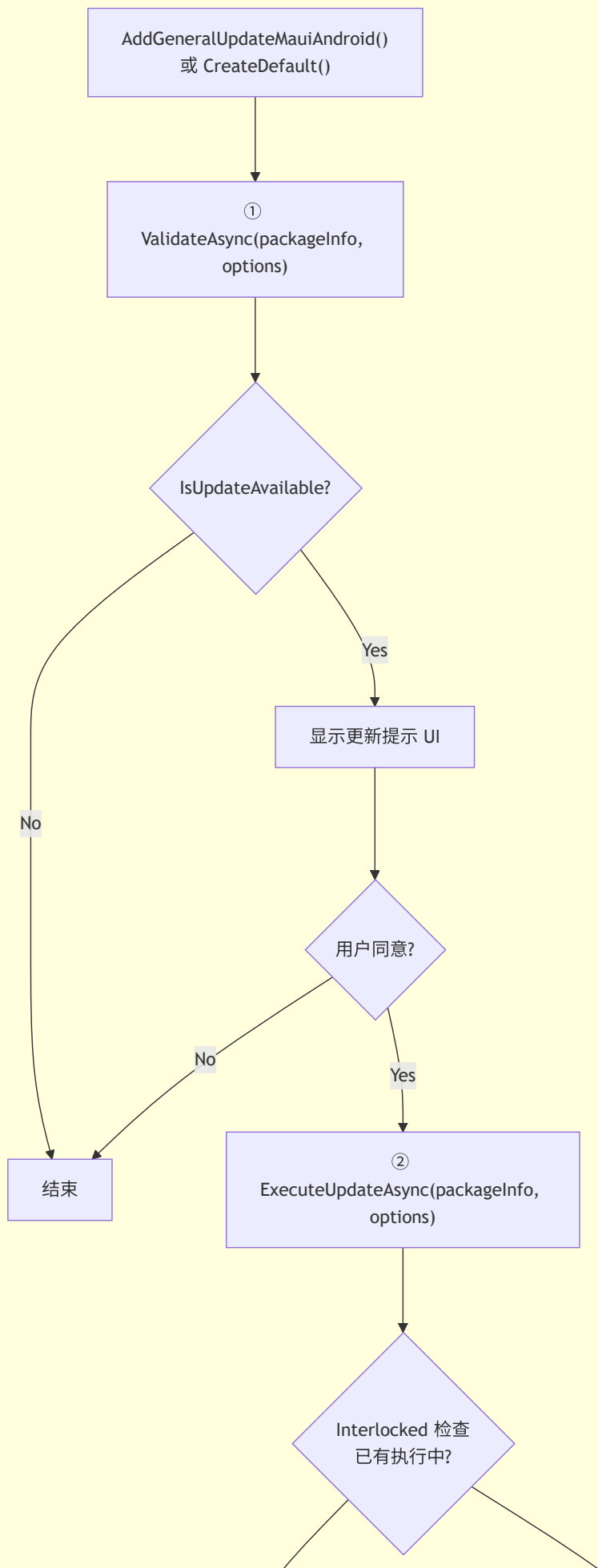
// 使用
public class UpdateService
{
    private readonly IAndroidBootstrap _bootstrap;

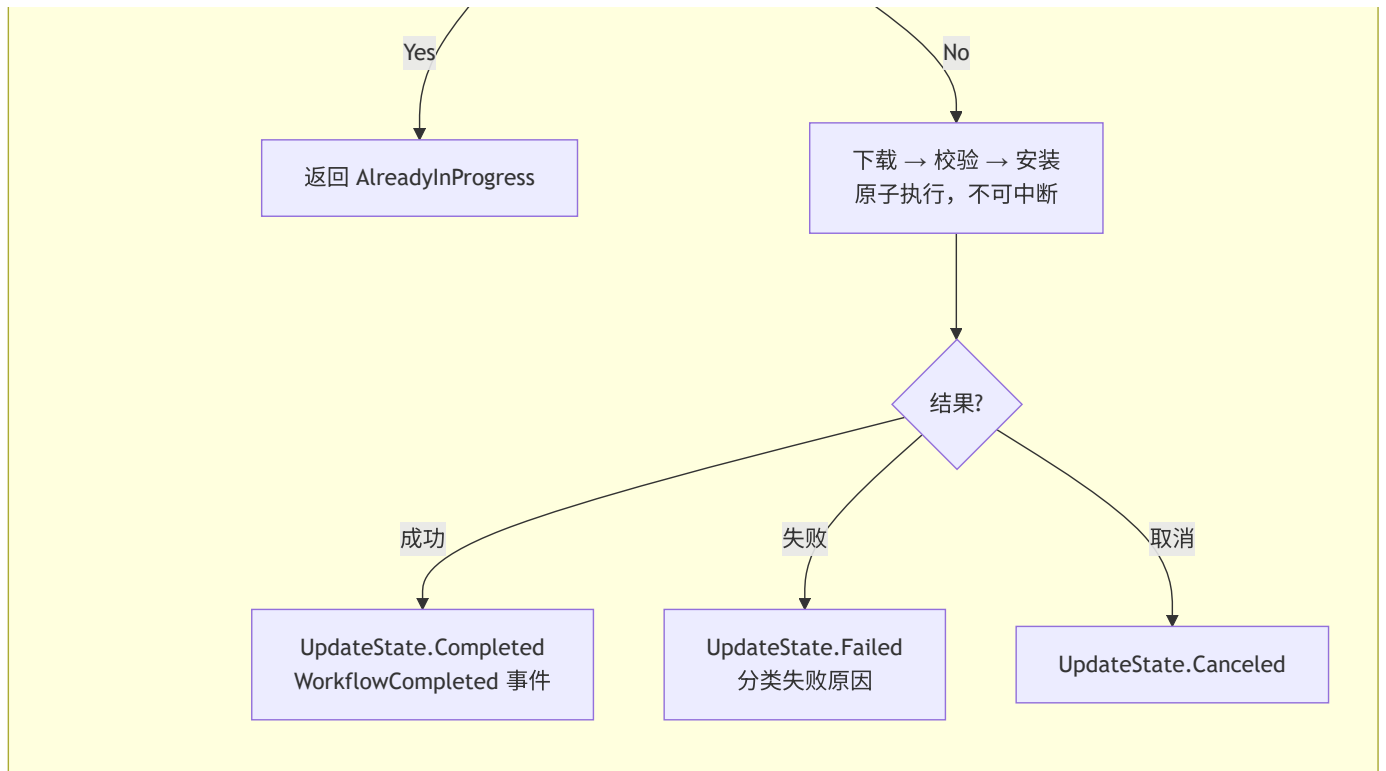
    public UpdateService(IAndroidBootstrap bootstrap)
    {
        _bootstrap = bootstrap;
    }
}
```

3. AndroidBootstrap: 合并式两步 API

3.1 完整生命周期

MAUI App 调用流程





3.2 状态机

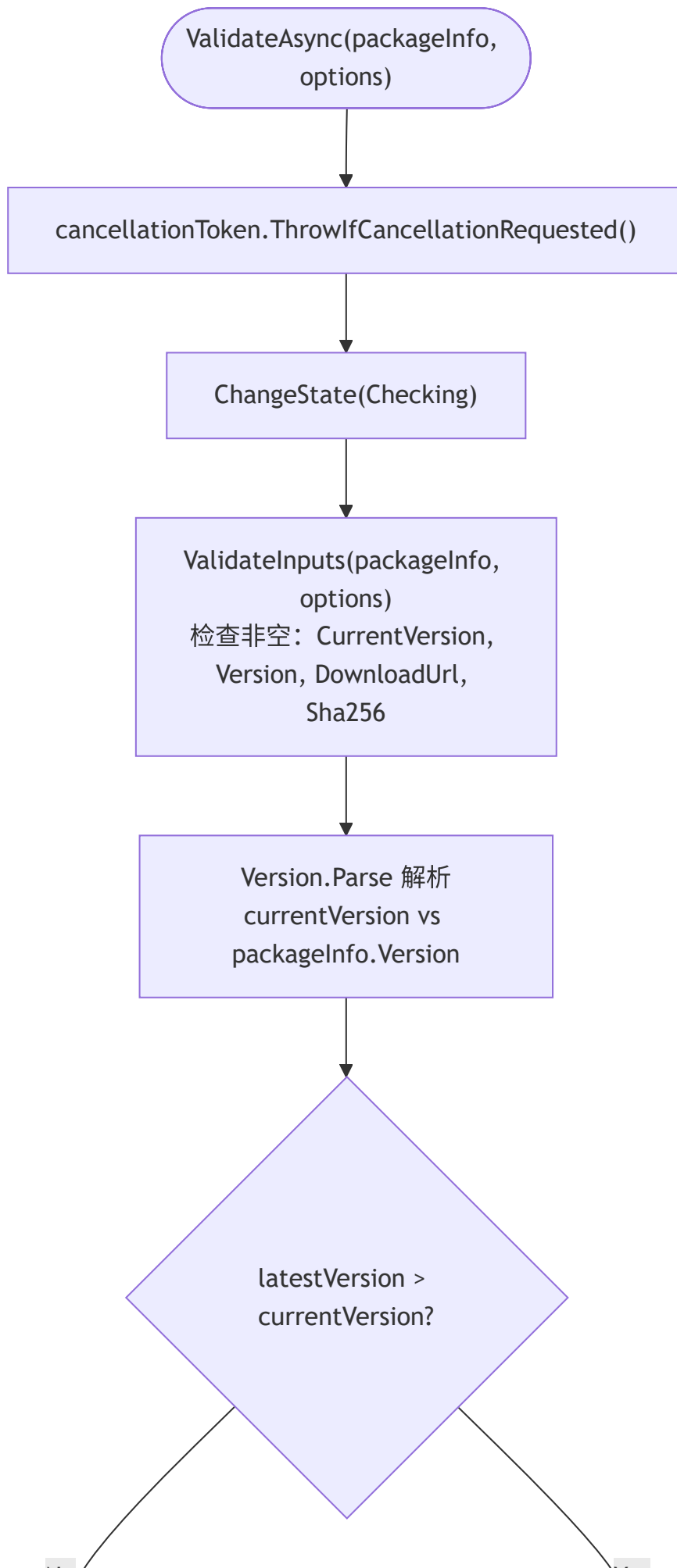


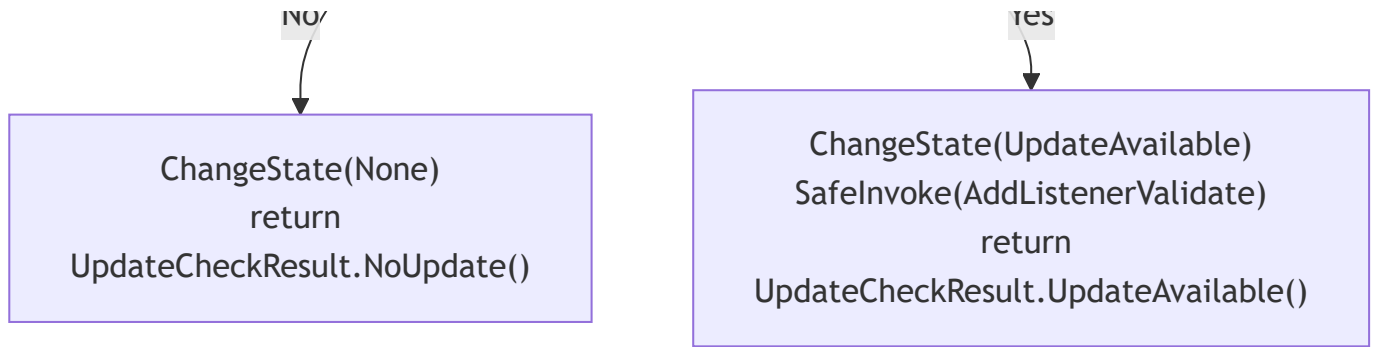
状态通过 `Interlocked.Exchange` 原子转换:

```
public UpdateState CurrentState => (UpdateState)Volatile.Read(ref _currentState);

private void ChangeState(UpdateState state)
{
    Interlocked.Exchange(ref _currentState, (int)state);
}
```

4. Step 1: ValidateAsync — 版本校验





4.1 输入验证

```
private static void ValidateInputs(UpdatePackageInfo packageInfo, UpdateOptions options)
{
    ArgumentNullException.ThrowIfNull(packageInfo);
    ArgumentNullException.ThrowIfNull(options);

    if (string.IsNullOrEmpty(options.CurrentVersion))
        throw new ArgumentException("Current version cannot be null or empty.");

    if (string.IsNullOrEmpty(packageInfo.Version))
        throw new ArgumentException("Update package version cannot be null or empty.");

    if (string.IsNullOrEmpty(packageInfo.DownloadUrl))
        throw new ArgumentException("Update package download url cannot be null or empty.");

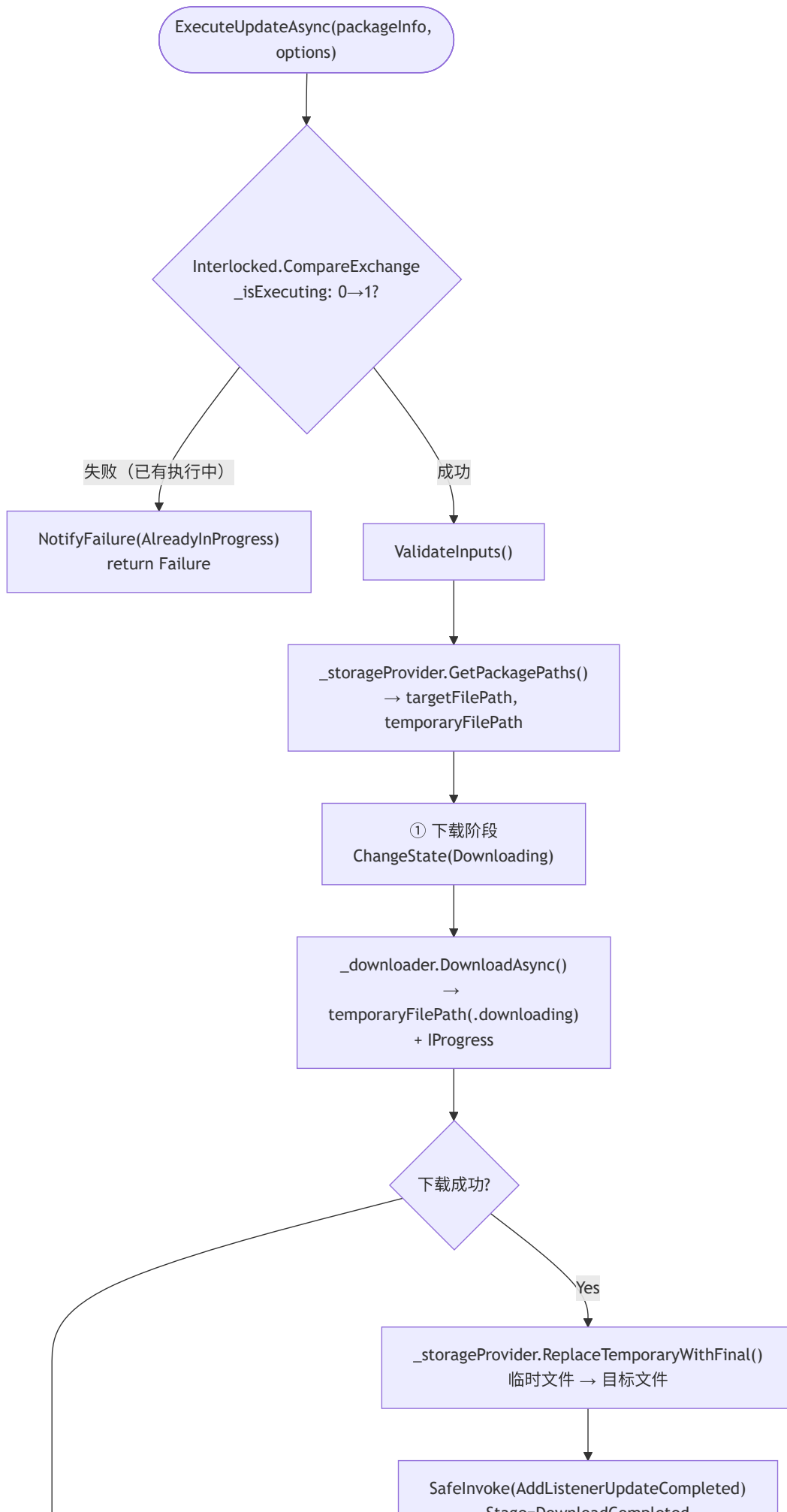
    if (string.IsNullOrEmpty(packageInfo.Sha256))
        throw new ArgumentException("Update package SHA256 cannot be null or empty.");
}
```

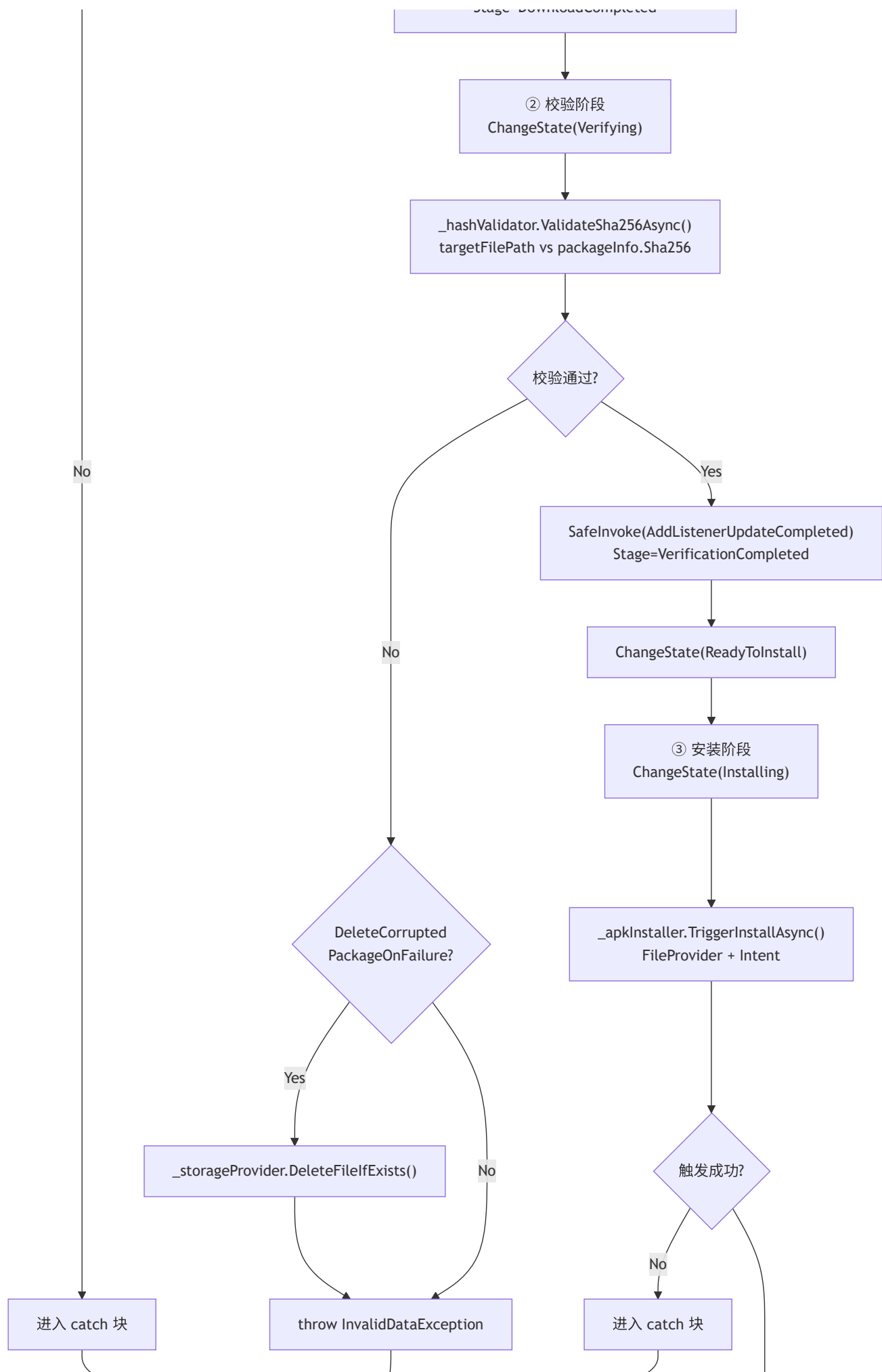
安全设计： SHA256 为必填项——Maui.Android 不允许跳过完整性校验。

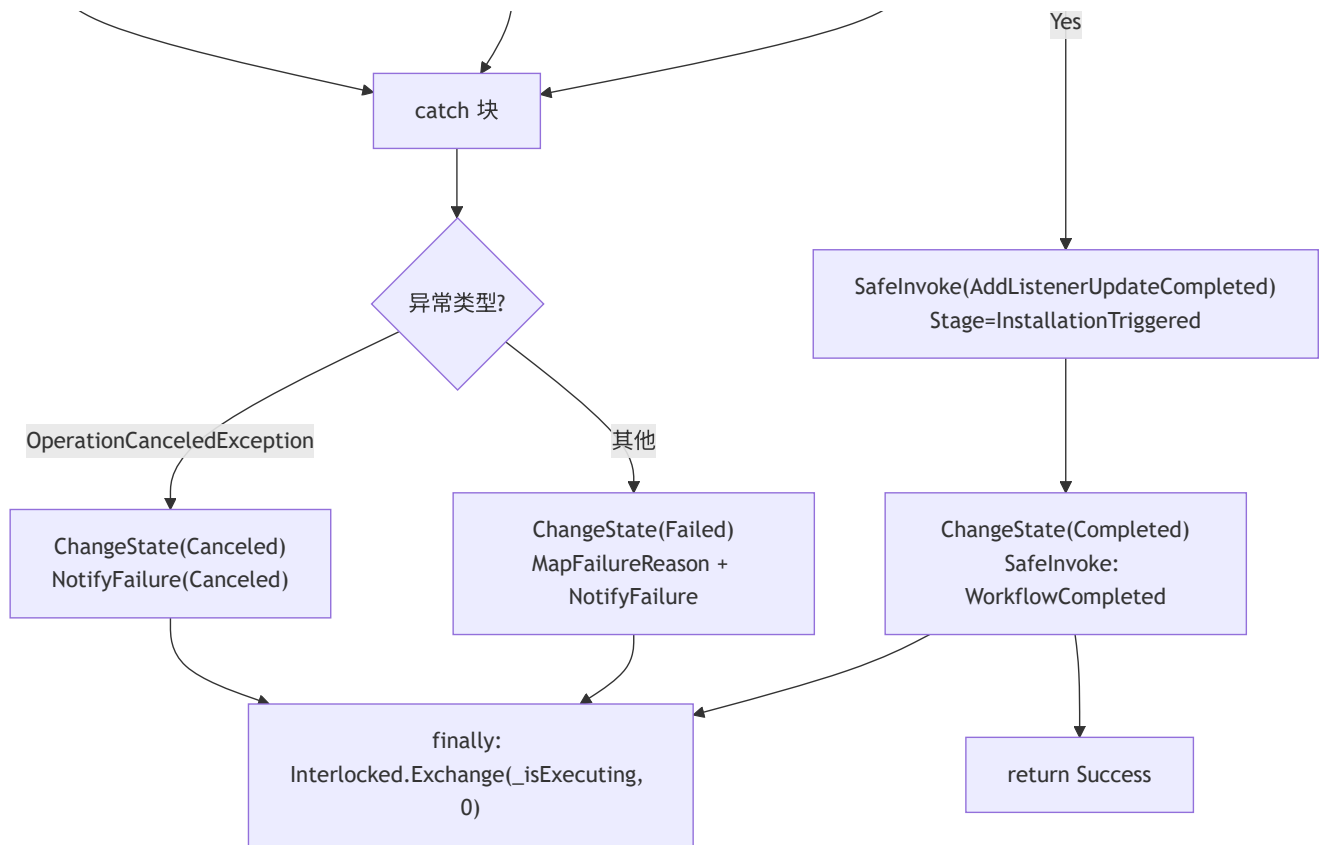
5. Step 2: ExecuteUpdateAsync — 原子执行完整更新

这是 Maui.Android 最核心的方法。它将下载、校验、安装合并在一个原子操作中。

5.1 全流程总图







5.2 四个完成阶段

阶段	UpdateCompletionStage	触发时机
下载完成	DownloadCompleted	文件下载完成 + 原子替换后
校验完成	VerificationCompleted	SHA256 校验通过后
安装触发	InstallationTriggered	Android Installer Intent 发出后
流程完成	WorkflowCompleted	所有步骤成功，状态设为 Completed

6. 可恢复下载：HttpRangeDownloader 深度解析

6.1 下载流程

```
HttpRangeDownloader.DownloadAsync()
|
|— HEAD 请求探测
|   Content-Length, Accept-Ranges, ETag
|
|— 检查已有部分下载
|   临时文件路径: {targetFilePath}.downloading
|
|— 续传判断
|   如果临时文件存在:
|       → 文件大小 ≤ Content-Length → Range: bytes={size}-
|       → 否则删除临时文件, 从头下载
|
|— GET with Range
|   流式写入临时文件 (追加模式)
|   实时报告进度: 字节数、总大小、速度
|
|— 进度报告
    通过 IProgress 回调
```

6.2 DownloadStatistics

```
public class DownloadStatistics
{
    public long BytesDownloaded { get; set; }
    public long? TotalBytes { get; set; }
    public double DownloadSpeedBytesPerSecond { get; set; }
    public int ProgressPercentage { get; set; }
    public TimeSpan? EstimatedTimeRemaining { get; set; }
}
```

6.3 原子文件替换

```
// UpdateFileStore.ReplaceTemporaryWithFinal
public void ReplaceTemporaryWithFinal(string temporaryPath, string targetPath)
{
    if (File.Exists(targetPath))
        File.Delete(targetPath);

    File.Move(temporaryPath, targetPath); // 原子重命名
}
```

7. SHA256 校验与损坏文件清理

7.1 校验流程

```
var hashResult = await _hashValidator.ValidateSha256Async(
    targetFilePath, packageInfo.Sha256, progress: null, cancellationTokens);

if (!hashResult.IsSuccess)
{
    // 清理策略：仅当配置允许时删除
    if (options.DeleteCorruptedPackageOnFailure)
    {
        _storageProvider.DeleteFileIfExists(targetFilePath);
    }

    throw new InvalidDataException(hashResult.FailureReason ?? "Integrity check failed.");
}
```

7.2 损坏文件处理策略

DeleteCorruptedPackageOnFailure	行为
true（默认）	删除损坏文件 → 下次更新重新下载
false	保留文件 → 方便开发者手动排查

8. APK 安装：平台守卫的 Installer

8.1 编译时平台守卫

```
public class AndroidApkInstaller : IApkInstaller
{
    public Task TriggerInstallAsync(string filePath, InstallOptions options, CancellationToken ct)
    {
#if ANDROID
        // Android 平台实现
        var context = Android.App.Application.Context;
        var file = new Java.IO.File(filePath);

        if (Android.OS.Build.VERSION.SdkInt >= Android.OS.BuildVersionCodes.O)
        {
            if (!context.PackageManager.CanRequestPackageInstalls())
                throw new UnauthorizedAccessException("Install permission not granted.");
        }

        var uri = AndroidX.Core.Content.FileProvider.GetUriForFile(
            context, options.FileProviderAuthority, file);

        var intent = new Android.Content.Intent(Android.Content.Intent.ActionView);
        intent.SetDataAndType(uri, "application/vnd.android.package-archive");
        intent.AddFlags(Android.Content.ActivityFlags.GrantReadUriPermission);
        intent.AddFlags(Android.Content.ActivityFlags.NewTask);

        context.StartActivity(intent);
        return Task.CompletedTask;
#else
        throw new PlatformNotSupportedException("APK installation is only supported on Android.");
#endif
    }
}
```

8.2 安装权限检查 (API 26+)

```
if (Android.OS.Build.VERSION.SdkInt >= Android.OS.BuildVersionCodes.O)
{
    if (!context.PackageManager.CanRequestPackageInstalls())
    {
        // 引导用户到设置页面开启"安装未知应用"权限
        var intent = new Android.Content.Intent(
            Android.Provider.Settings.ActionManageUnknownAppSources);
        context.StartActivity(intent);
        throw new UnauthorizedAccessException("Install permission not granted.");
    }
}
```

9. 并发安全：Interlocked 原子操作

9.1 执行互斥

```
private int _isExecuting;

public async Task ExecuteUpdateAsync(...)
{
    // 原子地尝试将 _isExecuting 从 0 改为 1
    if (Interlocked.CompareExchange(ref _isExecuting, 1, 0) != 0)
    {
        // 已有执行中 → 直接拒绝
        NotifyFailure(UpdateFailureReason.AlreadyInProgress, "An update execution is already in progress.", ...);
        return UpdateExecutionResult.Failure(UpdateFailureReason.AlreadyInProgress, "...");
    }

    try
    {
        // 执行更新...
    }
    finally
    {
        // 确保在任何情况下释放
        Interlocked.Exchange(ref _isExecuting, 0);
    }
}
```


9.2 状态原子读取

```
private int _currentState = (int)UpdateState.None;

public UpdateState CurrentState => (UpdateState)Volatile.Read(ref _currentState);
```

设计优势：相比 `SemaphoreSlim`，`Interlocked` 更轻量——没有异步等待开销，不会死锁，天然适合这种“要么执行要么拒绝”的场景。

10. SafeInvoke：防御式事件触发

```
private void SafeInvoke(
    EventHandler? eventHandler,
    TEventArgs eventArgs,
    string eventName) where TEventArgs : EventArgs
{
    if (eventHandler is null) return;

    foreach (EventHandler subscriber in eventHandler.GetInvocationList())
    {
        try
        {
            subscriber(this, eventArgs);
        }
        catch (Exception ex)
        {
            _logger.LogError($"Unhandled exception in {eventName} listener.", ex);
            // 一个订阅者的异常不影响其他订阅者
        }
    }
}
```

关键保障：如果事件有多个订阅者，其中一个抛出异常，其他订阅者仍能收到通知。

11. 异常映射：MapFailureReason 分类机制

```
private static UpdateFailureReason MapFailureReason(Exception ex)
{
    return ex switch
    {
        ArgumentException          => UpdateFailureReason.InvalidInput,
        HttpRequestException       => UpdateFailureReason.Network,
        InvalidDataException       => UpdateFailureReason.IntegrityCheckFailed,
        IOException               => UpdateFailureReason.FileAccess,
        UnauthorizedAccessException => UpdateFailureReason.InstallPermissionDenied,
        OperationCanceledException => UpdateFailureReason.Canceled,
        _                          => UpdateFailureReason.Unknown
    };
}
```

11.1 失败原因枚举

UpdateFailureReason	触发条件	建议操作
None	—	—
InvalidInput	参数为空或格式错误	检查 CurrentVersion / Version / DownloadUrl / Sha256
Network	HTTP 请求失败	检查网络连接和服务端可用性
IntegrityCheckFailed	SHA256 不匹配或文件损坏	重新下载；检查 CDN 缓存
FileAccess	读写文件失败	检查磁盘空间和目录权限
InstallPermissionDenied	缺少 INSTALL_PACKAGES 权限	引导用户到设置页面授权
AlreadyInProgress	并发调用 ExecuteUpdateAsync	等待当前更新完成或重启
Canceled	用户取消或超时	允许用户重试
Unknown	未分类异常	查看日志详情

12. 与 Avalonia.Android 的设计对比

12.1 架构哲学差异

维度	Maui.Android	Avalonia.Android
框架定位	MAUI DI 生态深度集成	框架无关的通用设计
API 粒度	合并式（少 API，内部原子化）	显式式（多 API，调用方控制）
并发模型	Interlocked 原子操作	SemaphoreSlim 异步门
事件模型	4 阶段完成事件	2 事件（Completed + Failed）
SHA256	强制必填	可选（但强烈建议）
文件大小验证	不单独验证	双重验证（大小 + SHA256）

维度	Maui.Android	Avalonia.Android
进度接口	IProgress	EventHandler
编译守卫	#if ANDROID	运行时检查
最低 API	API 21 (Android 5.0)	API 26 (Android 8.0)

12.2 选择建议

- 使用 **Maui.Android** 如果你：使用 MAUI 框架、需要 DI 集成、偏好简洁的合并式 API
- 使用 **Avalonia.Android** 如果你：使用 Avalonia 框架、需要在每步之间插入自定义逻辑、需要 Sidecar 断点续传

13. 关键代码路径索引

组件	文件	关键方法
DI 注册 + 工厂	Services/GeneralUpdateBootstrap.cs	AddGeneralUpdateMauiAndroid() / CreateDefault()
编排器	Services/AndroidBootstrap.cs	ValidateAsync() / ExecuteUpdateAsync() / SafeInvoke()
编排器接口	Abstractions/IAndroidBootstrap.cs	—
可恢复下载器	Services/HttpRangeDownloader.cs	DownloadAsync() / HEAD 探测 / Range 请求
SHA256 校验	Services/Sha256Validator.cs	ValidateSha256Async()
APK 安装器	Platform/Android/AndroidApkInstaller.cs	TriggerInstallAsync() / #if ANDROID
文件存储	Services/UpdateFileStore.cs	GetPackagePaths() / ReplaceTemporaryWithFinal() / DeleteFileIfExists()
速度计算	Utilities/SpeedCalculator.cs	下载速度计量
更新选项	Models/UpdateOptions.cs	CurrentVersion / DownloadDirectory / DeleteCorruptedPackageOnFailure
下载统计	Models/DownloadStatistics.cs	BytesDownloaded / TotalBytes / Speed / Percentage
完成阶段	Enums/UpdateCompletionStage.cs	DownloadCompleted / VerificationCompleted / InstallationTriggered / WorkflowCompleted
失败原因	Enums/UpdateFailureReason.cs	—
日志接口	Abstractions/IUpdateLogger.cs	—



视频演示: <https://www.bilibili.com/video/BV12P9dBiEEh>

版本与价格

TSLH™ GeneralSpacestation 采用**年度付费模式**，购买或续费即可获得当年版本的永久授权。所有版本均提供部署指导与用户操作手册。

订阅版

按需付费，灵活可控

项目	说明
付费模式	流量费 + 服务费
流量包	9 元 / 100 GB（有效期 1 个月）
服务费	流量包总价的 10%
席位	不限制
技术支持	公共频道
Beta 版本优先	✓

流量包价格随供应商调整可能变动，支持按需灵活采购。

个人版

轻量起步，独立开发者首选

项目	说明
首年	¥599
续费	¥299 / 年
永久授权	√（含后续一年更新）
席位	1 个
技术支持	公共频道

项目	说明
专属技术支持	—

企业版

专业服务，企业级全场景覆盖

项目	说明
首年	¥2599
续费	¥1299 / 年
永久授权	✓（含后续一年更新）
专属技术支持	1 个席位
Beta 版本优先	✓
公共频道支持	✓
离线版	可选

优惠政策

- **阶梯定价锁价：**所有用户首次购买后永久锁定该价格，续费不涨价
- **合作展示折扣：**使用本产品开发项目并授权展示合作关系，永久享受 **10% 折扣**
- **定制开发：**特殊需求支持单独洽谈付费定制

产品概述

TSLH™ GeneralSpacestation 是**客户端全生命周期升级管理服务**，聚焦企业客户端更新包管理、版本发布、推送管控、日志追踪等核心场景，一站式解决更新繁琐、流量浪费、发布混乱、追溯困难等问题。

- **适用场景：**桌面客户端、移动端应用的自动升级管理，包括版本发布、灰度推送、更新追踪
- **部署模式：**私有化本地部署，数据不出企业内网
- **核心价值：**降低客户端运维成本、精准控制版本发布范围、完整追溯升级链路

开源组件 vs 商业服务

GeneralUpdate（开源）解决了"客户端怎么更新"的问题；**GeneralSpacestation**（商业）解决了"服务端怎么管理更新"的问题。前者是客户端 SDK，后者是服务端管理平台。两者配合使用，形成完整的客户端升级管理体系。

体系构成

组件	说明	是否收费
GeneralSpacestation	自动升级管理服务解决方案（服务端）	是
GeneralUpdate.Admin	自动升级管理可视化客户端	是
GeneralUpdate	桌面客户端自动升级组件	否（开源）

组件	说明	是否收费
GeneralUpdate-Samples	GeneralUpdate 使用代码示例	否（开源）

在线文档：<https://www.justerzhu.cn/> 开源仓库：[GitHub - GeneralUpdate](#) · [GitHub - GeneralSpacestation](#)

核心功能

分组灰度发布

按区域、产品线、门店、试点客户灵活编组，精准控制更新范围，降低发布风险。支持**分组冻结**，被冻结的分组无法接收升级推送，用于误发补丁后的紧急阻止。

补丁包管理

三种制作模式覆盖全场景：

模式	说明	适用场景
自动构建	读取发布命令文件，系统自动执行发布并生成历史版本	标准化发布流程
手动构建	手动指定目录，通过文件差异对比生成差分补丁	灵活定制场景
全量构建	直接压缩指定目录全部文件	首次发布或完整覆盖

- **二进制差分压缩**，仅同步变更文件，补丁体积极小（KB 级），大幅节省带宽与存储成本
- 支持 ZIP 格式、断点续传，单次更新失败后下次启动可继续下载

三级管理体系

产品管理 → 分组管理 → 客户端管理

同一产品下按区域、门店或试点范围划分子组，各分组下管理实际客户端设备，通过应用密钥（App Key）唯一标识，实现精细化数据隔离。

灵活版本推送

推送方式	说明
立即推送	选择分组后即时下发更新通知
定时推送	指定发布日期和时间，到时自动推送
强制更新	标记补丁包为强制更新，客户端不可跳过
可选更新	客户端可自行选择是否执行升级

全链路日志追溯

- **升级记录**：客户端升级操作全程记录，含状态（成功/失败/升级中）、版本号、操作时间
- **推送记录**：管理员推送操作留痕，含操作人、推送时间、目标分组
- 支持按产品、版本、时间范围、状态等多维度查询

OSS 极简升级模式

一键生成版本配置文件，无需编写服务端代码。客户端直接根据 OSS 文件服务器的版本信息判断是否更新，大幅降低新手接入门槛。

扩展管理

支持将插件或扩展目录压缩为扩展包上传至服务端统一管理，支持依赖关系声明、平台筛选和预发布标记，满足插件化架构需求。

多语言与主题

支持中文 / 英文界面切换，明 / 暗双主题，可折叠菜单，操作空间更大。

可定制方案

除标准版本外，我们提供以下增值服务，满足企业个性化需求：

服务	说明
定制开发	根据业务需求定制功能模块，单独洽谈付费
项目对接	技术团队一对一接入指导，助企业快速上手
培训支持	线上 / 线下产品培训会议
部署指导	私有化部署全程技术支持

💡 商务咨询请扫描下方二维码（推荐微信添加），加好友请注明来意。



UpgradeHub

组件概览

UpgradeHubService 是基于 SignalR 的实时版本推送服务，集成在 `GeneralUpdate.Core` 中。服务端通过 `UpgradeHub` 向已连接的客户端主动推送版本更新通知，客户端通过 `UpgradeHubService` 订阅并接收消息，支持点对点（一对一）和广播（一对多）两种推送模式。

UpgradeHub 与 Core 的关系

`UpgradeHubService` 是 `GeneralUpdate.Core` 的**可选功能**，不依赖它也能完成常规更新。它的价值在于：服务端有紧急版本时可以**主动推送**通知给在线客户端，客户端无需等待下一次轮询就能立即触发更新——特别适合紧急 bug 修复和灰度发布场景。

命名空间： `GeneralUpdate.Core.Hubs` 程序集： `GeneralUpdate.Core.dll`

```
public class UpgradeHubService : IUpgradeHubService
```

核心特性

1. 实时推送

- 基于 SignalR 长连接，服务端主动推送，无需客户端轮询
- 支持点对点推送（按 appkey 定向推送）
- 支持广播推送（同时通知所有在线客户端）

2. 自动重连

- 内置随机退避重连策略（ `RandomRetryPolicy` ）
- 断线后自动尝试重连，保证连接稳定性

3. 完整生命周期管理

- 连接建立（ `StartAsync` ）
- 优雅停止（ `StopAsync` ）
- 资源释放（ `DisposeAsync` ）

4. 多事件订阅

- 接收版本推送消息（ `AddListenerReceive` ）
- 在线 / 离线状态通知（ `AddListenerOnline` ）
- 重连成功通知（ `AddListenerReconnected` ）
- 连接关闭通知（ `AddListenerClosed` ）

快速开始

安装

`UpgradeHubService` 包含在 `GeneralUpdate.Core` 中，无需单独安装：

```
dotnet add package GeneralUpdate.Core
```

初始化与使用

以下示例展示了如何在客户端应用中接收服务端推送的版本更新消息：

```
using GeneralUpdate.Core.Hubs;

// 创建 UpgradeHubService 实例
// url - SignalR Hub 订阅地址
// token - ID4 认证令牌（可选）
// appkey - 客户端唯一标识（推荐使用 GUID），服务端用此字段定向推送
var hub = new UpgradeHubService(
    "http://localhost:5000/UpgradeHub",
    token: null,
    appkey: "dfeb5833-975e-4afb-88f1-6278ee9aef6");

// 订阅服务端推送的版本更新消息
// message 为服务端推送的 JSON 字符串（通常为 Packet 对象序列化结果）
hub.AddListenerReceive((message) =>
{
    Console.WriteLine($"收到更新推送: {message}");
});

// 订阅在线 / 离线状态通知
hub.AddListenerOnline((info) =>
{
    Console.WriteLine($"在线状态: {info}");
});

// 订阅重连成功通知
hub.AddListenerReconnected((connectionId) =>
{
    Console.WriteLine($"已重连, connectionId={connectionId}");
    return Task.CompletedTask;
});

// 订阅连接关闭通知
hub.AddListenerClosed((exception) =>
{
    if (exception is not null)
        Console.WriteLine($"连接异常关闭: {exception.Message}");
    else
        Console.WriteLine("连接已正常关闭");
    return Task.CompletedTask;
});

// 建立 SignalR 连接
await hub.StartAsync();
```

```
Console.WriteLine("已连接，等待服务端推送...");
Console.ReadLine();

// 停止连接（可在应用进入后台时调用，保留重连能力）
await hub.StopAsync();

// 释放资源（不可再复用）
await hub.DisposeAsync();
```

在依赖注入项目中使用（以 Prism 为例）

```
// 注册服务
protected override void RegisterTypes(IContainerRegistry containerRegistry)
{
    containerRegistry.Register<IUpgradeHubService, UpgradeHubService>();
}

// 在 ViewModel 中使用
public MainWindowViewModel(IUpgradeHubService hubService)
{
    hubService.AddListenerReceive((message) =>
    {
        Console.WriteLine($"收到推送: {message}");
    });
    _ = hubService.StartAsync();
}
```

核心 API 参考

构造函数

```
public UpgradeHubService(string url, string? token = null, string? appkey = null)
```

参数：

参数	类型	说明
url	string	SignalR Hub 订阅地址，例如 http://127.0.0.1/UpgradeHub
token	string?	ID4 认证令牌字符串（可选）
appkey	string?	客户端唯一标识，推荐使用 GUID，服务端用于定向推送（可选）

方法

AddListenerReceive 方法

订阅服务端推送的版本更新消息。

```
public void AddListenerReceive(Action<string> receiveMessageCallback)
```

参数:

- `receiveMessageCallback` — 接收到消息时的回调, `string` 为消息内容 (建议为 JSON 格式的 `Packet` 对象)

AddListenerOnline 方法

订阅在线 / 离线状态变化通知。

```
public void AddListenerOnline(Action<string> onlineMessageCallback)
```

参数:

- `onlineMessageCallback` — 状态变化时的回调, `string` 为状态描述

AddListenerReconnected 方法

订阅断线重连成功通知。

```
public void AddListenerReconnected(Func<string?, Task>? reconnectedCallback)
```

参数:

- `reconnectedCallback` — 重连成功后的异步回调, `string?` 为新的 `connectionId` (可能为 `null`)

AddListenerClosed 方法

订阅连接关闭通知。

```
public void AddListenerClosed(Func<Exception?, Task> closeCallback)
```

参数:

- `closeCallback` — 连接关闭时的异步回调, `Exception?` 为关闭原因 (正常关闭时为 `null`)

StartAsync 方法

建立 SignalR 连接, 开始订阅推送通知。

```
public Task StartAsync()
```

StopAsync 方法

优雅停止连接。当前正在处理的消息会完成，不再接受新消息。适合在应用进入后台或暂停时调用，之后可重新调用 `StartAsync` 恢复连接。

```
public Task StopAsync()
```

DisposeAsync 方法

彻底释放 Hub 实例及所有资源，释放后不可再复用。

```
public Task DisposeAsync()
```

实际使用示例

示例 1：基本推送订阅

```
using GeneralUpdate.Core.Hubs;

var hub = new UpgradeHubService("http://localhost:5000/UpgradeHub",
    appkey: Guid.NewGuid().ToString());

hub.AddListenerReceive((message) =>
{
    Console.WriteLine($"[更新推送] {message}");
});

await hub.StartAsync();
```

示例 2：完整生命周期管理

```
using GeneralUpdate.Core.Hubs;

var hub = new UpgradeHubService(
    "http://localhost:5000/UpgradeHub",
    token: null,
    appkey: "dfeb5833-975e-4afb-88f1-6278ee9aeff6");

hub.AddListenerReceive(msg => Console.WriteLine($"推送: {msg}"));
hub.AddListenerOnline(info => Console.WriteLine($"在线状态: {info}"));
hub.AddListenerReconnected(id => { Console.WriteLine($"重连: {id}"); return Task.CompletedTask; });
hub.AddListenerClosed(ex => { Console.WriteLine(ex != null ? $"异常: {ex.Message}" : "关闭"); return Task.CompletedTask; });

try
{
    await hub.StartAsync();
    Console.ReadLine(); // 等待用户输入
}
finally
{
    await hub.StopAsync();
    await hub.DisposeAsync();
}
```

示例 3：搭配 GeneralClientBootstrap 使用

实际项目中，`UpgradeHubService` 通常与 `GeneralClientBootstrap` 一起使用。客户端先通过 `GeneralClientBootstrap` 完成常规版本检查和下载，同时用 `UpgradeHubService` 接收服务端的实时推送通知，无需等待下一次轮询即可立即触发更新。

```
using System.Text;
using GeneralUpdate.Core;
using GeneralUpdate.Core.Hubs;

// 1. 启动 SignalR 推送监听
var hub = new UpgradeHubService("http://localhost:5000/UpgradeHub",
    appkey: "dfeb5833-975e-4afb-88f1-6278ee9aeff6");
hub.AddListenerReceive(msg => Console.WriteLine($"实时推送: {msg}"));
await hub.StartAsync();

// 2. 常规更新流程
var config = new Configinfo
{
    UpdateUrl = "http://127.0.0.1:5000/Upgrade/Verification",
    ReportUrl = "http://127.0.0.1:5000/Upgrade/Report",
    AppName = "UpgradeSample.exe",
    MainAppName = "ClientSample.exe",
    ClientVersion = "1.0.0.0",
    UpgradeClientVersion = "1.0.0.0",
    InstallPath = AppDomain.CurrentDomain.BaseDirectory,
    ProductId = "2d974e2a-31e6-4887-9bb1-b4689e98c77a",
    AppSecretKey = "dfeb5833-975e-4afb-88f1-6278ee9aeff6"
};

await new GeneralClientBootstrap()
    .AddListenerException( (_, e) => Console.WriteLine(e.Exception))
    .SetConfig(config)
    .LaunchAsync();
```

注意事项

⚠️ 重要提示

1. 服务端配置

- 服务端需要安装 `Microsoft.AspNetCore.SignalR` 并注册 `UpgradeHub`
- 在 `Program.cs` 中: `app.MapHub("/UpgradeHub");`

2. appkey 唯一性

- `appkey` 用于服务端定向推送, 每个客户端实例应使用唯一值

- 推荐使用固定 GUID（与 `Configinfo.AppSecretKey` 保持一致）

3. StopAsync vs DisposeAsync

- `StopAsync`：暂停连接，之后可重新调用 `StartAsync` 恢复
- `DisposeAsync`：彻底释放，不可再复用

4. 消息格式

- `AddListenerReceive` 中接收的消息格式由服务端决定，建议使用 JSON 字符串（`Packet` 对象序列化结果）

💡 最佳实践

- 在应用启动时调用 `StartAsync`，在应用关闭时调用 `StopAsync` + `DisposeAsync`
- 使用依赖注入管理 `UpgradeHubService` 实例的生命周期
- 在 `AddListenerClosed` 回调中记录日志，便于排查连接问题

适用平台

产品	版本
.NET	5, 6, 7, 8, 9, 10
.NET Framework	4.6.1
.NET Standard	2.0
.NET Core	2.0
ASP.NET	Any

相关资源

- 示例代码：[查看 GitHub 示例](#)
- 主仓库：[GeneralUpdate 项目](#)
- 相关组件：[GeneralUpdate.Core](#) | [GeneralUpdate.Core](#)

发布日志 Release Log

Release log

📌 2026-06-24 — v10.5.0-rc.1

- **零配置 SetSource() API**: 自动从 manifest.json 获取更新源地址, 无需在启动代码中硬编码 URL
- **manifest.json 自动发现**: 自动扫描 manifest.json 解析更新配置, 简化多环境部署
- **IUpdateHooks 扩展点**: 新增生命周期钩子, 可在下载前后、安装前后、出错时注入自定义逻辑
- **LoadFromConfiguration() 支持**: 直接从 appsettings.json 读取更新设置, 零代码变更实现环境感知配置
- **SSL/HttpClient 生命周期修复**: 解决长时间运行过程中 HttpClient 套接字耗尽和 SSL 证书验证问题
- **OSS 更新流程改进**: 更可靠的 OSS 更新交付, 具备更好的错误恢复和进度上报
- **链式转全量回退策略**: 用数量优先启发式替代 80% 体积阈值, 增量补丁过多时智能回退到全量包安装
- **Bowl → BowlBootstrap 重命名**: 明确崩溃守护进程的引导角色
- **CI/CD 可靠性修复**: 规范化版本号输入、修复 publish 工作流 fetch-depth、提升 Linux 测试稳定性

📌 2026-06-21 — v10.5.0-beta.7

- chore: bump version to v10.5.0-beta.7

📌 2026-06-20 — v10.5.0-beta.6

- fix(core): resolve 16 audit findings — race conditions, design flaws, glue code, and performance issues
- feat(core): PackageType enum migration + chain/full fallback mechanism
- feat(core): unified HTTP auth provider + custom headers + CVP fix
- fix(core): skip redundant chain packages after full fallback
- fix(core): eliminate all AOT-incompatible patterns for Native AOT support
- fix: resolve CI failures and address Copilot review comments
- feat: align all components to SemVer 2.0 for version comparison (#521)
- chore: bump version to v10.5.0-beta.6 and fix CI pipeline

📌 2026-06-14

- **GeneralUpdate.Core**: 修复代码审计发现的 BSDIFF 溢出、Zip 路径遍历、ProcessExit 死锁、IPC 密钥硬编码等严重/高危安全问题; 修复下载管道失败后继续启动 Upgrade 的错误行为; 备份功能默认关闭 (不再默认备份); 移除 UpgradeMode 透传字段; 新增跨版本优先更新 (CVP-first) 策略, 自动降级为链式更新
- **GeneralUpdate.Differential**: BSDIFF 溢出长度校验修复
- **GeneralUpdate.Bowl**: 修复 Trace Listener 生命周期问题
- **GeneralUpdate.Drivelution**: 修复策略生命周期问题
- **GeneralUpdate.Extension**: 修复 HubConnection 生命周期和 RetryPolicy 问题
- **GeneralUpdate.Avalonia.Android: 首个公开发布** v0.0.1-beta.9 — Avalonia Android APK 自动更新组件, 提供版本对比、断点续传下载、SHA256 校验和 Package Installer 触发编排; 支持 HMAC/Bearer/ApiKey/Basic 多协议认证
- **GeneralUpdate.Maui.Android: 首个公开发布** v0.0.1-beta.3 — MAUI Android APK 自动更新组件, 提供 `ValidateAsync` + `ExecuteUpdateAsync` 双阶段 API、DI 容器集成、HTTP Range 断点续传和 SHA256 校验

- **GeneralUpdate.Tools:** 新增 Mobile 打包模块, 支持 APK/AAB 文件解析 (自动提取 AndroidManifest 元数据)、`dotnet publish` 项目构建、SHA256 计算、服务端上传和版本记录导出
- **文档:** 新增 GeneralUpdate.Avalonia.Android 和 GeneralUpdate.Maui.Android 完整组件参考文档 (中英文); 新增 Avalonia/Maui Android 快速入门教程; Tools 文档补充 Mobile 模块和双发布流水线说明

📌 2026-05-20 — v10.5.0-beta.2

- **GeneralUpdate.Core:** 统一 `GeneralUpdateBootstrap` 入口, 合并原 `ClientCore` 和 `Core` 能力; 新增 `SetSource()` 轻配置入口与 `generalupdate.manifest.json` 极简接入; 重构下载子系统为可替换的 5 层模型 (`Source/Policy/Executor/Pipeline/Orchestrator`); 扩展点体系增至 10 个可替换接口
- **GeneralUpdate.Bowl:** `BowlContext` 配置 API 稳定; 优化 Windows/Linux/macOS ProcDump 集成
- **GeneralUpdate.Differential:** 稳定 `IBinaryDiffer` 抽象; BSDIFF 4.0 + Streaming HDiff 双算法; BZip2 / Deflate 压缩
- **GeneralUpdate.Drivelution:** 稳定跨平台驱动更新流水线; Windows `pnputil` / Linux `insmod-dpkg-rpm` / macOS `kextload-installer`
- **GeneralUpdate.Extension:** 稳定 `IExtensionHost` 抽象; Zip Slip 安全解压; 拓扑排序依赖解析
- **文档:** 重构全部 5 个组件文档为统一模板; 新增入门实战手册; 新增软件外包服务页面
- **Samples:** Hub 示例浏览器 9 个场景覆盖完整更新/OSS/静默/推送/差分/压缩/扩展/Bowl/驱动

📌 2026-04-10

- Add `AddListenerUpdatePrecheck(Func)` to `GeneralClientBootstrap` — unified entry that receives the full version info and returns `true` to skip or `false` to proceed; replaces the combined use of `AddListenerUpdateInfo` + `SetCustomSkipOption`. Forced-update (`IsForcibly`) versions always proceed regardless of the callback return value.
- Remove deprecated `SetCustomSkipOption` from `GeneralClientBootstrap`.
- Add `AddListenerUpdateInfo(Action)` to `GeneralClientBootstrap` and `GeneralUpdateBootstrap` — dispatched immediately after version validation; each `VersionInfo` entry now carries a new `UpdateLog` field for release notes.
- Add opt-in silent update mode (`UpdateOption.EnableSilentUpdate`) to `GeneralClientBootstrap` — polls every 20 minutes in the background, prepares the update package silently, and launches the updater only after the host process exits.
- Add `ConfiginfoBuilder` — zero-configuration builder for `Configinfo`; only `UpdateUrl`, `Token`, and `Scheme` are required; all other fields are auto-detected from the `.csproj` file and the runtime platform (Windows / Linux / macOS).
- Add `DriverDirectory` field to `Configinfo` / `BaseConfigInfo` and propagate it through `ConfigurationMapper`, `ProcessInfo`, and `PipelineContext` to `DrivelutionMiddleware` for driver update support.
- Remove `SetFieldMappings` from `GeneralUpdateBootstrap`.
- Add full lifecycle tracing (`GeneralTracer`) across `GeneralUpdate.Bowl`, `GeneralUpdate.Extension`, `GeneralUpdate.Core`, `GeneralUpdate.ClientCore`, and `GeneralUpdate.Drivelution`.
- Fix `DefaultCleanMatcher.Match` incorrectly returning `null` for same-name files in different directories.
- Fix orphaned temp files in `BinaryHandler.Dirty` causing patch failures on repeated update runs.

📌 2026-01-06 10.0.0

- Adapt to .NET 10 upgrade and update the versions of referenced components.
- Add scripted execution of GeneralUpdate via `dotnet run x.cs` for .NET 10.
- Refactor part of the functions.

📌 2026-01-01 9.5.10

- Fix the file occupation issue in `BinaryHandler` caused by `File.Move(_newfilePath, _oldfilePath)`.
- Fix the null value issue of `Environments.GetEnvironmentVariable("ProcessInfo")` on Linux.
- Add the function to set executable permissions for updated files on Linux.
- Refactor part of the functions.

📌 2025-02-20 9.1.6

- Fix OSS to directly terminate the current process when no update is required.

📌 2025-01-14 9.1.5

- Fix the issue with incorrect value retrieval for LastVersion.

📌 2025-01-13 9.1.4

- Fix the execution slowness issue caused by the `Environment.SetEnvironmentVariable` function.
- Added the following files common to ClientCore and Core to the blacklist (these files will not be operated on during the entire packaging and upgrade process):

```
"Microsoft.Bcl.AsyncInterfaces.dll",  
"System.Collections.Immutable.dll",  
"System.IO.Pipelines.dll",  
"System.Text.Encodings.Web.dll",  
"System.Text.Json.dll"
```

📌 2025-01-05 Version 9.1.2

- Fixed the issue where updates would continue despite hash verification exceptions.
- Removed the references of the Common and Differential components from Bowl / ClientCore / Core to resolve the DLL usage issue during mutual upgrades.
- Note: The Common and Differential components will only be updated and iterated within the GeneralUpdate solution and will no longer be updated on the NuGet platform. The capabilities provided by the Common and Differential components will continue to exist in the Bowl / ClientCore / Core components through code file references. Developers no longer need to worry about whether the Common and Differential components are referenced.

📌 2025-01-04 9.1.0

- Add a feature to skip specified folders (specified parameters will be passed through Client and Upgrade).

- Add Scheme and Token parameters to the HTTP request headers.
- Fix the issue of OSS Samples failing to update.
- Modify the OSS feature to not delete the update package upon completion.
- Fix the issue in Samples where only the Client updates, not the Upgrade. (The looping update occurred because the local version number was not changed)
- Upgrade all Samples from version 9.0.0 to 9.1.0.

📌 2024-11-28 9.0.0

- Upgrade all components to C# 13 syntax.
- Optimize, refactor, and simplify code to achieve functionality with minimal code, reducing the difficulty of code reading.
- Simplified the complexity of input parameters for GeneralClientBootstrap.
- Removed Strategys parameter settings; these are now built into the component, automatically deducing the operating system platform to switch update strategies, so developers no longer need to manage update strategy settings.
- Added features for driver updates, backups, and installations.
- Optimized the automatic upgrade process with four types of update workflows:
 1. Client needs an upgrade, update server needs an upgrade.
 2. Client does not need an upgrade, update server does not need an upgrade.
 3. Client does not need an upgrade, update server needs an upgrade.
 4. Client needs an upgrade, update server does not need an upgrade.
- If an update fails, the version will be stored locally, and upon rollback, the failed version will be skipped on subsequent attempts.
- GeneralUpdate's OSS feature currently supports only Windows and only the zip compression format.
- Removed all event notifications from GeneralUpdateOSS.
- Introduced the GeneralUpdate.Bowl component.
- GeneralUpdate.Bowl includes rollback, monitoring, and dump export functionalities (Only for Windows; Linux support will be added gradually).
- Introduced the GeneralUpdate.Common component.
- Removed the GeneralUpdate.Zip component.
- Removed the GeneralUpdate.AspnetCore component.
- Removed the MultiDownloadProgressChangedEvent, merging its notification content into the MultiDownloadStatisticsEvent.
- Removed the legacy feature, replaced by GeneralUpdate.Bowl.
- Removed support for the 7z compression format; only zip compression is supported now.
- Removed ProgressType notification event parameters for various working modes.
- Replaced VersionHub with UpgradeHubService (push functionality).
- Updated all hash value-related checks and generation within components to the SHA256 algorithm, removing the MD5 algorithm.
- Added functionality to back up all current program files before updating.

- Added support for the Ubuntu operating system.
- Compatible with and supports AOT compilation, removing or refactoring all code detrimental to AOT compilation or usage.
- All component version numbers follow the .NET Core framework version number, sharing a unified version number instead of maintaining separate ones.
- Updated Sample example updates, allowing for one-click generation using bat scripts.
- Fixed several bugs reported in issues.
- Moved GeneralUpdate.Maui.OSS and GeneralUpdate.OSS libraries to a new repository GeneralUpdate.Maui.
- GeneralUpdate.Differential removed the GeneralUpdate.Zip reference and all compression package handling capabilities.
- Added reporting of the upgrade process to the server with statuses like pending update, update failed, and update successful.
- Refactored GeneralUpdate.Tools to an Avalonia version, adapting it for creating differential patch packages on Linux operating systems.

📌 2023-08-05

Before releasing an enterprise product to the market, it's crucial to test its features. In many companies, the auto-update feature often lacks documentation such as requirement documents or business specifications, unlike other product features. Typically, the main requirement is that the product can be updated smoothly or incrementally to save bandwidth. For testers with insufficient experience, this can lead to inadequate testing or overlooked issues, causing numerous problems. (This is just a testing approach and not professional testing guidance, so please consider it critically.)

1. Testing Version Upgrade Order

Assume all customers are currently using version v1.0.0.0, and we are about to release v2.0.0.0. Before release, both versions need to be tested in a test environment.

- While downloading the update package
- During file updates
- Simulate program interruption, network disconnection, power outages, weak network conditions, crashes, etc.

In GeneralUpdate, when extracting binary differential update patches between two versions, encrypted files may not be recognized by the differential algorithm. Consider adding encrypted files to a blacklist (SetBlacklist) or directly including them in the compressed package.

1. Failure Rollback or Re-upgrade?

This is a major concern in enterprises. While auto-updates offer many benefits, a failed update can render the client unusable, which is critical. I plan to add two strategies in GeneralUpdate to address this issue.

- **Strategy One:**

Back up files or directories to be updated before the upgrade. If the update fails, the backup files will be restored upon the second startup, and the auto-update will be disabled to prevent further updates after restoration.

- **Strategy Two:**

This is my preferred method. The essence of auto-updates is to upgrade, not roll back. The initial idea is to introduce a "last words" mechanism. To handle exceptions during updates that may corrupt files, the program should report the update status to the server after each update. If the client doesn't respond within 30 minutes, it is considered a destructive update failure (file corruption). Upon each startup, the upgrade program reads the last update's status. If the previous update failed, it automatically downloads and installs the client package (compressed package). Alternatively, introduce an update daemon process to receive real-time push notifications for automated downloads and installations.

If developers make structural changes to the project, such as moving or renaming modules in a client program based on IoC principles (e.g., Prism framework), it may lead to exceptions where the IoC container can't find the DLL after an update. Consider file path changes during updates, and testers should routinely inquire about such changes.

1. Final Testing as a Closed-Loop Process

This is crucial. If auto-updates are tested early, subsequent bug fixes or changes may introduce unknown issues. Therefore, conducting tests as the final closed-loop process is recommended (adjust as needed for special cases).

1. Weak Network Environment Testing

Use tools like NetLimiter and Clumsy to simulate weak network conditions. For detailed usage, refer to my article: [Link](#).

1. Gray Release

Before releasing a version, thorough testing is necessary.

- For large customer bases, conduct a small-scale gray release before full release to all customers.
- For gray upgrades, avoid setting the version as mandatory; let users choose whether to upgrade.

GeneralUpdate currently supports receiving push notifications of the latest version updates via Signal R. If a client with multiple devices has one device experiencing issues due to hardware or software environment problems, targeted fixes are necessary. In this case, use one-to-one upgrades for precise updates on the problematic device.

1. Compatibility of Old Configurations with New Versions

When different branches of a version exist in the market, consider local configuration files during auto-update upgrades. If the new version's program requires reading configuration files, incompatible old version configurations may cause issues. Testers should be aware of this scenario.

To avoid such issues, store frequently changing configurations on the server to be read at each login, while storing stable configurations locally.

1. Automated Testing

Writing scripts for automated testing of auto-updates is crucial. It saves time and increases accuracy in multi-branch and multi-version upgrade testing.

 **2023-07-23**

Developer Questions:

1. **What should be done if unexpected situations like network disconnection, power outage, computer crash, sudden blue screen, or program freeze occur during the update process, causing the application to fail to update or start properly? Can the update program roll back to the previous version?**

Answer: This question can be divided into two parts:

(1) In the event of unexpected situations, you can try restarting the application to resume the update download from the breakpoint. Currently, if files become corrupted during the update, reinstallation is the only option. In the future, we plan to introduce a mechanism to handle situations where the client application cannot start due to update anomalies.

(2) Rolling back is not possible because the core purpose of the auto-update program is to facilitate updates. Rolling back could negate the purpose of the update. It is preferable to quickly identify issues and urgently release a safe update package to allow clients to update incrementally until successful.

1. **What if the update package itself contains issues (e.g., abnormal files), causing the program to fail to start after the update? Can it roll back or back up?**

Answer: No, comprehensive testing must be conducted before the version is released. For a large customer base, a small-scale gray release should be conducted first. If no issues arise, then the release can be extended to all customers in the market.

1. **If the client locally stores some data files and needs to add new content while retaining previous data (e.g., updating a SQLite .db file without overwriting the old .db file), how can this be resolved? Can incremental updates handle such files?**

Answer: We previously developed such a feature, but it was ineffective and has been temporarily discontinued. It will require a redesign before being reinstated. Currently, the component's functionality cannot address this issue, and the only option is to overwrite. Incremental updates are also unlikely to handle this situation effectively.

📌 2023-04-23

Technical Update Overview

New Features: Blacklist Functionality and OSS Support for Windows and .NET MAUI Android Versions

This update focuses on bug fixes and a complete refactoring of the event management notification system across components.

Release Details

Component Name	Version (Old)	Version (New)	Status
GeneralUpdate.AspNetCore	1.4.1	-	-
GeneralUpdate.ClientCore	2.8.9	2.12.9	Updated
GeneralUpdate.Core	4.11.18	4.14.18	Updated
GeneralUpdate.Differential	1.3.0	1.4.1	Updated
GeneralUpdate.Zip	1.3.0	-	-
GeneralUpdate.Tool	2.2.5	2.3.5	Updated

Component Name	Version (Old)	Version (New)	Status
GeneralUpdate.Single	1.0.0	-	-
GeneralUpdate.Maui.OSS	1.0.0	1.0.0	New

Component Updates

1. GeneralUpdate.ClientCore

- Added OSS update functionality.
- Introduced blacklist functionality.
- Refactored events with a new event management mechanism.
- Fixed issue where incremental packages could only recognize additions, not deletions.

2. GeneralUpdate.Core

- Added OSS update functionality.
- Refactored events with a new event management mechanism.
- Fixed issue where incremental packages could only recognize additions, not deletions.

3. GeneralUpdate.Differential

- Fixed issue where incremental packages could only recognize additions, not deletions.

Changes in GeneralUpdate.ClientCore and GeneralUpdate.Core

1. Event Subscription Mechanism

- The event subscription mechanism is refactored. Instead of using `event +=`, events are now managed through an `EventManager`. This requires minimal code changes for developers and reduces the component's codebase by favoring composition over inheritance.

2. Blacklist Management

- Added the `SetBlacklist` method to manage files you don't want to update. The component has built-in default blacklist files: `{ "Newtonsoft.Json.dll" }` and default blacklist file extensions: `{ ".patch", ".7z", ".zip", ".rar", ".tar", ".json" }`.

```

Task.Run(async () =>
{
    var configinfo = GetWindowsConfiginfo();
    var generalClientBootstrap = await new GeneralClientBootstrap()
        .AddListenerMultiDownloadProgress(OnMultiDownloadProgressChanged)
        .AddListenerMultiDownloadStatistics(OnMultiDownloadStatistics)
        .AddListenerMultiDownloadCompleted(OnMultiDownloadCompleted)
        .AddListenerMultiAllDownloadCompleted(OnMultiAllDownloadCompleted)
        .AddListenerMultiDownloadError(OnMultiDownloadError)
        .AddListenerException(OnException)
        .Config(configinfo)
        .Option(UpdateOption.DownloadTimeOut, 60)
        .Option(UpdateOption.Encoding, Encoding.Default)
        .Option(UpdateOption.Format, Format.ZIP)
        .Strategy()
        .SetCustomOption>ShowCustomOption)
        .SetBlacklist(GetBlackFiles(), GetBlackFormats())
        .LaunchTaskAsync();
});

```

New OSS Functionality

OSS (Object Storage Service) is a familiar concept for backend developers, provided by major cloud service providers like Alibaba Cloud OSS, Tencent Cloud COS, and Huawei Cloud OBS. The update component adopts OSS for its naming and functionality, making it intuitive for developers to understand its purpose.

GeneralUpdate.OSS Operation Principle:

1. Prepare a `version.json` configuration file and an update file (`update.zip`) as usual.

```

[
  {
    "PubTime": 1680443321,
    "Name": "generalupdate.ossclient",
    "MD5": "9bf414990a67e74f11752d03f49b15d8",
    "Version": "1.0.4",
    "Url": "http://192.168.50.203/update.zip"
  },
  {
    "PubTime": 1680444916,
    "Name": "generalupdate.ossclient",
    "MD5": "JXC122DFXCZXZNMRFf11752d03f49b15d8",
    "Version": "1.0.5",
    "Url": "http://192.168.50.203/update2.zip"
  }
]

```

1. The client requests the OSS server or file server to download the `version.json` file at startup.
2. Upon downloading, it parses the version information to decide if an update is necessary. If so, it passes the info through the process to start `Upgrade` (the client self-closes).
3. `Upgrade` then downloads `update.zip`, extracts it, and overwrites local files.
4. Once the update completes, `Upgrade` restarts the client and self-closes. The update is finished.

GeneralUpdateOSS is easier to use compared to `GeneralUpdateBootstrap`. It downloads `version.json` and updates incrementally based on its content.

Quick Start

Client (Main Client) Code Example:

```
Task.Run(async () =>
{
    var url = "http://192.168.50.203";
    var appName = "GeneralUpdate.Client";
    var version = "1.0.0.0";
    var versionFileName = "version.json";
    ParamsOSS @params = new ParamsOSS(url, appName, version, versionFileName);
    await GeneralClientOSS.Start(@params);
});
```

Upgrade (Upgrade Assistant) Code Example:

```
private static void Main(string[] args)
{
    Task.Run(async () =>
    {
        ParamsOSS @params = SerializeUtil.Deserialize(args[0]);
        await GeneralUpdateOSS.Start(@params, Encoding.Default);
    });
}
```

Event Notification Subscription:

```
GeneralUpdateOSS.AddListenerMultiDownloadStatistics(OnMultiDownloadStatistics);

private static void OnMultiDownloadStatistics(object sender, MultiDownloadStatisticsEventArgs e)
{
    Console.WriteLine($" {e.Speed} , {e.Remaining.ToShortTimeString()}");
}
```

.NET MAUI OSS

The MAUI OSS functionality is similar to the OSS described above but is specifically for .NET MAUI, focusing on the Android platform.

1. Prepare a `version.json` configuration file and an update file (`update.apk`), which is the new version's APK.

```
{
  "PubTime": 1680444916,
  "Name": "com.companyname.generalupdate.ossclient",
  "MD5": "9bf414990a67e74f11752d03f49b15d8",
  "Version": "1.0.5",
  "Ur1": "http://192.168.50.203/com.companyname.generalupdate.ossclient.apk"
}
```

1. The client requests the OSS server or file server to download the `version.json` file at startup.
2. After downloading, it parses the version information to decide if an update is necessary.
3. If an update is needed, it downloads `update.apk` .
4. Upon download completion, the Android OS handles the installation, and the new app version runs.

Quick Start:

```
string url = "http://192.168.50.203";
string appName = "MainApplication.exe";
string currentVersion = "1.1.1.1";
string versionFileName = "versions.json";
GeneralUpdateOSS.AddListenerDownloadProcess(OnOSSDownload);
GeneralUpdateOSS.AddListenerException(OnException);
await GeneralUpdateOSS.Start(new ParamsAndroid(url, appName, "123456789", currentVersion, versionFileNam
e));
```

Event Notification Subscription:

```
GeneralUpdateOSS.AddListenerDownloadProcess(OnOSSDownload);
GeneralUpdateOSS.AddListenerException(OnException);

private void OnOSSDownload(object sender, OSSDownloadArgs e)
{
    Console.WriteLine($"{e.ReadLength},{e.TotalLength}");
}

private void OnException(object sender, ExceptionEventArgs exception)
{
    Console.WriteLine(exception.Exception.Message);
}
```

Guidance Articles for .NET MAUI Android Related Queries:

- [Blog Post on .NET MAUI Android](#)
- [Microsoft Documentation on .NET MAUI Android Deployment](#)
- [Stack Overflow: Android 8 Cleartext HTTP Traffic Not Permitted](#)

- [Youdao Note](#)

Current Testing Devices and Platforms:

1. Huawei Honor Px30 (non-HarmonyOS) - operational.
2. Visual Studio 2022 Preview on Pixel 5 - API 33 (Android 13.0 - API 33) - operational.

📌 2023-01-17

Open Source Project Reorganization

We have established the "General" open source organization on both GitHub and Gitee platforms. Contributors with significant code contributions will be invited to join the organization and receive rewards.

The original GeneralUpdate project has been split into three separate projects:

1. **GeneralLibrary on Gitee:**
 - Main organization page: [GeneralLibrary](#)
2. **Projects under GeneralLibrary:**
 - **Automatic Update Project:** [GeneralUpdate](#)
 - **Differential Patch Packaging Tool:** [GeneralUpdate.Tools](#)
 - **Usage Examples:** [GeneralUpdate-Samples](#)

📌 2022-10-09

Release Notes

Component Name	Version (old)	Version (new)	Status
GeneralUpdate.AspNetCore	1.1.0	1.4.1	Normal
GeneralUpdate.ClientCore	2.3.6	2.8.9	Normal
GeneralUpdate.Core	4.10.12	4.11.18	Normal
GeneralUpdate.Differential	1.0.0	1.3.0	Normal
GeneralUpdate.Zip	1.0.0	1.3.0	Normal
GeneralUpdate.Tool	1.0.0	2.1.5	Normal
GeneralUpdate.Single	1.0.0	-	Normal

[1] Component GeneralUpdate.AspNetCore

1. Refactor: Merged previous version and validate methods into a single Update method to perform version validation and return update information.
2. New: Added an upload method to support the automatic upload release function of GeneralUpdate.PackTool. This method completes the storage of version information and the placement of update packages on the file server.
3. New: Added a push method to support immediate push of important versions or updates between server services to the client, using Signal R to push update information after a single call.

[2] Component GeneralUpdate.ClientCore

1. Fix: Resolved file conflict issue between AutoUpdate.Core and AutoUpdate.ClientCore.
2. Fix: Resolved error when components do not require an update.
3. Refactor: Updated mechanism to support multiple operating system platforms.
4. Refactor: Refactored version validation mechanism.
5. New: Configuration adaptation for MAUI - Windows machine.
6. Fix: When the main program is already the latest, should it first check to avoid launching the upgrade software unnecessarily?
7. Refactor: Changed the method signature for configuration from `public GeneralClientBootstrap Config(ClientParmeter info)` to `public GeneralClientBootstrap Config(Configinfo info)`, removing the fields `validaterul` and `versionurl`.
8. New: Added a user-defined method to decide whether to skip the current update using a Task version method.
9. New: Added an enumeration to the update configuration, changing the original string ".zip" to the enumeration `Format.ZIP` to prevent user input errors.

[3] Component GeneralUpdate.Core

1. Refactor: Updated mechanism to support multiple operating system platforms.
2. Refactor: Refactored version validation mechanism.
3. Fix: When the main program is already the latest, should it first check to avoid launching the upgrade software unnecessarily?
4. Fix: Bug in "GeneralUpdate.Core\Utils\FileUtil.cs" `GetFileMD5` function.
5. Fix: Issue with multi-level folder structure updates failing.
6. New: Added a unique client identifier in the update system.
7. Offline: JSON configuration file update function will be directly overwritten. Will be back online after refactoring. Affected areas include:
 - Suggestions from users.
 - Error reports.
8. Research: Feasibility of running normally in a Mac environment.
9. Fix: Bug in "GeneralUpdate\src\GeneralUpdate.Core\Download\AbstractTask.cs" at line 143.

[4] Component GeneralUpdate.Differential

- Refactor: Recursively search for files to update to prevent missing update detections. Added a tree structure to manage the update file directory structure.
- Fix: Bug where only the first layer of files could be updated. Now all directories and subdirectories can be recursively updated.
- Fix: Error when the packaging tool includes two files with the same name but different extensions.

[5] Component GeneralUpdate.Zip

1. Fix: Several issues related to file compression.
2. Fix: Garbled characters in Word files.
3. Fix: Permission issues when extracting downloaded packages in C:\Program Files (x86).

[6] GeneralUpdate.PacketTool

1. Fix: Packaging tool did not consider subfolder issues during packaging.
2. Refactor: Refactored the file functionality system.
3. Refactor: Migrated packaging tool to MAUI.
4. New: Added upload functionality to GeneralUpdate.Tool.
5. Fix: PacketTool's packaging function was ineffective when only file content was modified.
6. Fix: Garbled characters in Chinese filenames after file extraction.

[7] Test Cases and Example Code

Example Code:

- src/GeneralUpdate.Api/Program.cs
- src/GeneralUpdate.Client/MainPage.xaml.cs
- src/GeneralUpdate.Upgrad/Program.cs

Test Cases (still in progress):

- src/TestClientCore/UnitTest1.cs
- src/TestDifferential/UnitTest1.cs
- src/TestMD5/UnitTest1.cs
- src/TestService/Program.cs
- src/TestZIP/UnitTest1.cs

2022-04-03

(1) Does the main program support mutual upgrades with the update program?

Answer: Yes, it does.

(2) Do developers need to write code to manage the timing of process closure or any other code?

Answer: No, they don't. The component already considers the entire update process, so developers don't need to write any additional auxiliary code beyond the component code.

(3) Does the update program need to be in the same directory as the main program?

Answer: Yes, it does. However, make sure that the update program does not reference any code from the main program; otherwise, the update will fail.

(4) Will the update package's patch files be deleted after the update is complete?

Answer: Yes, they will. The component ensures a clean file list after the update, avoiding redundant files and unnecessary disk space usage.

(5) Can this be used on the server side, for instance, for upgrades between services?

Answer: Theoretically, it is supported, although the author has not used it this way personally. Feedback indicates that some users have done this. This sharing is focused on C/S architecture scenarios.

(6) How can I get the MD5 checksum of the update package?

Answer: Use the AutoUpdate.MD5 project from the source code.

(7) How do I create an update package?

Answer: Use the GeneralUpdate.PacketTool to generate it. You can find the packaged installation program in the release section of the source code repository.

(8) How can I learn more about other aspects of the component?

Answer: You can learn more through the official website, relevant QQ groups, or by discussing with me on Gitee or GitHub issues.

(9) How to handle lack of permissions when extracting the download package in C:\Program Files (x86)?

Answer: [Link to issue](#)

(10) When the update file is small, the download speed shows as 0B/S.

Answer: [Link to issue](#)

📌 2022-03-23

Instructions

- Online Help Documentation: <http://justerzhu.cn/> (Will be divided into separate documents by component in the future)
- NuGet Versioning Standards Reference: [NuGet Package Versioning](#)
- Application Assembly Versioning Standards Reference: [Assembly Versioning](#) (The client program updated by the component is essentially your company's product; the component's operations will follow this standard.)
- This release includes many changes and is not a smooth upgrade (use with caution if stability is crucial or if modifications are undesirable). The project is gradually stabilizing, and future updates will aim for smooth upgrades to avoid requiring further developer modifications.
- In case of a "blunder" such as incorrect update package content or unforeseen events during the update, rollback is not currently supported. The current solutions are (1) request the update again, or (2) send an emergency fix update package via the push mechanism.
- Regarding the "non-rollback" issue, if you have better solutions, please share your thoughts in the issues section. Currently, there are two opinions in the community: (1) In case of unexpected events, the component should roll back the local program to the previous version to ensure usability even if the update fails. (2) The purpose of the update component is to push the component to a new version, making the release effective. If the update fails, it should continue until it succeeds; only forward, no retreat.

Release Content

Let's first look at the version numbers and the changes made in this major release (8 sections in total).

Component Name	Version (old)	Version (new)	Status
GeneralUpdate.AspNetCore	1.0.0	1.1.0	Normal
GeneralUpdate.ClientCore	1.1.2	2.3.6	Normal
GeneralUpdate.Core	3.6.10	4.10.12	Normal
GeneralUpdate.Differential	-	1.0.0	Normal
GeneralUpdate.Common	1.0.0	-	Removed

[1] Component GeneralUpdate.AspNetCore

1. [NEW] Added latest version push functionality with the VersionHub object. This primarily addresses the need to push urgent bug fix versions temporarily.

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddSingleton<IUpdateService, GeneralUpdateService>();
builder.Services.AddSignalR();
var app = builder.Build();

app.MapHub("/versionhub");

app.Use(async (context, next) =>
{
    var hubContext = context.RequestServices.GetRequiredService<IHubContext>();
    await CommonHubContextMethod((IHubContext)hubContext);
    if (next != null)
    {
        await next.Invoke();
    }
});

async Task CommonHubContextMethod(IHubContext context)
{
    await context.Clients.All.SendAsync("clientMethod", "");
}
```

[2] Component GeneralUpdate.ClientCore

1. [NEW] Overloaded configuration method in GeneralClientBootstrap, simplifying it to require only the remote server address, such as http://127.0.0.1, and the update program name, reducing developer concern over parameters as they are automatically obtained within the component.

```
public GeneralClientBootstrap Config(string url, string appName = "AutoUpdate.Core");
```

1. [NEW] Added latest version reception push functionality with the VersionHub object. This primarily addresses the need to receive urgent bug fix versions temporarily, allowing update operations in the GetMessage callback function.

```
VersionHub<string>.Instance.Subscribe($"{ baseUrl }/{ hubName }", "TESTNAME", new Action<string>(GetMessage));
```

[3] Component GeneralUpdate.Core

1. [FIX] Fixed issue where a download interruption due to network disconnection did not throw an exception after the timeout period.
2. [REFACTOR] Refactored multi-task download design.
3. [FIX] Fixed issue where FileUtil.Update32Or64Libs would throw exceptions under certain conditions.

4. [REFACTOR] Refactored default update strategy design.
5. [NEW] Added latest version reception push functionality with the VersionHub object. This primarily addresses the need to receive urgent bug fix versions temporarily, allowing update operations in the GetMessage callback function.

```
VersionHub<string>.Instance.Subscribe($"{ baseUrl }/{ hubName }", "TESTNAME", new Action<string>(GetMessage));
```

1. [NEW] Added support for extracting 7z compressed files.
2. [REMOVE] Removed duplicate code between GeneralUpdate.Core and GeneralUpdate.ClientCore, addressing redundancy through linked files.
3. [NEW] Added support for incremental updates. For example, if only one file is updated, only that file will be packaged.
4. [NEW] Added support for binary differential functionality. For example, if a file like temp.dll is modified to create a new version, a .patch file will be created from the differences between the new and old versions. With normal project iteration, this .patch file will typically be only a few kilobytes. Combined with 7z's high compression, the update package size is minimized to save bandwidth and data usage. It should be noted that if new files are added in the new version, they will be included in the update package as they are.

[4] Component [NEW] GeneralUpdate.Differential

Added the Differential component in GeneralUpdate, which mainly provides the following functionalities:

1. [NEW] Added binary differential updates, generating .patch files.
2. [NEW] Added incremental updates. For example, if version 1.1.1.1 originally had 10 files and this release modified 3 of them, only the modified files will be packaged.
3. [NEW] Updated configuration files (currently only supports .json configuration files with a depth of 1 level). For example, if the client program uses a .json file as a configuration file, the original configuration content will be preserved while updating the configuration file. Assuming the client's original configuration file (.json) contains:

```
{
  "ip": "123456789",
  "port": 9999
}
```

The server's new configuration file structure is:

```
{
  "ip": "0",
  "port": 0,
  "path": "c://"
}
```

The "path" field will be updated into the client's local configuration file, preserving the original content as:

```
{
  "ip": "123456789",
  "port": 9999,
  "path": "c://"
}
```

[5] Component **GeneralUpdate.Zip**

1. [REFACTOR] Refactored compression component design to support .zip and .7z archive formats.
2. [NEW] Added 7z extraction functionality.

[6] Component **[REMOVE] GeneralUpdate.Common**

1. Removed this component, decoupling GeneralUpdate.Core and GeneralUpdate.ClientCore as it was deemed unnecessary for the current version.

[7] Component **GeneralUpdate.Single**

No functional updates in this release.

[8] Test Cases Example

To simplify developers' work in writing test cases during validation and testing, the following test cases have been added to the GeneralUpdate solution:

- AutoUpdate.ClientCore
- AutoUpdate.Core
- AutoUpdate.MD5
- AutoUpdate.Differential
- AutoUpdate.ZIP
- AutoUpdate.MinimalService
- AutoUpdate.Test

 **2022-02-16**

Current Framework Support

Theoretical Support: [Microsoft .NET Standard Documentation](#)

Framework Name	Support Status
.NET Core 2.0	Supported
.NET 5, 6, 7	Supported
.NET Framework 4.6.1	Supported
Mono 5.4	Theoretically supported, unverified
Xamarin.iOS	Theoretically supported, unverified
Xamarin.Mac	Theoretically supported, unverified
Xamarin.Android	Theoretically supported, unverified
Universal Windows Platform 10.0.16299	Theoretically supported, unverified

Framework Name	Support Status
Unity 2018.1	Theoretically supported, unverified

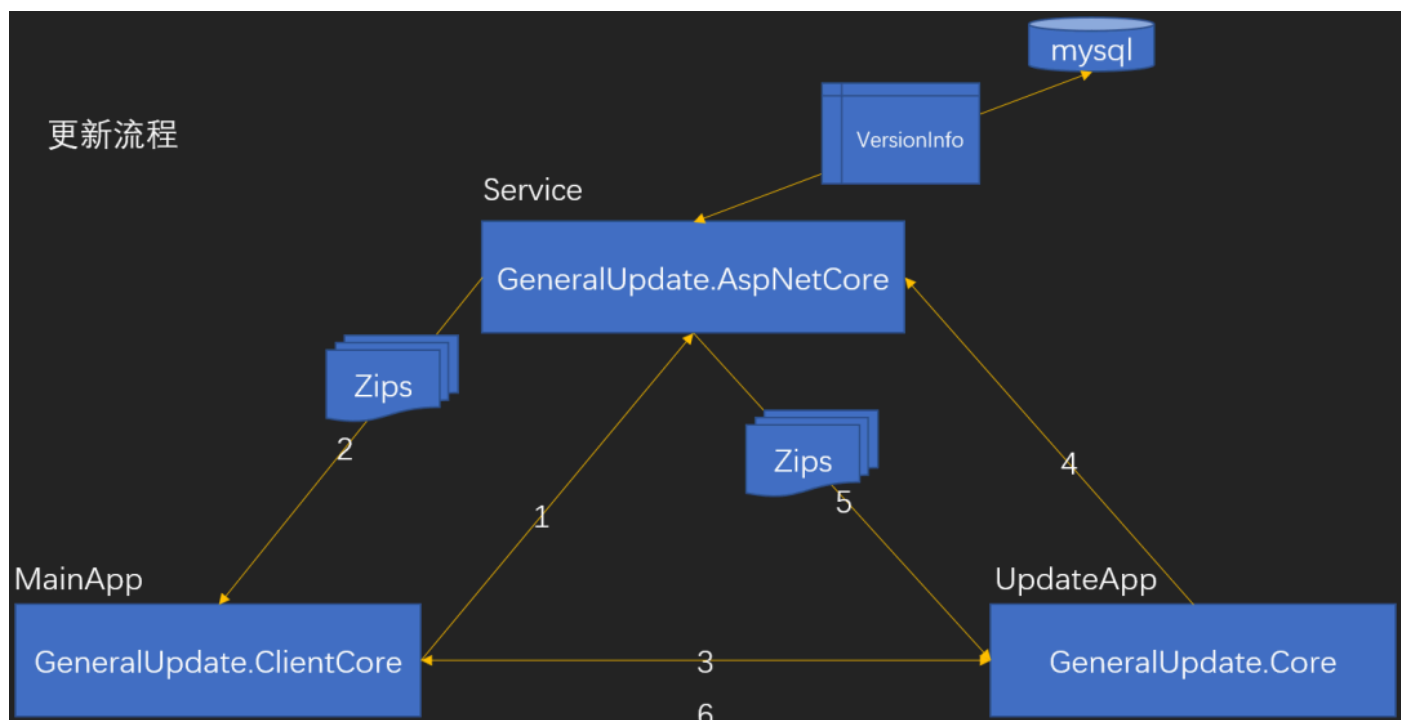
UI Framework Name	Support Status
WPF	Supported
UWP	Unverified, awaiting feedback
MAUI	Not supported, in progress for compatibility
Avalonia	Unverified, awaiting feedback
WinUI	Unverified, awaiting feedback
Console	Supported

Operating System Name	Support Status
Windows	Supported
Linux	Unverified
Mac	Unverified
iOS	Not supported
Android	Not supported

📌 2021-08-14

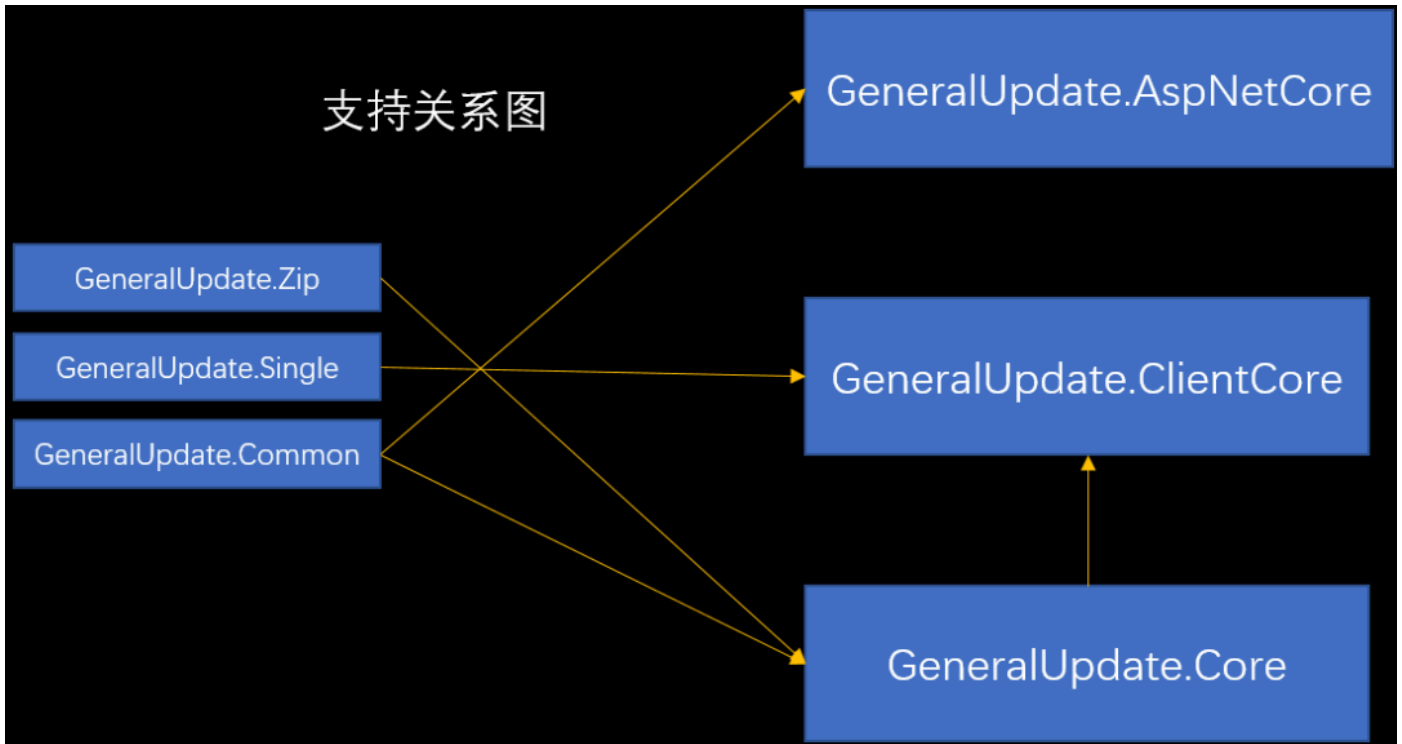
Notice

- The GeneralUpdate website is expected to be developed using Blazor. It will introduce component structure, update processes, quick start guides, parameters, and more.



1. When the main program starts, it checks if the updater needs to be updated.
2. If an update is needed, it uploads the version number of the updater and updates it incrementally.

3. Once the updater is updated or if no update is needed, it checks if the main program needs an update. If so, the updater is launched.
4. Requests the main program update version.
5. Receives multiple update packages for the main version and updates incrementally.
6. After updating, the updater is closed, and the main program is started.



- The above update and download processes support resuming from breakpoints and incremental updates.
- Incremental download functionality is sorted by the release date of versions, with earlier versions being updated first (see the SQL script field content in the source code for details).
- The maximum tested update content for incremental downloads is 1GB.
- Nuget
- (New) <https://www.nuget.org/packages/GeneralUpdate.Common/>
- (New) <https://www.nuget.org/packages/GeneralUpdate.ClientCore/>
- (New) <https://www.nuget.org/packages/GeneralUpdate.AspNetCore/>
- <https://www.nuget.org/packages/GeneralUpdate.Zip/>
- <https://www.nuget.org/packages/GeneralUpdate.Single/>
- (Update) <https://www.nuget.org/packages/GeneralUpdate.Core/>
- Issues & Git, Gitee

Feel free to raise issues at the following addresses. Please describe the cause of the anomaly or defect details as clearly as possible. The checking cycle is every Friday.

- <https://gitee.com/Juster-zhu/GeneralUpdate>
- <https://github.com/WELL-E/AutoUpdater/tree/autoupdate2>
- <https://github.com/WELL-E/AutoUpdater/issues>
- <https://gitee.com/Juster-zhu/GeneralUpdate/issues>
- New

- GeneralUpdate.Core adds incremental update functionality.
- GeneralUpdate.Core introduces new events: ExceptionEvent, MutiDownloadStatisticsEvent, MutiDownloadErrorEvent, MutiDownloadCompletedEvent, MutiDownloadProgressEvent, MutiAllDownloadCompletedEvent.
- GeneralUpdate.Core adds the RemoteAddressBase64 method.
- A new ClientParameter class is added for inter-process communication to pass parameters between components.
- A new component, GeneralUpdate.AspNetCore, is added, which provides update version information based on update type and supports pipeline dependency injection. However, you need to write your own methods for database queries.
- A new component, GeneralUpdate.ClientCore, is added:
 - (1) It can update the version of the update component itself.
 - (2) It supports incremental updates for update components (multiple update packages can be downloaded simultaneously).
 - (3) It simplifies the launch of the updater, eliminating the previous cumbersome process of starting and passing parameters.
- A new component, GeneralUpdate.Common, is added. This library integrates all common classes and helper methods used within the components (this component is mandatory, and its update frequency is very low, so it's not recommended to package it in update packages).
- A new MySQL script is added to create the update_version table for the GeneralUpdate.AspNetCore server.
- Remove
- All notification events in GeneralUpdate.Core are replaced with MutiXXXXEvent.
- The method RemoteAddress in GeneralUpdateBootstrap for passing parameters through processes is removed from GeneralUpdate.Core.
- The feature for rolling back failed updates is removed, as it caused fatal exceptions due to insufficient permissions when rolling back updates on the C drive. This feature may be reopened in the future.
- Fix
- Fixed the issue with HTTPS SSL access failures.
- Fixed notification exceptions related to event registration with begininvoke in other .NET framework versions.
- Corrected multiple class name spelling errors.
- Removed unnecessary fields from several Model classes.
- Quick Start

(1) Example GeneralUpdate.ClientCore

```

// Client version.
var mainVersion = "1.1.1";
var mianType = 1;

// Updater version
clientParameter = new ClientParameter();
clientParameter.ClientVersion = "1.1.1";
clientParameter.ClientType = 2;
clientParameter.AppName = "AutoUpdate.ConsoleApp";
clientParameter.MainAppName = "AutoUpdate.Test";
clientParameter.InstallPath = @"D:\update_test";
clientParameter.UpdateLogUrl = "https://www.baidu.com/";
clientParameter.ValidateUrl = $"https://127.0.0.1:5001/api/update/getUpdateValidate/{ clientParameter.ClientType }/{ clientParameter.ClientVersion }";
clientParameter.UpdateUrl = $"https://127.0.0.1:5001/api/update/getUpdateVersions/{ clientParameter.ClientType }/{ clientParameter.ClientVersion }";
clientParameter.MainValidateUrl = $"https://127.0.0.1:5001/api/update/getUpdateValidate/{ mianType }/{ mainVersion }";
clientParameter.MainUpdateUrl = $"https://127.0.0.1:5001/api/update/getUpdateVersions/{ mianType }/{ mainVersion }";

generalClientBootstrap = new GeneralClientBootstrap();
generalClientBootstrap.MutiDownloadProgressChanged += OnMutiDownloadProgressChanged;
generalClientBootstrap.MutiDownloadStatistics += OnMutiDownloadStatistics;
generalClientBootstrap.MutiDownloadCompleted += OnMutiDownloadCompleted;
generalClientBootstrap.MutiAllDownloadCompleted += OnMutiAllDownloadCompleted;
generalClientBootstrap.MutiDownloadError += OnMutiDownloadError;
generalClientBootstrap.Exception += OnException;
generalClientBootstrap.Config(clientParameter).
    Strategy();
await generalClientBootstrap.LaunchTaskAsync();

```

(2) Example GeneralUpdate.Core

```
static void Main(string[] args)
{
    var resultBase64 = args[0];
    var bootstrap = new GeneralUpdateBootstrap();
    bootstrap.Exception += OnException;
    bootstrap.MutiDownloadError += OnMutiDownloadError;
    bootstrap.MutiDownloadCompleted += OnMutiDownloadCompleted;
    bootstrap.MutiDownloadStatistics += OnMutiDownloadStatistics;
    bootstrap.MutiDownloadProgressChanged += OnMutiDownloadProgressChanged;
    bootstrap.MutiAllDownloadCompleted += OnMutiAllDownloadCompleted;
    bootstrap.Strategy().
        Option(UpdateOption.DownloadTimeOut, 60).
        RemoteAddressBase64(resultBase64).
        LaunchAsync();
}
```

(3) Example GeneralUpdate.AspNetCore

Startup.cs

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddControllers();
    services.AddSingleton<IUpdateService, GeneralUpdateService>();
}
```

UpdateController.cs

```
private readonly ILogger _logger;
private readonly IUpdateService _updateService;

public UpdateController(ILogger logger, IUpdateService updateService)
{
    _logger = logger;
    _updateService = updateService;
}

/// <summary>
/// https://localhost:5001/api/update/getUpdateVersions/1/1.1.1
/// </summary>
/// <param name="clientType"> 1:ClientApp 2:UpdateApp</param>
/// <param name="clientVersion"></param>
/// <returns></returns>
[HttpGet("getUpdateVersions/{clientType}/{clientVersion}")]
public async Task GetUpdateVersions(int clientType, string clientVersion)
{
    _logger.LogInformation("Client request 'GetUpdateVersions'.");
    var resultJson = await _updateService.UpdateVersionsTaskAsync(clientType, clientVersion, UpdateVersion
s);
    return Ok(resultJson);
}

/// <summary>
/// https://localhost:5001/api/update/getUpdateValidate/1/1.1.1
/// </summary>
/// <param name="clientType"> 1:ClientApp 2:UpdateApp</param>
/// <param name="clientVersion"></param>
/// <returns></returns>
[HttpGet("getUpdateValidate/{clientType}/{clientVersion}")]
public async Task GetUpdateValidate(int clientType, string clientVersion)
{
    _logger.LogInformation("Client request 'GetUpdateValidate'.");
    var lastVersion = GetLastVersion();
```

```
var resultJson = await _updateService.UpdateValidateTaskAsync(clientType, clientVersion, lastVersion,
true, GetValidateInfos);
return Ok(resultJson);
}
```

2021-03-18

Statement

1. This component will support applications developed with the following frameworks: .NET Framework 4.6.1, .NET Core 2.0, and .NET 5.
2. The GeneralUpdate.Single component currently only supports WPF and the .NET Framework.

Additions

1. GeneralUpdate.Core version 3.2.1 introduces a new feature for rollback on update failure.
2. A new component, GeneralUpdate.Zip version 1.0.0, has been added, providing functionality for compressing and decompressing files, and it can be used independently.

Fixes and Modifications

1. Fixed the issue where events were not being notified during multithreaded operations.
2. The component framework version has been changed from .NET Framework 4.5.2 to .NET Standard 2.0.
(Supported frameworks reference: <https://docs.microsoft.com/en-us/dotnet/standard/net-standard>)

Removals

1. Removed the dependency on the third-party component 7zip.
2. Removed the RegistryUtil utility class.

GeneralUpdate Quick start

```

args = new string[6] {
    "0.0.0.0",
    "1.1.1.1",
    "https://github.com/WELL-E",
    "http://192.168.50.225:7000/update.zip",
    @"E:\PlatformPath",
    "509f0ede227de4a662763a4abe3d8470",
};

GeneralUpdateBootstrap bootstrap = new GeneralUpdateBootstrap();
bootstrap.DownloadStatistics += OnDownloadStatistics;
bootstrap.ProgressChanged += OnProgressChanged;
bootstrap.Strategy().
    Option(UpdateOption.Format, "zip").
    Option(UpdateOption.MainApp, "your application name").
    Option(UpdateOption.DownloadTimeOut, 60).
    RemoteAddress(args).
    Launch();

```

📌 2020-08-30

New Additions

1. In the new release, GeneralUpdate.Core version 2.1.0 adds support for resuming downloads.
2. In the new release, a new component, GeneralUpdate.Single version 1.0.0, has been added. It provides singleton functionality to prevent multiple instances of the auto-update program from starting.

Update Process

1. The client program starts and retrieves update information from the server, then parses and compares it to determine if an update is needed.
2. Parses process parameters, such as local version number, latest version number, download URL, extraction path, installation path, etc.
3. The client program launches the updater (GeneralUpdate) and closes itself.
4. The auto-update program (GeneralUpdate) performs the following based on the passed update information:
 - (1) download, (2) MD5 checksum, (3) extraction, (4) delete update files, (5) replace update files, (6) close the update program itself, (7) start the client program.
5. Update is completed.

Inter-process Communication

```
using System;
using System.Diagnostics;
using System.ComponentModel;

namespace MyProcessSample
{
    class MyProcess
    {
        //此段代码来自于msdn
        // Opens the Internet Explorer application.
        void OpenApplication(string myFavoritesPath)
        {
            // Start Internet Explorer. Defaults to the home page.
            Process.Start("IExplore.exe");

            // Display the contents of the favorites folder in the browser.
            Process.Start(myFavoritesPath);
        }

        // Opens urls and .html documents using Internet Explorer.
        void OpenWithArguments()
        {
            // url's are not considered documents. They can only be opened
            // by passing them as arguments.
            Process.Start("IExplore.exe", "www.northwindtraders.com");

            // Start a Web page using a browser associated with .html and .asp files.
            Process.Start("IExplore.exe", "C:\\myPath\\myFile.htm");
            Process.Start("IExplore.exe", "C:\\myPath\\myFile.asp");
        }

        // Uses the ProcessStartInfo class to start new processes,
        // both in a minimized mode.
        void OpenWithStartInfo()
        {
            ProcessStartInfo startInfo = new ProcessStartInfo("IExplore.exe");
            startInfo.WindowStyle = ProcessWindowStyle.Minimized;

            Process.Start(startInfo);

            startInfo.Arguments = "www.northwindtraders.com";

            Process.Start(startInfo);
        }
    }
}
```

```
static void Main()
{
    // Get the path that stores favorite links.
    string myFavoritesPath =
        Environment.GetFolderPath(Environment.SpecialFolder.Favorites);

    MyProcess myProcess = new MyProcess();

    myProcess.OpenApplication(myFavoritesPath);
    myProcess.OpenWithArguments();
    myProcess.OpenWithStartInfo();
}
}
```

Using GeneralUpdate.Core-2.1.0

```

#region Launch1

args = new string[6] {
    "0.0.0.0",
    "1.1.1.1",
    "https://github.com/WELL-E",
    "http://192.168.50.225:7000/update.zip",
    @"E:\PlatformPath",
    "509f0ede227de4a662763a4abe3d8470",
};

GeneralUpdateBootstrap bootstrap = new GeneralUpdateBootstrap(); // Auto-update bootstrap class
bootstrap.DownloadStatistics += OnDownloadStatistics; // Download progress notification event
bootstrap.ProgressChanged += OnProgressChanged; // Update progress notification event
bootstrap.Strategy() // Register strategy, can customize update process
    .Option(UpdateOption.Format, "zip") // Specify the format of the update package, currently only zip is
supported
    .Option(UpdateOption.MainApp, "your application name") // Specify the main program name to start after
the update, no need to add .exe, just the name
    .Option(UpdateOption.DownloadTimeout, 60) // Download timeout (seconds), default is 30 seconds if not
specified
    .RemoteAddress(args) // Use the previous parameter array
    .Launch(); // Start the update
#endregion

#region Launch2

/*
 * Launch2
 * Added a second launch method
 * Process:
 * 1. Specify the update address, e.g., https://api.com/GeneralUpdate?version=1.0.0.1, passing the current
client version in the web API
 * 2. If an update is needed, the API returns all update information (details in /Models/UpdateInfo.cs)
 * 3. After obtaining the update information, start the HTTP request for the update package
 * 4. Download
 * 5. Extract
 * 6. Update local files
 * 7. Close the update program
 * 8. Start the configured main program
 * The update program must be placed in the same directory as the main program
 */

//GeneralUpdateBootstrap bootstrap2 = new GeneralUpdateBootstrap();
//bootstrap2.DownloadStatistics += OnDownloadStatistics;

```

```

//bootstrap2.ProgressChanged += OnProgressChanged;
//bootstrap2.Strategy()
//    .Option(UpdateOption.Format, "zip")
//    .Option(UpdateOption.MainApp, "")
//    .Option(UpdateOption.DownloadTimeOut, 60) // Download timeout (seconds), default is 30 seconds if no
t specified
//    .RemoteAddress(@"https://api.com/GeneralUpdate?version=1.0.0.1") // Specify update address
//    .Launch();

#endregion

private static void OnProgressChanged(object sender, ProgressChangedEventArgs e)
{
    if (e.Type == ProgressType.Updatefile)
    {
        var str = $"Currently updating file: {e.ProgressValue}, total files to update: {e.TotalSize}";
        Console.WriteLine(str);
    }

    if (e.Type == ProgressType.Done)
    {
        Console.WriteLine("Update completed");
    }
}

private static void OnDownloadStatistics(object sender, DownloadStatisticsEventArgs e)
{
    Console.WriteLine($"Download speed: {e.Speed}, remaining time: {e.Remaining.Minute}:{e.Remaining.Secon
d}");
}

```

Using GeneralUpdate.Single-1.0.0

```

/// <summary>
/// Interaction logic for App.xaml
/// </summary>
public partial class App : Application, ISingletonApp
{
    private const string AppId = "{7F280539-0814-4F9C-95BF-D2BB60023657}";

    [STAThread]
    protected override void OnStartup(StartupEventArgs e)
    {
        string[] resultArgs = null;

        if (e.Args == null || e.Args.Length == 0)
        {
            resultArgs = new string[6] {
                "0.0.0.0",
                "1.1.1.1",
                "https://github.com/WELL-E",
                "http://192.168.50.225:7000/update.zip",
                @"E:\PlatformPath",
                "509f0ede227de4a662763a4abe3d8470",
            };
        }
        else
        {
            resultArgs = e.Args;
        }

        if (resultArgs.Length != 6) return;
        if (Singleton.InitializeAsFirstInstance(AppId))
        {
            var win = new MainWindow();
            var vm = new MainViewModel(resultArgs, win.Close);
            win.DataContext = vm;

            var application = new App();
            application.InitializeComponent();
            application.Run(win);
            Singleton.Cleanup();
        }
    }

    public bool SignalExternalCommandLineArgs(IList<string> args)
    {

```



```

        if (this.MainWindow.WindowState == WindowState.Minimized)
        {
            this.MainWindow.WindowState = WindowState.Normal;
        }
        this.MainWindow.Activate();

        return true;
    }
}

```

Q&A

1. What should be done if there are multiple version iterations and cross-version updates?

As long as it's not a framework-level update, updates can be applied. Regardless of how many iterations or versions you cross, you just need to place the final package on the server. There is no incremental update involved here, so if there are many updates, you can simply overwrite all the new files.

1. Is GeneralUpdate integrated with the client as a whole?

No, GeneralUpdate is a program independent of the client.

1. Can it perform incremental updates, automatic rollback on failure, or update local data or configuration files?

Currently, it cannot. (This feature is in the development plan.)

1. How does GeneralUpdate perform updates?

The update method involves overwriting the existing client files locally.

2020-05-03

Here is the translation of your content into English:

1. Concise launch code as follows: Launch1 Launch2

Name	Type	Remarks
UpdateOption.Format	Config Parameter	Compression format of the update package (currently only supports zip)
UpdateOption.MainApp	Config Parameter	Name of the main application to launch after the update
DownloadStatistics	Event	Event notification for update package download
ProgressChanged	Event	Event notification for update progress
Strategy()	Method	Strategy injection
RemoteAddress()	Method	Remote address configuration, pass in args[] if not available
Launch	Method	Start update
GeneralUpdateBootstrap	Class	Update bootstrap class

1. Added Strategy (update strategy), which is open for everyone to freely extend the update method without modifying the source code, not limited to the default update strategy.
2. When updating local files, there will be an update notification event, clearly indicating the total number of files to update and the current file being updated.

3. Added new update statuses: Check (check for updates), Download (download update package), UpdateFile (update files), Done (update completed), Fail (update failed). Developers can directly use a series of enumeration values to determine the current running status and make corresponding handling.
4. Added validation for several startup configuration parameters.

```
if (args != null)
{
    if (args.Length == 0)
    {
        throw new NullReferenceException("Args does not contain any elements.");
    }

    if (args.Length > elementNum)
    {
        throw new Exception($"The number of args cannot be greater than { elementNum }");
    }
}

if (string.IsNullOrEmpty(PacketName))
{
    throw new NullReferenceException("packet name not set");
}

if (string.IsNullOrEmpty(DownloadPath))
{
    throw new NullReferenceException("download path not set");
}

if (string.IsNullOrEmpty(InstallPath))
{
    throw new NullReferenceException("install path not set");
}

if (string.IsNullOrEmpty(MD5))
{
    throw new NullReferenceException("MD5 not set");
}
```

1. Download update progress notification event will provide remaining download time (Remaining) and download speed (Speed).

```

#region Launch1

args = new string[6] {
    "0.0.0.0",
    "1.1.1.1",
    "https://github.com/WELL-E",
    "http://192.168.50.225:7000/update.zip",
    @"E:\PlatformPath",
    "509f0ede227de4a662763a4abe3d8470",
};

GeneralUpdateBootstrap bootstrap = new GeneralUpdateBootstrap();
bootstrap.DownloadStatistics += OnDownloadStatistics;
bootstrap.ProgressChanged += OnProgressChanged;
bootstrap.Strategy().
    Option(UpdateOption.Format, "zip"). // Specify the format of the update package, currently only support
    ts zip
    Option(UpdateOption.MainApp, "your application name"). // Specify the name of the main application to
    launch after the update, no need to add .exe, just the name
    RemoteAddress(args). // Retains the previous parameter array collection
    Launch();

#endregion

#region Launch2

/*
 * Launch2
 * Added a second launch method
 * Process:
 * 1. Specify the update address, e.g., https://api.com/GeneralUpdate?version=1.0.0.1, passing the current
    client version to the web API
 * 2. If an update is needed, the API returns all update information (refer to /Models/UpdateInfo.cs for d
    etails)
 * 3. After obtaining the update information, start an HTTP request for the update package
 * 4. Download
 * 5. Unzip
 * 6. Update local files
 * 7. Close the update program
 * 8. Start the configured main program
 * The update program must be in the same directory as the main program
 */

// GeneralUpdateBootstrap bootstrap2 = new GeneralUpdateBootstrap();

```

```

// bootstrap2.DownloadStatistics += OnDownloadStatistics;
// bootstrap2.ProgressChanged += OnProgressChanged;
// bootstrap2.Strategy().
//     Option(UpdateOption.Format, "zip").
//     Option(UpdateOption.MainApp, "KGS.CPP").
//     RemoteAddress(@"https://api.com/GeneralUpdate?version=1.0.0.1"). // Specify the update address
//     Launch();

#endregion

private static void OnProgressChanged(object sender, ProgressChangedEventArgs e)
{
    if (e.Type == ProgressType.Updatefile)
    {
        var str = $"Currently updating file: {e.ProgressValue}, Total number of files to update: {e.TotalSize}";
        Console.WriteLine(str);
    }

    if (e.Type == ProgressType.Done)
    {
        Console.WriteLine("Update completed");
    }
}

private static void OnDownloadStatistics(object sender, DownloadStatisticsEventArgs e)
{
    Console.WriteLine($"Download speed: {e.Speed}, Remaining time: {e.Remaining.Minutes}:{e.Remaining.Seconds}");
}

```

Let me know if you need further assistance!

Agent 技能 Agent Skills

GeneralUpdate Agent Skills 套件

Claude Code 技能套件 — 帮助 .NET 开发者在 5 分钟内将 **GeneralUpdate** 自动更新系统集成到任意 .NET 应用中。

覆盖 50+ 真实 Issue 发现的已知问题，提供即用型代码生成 + 深度故障排查。

Current Version: 0.0.2-beta.1 — targets NuGet `GeneralUpdate.Core 10.5.0-rc.1` 兼容性: `v10.5.0-rc.1`
(NuGet 最新预览版) 所有模板已通过 `dotnet build` 编译验证 (0 errors) 。

什么是 Agent Skill?

Agent Skill (智能体技能) 是一种**可复用指令模板**，让 AI 编程助手获得特定领域的专业知识和工作流程。

用一句话概括: Skill = 领域专家知识 + 标准化 workflow，注入到 AI 助手中，让它能像资深开发者一样处理该领域的问题。

Skill 解决了什么问题?

在没有 Skill 的情况下，让 AI 助手帮你做 GeneralUpdate 集成，你需要：

- 1. 详细解释 GeneralUpdate 的双进程架构、API、策略.....AI 才能理解上下文
- 2. 每次对话都要重复这些背景知识
- 3. AI 生成的代码可能遗漏边界情况、安全注意事项

有了 Skill 之后：

- 1. 一条命令（如 `/generalupdate-init`）就能触发完整的工作流
- 2. AI 自动加载领域知识（架构、API、已知问题、最佳实践）
- 3. 输出标准化、经实战验证的代码

本套件包含 7 个 Skill

Skill	作用一句话
<code>generalupdate-init</code>	从零搭建双项目脚手架 + Bootstrap 配置
<code>generalupdate-ui</code>	自动识别 UI 框架，生成全状态更新窗口
<code>generalupdate-strategy</code>	按场景选择最优更新策略（OSS/静默/差分等）
<code>generalupdate-advanced</code>	Bowl 崩溃守护、IPC、Pipeline、自定义 Hook
<code>generalupdate-troubleshoot</code>	诊断 50+ 已知问题，匹配修复方案
<code>generalupdate-migration</code>	v9.x → v10 / dev-branch → stable 迁移
<code>generalupdate-security-audit</code>	14 项安全检查 + 修复建议

安装 Skills

获取 Skill 文件

所有 Skill 文件托管在独立仓库：

```
https://github.com/GeneralLibrary/generalupdate-skill-codegen
```

仓库包含：

```
generalupdate-skill-codegen/  
├─ SKILL.md                               ← 套件入口  
└─ .claude/skills/  
    ├─ generalupdate-init/                ← 🚀 集成指南  
    ├─ generalupdate-ui/                  ← 🎨 界面生成  
    ├─ generalupdate-strategy/            ← ⚙️ 策略指南  
    ├─ generalupdate-advanced/            ← 🔧 高级定制  
    ├─ generalupdate-troubleshoot/        ← 🩺 故障排查  
    ├─ generalupdate-migration/           ← 🔄 迁移指南  
    └─ generalupdate-security-audit/ ← 🔒 安全审计
```

方式一：直接克隆到项目

```
# 在你的项目根目录下执行  
git clone https://github.com/GeneralLibrary/generalupdate-skill-codegen.git  
cp -r generalupdate-skill-codegen/.claude/skills .claude/  
cp generalupdate-skill-codegen/SKILL.md .claude/  
rm -rf generalupdate-skill-codegen
```

方式二：作为 git submodule 管理（推荐团队协作）

```
# 在你的项目根目录下执行  
git submodule add https://github.com/GeneralLibrary/generalupdate-skill-codegen.git .claude/generalupdate-skill-codegen  
ln -s .claude/generalupdate-skill-codegen/.claude/skills .claude/skills 2>/dev/null || \  
cp -r .claude/generalupdate-skill-codegen/.claude/skills .claude/
```

submodule 方式的好处：通过 `git submodule update --remote` 即可获取 Skill 的最新更新。

方式三：仅复制需要的 Skill

```
# 只复制你需要的 skill  
git clone https://github.com/GeneralLibrary/generalupdate-skill-codegen.git /tmp/gussc  
cp -r /tmp/gussc/.claude/skills/generalupdate-init .claude/skills/  
rm -rf /tmp/gussc
```

集成到各种 AI 编程助手

本套件支持以下 AI 编程助手。根据你使用的工具选择对应的集成方式：

Claude Code (桌面版 / CLI)

桌面版：

1. 在项目根目录放置 `.claude/skills/` 目录（参考上面的安装步骤）
2. 重启 Claude Code 桌面版
3. 输入 `/generalupdate-init` 即可使用

CLI 版：

1. 在项目根目录放置 `.claude/skills/` 目录
2. 重新启动 Claude Code CLI
3. 输入 `/generalupdate-init` 测试

验证方式：输入 `/generalupdate-init`，看到集成指南输出即为成功。

GitHub Copilot (VS Code)

安装到 `.github/copilot-instructions.md`（或项目 `instructions`）：

1. 将 `generalupdate-skill-codegen/SKILL.md` 的内容整理为 `instructions`
2. 放置在项目根目录的 `.github/copilot-instructions.md`
3. 或在 GitHub 仓库设置中添加为 Copilot 知识库

注意：GitHub Copilot 使用 `instructions` 机制而非 `slash` 命令，因此需要将 Skill 内容转换为 `instructions` 格式。

Cursor

Cursor 支持 `.cursorrules` 文件，类似 Claude Code 的 `skills`：

1. 在项目根目录创建或编辑 `.cursorrules`
2. 将需要的 Skill 内容（如 `generalupdate-init/SKILL.md`）的核心指令追加到 `.cursorrules`
3. 或使用 Cursor 的 Docs 功能，将 `skill-codegen` 仓库添加为参考文档

Windsurf

1. 在项目根目录创建 `.windsurfrules` 文件
2. 将需要的 Skill 知识嵌入该文件
3. 引用方式：在对话中直接描述需求，AI 会基于规则文件中的知识回答

Cline / Continue.dev (VS Code 插件)

1. 在 `.clinerules` 或 `.cursorrules` / `.continuerc` 中引用 Skill 知识
2. 或直接将 Skill 内容添加到项目的 AI 规则文件中

DeepSeek / 通义灵码 等国内 AI 助手

- 在项目根目录建议放置一个 `AI_INSTRUCTIONS.md` 或 `CODEGEN.md`
- 将关键 Skill 指令以自然语言描述写入该文件
- 在对话开始时引用该文件作为上下文

✅ 验证安装

无论在哪个 AI 助手中使用，验证方式都是：

尝试让 AI 帮你 "给我的 .NET 应用添加自动更新"

如果 AI 输出了 GeneralUpdate 双项目结构 + Bootstrap 配置代码 → 安装成功

如果 AI 只给出了通用建议 → 需要补充 Skill 上下文

先决条件

1. **AI 编程助手**: 需要一个支持自定义指令/规则的 AI 编程工具
2. **.NET SDK**: 目标项目需基于 .NET 8+（推荐 .NET 10）
3. **GeneralUpdate 服务端**: 对于标准策略，需要部署 [GeneralSpacestation](#) 或兼容的后端服务
4. **双进程架构**: 需要理解 Client + Upgrade 双进程核心理念

🛠️ 开发者集成路线图

你是哪种情况？找到你的入口，按步骤推进：

你的场景	从哪开始	做什么	完成后下一步
 第一次加更新，从零开始	<code>/generalupdate-init</code>	① 选集成模式 → ② 生成 Bootstrap → ③ 部署	<code>/generalupdate-ui</code> （加界面）
 已有集成，需要更新界面	<code>/generalupdate-ui</code>	① 自动检测框架 → ② 生成窗口 → ③ 桥接事件	<code>/generalupdate-strategy</code> （选策略）
 要选更新策略（OSS/静默/差分）	<code>/generalupdate-strategy</code>	① 决策树选策略 → ② 配置服务端 → ③ 示例代码	<code>/generalupdate-init</code> （配置 Bootstrap）
 需要高级定制（Bowl/IPC/Hooks）	<code>/generalupdate-advanced</code>	① 选扩展点 → ② 生成模板代码 → ③ 集成	部署验证
 更新失败/报错/异常	<code>/generalupdate-troubleshoot</code>	① 症状收集 → ② 匹配已知问题 → ③ 修复	回到对应 skill 改配置
 已有 v9.x 要迁移到 v10	<code>/generalupdate-migration</code>	参考迁移路径 + API 对照表	<code>/generalupdate-troubleshoot</code> （检查迁移问题）
 需要安全审计	<code>/generalupdate-security-audit</code>	14 项安全检查 + 审计报告	<code>/generalupdate-init</code> （修复发现的问题）

Skills 总览

Skill	命令	描述	覆盖范围
 generalupdate-init	<code>/generalupdate-init</code>	双项目脚手架 + Bootstrap 配置 (4 种方式)	4 大场景 + 4 种配置方式 + 完整 API
 generalupdate-ui	<code>/generalupdate-ui</code>	自动识别 UI 框架, 生成全状态更新窗口 (11 种状态)	6 UI 框架 + 全状态机 + 桥接代码
 generalupdate-strategy	<code>/generalupdate-strategy</code>	6 种策略决策树 + 混合组合 + 平台差异	6 策略 + 4 组合 + 平台对照
 generalupdate-advanced	<code>/generalupdate-advanced</code>	10+ 扩展点 + 4 种 IPC + Bowl + AOT	10+ 扩展点 + 完整架构图
 generalupdate-troubleshoot	<code>/generalupdate-troubleshoot</code>	50+ 已知问题诊断 + BM25 搜索引擎	8 致命 + 11 高 + 20 中 + 12 低
 generalupdate-migration	<code>/generalupdate-migration</code>	v9.x → v10 / dev-branch → stable 迁移	2 条迁移路径 + API 对照表
 generalupdate-security-audit	<code>/generalupdate-security-audit</code>	安全审计 + 修复建议	14 项安全矩阵 + 审计报告模板

快速开始

在 AI 编程助手中, 只需描述你的需求, 对应的 Skill 会自动激活:

"给我的 WPF 应用添加自动更新"
→ 自动激活 `generalupdate-init` + `generalupdate-ui`

"更新成功了但启动报错"
→ 自动激活 `generalupdate-troubleshoot`

"配置 OSS 静默更新"
→ 自动激活 `generalupdate-strategy`

"添加 Bowl 崩溃守护 + 自定义 Hooks"
→ 自动激活 `generalupdate-advanced`

"把 v9.x 的项目迁移到 v10"
→ 自动激活 `generalupdate-migration`

通用集成验证清单

无论使用哪个 skill, 完成集成后请逐项检查:

Bootstrap 配置

- ☐ `UpdateRequest` 的 6 个必填字段都已设置 (`UpdateUrl`, `AppSecretKey`, `MainAppName`, `ClientVersion`, `ProductId`, `InstallPath`)
- ☐ `UpdateUrl` 指向的服务端 API 可正常返回版本信息
- ☐ `AppSecretKey` 长度 ≥ 16 字符, 与服务端一致
- ☐ `InstallPath` 指向正确的安装目录 (生产环境用 `AppDomain.CurrentDomain.BaseDirectory`)
- ☐ `AppType` 设置正确 (`Client = 1`, `Upgrade = 2`)

NuGet & 编译

- ☐ `Client` 和 `Upgrade` 项目使用**完全相同**的 `GeneralUpdate` NuGet 版本
- ☐ 项目能正常 `dotnet build` (0 errors)

部署结构

- ☐ `UpgradeApp.exe` 存在于发布目录 (首个版本就必须有)
- ☐ `generalupdate.manifest.json` 的 `UpdateAppName` 包含 `.exe`
- ☐ IPC 文件 (`UpdateInfo.msg`) 路径在 `Client/Upgrade` 间一致
- ☐ `Encoding` 设置为 `Encoding.UTF8` (防止 Linux/macOS 中文乱码)

数据来源

所有技能的内容基于以下真实数据:

- **GitHub Issues:** #308–#517 (重构、Bug、功能、测试)
 - **Gitee Issues:** 30 个真实用户反馈 (中文社区痛点)
 - **全面代码审计:** 17 CRITICAL/HIGH + 14 MEDIUM + 10 INFO 发现
 - **Samples 源码:** `CompleteUpdateSample`、`SilentUpdateSample`、`OssSample`、`DifferentialSample`、`PushSample`、`BowlSample`、`ExtensionSample`、`CompressSample`、`ImDiskQuickInstallSample`
 - **UI Samples:** `SemiUrsa`、`LayUI`、`AntdUI`、`WPFDevelopers`、`MauiUpdate`、`AndroidUpdate`
-

Skill 文件结构

```
.claude/skills/
├─ generalupdate-init/          (7 files)
│  ├─ SKILL.md                 ← 4 大场景 + 4 种配置 + API 详解
│  ├─ reference.md             ← NuGet/API/协议/框架兼容性
│  └─ templates/
│     ├─ MinimalIntegration.cs
│     ├─ FullIntegration.cs
│     ├─ generalupdate.manifest.json
│     └─ project-scaffold/
│
├─ generalupdate-ui/            (10 files)
│  ├─ SKILL.md                 ← 11 状态 UI 状态机 + 框架检测逻辑
│  └─ templates/
│     ├─ RealDownloadService.cs
│     ├─ DownloadViewModels.cs
│     ├─ SemiUrsaClientView.axaml / SemiUrsaUpgradeView.axaml
│     ├─ LayUIStyle.xaml / WPFDevelopersStyle.xaml
│     ├─ AntdUIStyle.cs
│     └─ MauiUpdatePage.xaml/.cs
│
├─ generalupdate-strategy/      (7 files)
│  ├─ SKILL.md                 ← 决策树 + 6 策略详解 + 混合 + 平台对照
│  └─ examples/
│     ├─ ClientServerStrategy.cs / OssStrategy.cs
│     ├─ SilentStrategy.cs / DifferentialStrategy.cs
│     └─ CrossVersionStrategy.cs / PushStrategy.cs
│
├─ generalupdate-advanced/      (6 files)
│  ├─ SKILL.md                 ← 10+ 扩展点 + 4 IPC + Bowl + 事件系统
│  ├─ reference.md
│  └─ templates/
│     ├─ CustomHooks.cs / CustomStrategy.cs
│     └─ BowlIntegration.cs / NamedPipeIPC.cs
│
├─ generalupdate-troubleshoot/  (5+ files)
│  ├─ SKILL.md                 ← 诊断 workflow
│  ├─ reference.md             ← 50+ 症状清单 (C/H/M/L 四级)
│  ├─ scripts/search.py        ← BM25 搜索引擎
│  ├─ scripts/core.py          ← BM25 算法核心
│  └─ data/issues.csv          ← 51 条已知问题数据库
│
├─ generalupdate-migration/     (1 file)
```

```
| └─ SKILL.md                ← v9.x→v10 / dev-branch→stable 迁移
|
└─ generalupdate-security-audit/ (1 file)
    └─ SKILL.md                ← 14 项安全审计矩阵
```

API 兼容性说明

⚠️ NuGet 引用规则 (v10.5.0-rc.1) :

- Core only: `dotnet add package GeneralUpdate.Core --version 10.5.0-rc.1`
- With Bowl: 同时引用 `GeneralUpdate.Core` 和 `GeneralUpdate.Bowl` (v10.5.0-rc.1 中无类型冲突)
- Differential 已嵌入 Core, **无需**额外引用 `GeneralUpdate.Differential`

⚠️ API Surface: v10.5.0-rc.1 采用了全新的配置系统:

- ☒ `UpdateRequest` / `UpdateRequestBuilder` — 替代旧的 `Configinfo`
- ☒ `SetSource(updateUrl, appSecretKey)` — 零配置入口
- ☒ `SetOption(Option, T)` — 可编程配置系统
- ☒ `IUpdateHooks` — 生命周期钩子 (`Hooks()`)
- ☒ `IStrategy` — 可替换策略接口 (`Strategy()`)
- ☒ `UseDiffPipeline(Action)` — 差分管道配置
- ☒ `SilentPollOrchestrator` — 静默轮询
- ☒ `AddListenerProgress` — 第 7 个事件监听器
- ☒ `Configinfo` 类已被移除

反馈与建议

使用过程中遇到任何问题或有改进建议, 欢迎提交 Issue:

- **GitHub Issues:** [GeneralUpdate-Samples/issues](#) — 报告 Bug、提出功能需求
- **Skill 仓库:** [generalupdate-skill-codegen/issues](#) — Skill 内容相关的问题和建议
- **GeneralUpdate Issues:** [GeneralUpdate/issues](#) — 核心库 Bug 和功能请求

提交时请附上以下信息以便快速排查:

- GeneralUpdate 版本号 (如 v10.5.0-rc.1)
- 平台 (Windows / Linux / macOS)
- 更新策略 (标准 / OSS / 静默 / 差分 / 推送)
- 完整错误日志 (可从 `Logs/generalupdate-trace-*.log` 获取)

版本历史

0.0.2-beta.1 — 2026-06-16

Updated for GeneralUpdate v10.5.0-rc.1 API:

- Configinfo → UpdateRequest (namespace: `GeneralUpdate.Core.Configuration`)
- Event args moved to `GeneralUpdate.Core.Download` and `GeneralUpdate.Core.Event`
- Added `SetSource()`, `SetOption()`, `Hooks()` , `Strategy()` API coverage
- Updated all strategy examples to use the new API
- Updated `CustomHooks.cs` and `CustomStrategy.cs` to show v10.5 capabilities
- Fixed `IsComplated` → `IsCompleted`
- NuGet version bumped to `10.5.0-rc.1`
- Added 2 new skills: `generalupdate-migration`, `generalupdate-security-audit`

0.0.1-beta.1 — 2026-06-16

Initial beta release. All templates rewritten for NuGet v10.4.6 stable API.

相关项目

- [GeneralUpdate](#) — .NET 自动更新核心库
- [GeneralSpacestation](#) — 更新服务端
- [generalupdate-skill-codegen](#) — Agent Skills 套件仓库
- [GeneralUpdate-Samples](#) — 示例项目合集

GeneralUpdate 集成完全指南

帮助开发者在任意 .NET 应用中集成 GeneralUpdate 自动更新。从零开始，不需要提前了解 GeneralUpdate 的内部细节。

先理解：为什么更新需要两个程序？

GeneralUpdate 采用**双进程架构**，这是理解一切配置的前提：

你的 App.exe (Client — 主程序)

- |
- ├— 平时：正常运行你的业务逻辑
- |
- ├— 发现新版本时：
 - | 1. 下载更新包
 - | 2. 把下载信息写成 IPC 文件（一种临时文件）
 - | 3. 启动 "升级程序" (UpgradeApp.exe)
 - | 4. 自己退出（因为 Windows 不允许程序覆盖自己）
- |
- ▼

UpgradeApp.exe (Upgrade — 升级程序)

- |
- ├— 读取 IPC 文件
- ├— 解压/替换文件（此时主程序已退出，文件不占用）
- ├— 启动主程序
- └— 自己退出

IPC 文件 (Inter-Process Communication) = 临时通讯文件，Client 把下载好的更新信息写进去，Upgrade 读出来执行更新。

manifest.json = 更新清单文件，告诉 GeneralUpdate 你的主程序叫什么、升级程序在哪。

这种设计的原因很简单：**Windows 不允许正在运行的程序覆盖自己的 .exe 文件**，所以需要有一个"替身"来完成替换工作。

你需要准备什么？

1. 你的 .NET 项目 (WPF / WinForms / Avalonia / MAUI / 控制台 都可以)
2. .NET SDK (8.0+)
3. 一个后端服务 (可选) — 没有后端也可以用 OSS 对象存储

安装 NuGet

```
dotnet add package GeneralUpdate.Core --version 10.5.0-beta.7
```

如果要用崩溃守护（Bowl），再加一个包：`dotnet add package GeneralUpdate.Bowl --version 10.5.0-beta.7` 差分更新功能已经内置在 Core 里，**不需要**额外装 `GeneralUpdate.Differential`。

最低配置：3 行代码跑起来

这是最简单的集成方式，适用于控制台应用或想先验证功能：

```
using GeneralUpdate.Core;
using GeneralUpdate.Core.Configuration;

var config = new UpdateRequest
{
    UpdateUrl = "https://your-server.com/api",    // 服务端地址
    AppSecretKey = "your-secret-key",             // 密钥（长度 ≥ 16 字符）
    MainAppName = "MyApp.exe",                   // 你的主程序文件名
    ClientVersion = "1.0.0.0",                    // 当前版本号
    ProductId = "my-product-001",                // 你的产品 ID
    InstallPath = "."                             // 安装目录（. 表示当前目录）
};

await new GeneralUpdateBootstrap().SetConfig(config).LaunchAsync();
```

这 6 个字段是 UpdateRequest 的**必填项**，少一个都会报错：

字段	填什么	为什么需要
UpdateUrl	服务端 API 地址	程序从这里查询有没有新版本
AppSecretKey	你设定的密钥	用于 IPC 加密通讯，Client 和 Upgrade 必须一致
MainAppName	"MyApp.exe"	告诉升级程序"更新完要启动哪个 exe"
ClientVersion	"1.0.0.0"	服务端根据这个判断哪些版本需要更新给你
ProductId	"my-app-001"	区分不同产品（如果你有多个应用）
InstallPath	"."	更新包解压到哪个目录

完整配置：带事件监听

如果你想看到更新进度、处理错误、做日志，加上事件监听：


```
using GeneralUpdate.Core;
using GeneralUpdate.Core.Configuration;
using GeneralUpdate.Core.Download;

var config = new UpdateRequest
{
    UpdateUrl = "https://your-server.com/Upgrade/Verification",
    AppSecretKey = "your-secret-key",
    MainAppName = "MyApp.exe",
    ClientVersion = "1.0.0.0",
    ProductId = "my-product-001",
    InstallPath = AppDomain.CurrentDomain.BaseDirectory, // 生产环境用这个
};

await new GeneralUpdateBootstrap()
    .SetConfig(config)
    .AddListenerUpdateInfo((_, e) =>
    {
        // 有更新时触发
        Console.WriteLine($"发现 {e.Info?.Body?.Count ?? 0} 个新版本");
    })
    .AddListenerMultiDownloadStatistics((_, e) =>
    {
        // 下载进度（可绑定到进度条）
        Console.WriteLine($"进度: {e.ProgressPercentage}% | 速度: {e.Speed}");
    })
    .AddListenerMultiDownloadCompleted((_, e) =>
    {
        Console.WriteLine($"版本 {e.Version} 下载完成");
    })
    .AddListenerMultiDownloadError((_, e) =>
    {
        Console.WriteLine($"下载失败: {e.Exception?.Message}");
    })
    .AddListenerMultiAllDownloadCompleted((_, e) =>
    {
        Console.WriteLine("全部下载完成，即将开始更新");
    })
    .AddListenerException((_, e) =>
    {
        Console.WriteLine($"异常: {e.Message}");
    })
    .AddListenerProgress((_, e) =>
    {
```

```
// v10.5.0-beta.7 新增的第 7 个事件
}))
.LaunchAsync();
```

通常你只需要关注 3 个事件：`MultiDownloadStatistics`（进度条）、`MultiDownloadError`（下载失败）、`Exception`（异常）。其他事件按需添加。

UpdateRequest 完整配置说明

除了 6 个必填字段，还有这些可选配置：

```
var config = new UpdateRequest
{
    // === 必填（上面已经列出了） ===
    UpdateUrl = "...", AppSecretKey = "...", MainAppName = "...",
    ClientVersion = "1.0.0.0", ProductId = "...", InstallPath = ".",

    // === 可选：认证方式 ===
    AuthScheme = AuthScheme.Hmac,           // Hmac / Bearer / ApiKey / Basic
    Token = "your-token",
    BasicUsername = "user",
    BasicPassword = "pass",

    // === 可选：备份时排除的文件 ===
    Files = new List<string> { "*.log", "*.tmp" },           // 排除日志/临时文件
    Formats = new List<string> { ".pdb" },                 // 排除调试符号文件
    Directories = new List<string> { "logs", "cache" },     // 排除整个目录
};
```

也可以使用**建造者模式**（效果一样，写法更流畅）：

```
var config = UpdateRequestBuilder.Create()
    .SetUpdateUrl("https://your-server.com/api")
    .SetAppSecretKey("your-secret-key")
    .SetMainAppName("MyApp.exe")
    .SetClientVersion("1.0.0.0")
    .SetProductId("my-product-001")
    .SetInstallPath(".")
    .Build();
```

或者**零配置模式**（从 manifest.json 自动发现，更简洁）：

```
await new GeneralUpdateBootstrap()
    .SetSource(
        updateUrl: "https://your-server.com/api",
        appSecretKey: "your-secret-key")
    .AddListenerUpdateInfo(...)
    .LaunchAsync();
```

4 大更新场景（服务端返回什么）

GeneralUpdate 根据服务端返回的更新包类型自动选择动作：

场景	发生了什么
None	无需更新 → 直接启动主程序
UpgradeOnly	只更新升级程序自身 → 主程序原地解压覆盖升级程序
MainOnly	只更新主程序 → Client 下载 → IPC → 启动 Upgrade 替换主程序
Both	两者都更新

Upgrade 进程配置

Upgrade（升级程序）不需要填配置，它从 IPC 文件自动读取：

```
using GeneralUpdate.Core;

await new GeneralUpdateBootstrap()
    .AddListenerException( (_, e) =>
        Console.WriteLine($"错误: {e.Message}"))
    .LaunchAsync();
```

你的项目里需要**两个独立的项目**：一个 Client（主程序），一个 Upgrade（升级程序）。

部署结构检查清单

发布后你的目录应该长这样：

```
publish/
├─ MyApp.exe           ← 你的主程序
├─ generalupdate.manifest.json ← 更新清单（告诉 GeneralUpdate 升级程序在哪）
└─ update/
    └─ UpgradeApp.exe   ← 升级程序（首个版本就必须有！）
```

首次部署时最常见的错误就是**忘了放 UpgradeApp.exe**，导致检测到更新后无法执行升级。

应用角色说明

AppType 区分两个进程的角色：

值	枚举	谁在用
1	AppType.Client	主程序（你的业务代码）
2	AppType.Upgrade	升级程序（UpgradeApp.exe）
3	AppType.OssClient	OSS 模式的主程序
4	AppType.OssUpgrade	OSS 模式的升级程序

新手常见错误

#	错误	后果	正确做法
1	Client 和 Upgrade 用了 不同版本 的 NuGet 包	运行时报 <code>Method not found</code>	两个项目用 完全一样 的版本号
2	忘了放 UpgradeApp.exe	检测到更新但无法执行	从第一个版本就放入 update/ 目录
3	IPC 文件编码没设 UTF-8	Linux/macOS 中文乱码	设置 <code>Encoding.UTF8</code>
4	版本号不是 4 段式（如 <code>1.0</code> ）	版本比较出错	始终用 <code>1.0.0.0</code> 格式
5	manifest.json 写的 exe 名不对	更新后找不到主程序	和实际文件名完全一致

下一步

- 需要更新 UI → [generalupdate-ui](#)
- 选哪种更新策略 → [generalupdate-strategy](#)
- 需要崩溃守护 / 高级定制 → [generalupdate-advanced](#)
- 遇到问题 → [generalupdate-troubleshoot](#)

GeneralUpdate 更新界面生成指南

如果你的应用需要**让用户看到更新进度**（而不是在后台静默更新），这个页面会帮你生成合适的 UI 代码。

先理解：两个程序各自显示什么

GeneralUpdate 是双进程架构，更新时两个程序都会显示界面：

你的主程序（Client）

显示 "检查更新 → 下载进度"

用户能看到：正在下载 45% / 下载速度 / 剩余时间

↓ 下载完成后启动 ↓

升级程序（Upgrade）

显示 "正在安装更新，请稍候..."

用户能看到：安装进度条 / 安装失败提示

Client 端：负责下载，适合显示进度条、速度、版本信息 **Upgrade 端**：负责安装替换文件，过程很快，通常只显示等待界面

你属于哪种情况？

情况	需要做什么
已有主程序，想要加更新窗口	在现有项目上加一个更新窗口（本页会帮你生成）
从零开始搭建	先用 generalupdate-init 搭建双项目结构，再回来加 UI
不需要进度 UI（静默更新）	走 generalupdate-strategy 的静默模式

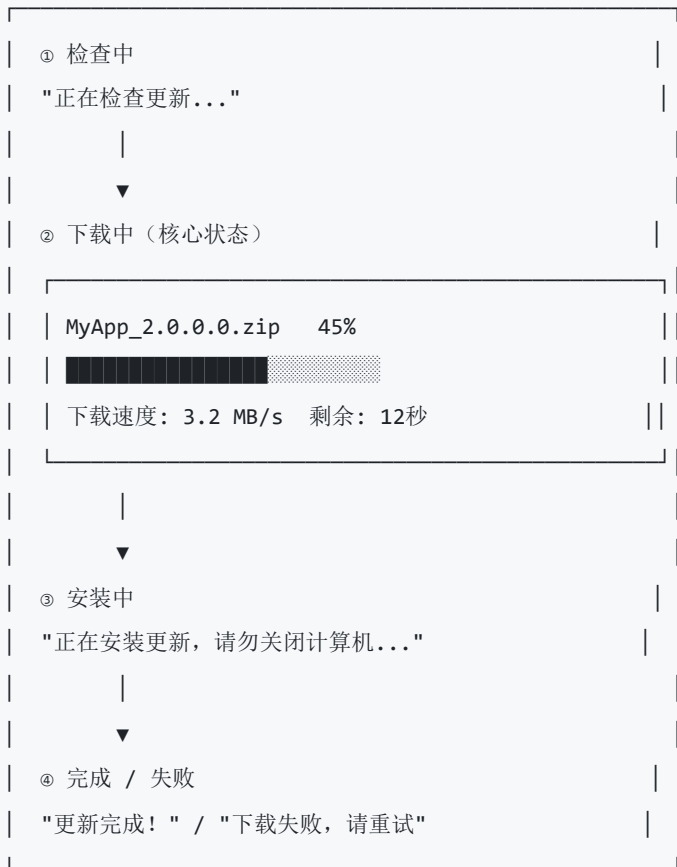
支持哪些 UI 框架

框架	模板文件	特点
WPF + LayUI.Wpf	LayUIStyle.xaml	玻璃效果进度条
WPF + WPFDevelopers	WPFDevelopersStyle.xaml	圆形进度 + 呼吸灯动画
Avalonia + SemiUrsa	SemiUrsaClientView.axaml	跨平台，暗黑切换
WinForms + AntdUI	AntdUIStyle.cs	暗黑主题
MAUI	MauiUpdatePage.xaml	移动端 + 桌面端

无论哪种框架，更新窗口的**逻辑都是相同的**，只是界面样式不同。

更新窗口的 4 个关键状态

一个更新窗口本质上就是一个**状态机**，只有 4 种主要状态：



除了这 4 种主要状态，还可能有"已是最新版本"、"暂停"、"重试"等子状态，但**新手只需要关注 ① 和 ②**。

如何把更新进度连到 UI 上？

GeneralUpdate 的事件系统会告诉你进度变化，你只需要把事件里的数字填到 UI 控件上：

```
// GeneralUpdate 更新进度事件 → 更新你的 UI 控件
bootstrap.AddListenerMultiDownloadStatistics((_, e) =>
{
    // e.ProgressPercentage → 你的进度条 Value (0-100)
    // e.Speed                → 你的下载速度标签
    // e.Remaining            → 你的剩余时间标签
    // e.BytesReceived        → 已下载字节
    // e.TotalBytesToReceive → 总字节
});
```

就是这么简单。**核心工作就是把事件里的 5 个数字绑定到 UI 控件上。**

生成 UI 代码前的信息收集

你的项目

- UI 框架：_____（WPF / WinForms / Avalonia / MAUI / 控制台）

- 是否需要暗黑模式：_____（是/否）

更新窗口角色（看上面的"各自显示什么"）

- Client 端（主程序）：_____（需要显示下载进度？）

- Upgrade 端（升级程序）：_____（需要显示安装等待？）

简单示例：控制台进度条

即使你没有图形界面，一个控制台进度条也很简单：

```
await new GeneralUpdateBootstrap()
    .SetConfig(config)
    .AddListenerMultiDownloadStatistics((_, e) =>
    {
        // 控制台进度条：3 行代码

        Console.WriteLine($"{r}下载进度: {e.ProgressPercentage}% ");
        Console.WriteLine($"[{new string('■', (int)(e.ProgressPercentage / 5))}]");
        Console.WriteLine($"[{new string('□', 20 - (int)(e.ProgressPercentage / 5))}]");
        Console.WriteLine($"速度: {e.Speed}/s");
    })
    .LaunchAsync();
```

已知问题和注意事项

#	问题	说明
1	更新 UI 不要做耗时操作	事件回调里只更新 UI，不要写文件或发网络请求
2	跨线程更新控件	WPF 用 Dispatcher.Invoke，WinForms 用 Control.Invoke
3	Upgrade 端不需要下载进度	Upgrade 只负责安装，显示 "请稍候" 即可
4	下载失败要显示重试按钮	否则用户卡在 99% 不知道怎么办
5	进度条不要一次跳到 100%	用户需要看到中间过程才有安全感

进阶：全状态 UI 模板

如果你需要更完整的 UI（包括暂停、恢复、重试、暗黑切换等 11 种状态），Skill 仓库提供了完整的模板代码。
生成方式：在 Claude Code 中描述你的 UI 框架，AI 会自动选择对应的模板并生成代码。

相关页面

- [generalupdate-init](#) — 先搭建双项目结构
- [generalupdate-strategy](#) — 如果不想要 UI (静默更新)
- [generalupdate-advanced](#) — 高级定制

⚙️ 选择你的更新策略

更新策略 = GeneralUpdate 用什么方式来发现和下载新版本。

根据你有没有后端服务、是否需要节省带宽、是否需要用户感知，选择一个适合你的策略。

先理解：更新策略的本质

不管哪种策略，最终都是回答 3 个问题：

- | | | |
|--------------|------------|-----------|
| ① 去哪查有没有新版本？ | ② 去哪下载更新包？ | ③ 怎么下载？ |
| | | |
| └ 自己查（轮询） | └ 自己的服务器 | └ 全量下载 |
| └ 服务端通知（推送） | └ 对象存储 OSS | └ 只下载差异部分 |
| └ | └ | |

策略快速选择表

看不懂术语没关系，先看这一列就行：

你的情况	推荐策略	一句话说明
有后端服务，新手入门	① 标准	最简单的方案，服务端返回版本信息，客户端下载更新
没有后端，只想把包放云存储	② OSS	把更新包上传到 S3/MinIO，零服务端成本
带宽有限，用户量大	④ 差分	每次只下载有变化的部分，省 60-90% 流量
需要强制用户升到最新（跳过中间版本）	⑤ 跨版本	用户从 v1.0 直升 v3.0，不用逐个版本升
用户无感知自动更新	③ 静默	后台下载完，下次启动自动生效
需要紧急推送安全补丁	⑥ 推送	服务端主动告诉客户端"立刻更新"

逐个策略详解

① 标准客户端-服务端（推荐新手入门）

适用场景：你有后端服务（如 GeneralSpacestation），想快速实现自动更新。

流程：客户端 → 问服务端"有新版本吗？"
服务端 → 返回版本列表
客户端 → 下载更新包 → 启动升级

优点：实现最简单，1 个 API 接口就能跑起来 **缺点：**需要部署和维护后端服务

② OSS 对象存储

适用场景：你没有后端服务，但有对象存储（阿里云 OSS / AWS S3 / MinIO）。

流程：客户端 → 定期读取 OSS 上的 versions.json
OSS → 返回版本列表
客户端 → 从 OSS 下载更新包 → 启动升级

优点：零服务端，成本最低 **缺点：**不区分主程序和升级程序的更新包；Bucket 要设置好权限

③ 静默更新

适用场景：用户不需要看到更新过程，后台静默完成。

流程：客户端在后台定期检查 → 发现新版本 → 静默下载
下载完成后 → 通知用户"更新已下载，是否重启？"
或：下次启动时自动生效

优点：用户体验好，不打扰 **缺点：**需要处理"下载完但用户不重启"的版本状态 **轮询频率建议：**30-60 分钟一次（太短耗电/流量，太长用户等得久）

④ 差分更新

适用场景：你的更新包很大（>100MB），用户量多，想省带宽。

流程：服务端生成"补丁"（只记录新旧版本之间的差异）
客户端下载补丁 → 本地合成新版本

优点：节省 60-90% 的下载量 **缺点：**服务端需要额外构建差分包；大文件（>2GB）可能触发整数溢出

⑤ 跨版本更新（CVP）

适用场景：你的用户版本分散（有人还在 v1.0，有人是 v2.5），都需要升到 v3.0。

流程：服务端保留所有版本的升级路径
v1.0 用户 → 直接下载 v3.0 全量包
v2.5 用户 → 只下载 v2.5→v3.0 的差分包

优点：用户不用逐个版本升级 **缺点：**服务端需要额外构建和维护版本跳转逻辑

⑥ SignalR 推送

适用场景：需要紧急推送安全修复，不能让用户等轮询。

流程：服务端通过 SignalR 连接主动推送"有新版本"
客户端立刻开始下载更新

优点：实时推送，秒级响应 **缺点：**需要额外部署 SignalR Hub；断线后要回退到轮询

混合策略组合

实际项目中，这些策略可以组合使用：

组合	适合谁	怎么做
① 标准 + UI	几乎所有人	标准检查 + 显示下载进度条
② OSS + ④ 差分	无后端 + 省带宽	更新包放 OSS，只下载增量
③ 静默 + ① 标准	后台服务	定时间隔检查，后台下载
⑤ 跨版本 + ⑥ 推送	强制升级	服务端推送"请立即更新"，跳过所有中间版本

如果你不确定选哪个

有后端服务？

└ 是 → ① 标准（先跑起来再说）

└ 以后需要时再加 ④ 差分 或 ③ 静默

└ 否 → ② OSS（零成本起步）

└ 把更新包上传到 S3/MinIO

这个方案不会错。先跑通再优化。

需要留意的事情

#	问题	建议
1	NuGet 版本不一致导致 "Method not found"	Client 和 Upgrade 的 NuGet 版本必须 完全一致
2	OSS 模式不区分 Main/Upgrade 更新	这是 OSS 的正常限制，接受即可
3	升级程序必须放在 update/ 子目录	从第一个版本就建好这个目录结构
4	Linux/macOS 不支持 Bowl 崩溃守护	只有 Windows 可以用 Bowl
5	差分包不要超过 2GB	大文件用全量包

相关页面

- [generalupdate-init](#) — Bootstrap 配置
- [generalupdate-ui](#) — 更新界面
- [generalupdate-advanced](#) — 高级定制
- [generalupdate-troubleshoot](#) — 遇到问题

GeneralUpdate 高级定制参考

涵盖扩展点架构、Pipeline 管道、差分引擎、Bowl 崩溃守护、事件系统、文件系统工具等。

 **API 版本说明：**本指南基于 **NuGet v10.5.0-beta.7**。以下功能在 v10.5.0-beta.7 中全部**可用**：

-  `IUpdateHooks` 生命周期钩子 (`Hooks()`)
-  `IStrategy` 自定义策略注入 (`Strategy()`)
-  `SilentPollOrchestrator` 静默轮询器 (`Option.Silent`)
-  `Option` 可编程配置系统
-  `ISslValidationPolicy` SSL 策略接口
-  `IHttpAuthProvider` HTTP 认证提供者
-  `DiffPipelineBuilder` 差分管道配置

各功能的命名空间和用法在文中已标注。

用户需求提取

定制目标（必需）

- 需要什么定制：_____（Bowl 崩溃守护 / IPC 替换 / Pipeline 定制 / 自定义策略 / AOT / Drivelution / 黑名单 / 认证提供者 / 差分引擎）
- 使用的 GeneralUpdate 版本：_____（v10.4.6 稳定版 / v10.5.0+ 开发分支）
- .NET 版本：_____（.NET 6/8/9/10）

Bowl（如果选择）

- 被监控进程名：_____
- 工作模式：_____（Normal / Upgrade）
- 是否需要崩溃 Dump：_____（是/否）
- 备份目录路径：_____

IPC 替换（如果选择）

- 替换方式：_____（NamedPipe / SharedMemory / 自定义）
- 目标平台：_____（Windows / Linux / macOS / 跨平台）
- 安全要求：_____（加密 / 签名 / 无额外安全）

AOT（如果选择）

- 当前剪裁警告：_____（有/无）
- 是否使用反射：_____（是/否）
- JSON 序列化需求：_____（有/无）

1. Pipeline 管道系统（v10.5.0-beta.7 可用）

GeneralUpdate 使用 Pipeline 管道模式处理更新包的校验、解压、补丁应用。

PipelineBuilder API

```
using GeneralUpdate.Core.Pipeline;

// 创建管道上下文
var context = new PipelineContext();
context.Add("ZipFilePath", @"C:\temp\update.zip");
context.Add("Hash", "sha256-hex-value");
context.Add("Format", 0); // 0=Zip
context.Add("Encoding", System.Text.Encoding.UTF8);
context.Add("SourcePath", @"C:\Program Files\MyApp");
context.Add("PatchEnabled", true);

// 构建并执行管道
await new PipelineBuilder(context)
    .UseMiddleware() // 哈希校验
    .UseMiddleware() // 解压
    .UseMiddleware() // 差分补丁
    .Build();
```

中间件	类名	命名空间	功能
哈希校验	HashMiddleware	GeneralUpdate.Core.Pipeline	SHA256 完整性校验
解压	CompressMiddleware	GeneralUpdate.Core.Pipeline	解压 ZIP 包
差分补丁	PatchMiddleware	GeneralUpdate.Core.Pipeline	应用 BSDIFF/HDiffPatch 补丁
驱动更新	DrivelutionMiddleware	GeneralUpdate.Core.Pipeline	Windows 驱动安装

2. 策略系统 (v10.5.0-beta.7 可用)

GeneralUpdate 内置三种平台策略，通过 `IStrategy` 接口实现：

策略	类名	平台
Windows	WindowsStrategy	Windows
Linux	LinuxStrategy	Linux
OSS	OSSStrategy	跨平台（对象存储）

✔ 支持通过 `bootstrap.Strategy()` 注入自定义策略。自定义策略需要实现 `IStrategy` 接口。

3. Bowl 崩溃守护 (v10.5.0-beta.7)

Bowl 是一个崩溃监控组件，通过 `BowlContext` 配置。

```
using GeneralUpdate.Bowl;

var context = new BowlContext
{
    ProcessNameOrId = "MyApp.exe",
    DumpFileName = "v1.0.0.0_fail.dmp",
    FailFileName = "v1.0.0.0_fail.json",
    TargetPath = @"C:\Program Files\MyApp",
    FailDirectory = @"C:\Program Files\MyApp\fail",
    BackupDirectory = @"C:\Program Files\MyApp\backup",
    WorkModel = "Upgrade",
    TimeoutMs = 30_000,
    AutoRestore = true,
    OnCrash = async (info, ct) => Console.WriteLine($"Crash: {info.DumpFilePath}"),
};

var bowl = new Bowl();
var result = await bowl.LaunchAsync(context);
Console.WriteLine($"Result: Success={result.Success}, Restored={result.Restored}");
```

属性	类型	说明
ProcessNameOrId	string	被监控的进程名或 PID（必填）
TargetPath	string	应用安装根目录（必填）
DumpFileName	string	Dump 文件名（必填）
FailFileName	string	故障报告文件名（必填）
FailDirectory	string	崩溃报告输出目录（必填）
BackupDirectory	string	备份目录（必填）
WorkModel	string	"Upgrade" 或 "Normal"
TimeoutMs	int	监控超时(毫秒)，默认 30000
AutoRestore	bool	崩溃后自动回滚
DumpType	DumpType	Mini / Full
OnCrash	delegate	崩溃回调

 NuGet v10.5.0-beta.7 中 Bowl 和 Core 无类型冲突，可以同时引用。

4. EventManager 事件系统（v10.5.0-beta.7 可用）

EventManager 是一个全局单例，提供事件的发布和订阅：

```
using GeneralUpdate.Core.Event;

// 添加监听
EventManager.Instance.AddListener((object? sender, UpdateInfoEventArgs e) =>
{
    // 处理版本发现事件
});

// 手动分发事件
EventManager.Instance.Dispatch(this, new ExceptionEventArgs(ex, "自定义错误"));

// 清空所有监听
EventManager.Instance.Clear();

// 释放
EventManager.Instance.Dispose();
```

⚠ `EventManager` 是全局单例，`Dispose()` 后 `Instance` 仍然可访问。

5. 文件系统工具 (v10.5.0-beta.7 可用)

BlackList (黑名单)

`UpdateRequest` 支持通过以下属性排除文件：

```
var config = new UpdateRequest
{
    // ...
    Files = new List<string> { "*.log", "*.tmp" },
    Formats = new List<string> { ".pdb", ".vshost.exe" },
    Directories = new List<string> { "logs", "cache", "temp" },
};
```

黑名单内部通过 `ToBlackPolicy()` 转换为 `BlackPolicy` 记录。

FileTree (文件树对比)

```
using GeneralUpdate.Core.FileSystem;

var tree = new FileTree();
var snapshot = tree.CreateSnapshot(@"C:\Program Files\MyApp");
```

6. 差分引擎 (v10.5.0-beta.7 可用, 无需额外安装包)

差分类型已内嵌在 `GeneralUpdate.Core` 中, 无需额外安装 `GeneralUpdate.Differential` 包。

DiffPipelineBuilder 方式 (推荐)

```
using GeneralUpdate.Core.Pipeline;

var pipeline = new DiffPipelineBuilder()
    .UseDiffer(new StreamingHdiffDiffer()) // 差分算法
    .UseCleanMatcher(new DefaultCleanMatcher()) // 文件匹配器 (服务端)
    .UseDirtyMatcher(new DefaultDirtyMatcher()) // 文件匹配器 (客户端)
    .WithParallelism(4)
    .WithStopOnFirstError(true)
    .WithProgress(new Progress(p =>
        Console.WriteLine($"{p.Completed}/{p.Total} [{p.FileName}]")))
    .Build();

// 服务端: 生成补丁
await pipeline.CleanAsync(oldDir, newDir, patchDir);

// 客户端: 应用补丁
await pipeline.DirtyAsync(appDir, patchDir);
```

Bootstrap 集成方式

```
new GeneralUpdateBootstrap()
    .SetConfig(config)
    .UseDiffPipeline(pipeline =>
    {
        pipeline.WithParallelism(2)
            .WithStopOnFirstError(true);
    })
    .LaunchAsync();
```

自定义匹配器

```
using GeneralUpdate.Core.Differential;

// 自定义清理匹配器
var cleanMatcher = new DefaultCleanMatcher(); // 或实现 ICleanMatcher
var dirtyMatcher = new DefaultDirtyMatcher(); // 或实现 IDirtyMatcher
```

7. AOT / NativeAOT 兼容性

GeneralUpdate.Core v10.5.0-beta.7 支持 .NET Native AOT (`net8.0` 和 `net10.0`) :

```
true
true
```

JSON 序列化上下文 (减少 AOT 大小) :

```
using GeneralUpdate.Core.JsonContext;

// 使用内置的 JsonSerializerContext
// VersionRespJsonContext, ProcessContractJsonContext, HttpParameterJsonContext 等
```

8. Drivelution (Windows 驱动更新)

`GeneralUpdate.Drivelution` 包提供 Windows 驱动管理:

```
using GeneralUpdate.Drivelution;

// 扫描驱动目录
var allDrivers = GeneralDrivelution.ScanDirectory(driverDir);

// 验证驱动
var isValid = GeneralDrivelution.ValidateDriver(driverPath);

// 安装驱动 (DIFx → SetupAPI → PnPUTil 级联)
var result = GeneralDrivelution.InstallDriver(driverPath);
```

内容索引

主题	可用性	参考
Pipeline 管道	✔ v10.5.0-beta.7	<code>GeneralUpdate.Core.Pipeline</code>
策略系统	✔ v10.5.0-beta.7	<code>GeneralUpdate.Core.Strategy</code>
FileTree	✔ v10.5.0-beta.7	<code>GeneralUpdate.Core.FileSystem</code>
BlackList	✔ v10.5.0-beta.7	<code>UpdateRequest.Files/Formats/Directories</code> → <code>ToBlackPolicy()</code>
差分引擎	✔ 内嵌 Core	<code>DiffPipelineBuilder</code> / <code>DiffPipeline</code>
AOT	✔ v10.5.0-beta.7	<code>JsonSerializerContext</code> 子类
EventManager	✔ v10.5.0-beta.7	<code>GeneralUpdate.Core.Event</code>
Bowl 崩溃守护	✔ v10.5.0-beta.7	<code>GeneralUpdate.Bowl.Bowl</code>

主题	可用性	参考
IUpdateHooks	✔ v10.5.0-beta.7	<code>GeneralUpdate.Core.Hooks</code> — <code>Hooks()</code>
自定义 Strategy 注入	✔ v10.5.0-beta.7	<code>Strategy()</code>
IPC 替换接口	✘ 暂不支持	使用 NamedPipe 替代方案
SilentPollOrchestrator	✔ v10.5.0-beta.7	<code>Option.Silent</code> + <code>SetOption()</code>
Option 系统	✔ v10.5.0-beta.7	<code>SetOption(Option, T)</code>

✔ 高级定制验证清单

Bowl 崩溃守护

- ❑ 如果用 Bowl：项目中同时引用 `GeneralUpdate.Core` 和 `GeneralUpdate.Bowl` (v10.5.0-beta.7 无冲突)
- ❑ `BowlContext` 的 `ProcessNameOrId` 与实际进程名匹配
- ❑ `TargetPath` 设置为应用安装根目录，非子目录
- ❑ `WorkModel` 根据场景选择 `Correct` (`Normal/Upgrade`)
- ❑ `FailDirectory` 有写入权限
- ❑ Linux/macOS 无此功能 (Bowl 仅 Windows)

Pipeline 定制

- ❑ `PipelineContext` 中的 Key 名称使用字符串常量拼写正确
- ❑ 中间件注册顺序正确：`Hash` → `Compress` → `Patch` → `Drivelution`
- ❑ `Encoding` 设置为 `Encoding.UTF8`

AOT/NativeAOT

- ❑ 启用了 `true`
- ❑ 对反射路径添加了 `[DynamicDependency]` 或 `[RequiresUnreferencedCode]`
- ❑ 使用了内置的 `JsonSerializerContext` 子类 (减少裁剪)

⚠ 反模式清单

#	反模式	后果	正确做法
1	在 v10.4.6 稳定版上使用开发分支 API (IUpdateHooks 等)	编译失败 / 运行时 <code>MissingMethodException</code>	检查 API 可用性表
2	PipelineContext Key 拼写错误	Pipeline 运行异常，值未传递	使用类库公开的常量或文档中的 Key 名
3	Bowl 的 WorkModel 设为 Upgrade 但进程是主程序	监控逻辑错误	<code>Normal</code> =主线进程， <code>Upgrade</code> =升级进程
4	Windows 上 IPC 使用默认加密密钥	加密可被破解	使用强密钥 (≥ 32 字符)
5	差分包生成时使用不同版本的源文件结构	补丁应用失败	源和目标版本的文件结构必须一致

#	反模式	后果	正确做法
6	AOT 项目中使用了大量反射且未标记 DynamicDependency	运行时 TypeLoadException	使用源代码生成器或显式标记保留
7	Pipeline 中 PatchMiddleware 排在 CompressMiddleware 前面	未解压就试图打补丁	顺序必须是 Compress→Patch

相关技能

- `/generalupdate-init` — Bootstrap 配置
- `/generalupdate-strategy` — 更新策略选择
- `/generalupdate-troubleshoot` — 问题诊断

综合性诊断系统 — 覆盖 50+ 已知问题，均可追溯到 GitHub/Gitee Issue 或代码审计发现。

用户症状提取

必填信息

- 症状描述：_____
- 错误信息/堆栈：_____
- GeneralUpdate 版本：_____
- 平台：_____（Windows / Linux / macOS）
- .NET 版本：_____
- 更新策略：_____（标准 / OSS / 静默 / 差分 / 跨版本 / 推送）
- 最近是否改过配置：_____（是/否，改了啥）

可选信息

- 事件监听中是否有异常（ExceptionEventArgs）：_____
- 是否有日志（Logs/generalupdate-trace-*.log）：_____
- 问题是否可复现：_____（是/否，频率）
- 首次出现时间点：_____

工作流程

1. 症状收集

- └─ 用户描述的症状是什么？
- └─ 错误信息/堆栈是什么？
- └─ GeneralUpdate 版本号？
- └─ 平台（Windows/Linux/macOS）？
- └─ 更新策略（标准/OSS/静默）？

2. 症状匹配

- └─ 优先：python3 scripts/search.py "<症状>" --domain issue
 - └─ 匹配到 → 给出根因 + 修复 + 代码
- └─ 未匹配 → 降级到 reference.md 全文搜索

3. 提供修复

- └─ 具体的代码修改、配置调整、版本升级建议
- └─ 预防措施（如何避免再发生）

4. 验证

- └─ 确认修复后问题解决

症状搜索（推荐）

优先使用 BM25 搜索引擎精确匹配已知问题，而不是在 reference.md 中手动查找：

```
# 自然语言搜索已知问题
python3 .claude/skills/generalupdate-troubleshoot/scripts/search.py "升级后应用启动不了" --domain issue
python3 .claude/skills/generalupdate-troubleshoot/scripts/search.py "方法找不到 MethodNotFound" --domain issue
python3 .claude/skills/generalupdate-troubleshoot/scripts/search.py "中文乱码 garbled" --domain issue

# 搜索策略相关问题
python3 .claude/skills/generalupdate-troubleshoot/scripts/search.py "OSS 权限问题" --domain strategy
```

症状分级

reference.md 中的问题按严重程度分级：

级别	颜色	含义	数量
C	致命	阻断性故障、数据损坏、安全漏洞	8
H	高	场景阻断、功能失效、需要升级	11
M	中	功能异常、需要配置调整	20
L	低	代码气味、边缘情况、已知行为	12

完整清单请查阅 `reference.md`

通用诊断前检查清单

运行环境检查

- ☐ 目标机器安装了正确的 .NET 运行时（版本与发布框架匹配）
- ☐ 目标机器上有写入权限（InstallPath 目录可写）
- ☐ 防火墙未阻断 UpdateUrl 的通信端口
- ☐ 磁盘空间充足（至少 2× 更新包大小）
- ☐ Linux/macOS: UpgradeApp 有 `chmod +x` 执行权限

版本检查

- ☐ Client 和 Upgrade 项目 NuGet 版本**完全一致**
- ☐ 服务端返回的版本号是 4 段式（如 1.0.0.0）
- ☐ manifest.json 中 `mainAppName` 与实际进程名匹配
- ☐ `AppType` 设置正确（Client = 1, Upgrade = 2）

配置检查

- ☐ `UpdateRequest` 的 6 个必填字段都已设置
- ☐ `UpdateUrl` 可通过 HTTP GET 访问并返回合法 JSON

- AppSecretKey 与服务端配置一致（长度 ≥ 16 字符）
- UpgradeApp.exe 存在于发布目录的 update/ 子目录中

日志检查

- 查看 Logs/generalupdate-trace-*.log （如有）
- 检查事件监听中的 ExceptionEventArgs
- 检查 MultiDownloadErrorEventArgs 中的异常

C 级（Critical） — 阻塞升级

问题	原因	排查/解决
升级没启动	LaunchAsync() 未调用 / UpgradeApp.exe 未部署	确认 Bootstrap.LaunchAsync() 在 Main() 中调用
Method not found	Client 和 Upgrade NuGet 版本不一致	统一 NuGet 版本, 清理 bin/obj 后重新生成
路径超长 (>260)	Windows 路径限制	缩短安装路径
IPC 暴露	IPC 加密密钥硬编码	使用强 AppSecretKey; 更新到 v10.4.6+
跨租户泄露	服务端多租户隔离缺失	每个租户独立 ProductId + AppSecretKey
ZIP 遍历写入	恶意 ZIP 含 ../ 路径	v10.4.6+ 已修复
BSDIFF 整数溢出	大文件差分补丁计算溢出	使用 HDiffPatch 算法
静默不生效	进程退出时未触发	确保正确调用 Close() 或 Dispose()

H 级（High） — 严重但不阻塞启动

问题	原因	排查/解决
无限循环更新	manifest.json 版本号未回写	更新到 v10.4.6+ （已修复 WriteBack）
OSS 无更新	Bucket 配置错误 / versions.json 格式不对	curl 测试 OSS URL 是否可下载
文件占用	目标文件被占用	关闭主进程后更新; 排除杀软扫描目录
SignalR 推送无响应	连接断开或认证失败	检查 SignalR Hub 状态和 Token 配置
Bowl 不守护	进程名配置错误	确认 ProcessNameOrId 与实际进程名一致
差分更新失败	基准文件不匹配	校验原始文件哈希匹配
下载进度不动	IDownloadService 未正确绑定	确认桥接实现是否正确
跨版本跳跃失败	中间版本包缺失	确保服务端保留了所有版本的补丁

M 级（Medium） — 功能降级

问题	原因	排查/解决
AOT 编译失败	反射代码未适配 NativeAOT	添加 [DynamicDependency] 属性
SignalR 重连慢	RetryDelay 配置过长	调整重连参数

问题	原因	排查/解决
日志不输出	日志路径权限不足	检查 %TEMP%/GeneralUpdate/logs/ 权限
多租户配置错误	ProductId 冲突	确保每个租户唯一 ProductId
黑名单不生效	格式配置错误	确认文件名和扩展名格式正确

L 级（Low） — 非关键

问题	原因	排查/解决
分发包过大	未使用差分	差分已内嵌在 Core，启用 PatchEnabled 即可
首次更新慢	CDN 冷启动	预热 CDN
更新后配置丢失	黑名单未包含配置目录	确认 Directories 包含配置文件夹

通用诊断流程（6 步）

当问题无法直接匹配到已知症状时，执行以下 6 步排查：

1. **版本一致性检查** — Client 和 Upgrade 的 NuGet 版本是否一致？
2. **manifest.json 验证** — 文件是否存在？字段值是否正确？
3. **UpgradeApp 存在性** — UpgradeApp.exe 是否在预期目录？
4. **网络可访问性** — UpdateUrl 能否用 curl 访问？
5. **日志分析** — 查看 Logs/generalupdate-trace-*.log 下的日志文件
6. **最小重现** — 从 Minimal 集成开始，逐步增加复杂度

日志文件位置

平台	默认路径
Windows	%TEMP%/GeneralUpdate/logs/
Linux	/tmp/GeneralUpdate/logs/

安全注意事项

- AppSecretKey 管理 — 硬编码在客户端是最后手段; 优先从启动参数或环境变量注入
- 定期轮换 IPC 加密密钥
- 生产环境禁用调试日志

⚠ 诊断阶段的反模式

#	反模式	后果	正确做法
1	只看错误信息不看事件	错过 ExceptionEventArgs 中的详细信息	订阅所有 7 个事件
2	日志文件路径不对就认为无日志	漏掉关键诊断信息	在 InstallPath/Logs 下查找
3	只检查 Client 不检查 Upgrade 进程	问题在 Upgrade 端但诊断方向全错	两端都要检查

#	反模式	后果	正确做法
4	升级问题直接改代码	可能是服务端配置问题而非客户端 Bug	优先检查服务端返回的版本信息
5	忽略 NuGet 版本一致性	方向错, "Method not found" 根因是版本不一致	第一个就要检查版本
6	只在 Debug 环境测试	Release 环境可能缺少运行时文件	在发布/生产环境复现

相关技能

- `/generalupdate-init` — Bootstrap 配置
- `/generalupdate-ui` — 更新界面诊断
- `/generalupdate-strategy` — 策略相关故障
- `/generalupdate-advanced` — 高级功能故障

GeneralUpdate 迁移指南

帮助开发者从旧版本 GeneralUpdate 迁移到最新稳定版 API (v10.5.0-beta.7)。

 **目标版本：NuGet v10.5.0-beta.7** 开发分支 (v10.5.0-beta.x) API 与稳定版有根本性差异。

迁移前需求提取

当前状态

- 当前 GeneralUpdate 版本：_____ (v9.x / v10.0-10.3 / v10.5.0-beta.x / 不确定)
- 当前 .NET 版本：_____
- UI 框架：_____
- 是否使用了 Bowl：_____ (是/否)
- 是否使用了 Differential：_____ (是/否)

迁移后目标

- 目标版本：_____ (v10.5.0-beta.7 / 继续用开发分支)
- 是否需要新的功能 (Bowl/IPC 替换/AOT)：_____

迁移路径

路径 A：v9.x → v10.5.0-beta.7

这是最大的跳跃。v9.x 和 v10 的架构完全不同。

v9.x (单进程, HttpClient 直连)

↓

Breaking Changes:

- └─ 单进程 → 双进程架构 (Client + Upgrade)
- └─ HttpClient 直连 → GeneralSpacestation 服务端
- └─ 无 IPC → AES 加密 IPC 文件
- └─ 无 manifest.json → 必须携带 manifest
- └─ API 命名空间全部重命名

↓

v10.5.0-beta.7 (双进程, UpdateRequest + Bootstrap)

迁移步骤：

```
// ❌ v9.x 写法（不复存在）
// var updater = new GeneralUpdater("https://api/method");
// updater.Start();

// ✅ v10.5.0-beta.7 写法
await new GeneralUpdateBootstrap()
    .SetConfig(new UpdateRequest
    {
        UpdateUrl = "https://your-server.com/Upgrade/Verification",
        AppSecretKey = "your-secret-key",
        MainAppName = "MyApp.exe",
        ClientVersion = "1.0.0.0",
        ProductId = "my-product-001",
        InstallPath = "."
    })
    .LaunchAsync();
```

v9.x API	v10.5.0-beta.7 对应	说明
GeneralUpdater	GeneralUpdateBootstrap	完全重命名
SetApiUrl() / SetMethod()	UpdateRequest.UpdateUrl	统一到 UpdateRequest
CheckUpdateAsync()	.LaunchAsync()	异步改为返回 Bootstrap 实例
单进程直接更新	Client + Upgrade 双进程	必须创建独立 Upgrade 项目
N/A	generalupdate.manifest.json	必须随首发版本发布

路径 B: v10.5.0-beta.x (开发分支) → v10.5.0-beta.7

| 开发分支 API (v10.5.0-beta.x) | 稳定版替代 (v10.5.0-beta.7) | 处理方式 |

开发分支 API (v10.5.0-beta.x)	稳定版替代 (v10.5.0-beta.7)	处理方式
new Option() / SetOption()	存在且兼容	直接使用，命名空间有调整
.Hooks() / IUpdateHooks	存在且兼容	直接使用
.Strategy() / IStrategy	存在且兼容	直接使用
SilentPollOrchestrator	存在	Option.Silent + SetOption()
ISslValidationPolicy	存在	直接使用
Configinfo	已移除	改用 UpdateRequest
OssClient (AppType=3,4)	存在	使用 AppType.OssClient 和 AppType.OssUpgrade
硬编码版本号	UpdateRequest.ClientVersion	建议使用 Assembly.GetEntryAssembly()?.GetName()?.Version

迁移验证清单

编译验证

- ☐ `dotnet build` 无错误
- ☐ 无 `MissingMethodException` 的风险
- ☐ 无 `CS0433` 类型冲突 (v10.5.0-beta.7 中 Core + Bowl 无冲突, 可同时引用)

架构验证

- ☐ 项目已拆分为 Client + Upgrade 两个独立项目
- ☐ Upgrade 项目 `AppType = 2`
- ☐ Client 项目 `AppType = 1`
- ☐ `generalupdate.manifest.json` 存在且配置正确

运行验证

- ☐ 版本检查 API 可正常返回
- ☐ 下载后 Upgrade 进程可正常启动
- ☐ 更新完成后主程序可正常重启
- ☐ IPC 文件编码设为 `Encoding.UTF8`

⚠ 迁移反模式

#	反模式	后果	正确做法
1	直接在项目中替换 NuGet 版本不修改代码	大量编译错误	先清理旧 API 引用再升级 NuGet
2	认为 v9.x 的配置对象就是 <code>UpdateRequest</code>	<code>UpdateRequest</code> 属性名完全不同	对照文档重新写 <code>UpdateRequest</code>
3	试图在 v10.4.6 中使用 dev-branch 的 API	<code>MissingMethodException</code>	检查 API 可用性表
4	迁移后不测试 Upgrade 进程	主程序能更新但 Upgrade 崩溃	两端都要测试
5	保留旧的 v9.x 引用不删除	类型冲突	清空 csproj 重新添加引用

相关技能

- `/generalupdate-init` — 从零开始的集成指南
- `/generalupdate-troubleshoot` — 迁移后问题诊断
- `/generalupdate-security-audit` — 迁移后安全审计

GeneralUpdate 安全审计指南

全面覆盖 GeneralUpdate 部署的安全风险面。基于代码审计发现（17 CRITICAL/HIGH 项）和最佳实践。

审计前信息收集

- ### 部署环境
- 部署模式：_____（内网 / 公网 / 混合）
 - 租户模式：_____（单租户 / 多租户）
 - 客户端数量：_____
 - 客户端操作系统：_____（Windows / Linux / macOS / 混合）
- ### 服务端
- 后端类型：_____（GeneralSpacestation / 自定义 / OSS）
 - 传输协议：_____（HTTP / HTTPS）
 - 认证方式：_____（Bearer / Basic / HMAC / 无）
 - API 是否公开访问：_____（是 / 否，有网络隔离）
- ### 客户端
- GeneralUpdate 版本：_____
 - 是否使用 IPC：_____（是 / 否）
 - 是否使用 Bowl：_____（是 / 否）
 - 是否使用 Differential：_____（是 / 否）

安全审计矩阵

#	检查项	严重度	描述	修复措施
S01	AppSecretKey 强度	 CRITICAL	密钥长度不足、纯字母、与示例代码相同	使用 ≥ 32 字符，大小写+数字+符号的随机密钥
S02	IPC 加密	 CRITICAL	默认 IPC 加密密钥硬编码在二进制中	确保 AppSecretKey 唯一且服务端/客户端一致
S03	HTTPS 传输	 HIGH	UpdateUrl 使用 HTTP 而非 HTTPS	生产环境强制 HTTPS；配置 HSTS
S04	ZipSlip 路径穿越	 CRITICAL	解压 ZIP 时未验证 ../ 路径	验证压缩包条目路径是否在目标目录内
S05	多租户隔离	 CRITICAL	服务端未按 ProductId 隔离租户	服务端添加租户身份验证中间件
S06	事件日志泄露	 MEDIUM	ExceptionEventArgs 日志可能包含敏感路径	脱敏后记录，过滤路径和密钥
S07	差分包签名	 HIGH	差分补丁无数字签名验证	对更新包进行 Authenticode 签名

#	检查项	严重度	描述	修复措施
S08	临时目录权限	 MEDIUM	临时解压目录权限可能过大	设置仅为当前用户可读写
S09	OSS Bucket 权限	 HIGH	更新包存储 Bucket 设为公共读	设置为私有，使用预签名 URL
S10	依赖版本漏洞	 MEDIUM	GeneralUpdate 及其依赖可能存在已知 CVE	定期检查 NuGet 依赖安全公告
S11	回滚攻击	 HIGH	攻击者可提交降级版本号强制安装旧版本	服务端校验版本号单调递增
S12	下载完整性	 HIGH	下载的更新包无完整性校验	确保 Pipeline 包含 HashMiddleware
S13	Bowl 提权	 MEDIUM	Bowl 崩溃守护以高权限运行可能被滥用	以最小必要权限运行 Bowl
S14	信息泄露通过 manifest	 LOW	manifest.json 中的 ProductId、版本号可被枚举	非公开环境下不暴露 manifest 文件

审计报告输出格式

完成审计后按以下格式输出：

🛡️ GeneralUpdate 安全审计报告

概要

- 项目：_____
- 审计日期：_____
- 总体评分：A/B/C/D/F
- 严重问题：_____ 个
- 高风险：_____ 个
- 中风险：_____ 个
- 低风险：_____ 个

严重问题（必须立即修复）

- S01 AppSecretKey 强度：⚠️ 当前密钥长度为 X，需要 ≥ 32
修复：_____

高风险（建议尽快修复）

...

中风险（评估后修复）

...

低风险（记录在案）

...

修复建议优先级

1. 立即：S01, S03, S04
2. 本周：S05, S07, S09
3. 本月：S08, S10, S11

安全配置检查清单

- ☐ AppSecretKey 长度 ≥ 32 字符，混合大小写+数字+符号
- ☐ 生产环境使用 HTTPS
- ☐ IPC 文件编码设为 Encoding.UTF8
- ☐ Pipeline 包含 HashMiddleware 做完整性校验
- ☐ OSS Bucket 权限设为私有
- ☐ 服务端按 ProductId 隔离租户
- ☐ 版本号严格单调递增
- ☐ 更新包进行 Authenticode 签名
- ☐ Zip 解压有路径穿越防护
- ☐ 日志中不记录敏感信息

相关技能

- `/generalupdate-init` — 修复审计发现的问题
- `/generalupdate-advanced` — IPC 替换、自定义认证
- `/generalupdate-troubleshoot` — 已知安全问题参考